



CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

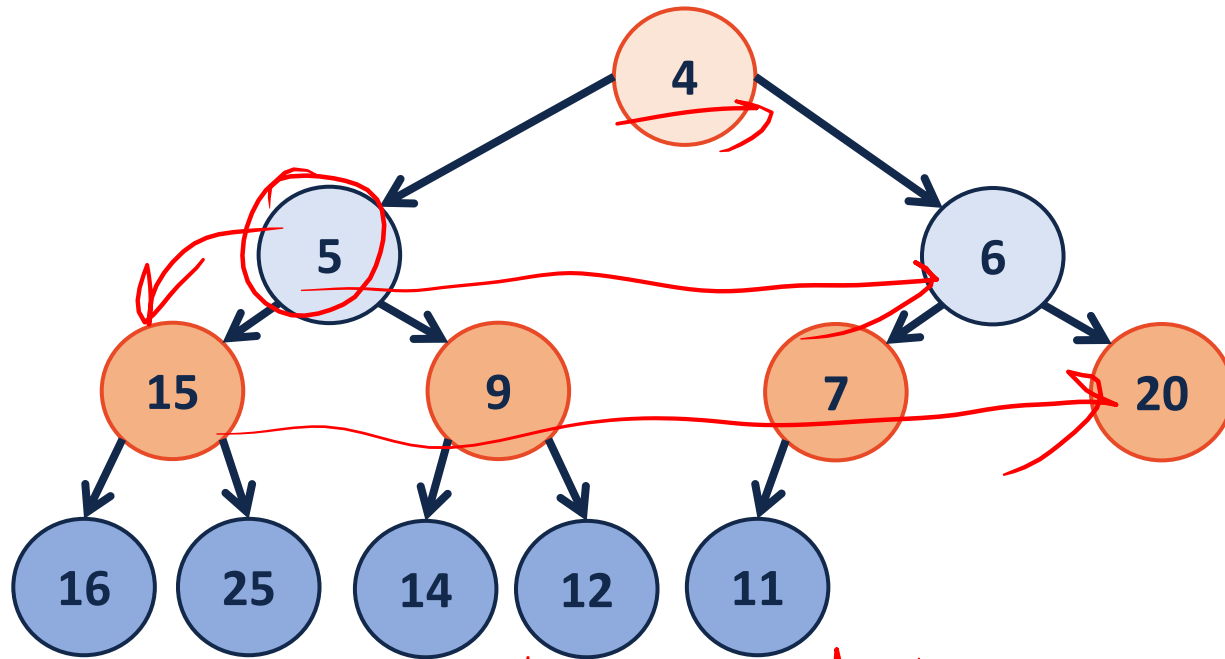
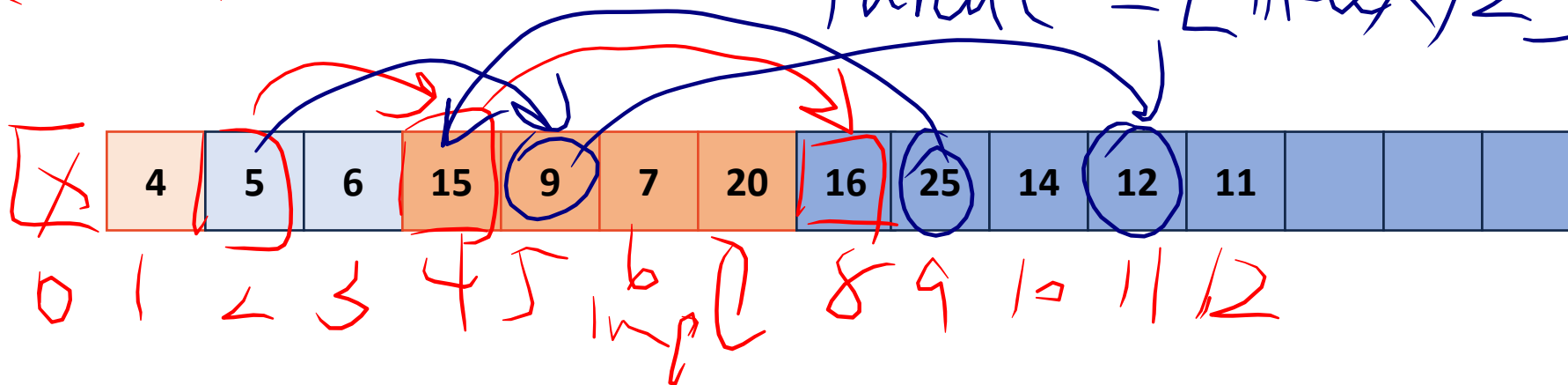
(min)Heap

Level order TrAV
★ In a heap impl
[0] is not used

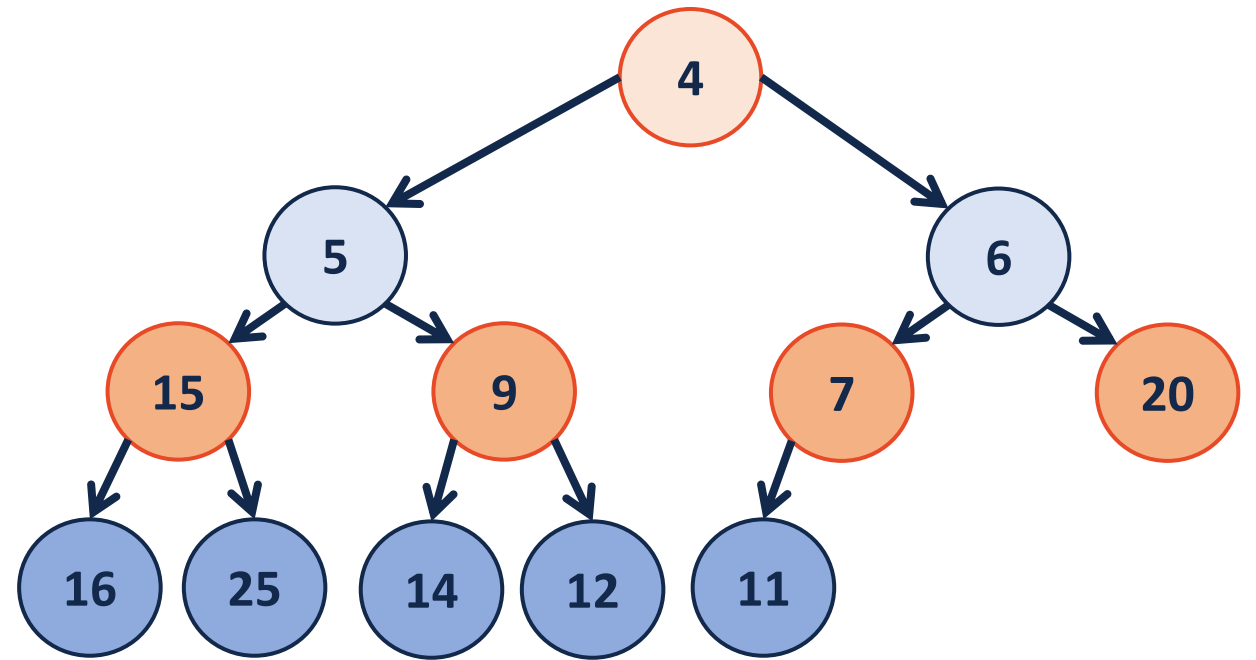
Left child = $2 * \text{index}$
Right child = $2 * \text{index} + 1$

visual model

Parent = $\lfloor \text{index} / 2 \rfloor$



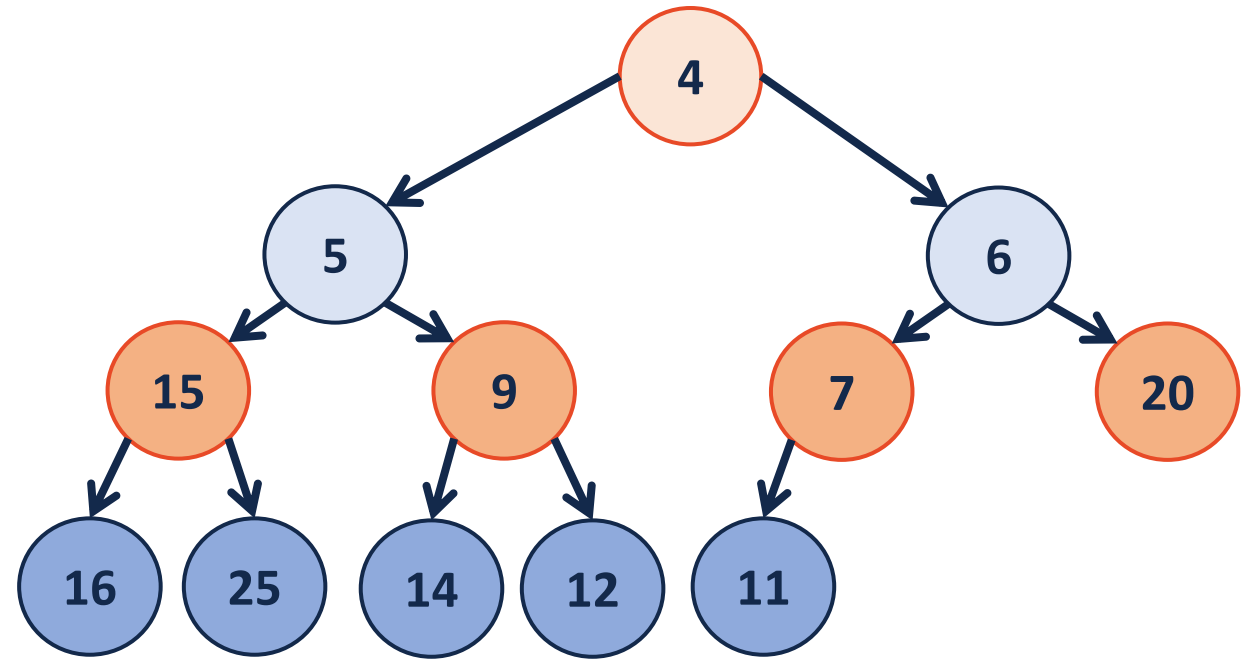
insert



	4	5	6	15	9	7	20	16	25	14	12	11			
--	---	---	---	----	---	---	----	----	----	----	----	----	--	--	--

insert

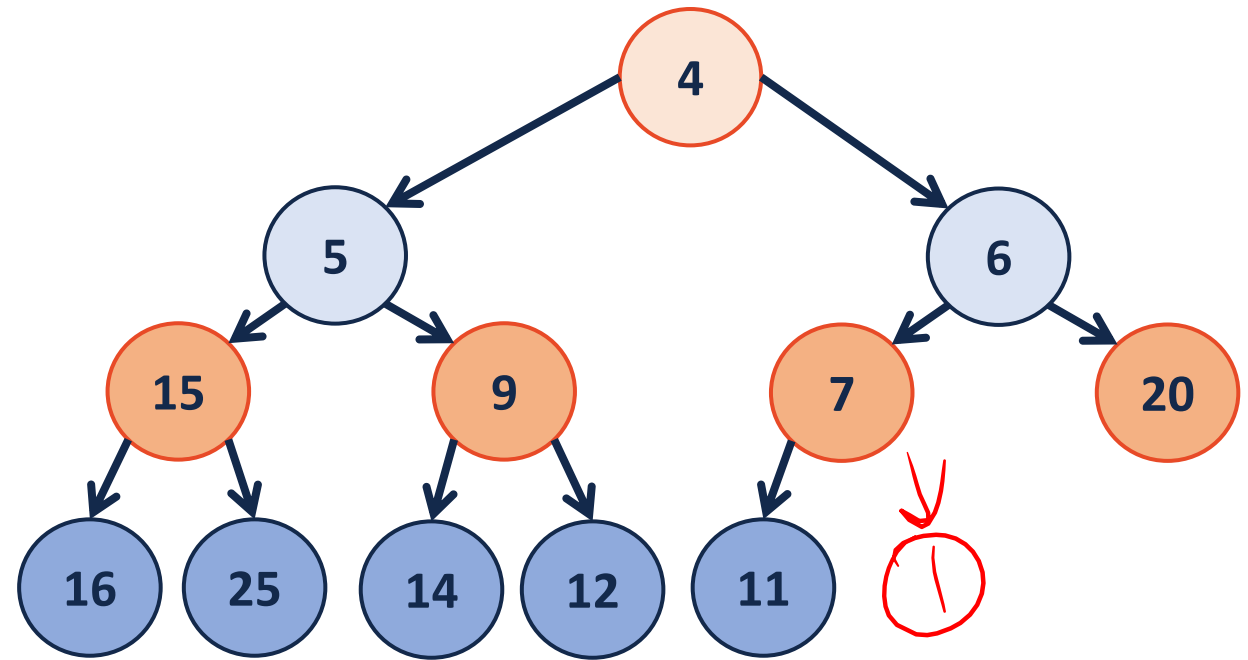
insert (1)



	4	5	6	15	9	7	20	16	25	14	12	11			
--	---	---	---	----	---	---	----	----	----	----	----	----	--	--	--

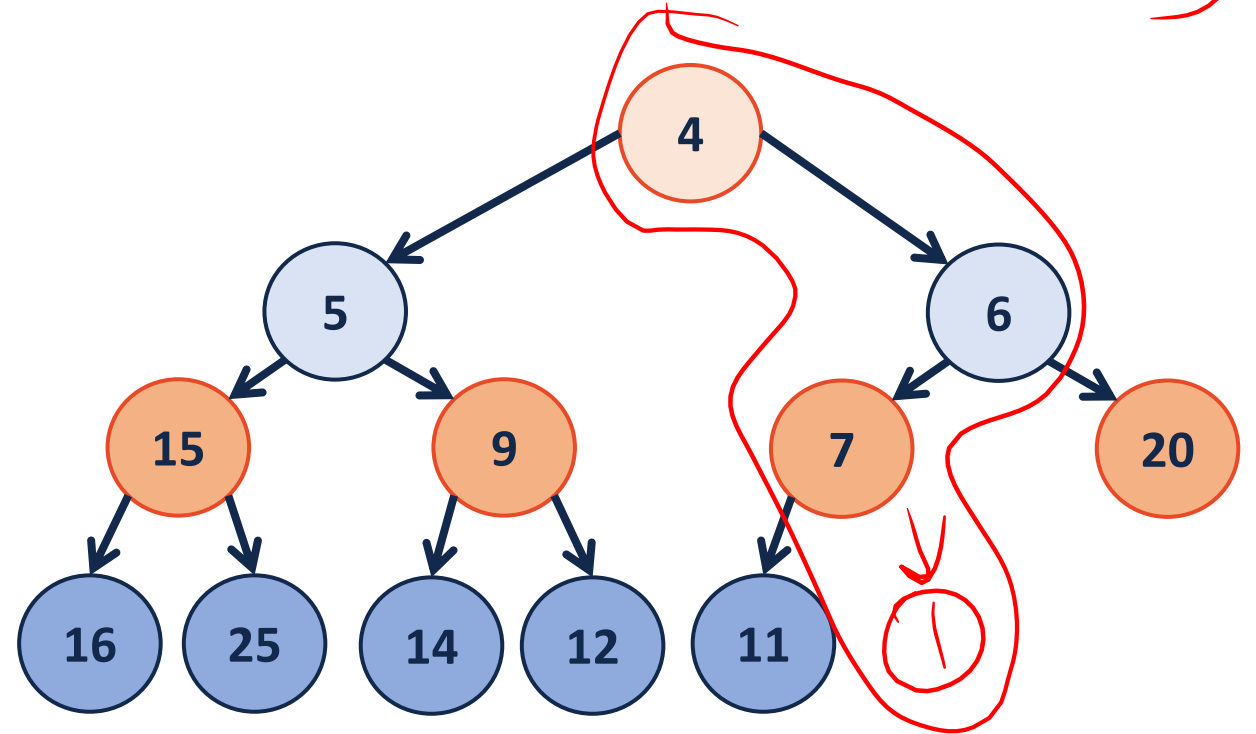
insert

insert (1)



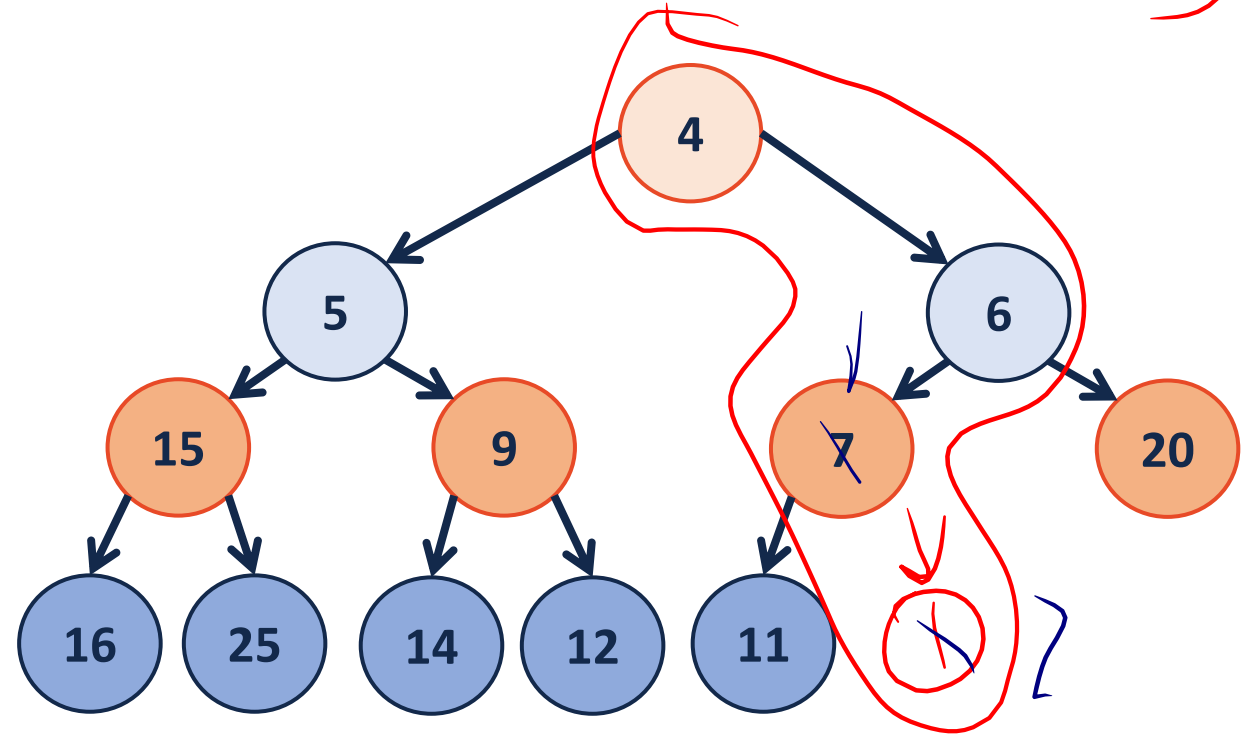
	4	5	6	15	9	7	20	16	25	14	12	11	/			
--	---	---	---	----	---	---	----	----	----	----	----	----	---	--	--	--

insert



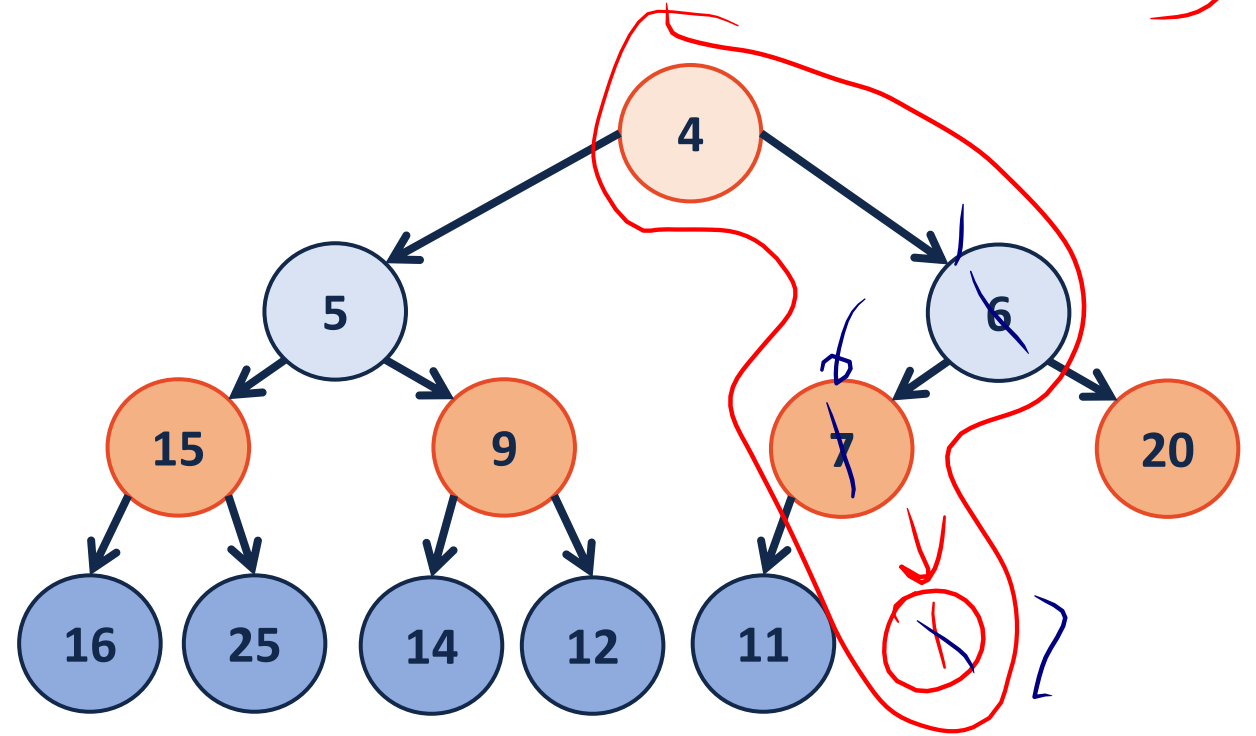
	4	5	6	15	9	7	20	16	25	14	12	11	/			
--	---	---	---	----	---	---	----	----	----	----	----	----	---	--	--	--

insert



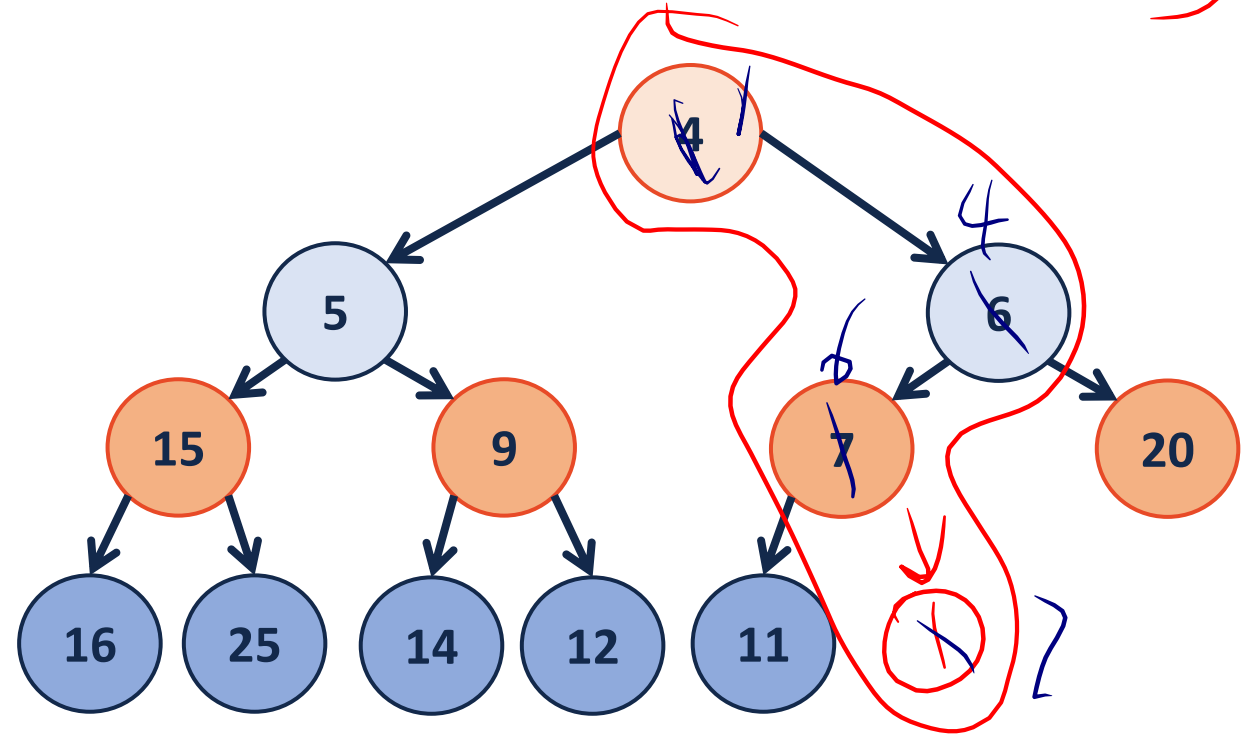
	4	5	6	15	9	7	20	16	25	14	12	11			
--	---	---	---	----	---	---	----	----	----	----	----	----	---	--	--

insert



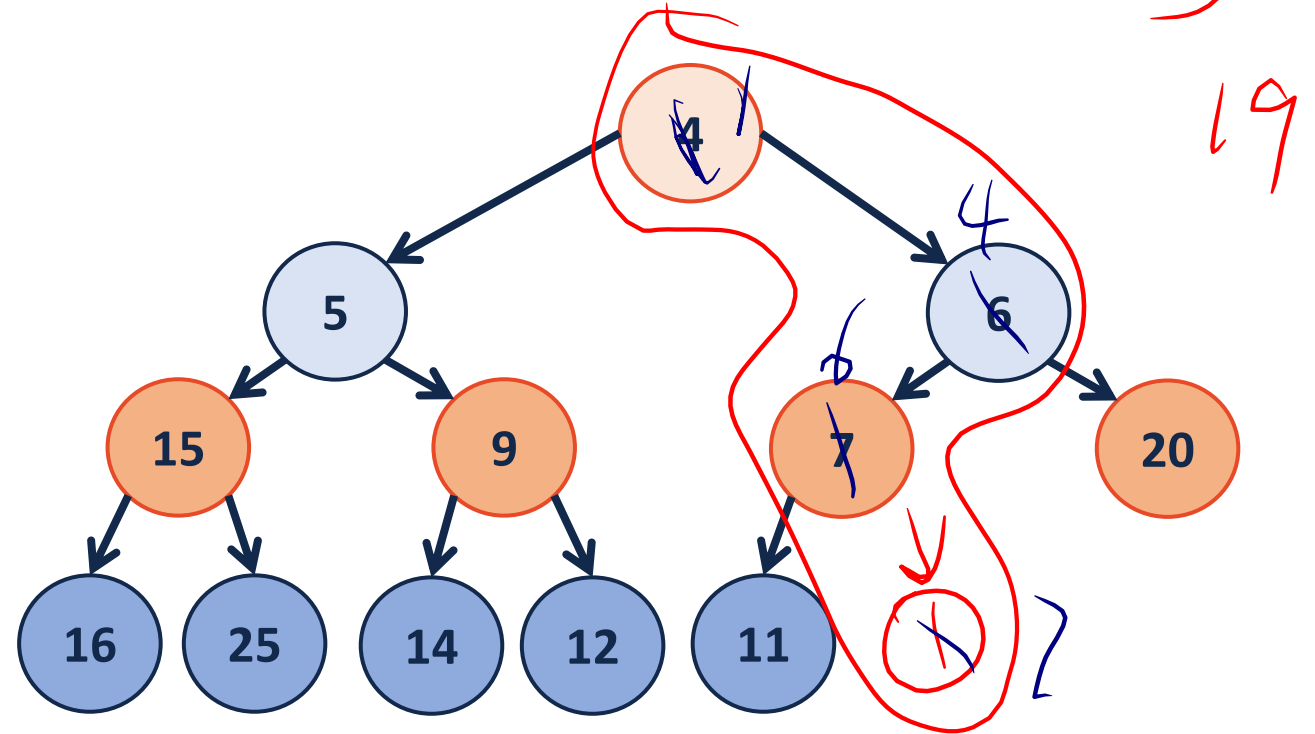
	4	5	6	15	9	7	20	16	25	14	12	11	/			
--	---	---	---	----	---	---	----	----	----	----	----	----	---	--	--	--

insert



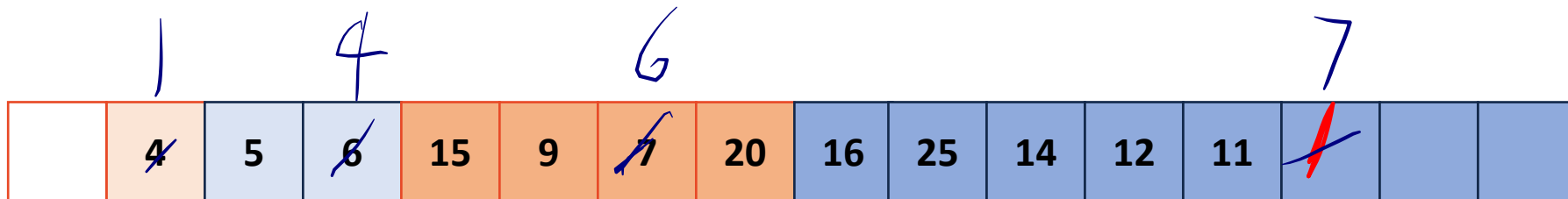
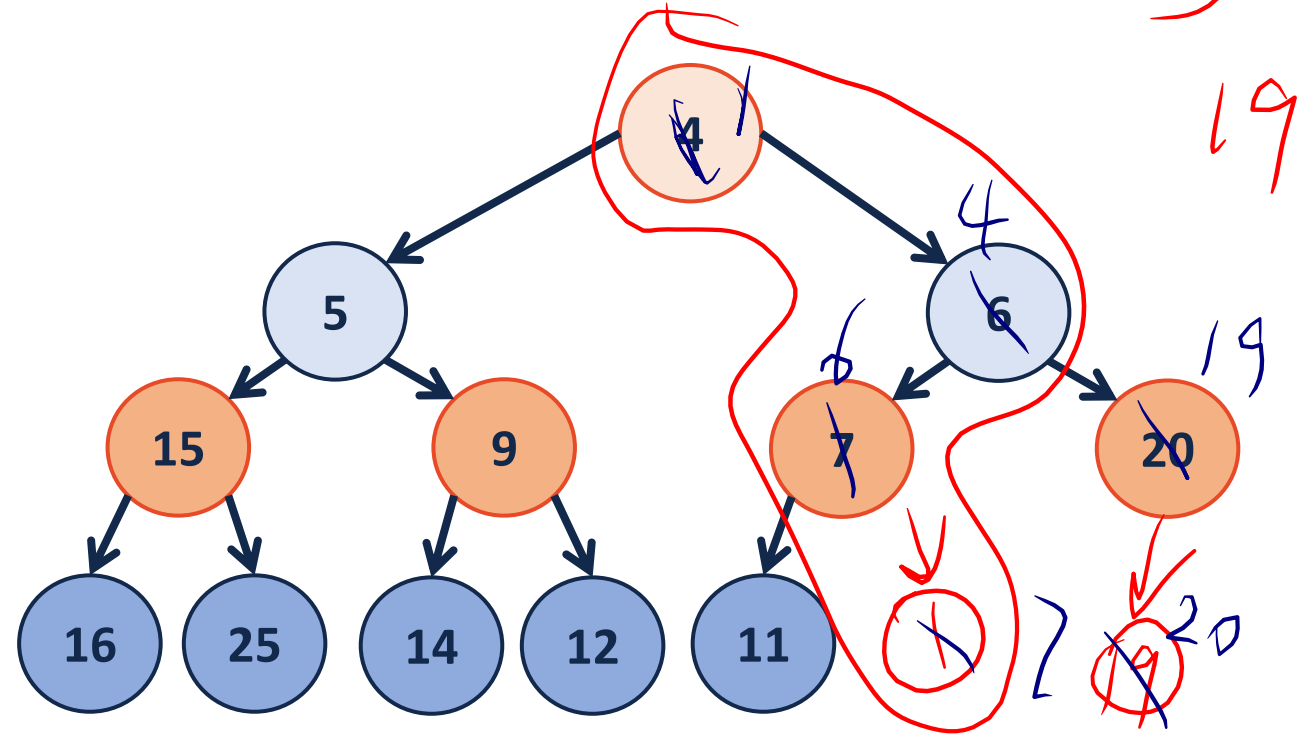
	4	5	6	15	9	7	20	16	25	14	12	11			
--	---	---	---	----	---	---	----	----	----	----	----	----	--	--	--

insert

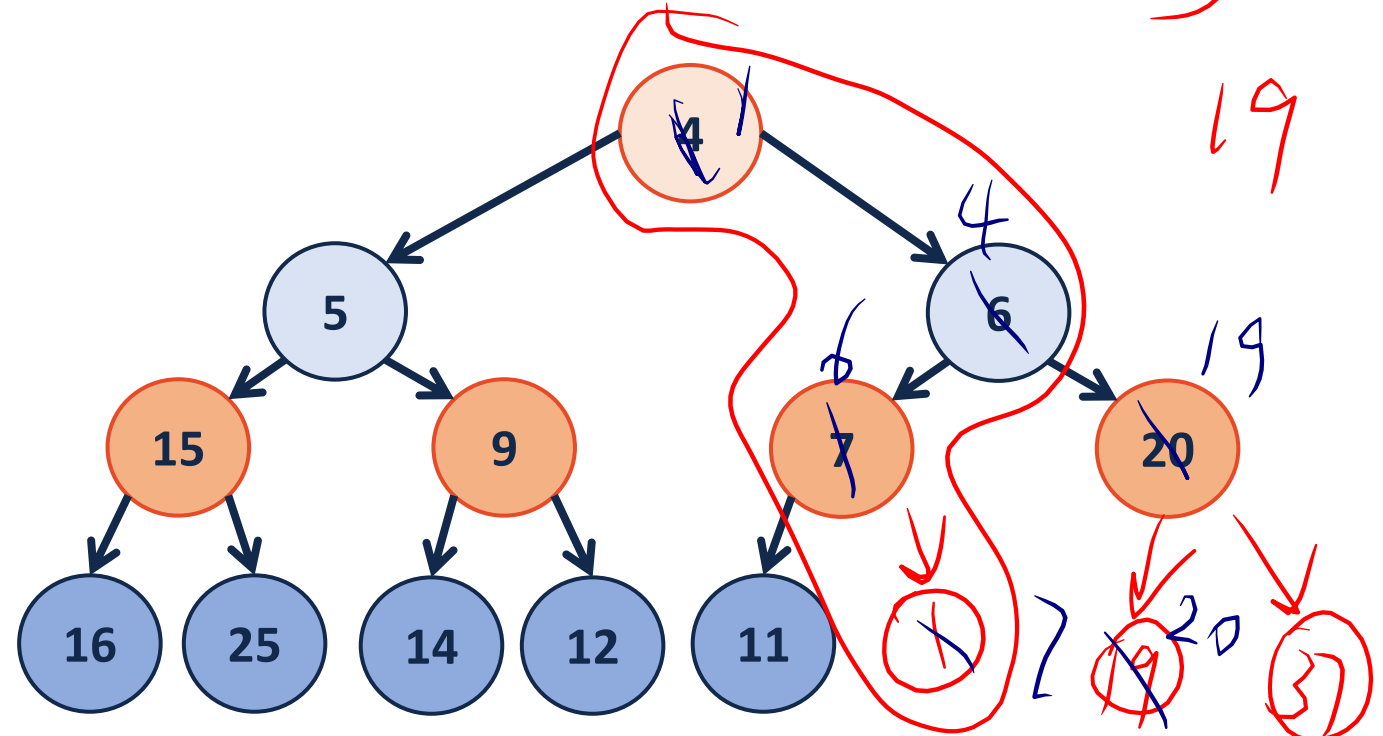


	1		4			6						7			
	4	5	6	15	9	7	20	16	25	14	12	11	7		

insert

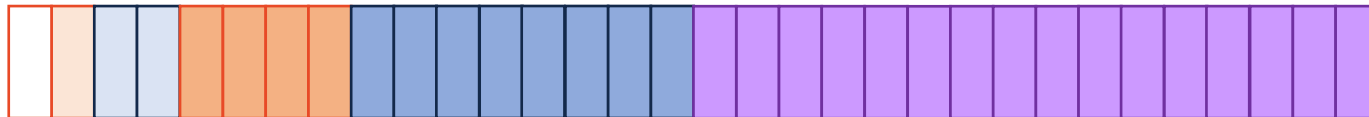
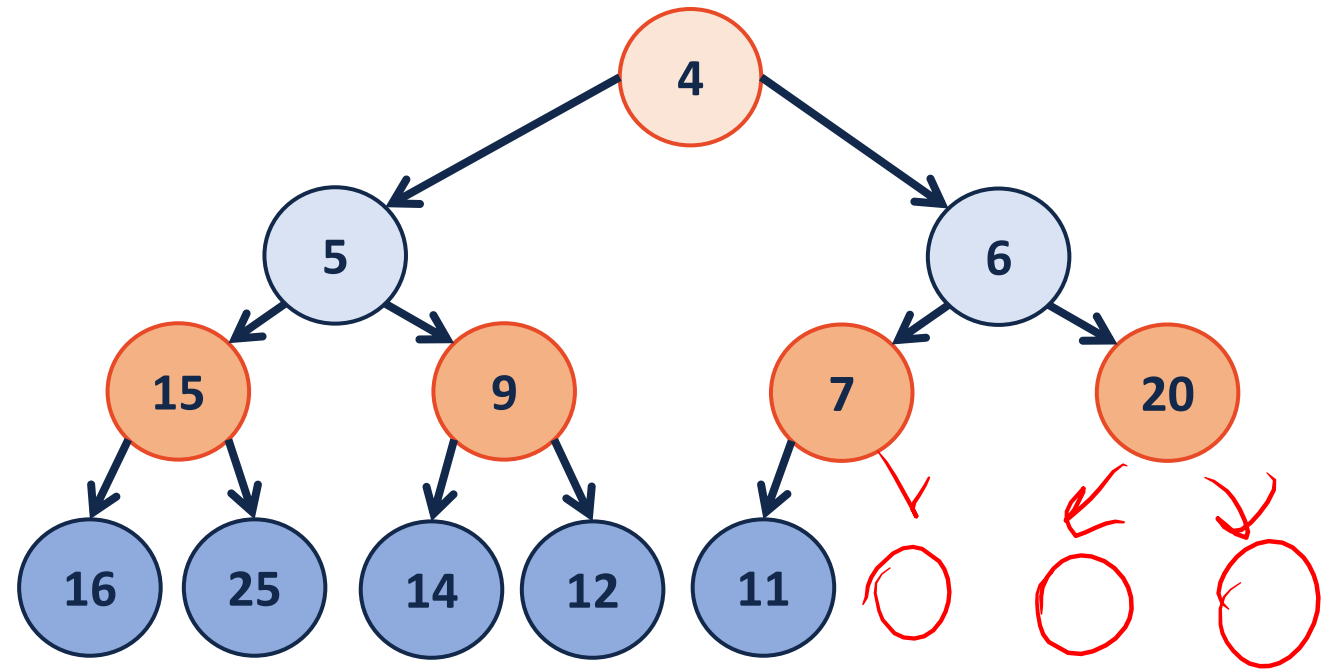


insert



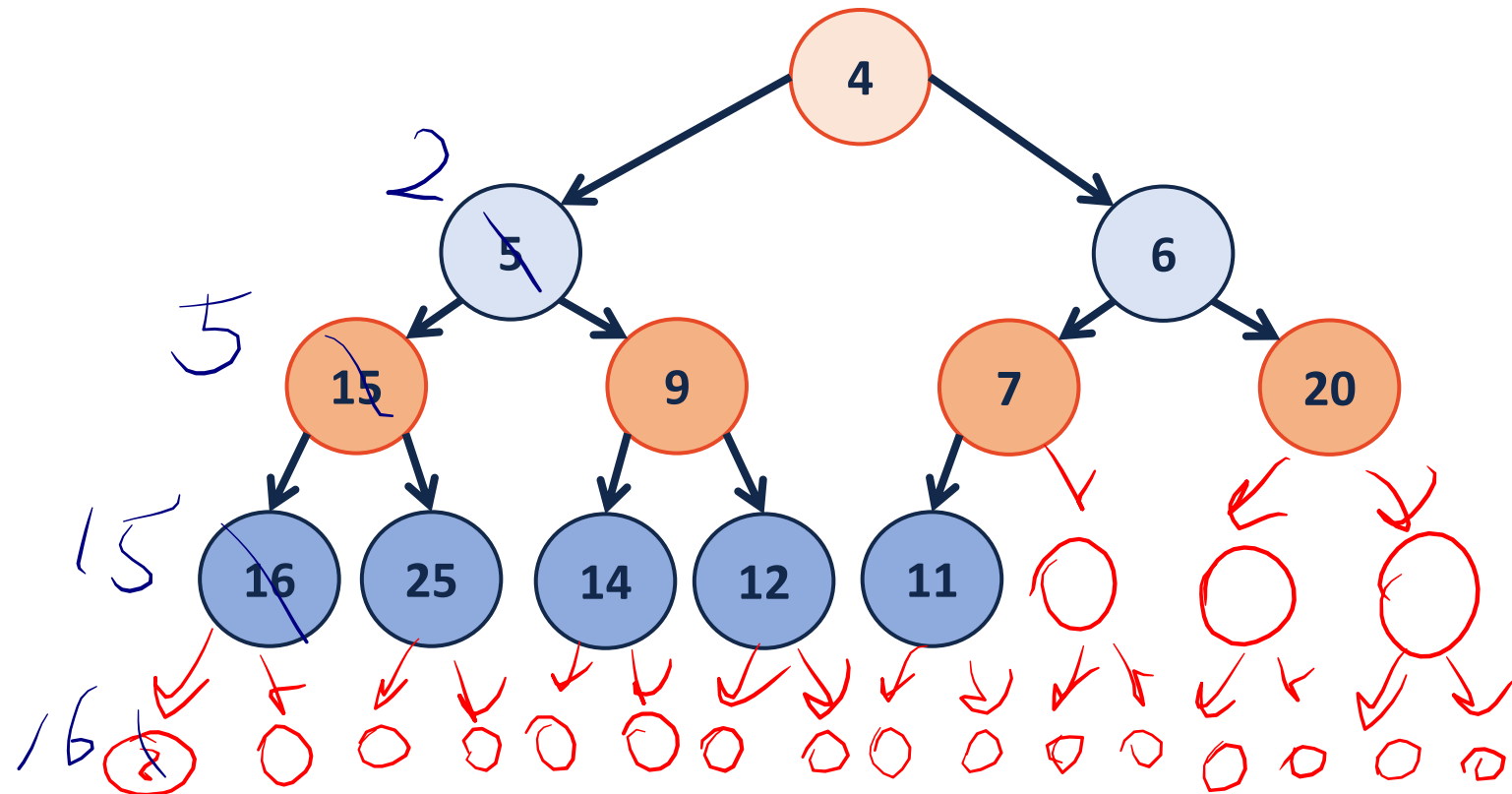
	1		4			6						7			
	4	5	6	15	9	7	20	16	25	14	12	11	1		

growArray



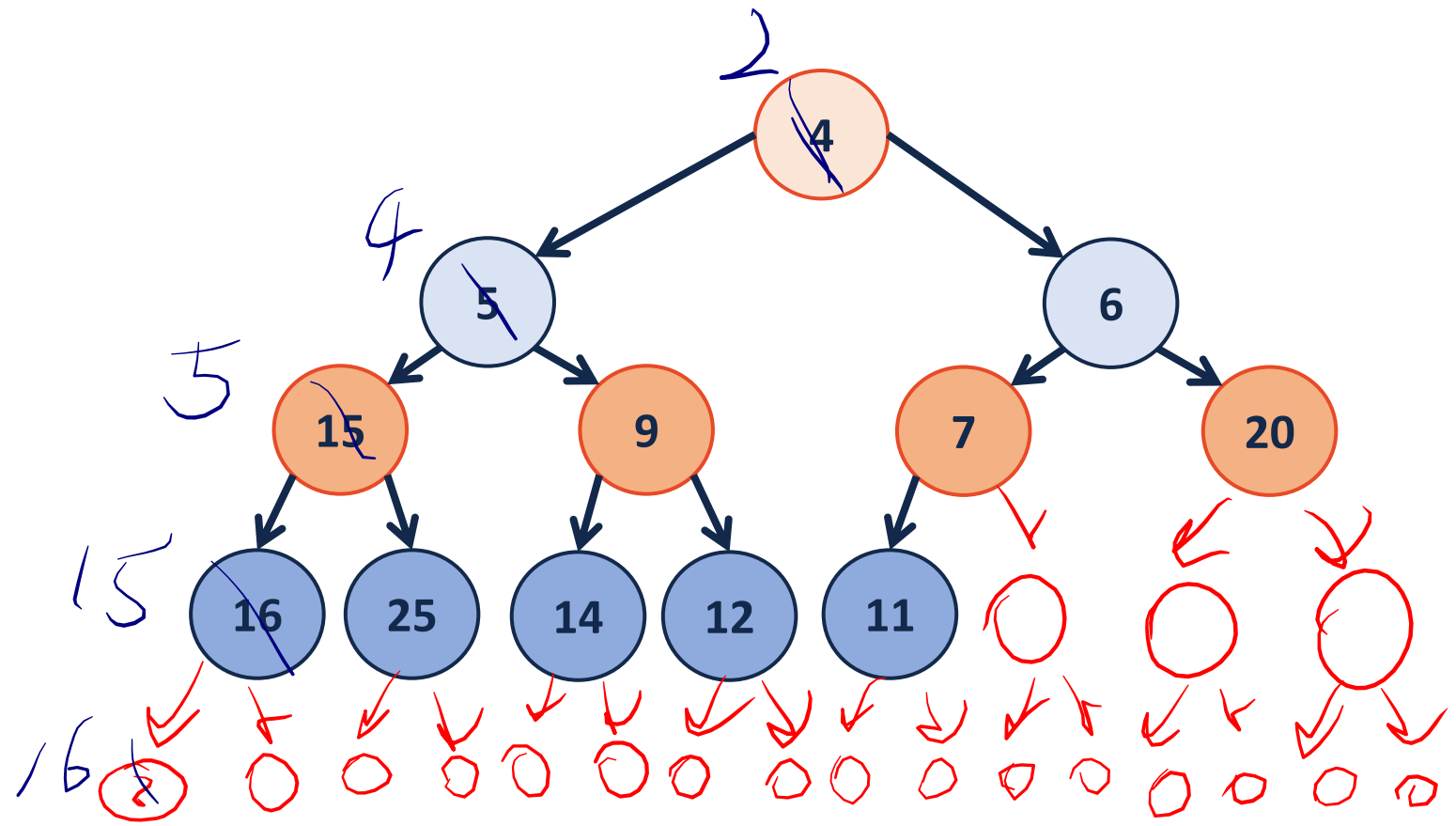
growArray

Double Size



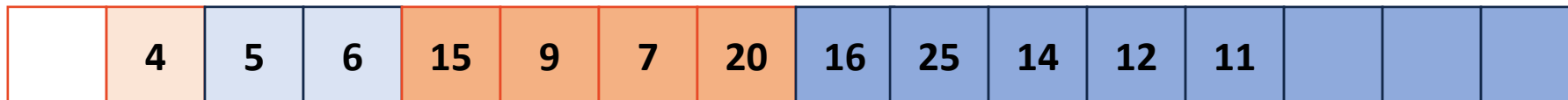
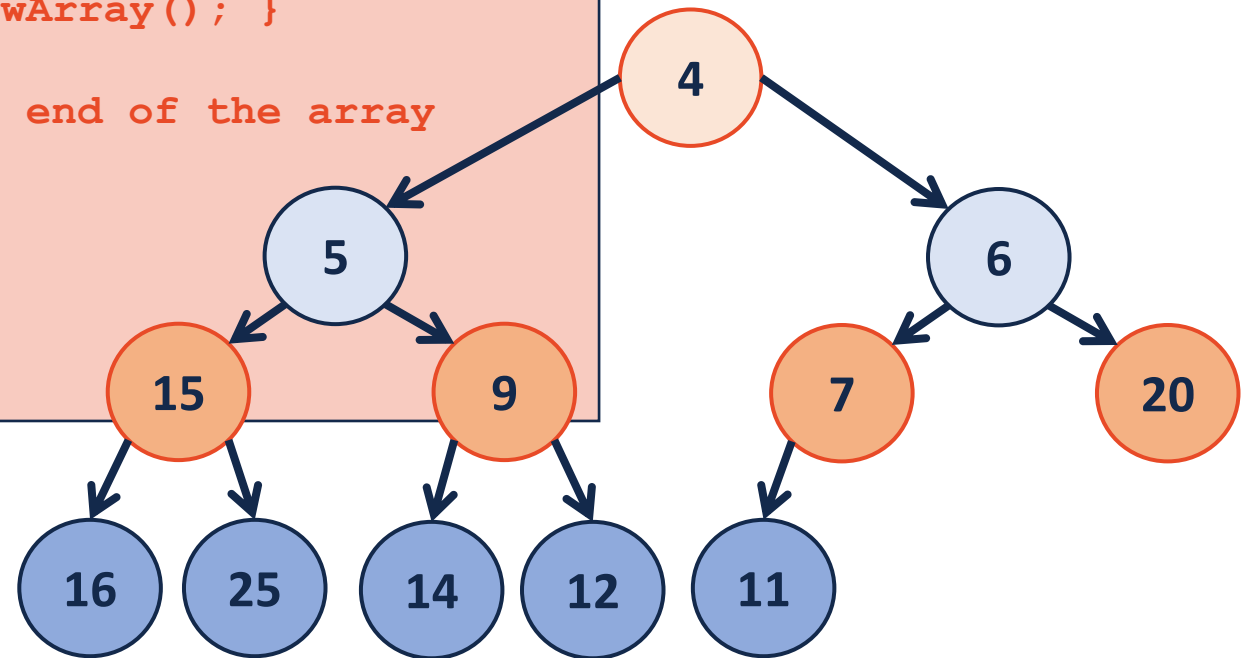
growArray

Double Size



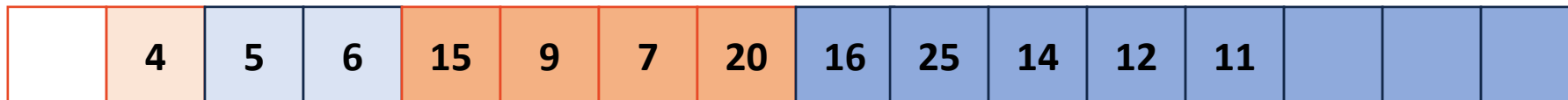
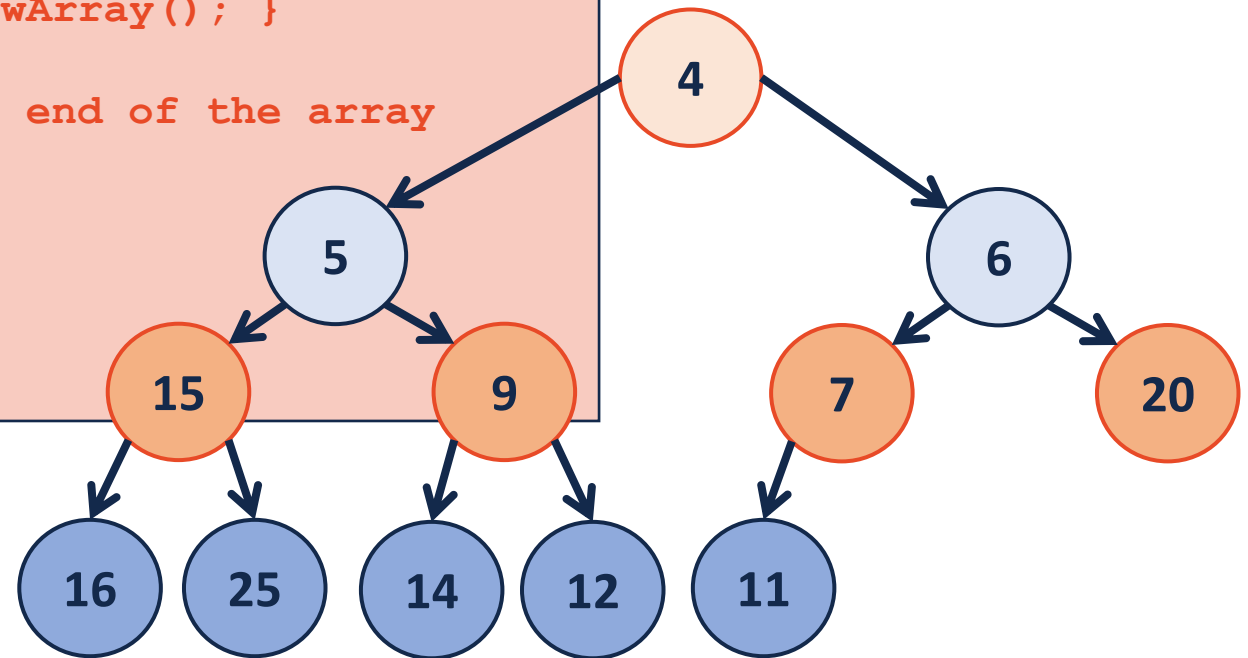
insert

```
1  template <class T>
2  void Heap<T>::_insert(const T & key) {
3      // Check to ensure there's space to insert an element
4      // ...if not, grow the array
5      if ( size_ == capacity_ ) { _growArray(); }
6
7      // Insert the new element at the end of the array
8      item_[++size_] = key;
9
10     // Restore the heap property
11     _heapifyUp(size_);
12 }
```



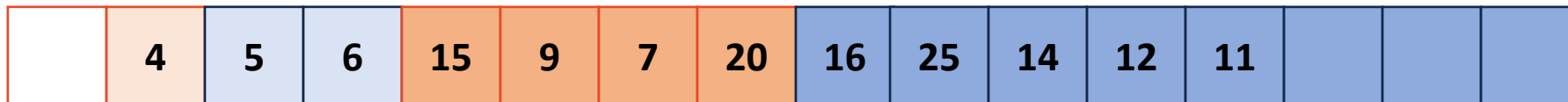
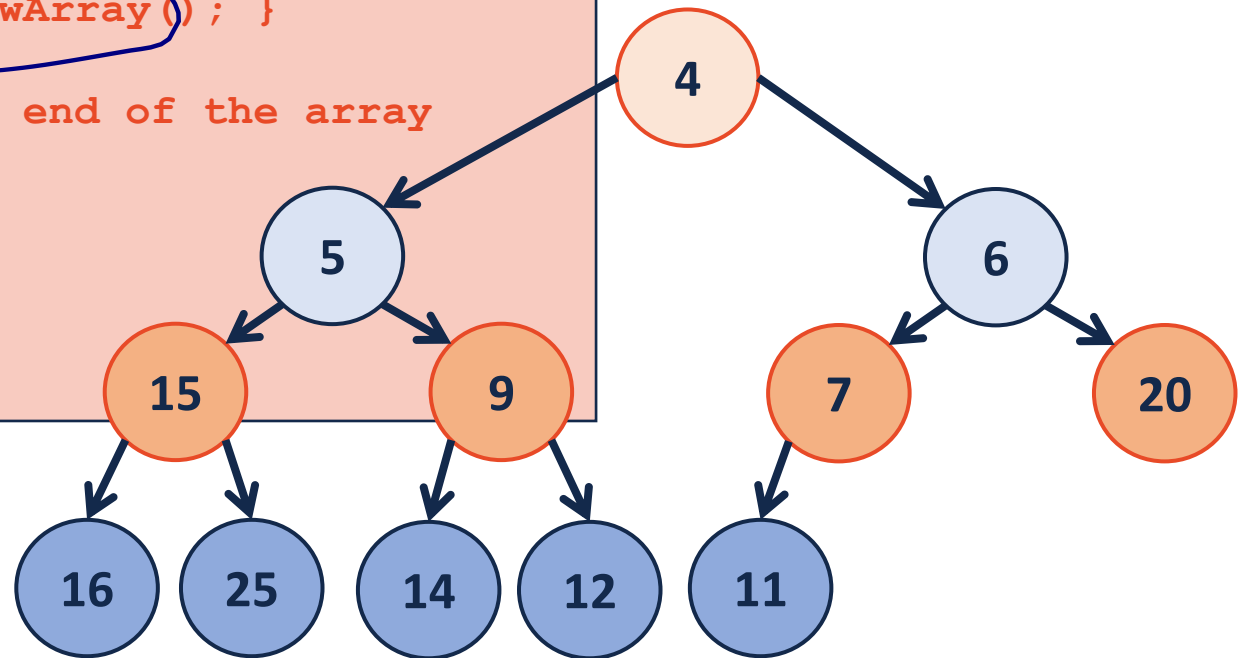
insert

```
1 template <class T>
2 void Heap<T>::_insert(const T & key) {
3     // Check to ensure there's space to insert an element
4     // ...if not, grow the array
5     if ( size_ == capacity_ ) { _growArray(); }
6
7     // Insert the new element at the end of the array
8     item_[++size_] = key;
9
10    // Restore the heap property
11    _heapifyUp(size_);
12 }
```



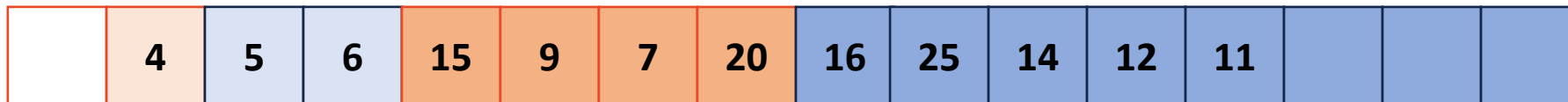
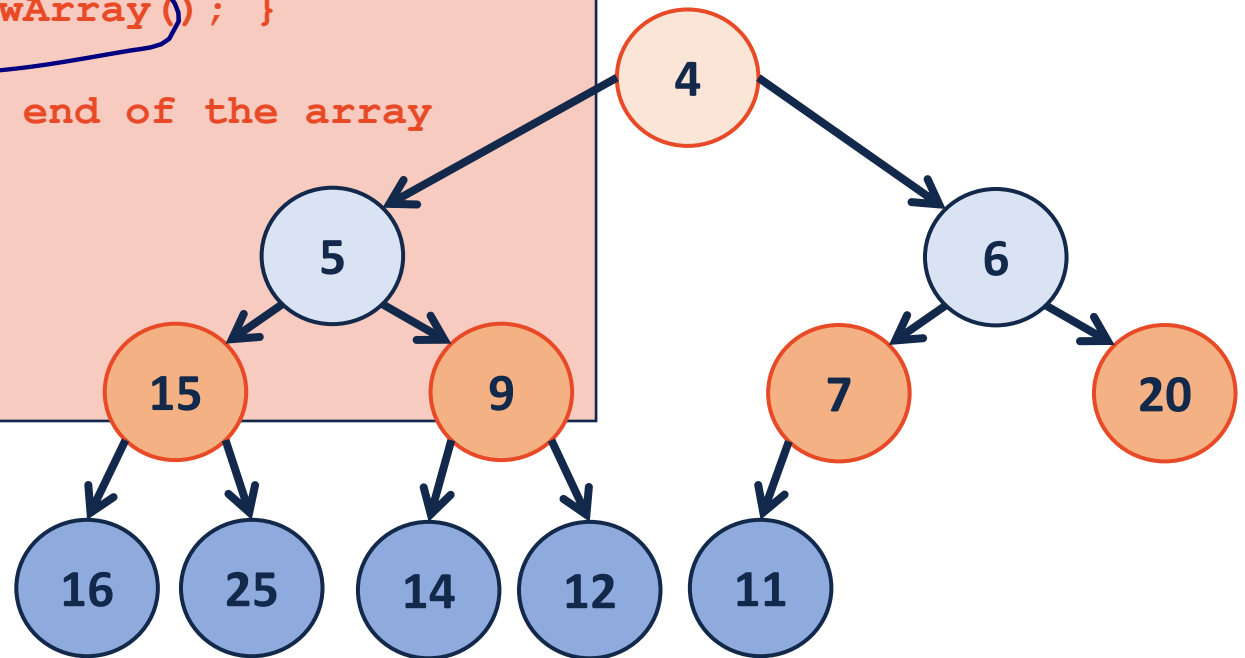
insert

```
1 template <class T>
2 void Heap<T>::_insert(const T & key) {
3     // Check to ensure there's space to insert an element
4     // ...if not, grow the array
5     if ( size_ == capacity_ ) { _growArray(); }
6
7     // Insert the new element at the end of the array
8     item_[++size_] = key;
9
10    // Restore the heap property
11    _heapifyUp(size_);
12 }
```



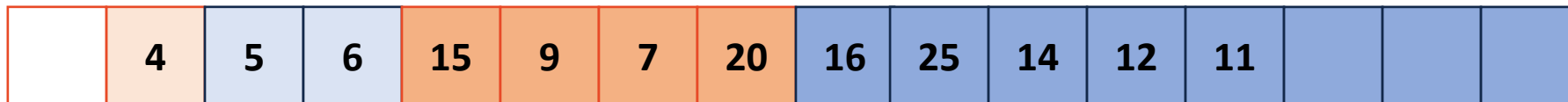
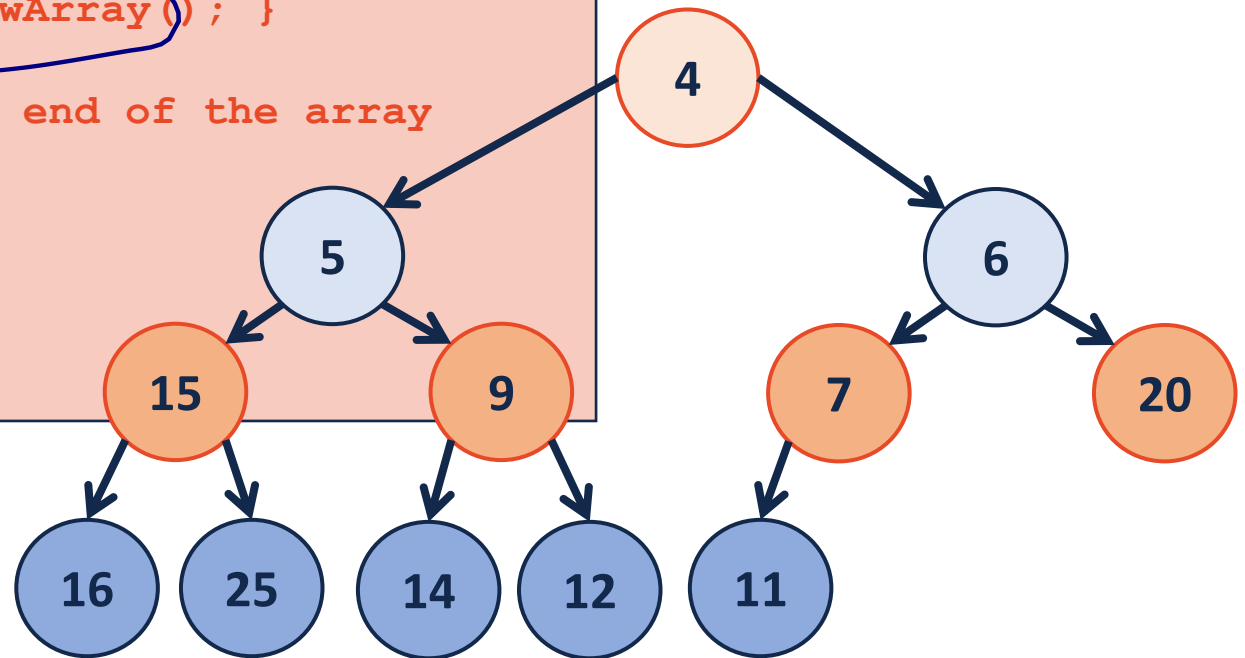
insert

```
1 template <class T>
2 void Heap<T>::_insert(const T & key) {
3     // Check to ensure there's space to insert an element
4     // ...if not, grow the array
5     if ( size_ == capacity_ ) { _growArray(); }
6
7     // Insert the new element at the end of the array
8     item_[++size_] = key;
9
10    // Restore the heap property
11    _heapifyUp(size_);
12 }
```



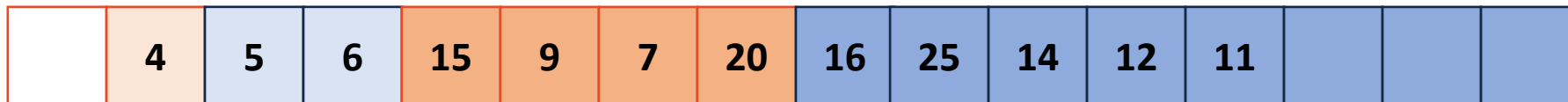
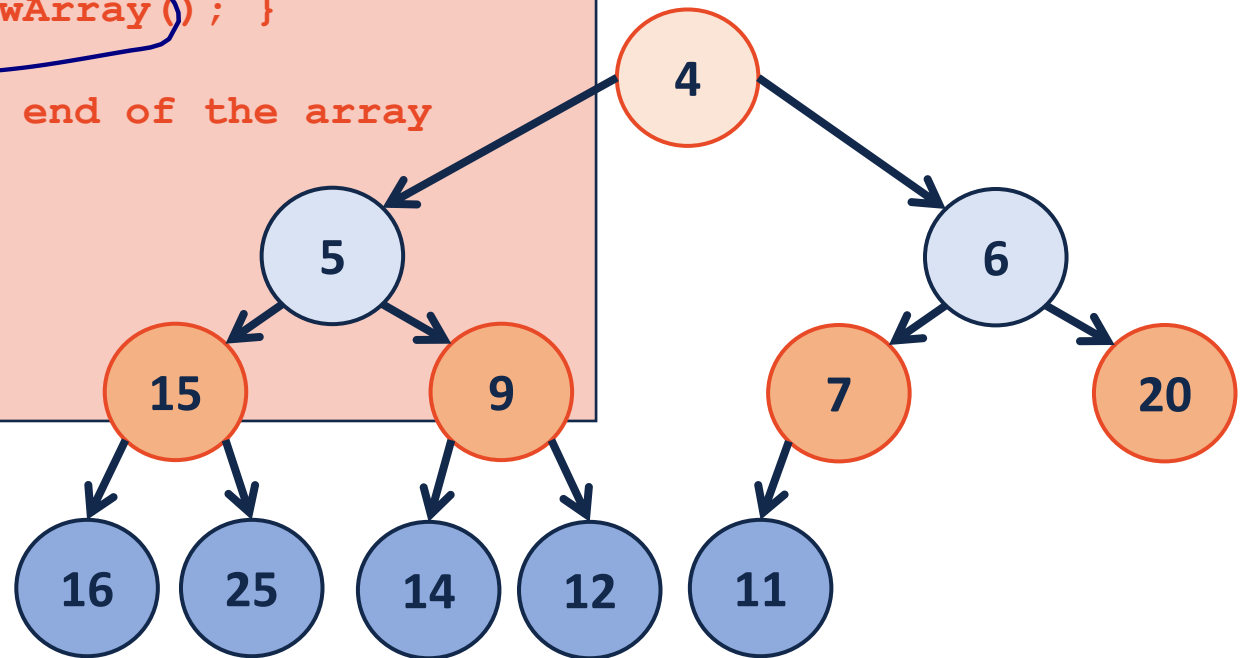
insert

```
1 template <class T>
2 void Heap<T>::_insert(const T & key) {
3     // Check to ensure there's space to insert an element
4     // ...if not, grow the array
5     if ( size_ == capacity_ ) { _growArray(); }
6
7     // Insert the new element at the end of the array
8     item_[++size_] = key;
9
10    // Restore the heap property
11    _heapifyUp(size_);
12 }
```



insert

```
1 template <class T>
2 void Heap<T>::_insert(const T & key) {
3     // Check to ensure there's space to insert an element
4     // ...if not, grow the array
5     if ( size_ == capacity_ ) { _growArray(); }
6
7     // Insert the new element at the end of the array
8     item_[++size_] = key;
9
10    // Restore the heap property
11    _heapifyUp(size_);
12 }
```



insert - heapifyUp

```
1  template <class T>
2  void Heap<T>::_insert(const T & key) {
3      // Check to ensure there's space to insert an element
4      // ...if not, grow the array
5      if ( size_ == capacity_ ) { _growArray(); }
6
7      // Insert the new element at the end of the array
8      item_[++size] = key;
9
10     // Restore the heap property
11     _heapifyUp(size);
12 }
```

```
1  template <class T>
2  void Heap<T>::_heapifyUp( _____ ) {
3      if ( index > _____ ) {
4          if ( item_[index] < item_[ parent(index) ] ) {
5              std::swap( item_[index], item_[ parent(index) ] );
6              _heapifyUp( _____ );
7          }
8      }
9  }
```

insert - heapifyUp

```
1  template <class T>
2  void Heap<T>::_insert(const T & key) {
3      // Check to ensure there's space to insert an element
4      // ...if not, grow the array
5      if ( size_ == capacity_ ) { _growArray(); }
6
7      // Insert the new element at the end of the array
8      item_[++size] = key;
9
10     // Restore the heap property
11     _heapifyUp(size);
12 }
```

```
1  template <class T>
2  void Heap<T>::_heapifyUp( unsigned index ) {
3      if ( index > _____ ) {
4          if ( item_[index] < item_[ parent(index) ] ) {
5              std::swap( item_[index], item_[ parent(index) ] );
6              _heapifyUp( _____ );
7          }
8      }
9  }
```

insert - heapifyUp

```
1  template <class T>
2  void Heap<T>::_insert(const T & key) {
3      // Check to ensure there's space to insert an element
4      // ...if not, grow the array
5      if ( size_ == capacity_ ) { _growArray(); }
6
7      // Insert the new element at the end of the array
8      item_[++size] = key;
9
10     // Restore the heap property
11     _heapifyUp(size);
12 }
```

```
1  template <class T>
2  void Heap<T>::_heapifyUp( unsigned index ) {
3      if ( index > 1 ) {
4          if ( item_[index] < item_[ parent(index) ] ) {
5              std::swap( item_[index], item_[ parent(index) ] );
6              _heapifyUp( parent(index) );
7          }
8      }
9  }
```

insert - heapifyUp

```
1  template <class T>
2  void Heap<T>::_insert(const T & key) {
3      // Check to ensure there's space to insert an element
4      // ...if not, grow the array
5      if ( size_ == capacity_ ) { _growArray(); }
6
7      // Insert the new element at the end of the array
8      item_[++size] = key;
9
10     // Restore the heap property
11     _heapifyUp(size);
12 }
```

```
1  template <class T>
2  void Heap<T>::_heapifyUp( unsigned index ) {
3      if ( index > 1 ) {
4          if ( item_[index] < item_[ parent(index) ] ) {
5              std::swap( item_[index], item_[ parent(index) ] );
6              _heapifyUp( parent(index) );
7          }
8      }
9  }
```

insert - heapifyUp

```
1  template <class T>
2  void Heap<T>::_insert(const T & key) {
3      // Check to ensure there's space to insert an element
4      // ...if not, grow the array
5      if ( size_ == capacity_ ) { _growArray(); }
6
7      // Insert the new element at the end of the array
8      item_[++size] = key;
9
10     // Restore the heap property
11     _heapifyUp(size);
12 }
```

```
1  template <class T>
2  void Heap<T>::_heapifyUp( unsigned index ) {
3      if ( index > 1 ) {
4          if ( item_[index] < item_[parent(index)] ) {
5              std::swap( item_[index], item_[parent(index)] );
6              _heapifyUp( parent(index) );
7          }
8      }
9  }
```

parent(index):
return index/2;

insert - heapifyUp

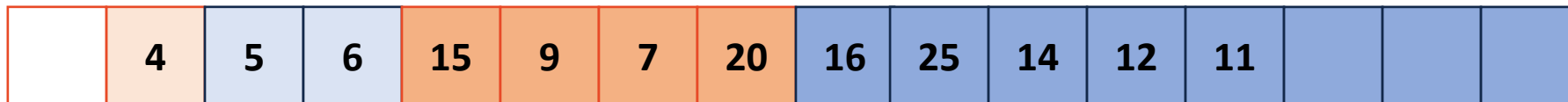
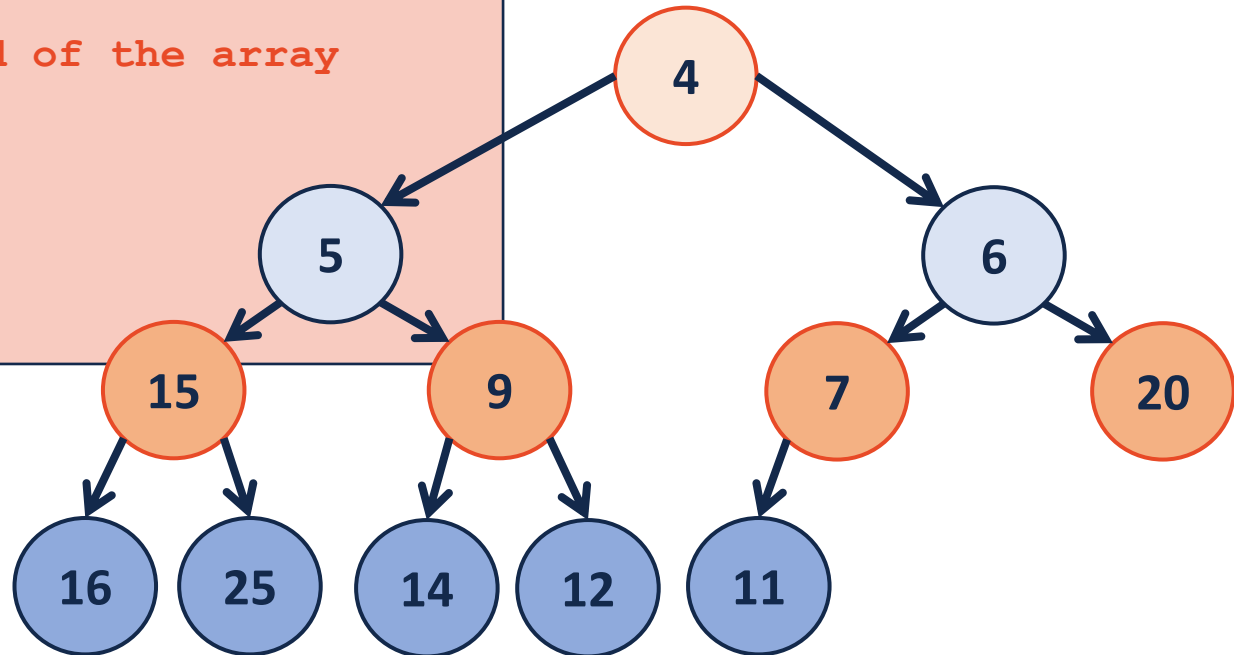
```
1 template <class T>
2 void Heap<T>::_insert(const T & key) {
3     // Check to ensure there's space to insert an element
4     // ...if not, grow the array
5     if ( size_ == capacity_ ) { _growArray(); }
6
7     // Insert the new element at the end of the array
8     item_[++size] = key;
9
10    // Restore the heap property
11    _heapifyUp(size);
12 }
```

```
1 template <class T>
2 void Heap<T>::_heapifyUp( unsigned index ) {
3     if ( index > 1 ) {
4         if ( item_[index] < item_[parent(index)] ) {
5             std::swap( item_[index], item_[parent(index)] );
6             _heapifyUp( parent(index) );
7         }
8     }
9 }
```

parent(index):
return index/2;

insert- running time

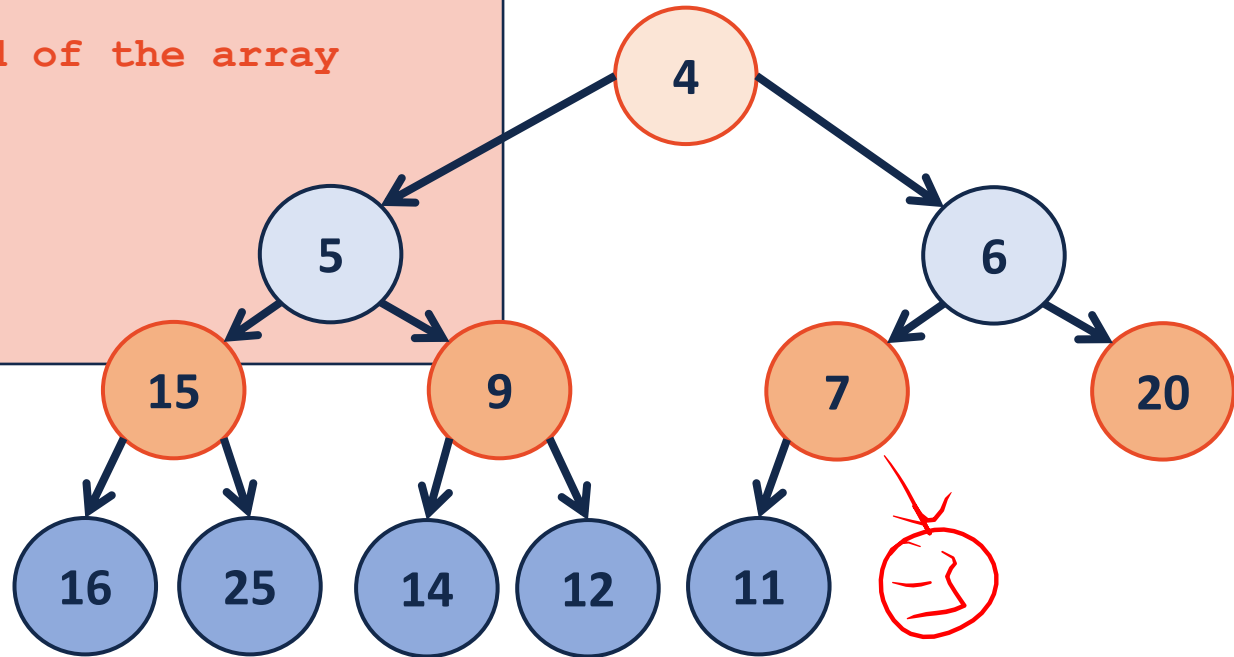
```
1  template <class T>
2  void Heap<T>::_insert(const T & key) {
3      // Check to ensure there's space to insert an element
4      // ...if not, grow the array
5      if ( size_ == capacity_ ) { _growArray(); }
6
7      // Insert the new element at the end of the array
8      item_[++size] = key;
9
10     // Restore the heap property
11     _heapifyUp(size);
12 }
```



insert- running time

add: $O(1)$

```
1 template <class T>
2 void Heap<T>::_insert(const T & key) {
3     // Check to ensure there's space to insert an element
4     // ...if not, grow the array
5     if ( size_ == capacity_ ) { _growArray(); }
6
7     // Insert the new element at the end of the array
8     item_[++size] = key;
9
10    // Restore the heap property
11    _heapifyUp(size);
12 }
```

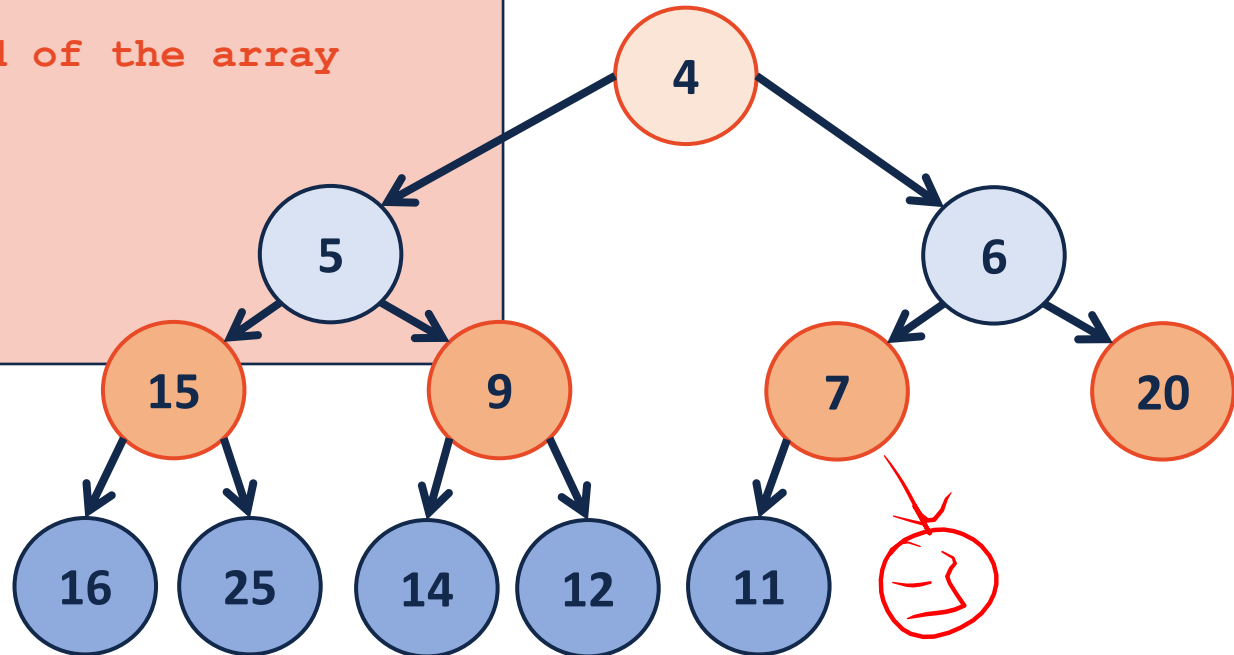


	4	5	6	15	9	7	20	16	25	14	12	11			
--	---	---	---	----	---	---	----	----	----	----	----	----	--	--	--

insert- running time

add: $O(1)$
swap: $O(1)$

```
1 template <class T>
2 void Heap<T>::_insert(const T & key) {
3     // Check to ensure there's space to insert an element
4     // ...if not, grow the array
5     if ( size_ == capacity_ ) { _growArray(); }
6
7     // Insert the new element at the end of the array
8     item_[++size] = key;
9
10    // Restore the heap property
11    _heapifyUp(size);
12 }
```

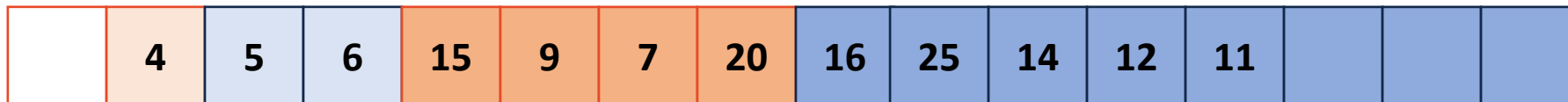
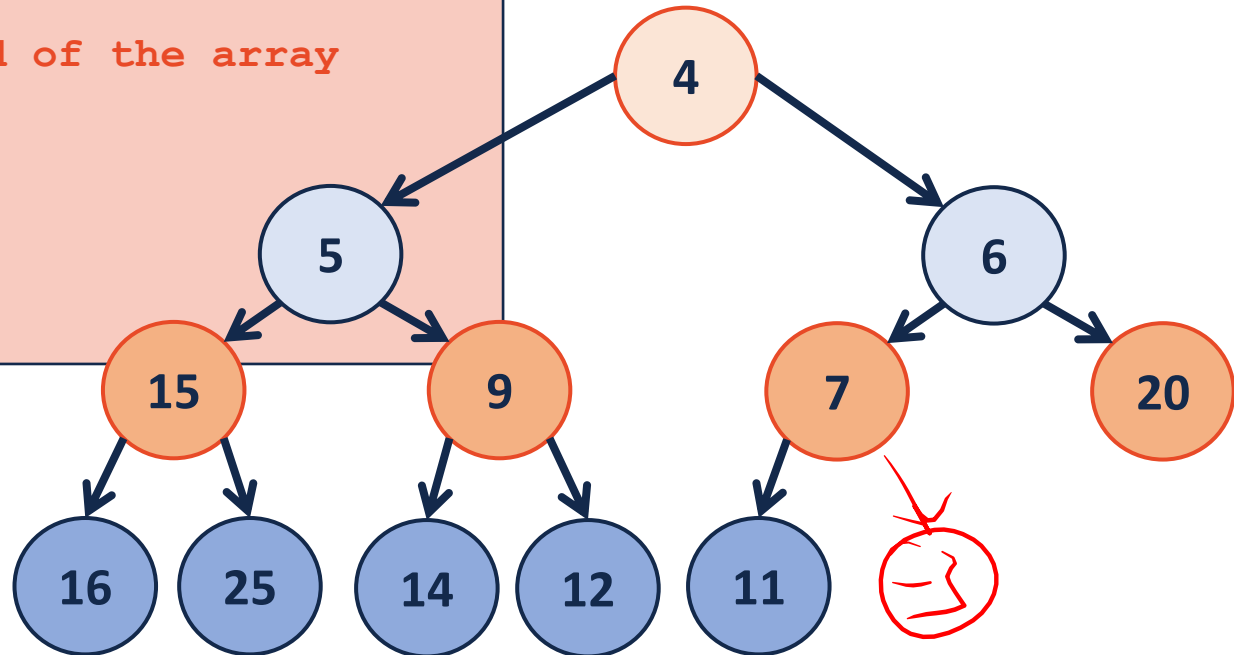


	4	5	6	15	9	7	20	16	25	14	12	11			
--	---	---	---	----	---	---	----	----	----	----	----	----	--	--	--

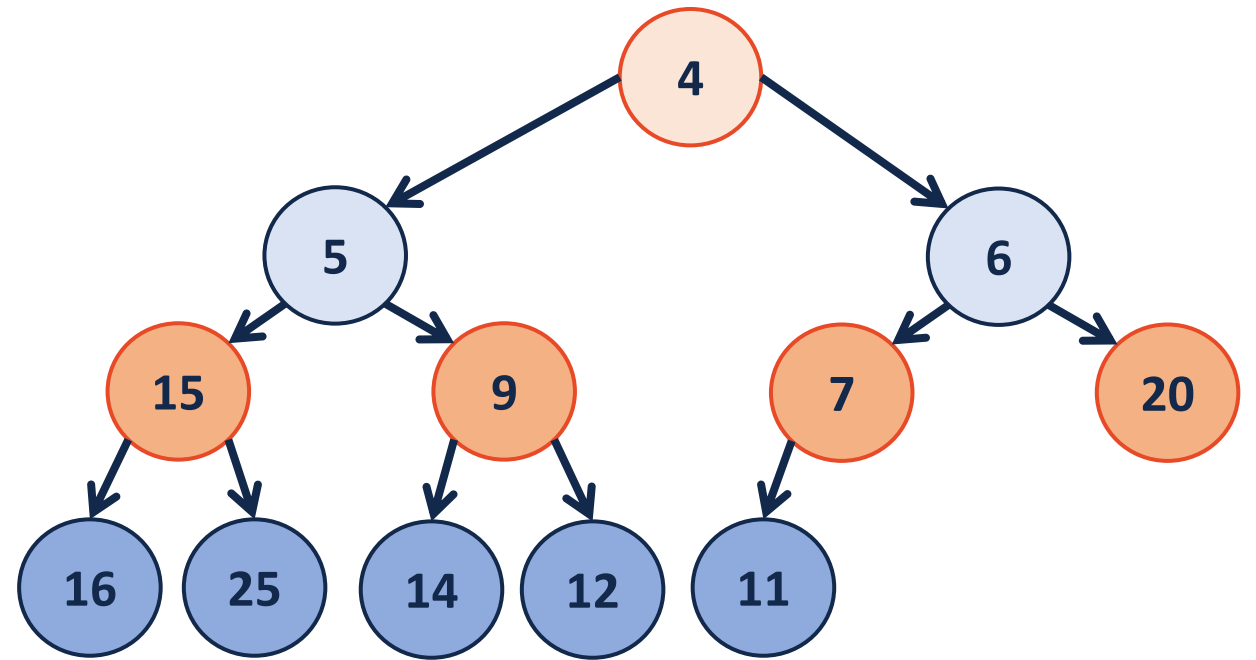
insert- running time

```
1 template <class T>
2 void Heap<T>::_insert(const T & key) {
3     // Check to ensure there's space to insert an element
4     // ...if not, grow the array
5     if ( size_ == capacity_ ) { _growArray(); }
6
7     // Insert the new element at the end of the array
8     item_[++size] = key;
9
10    // Restore the heap property
11    _heapifyUp(size);
12 }
```

add: $O(1)$
swap: $O(1)$
 $O(\lg(n))$

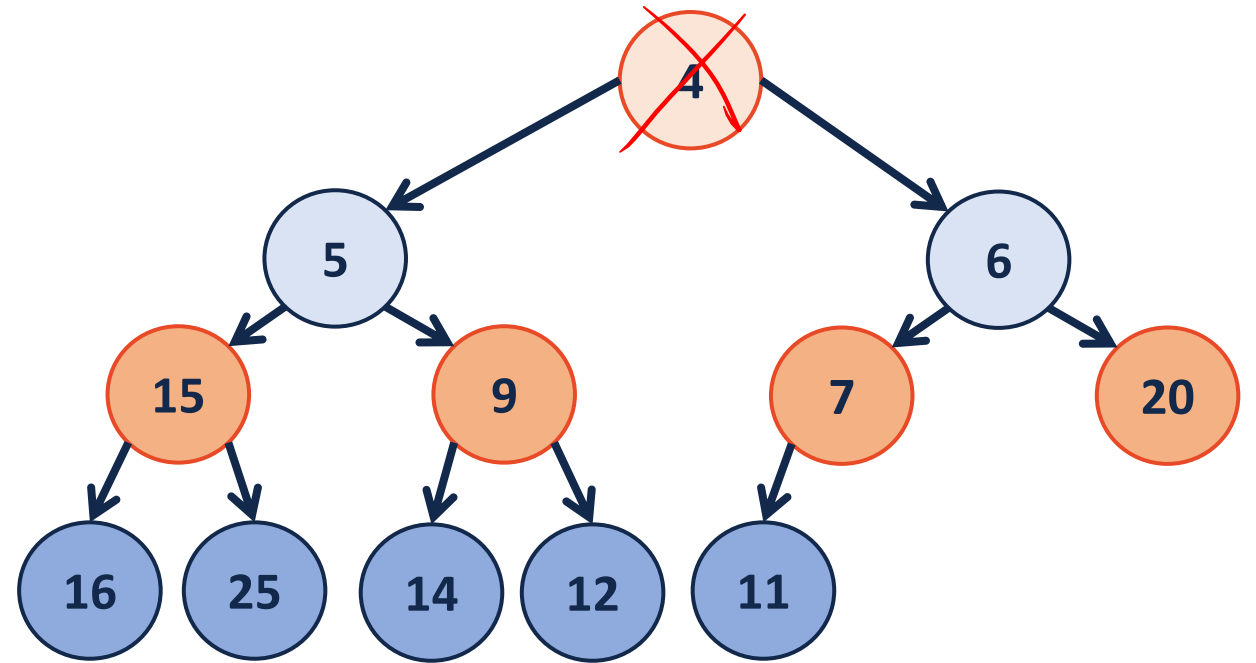


removeMin



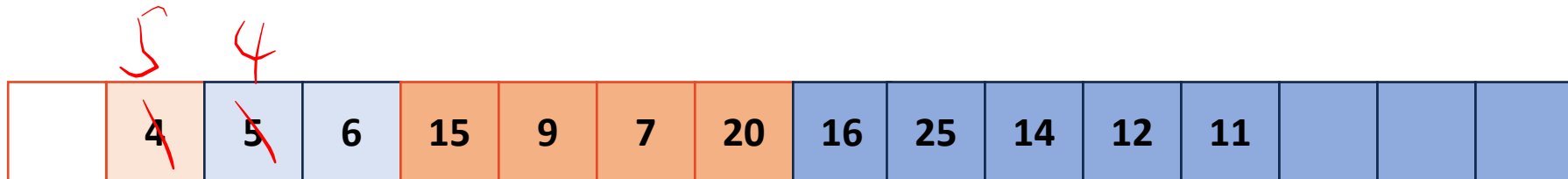
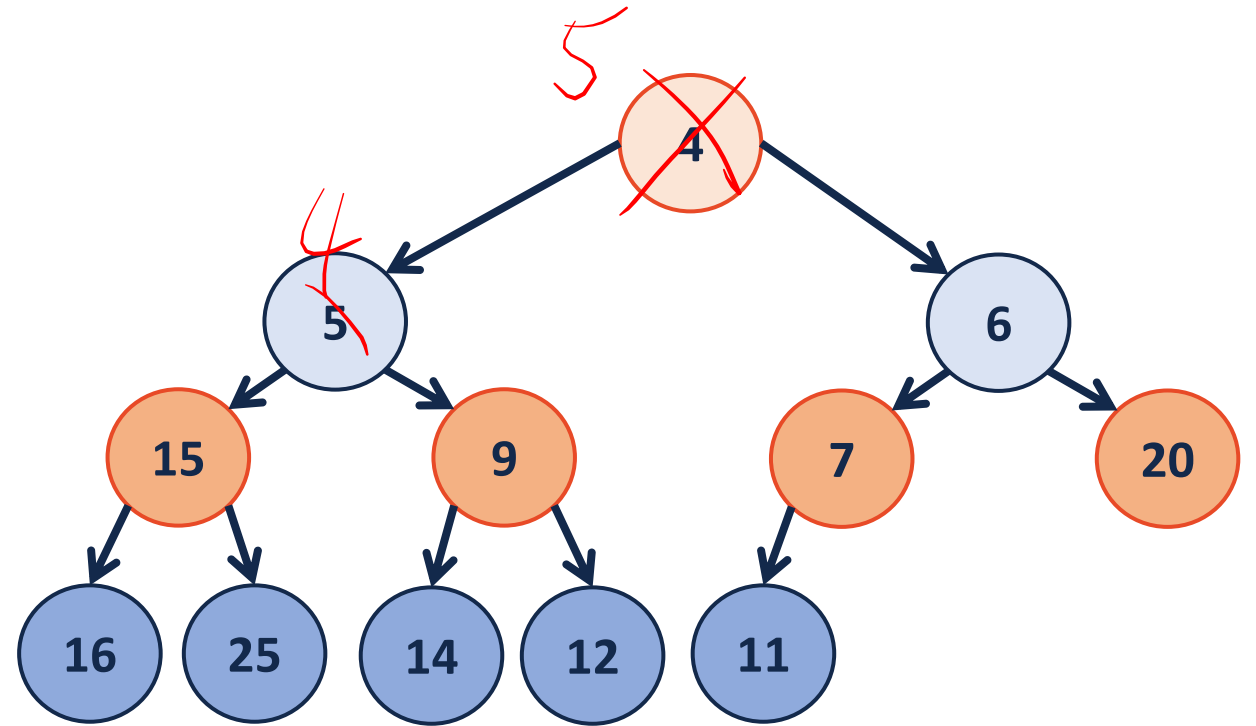
	4	5	6	15	9	7	20	16	25	14	12	11			
--	---	---	---	----	---	---	----	----	----	----	----	----	--	--	--

removeMin

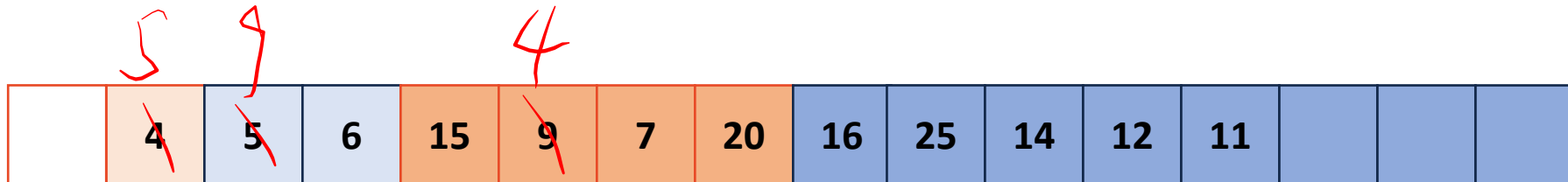
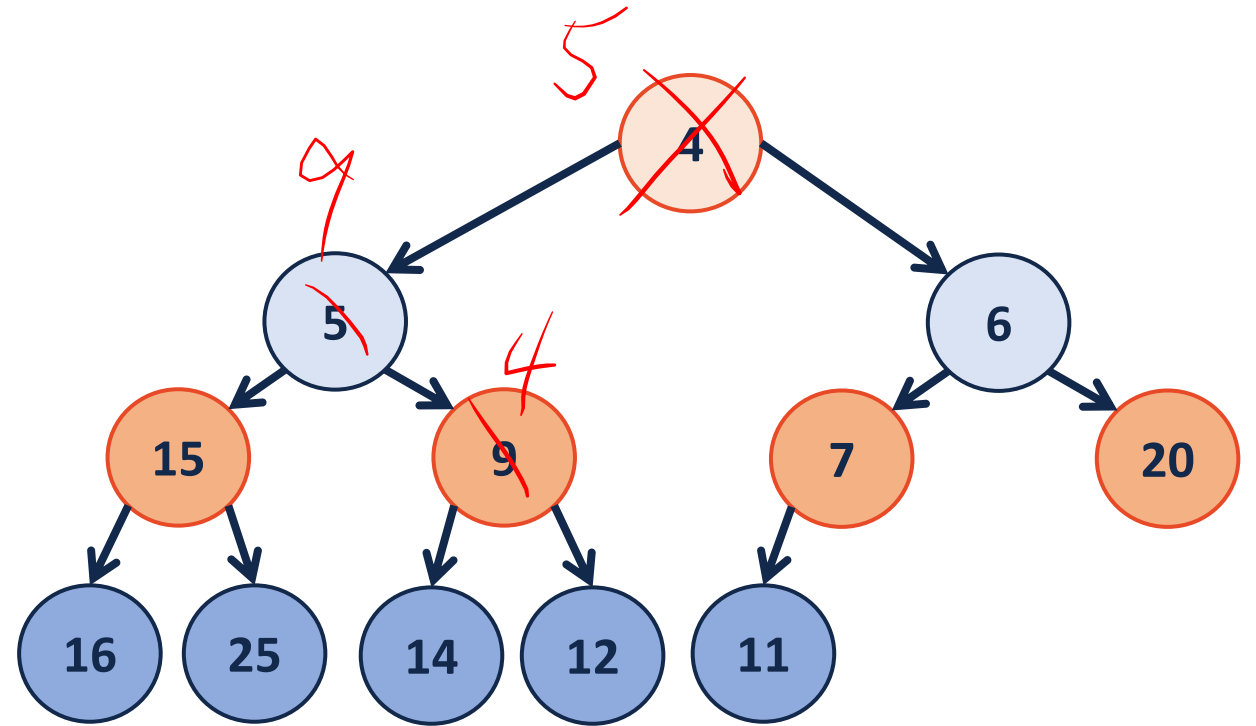


	4	5	6	15	9	7	20	16	25	14	12	11			
--	---	---	---	----	---	---	----	----	----	----	----	----	--	--	--

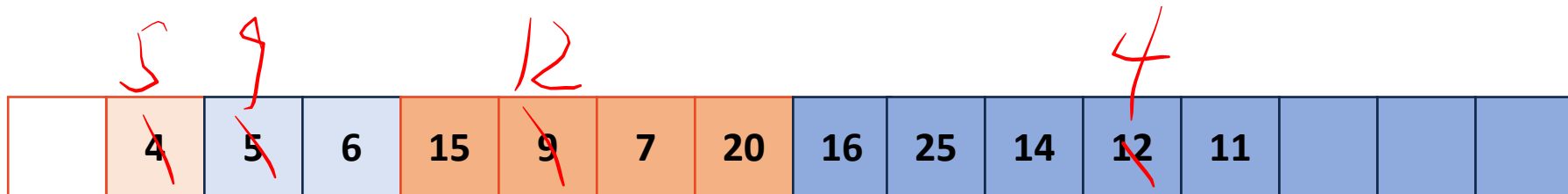
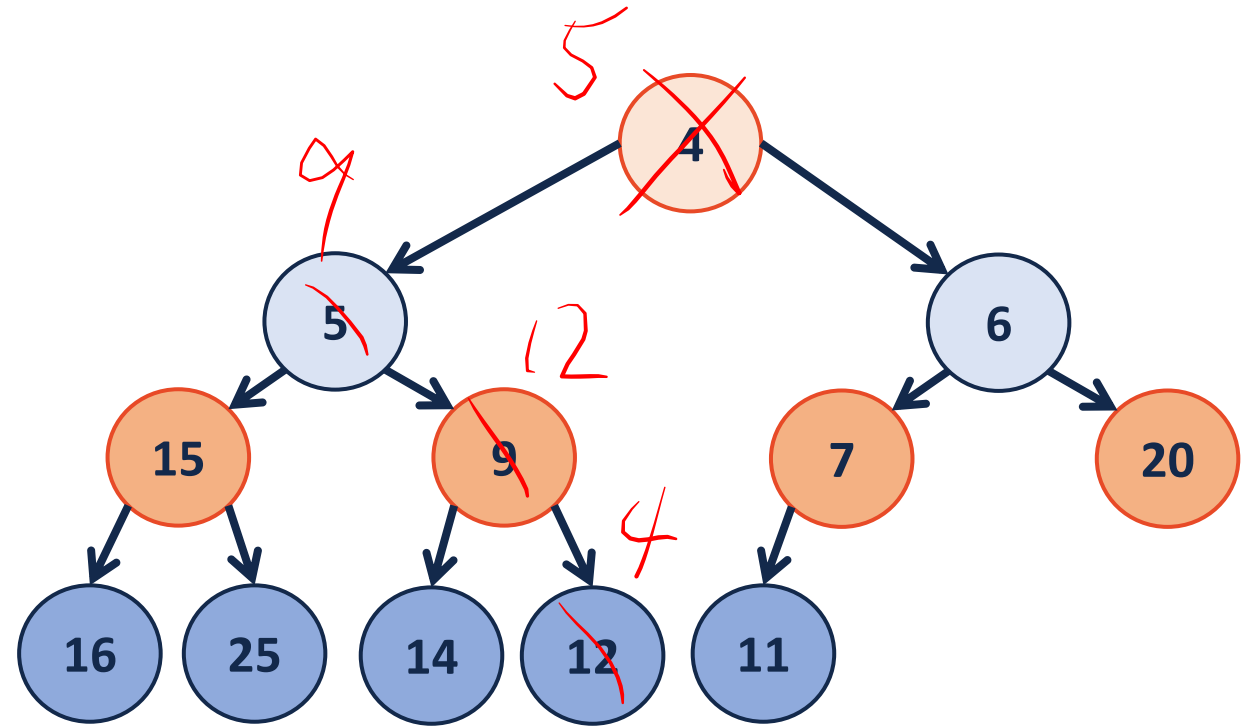
removeMin



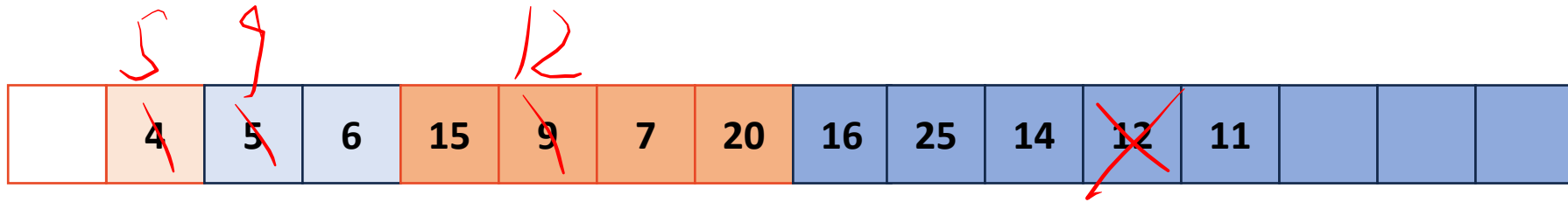
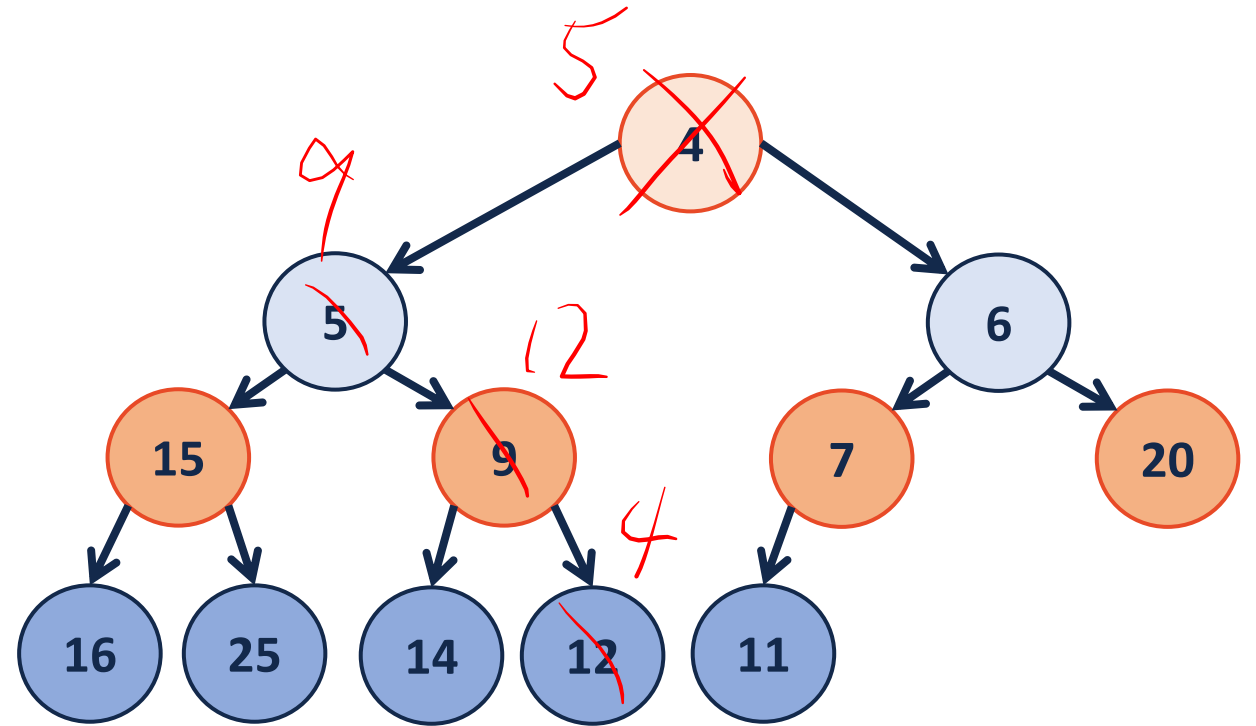
removeMin



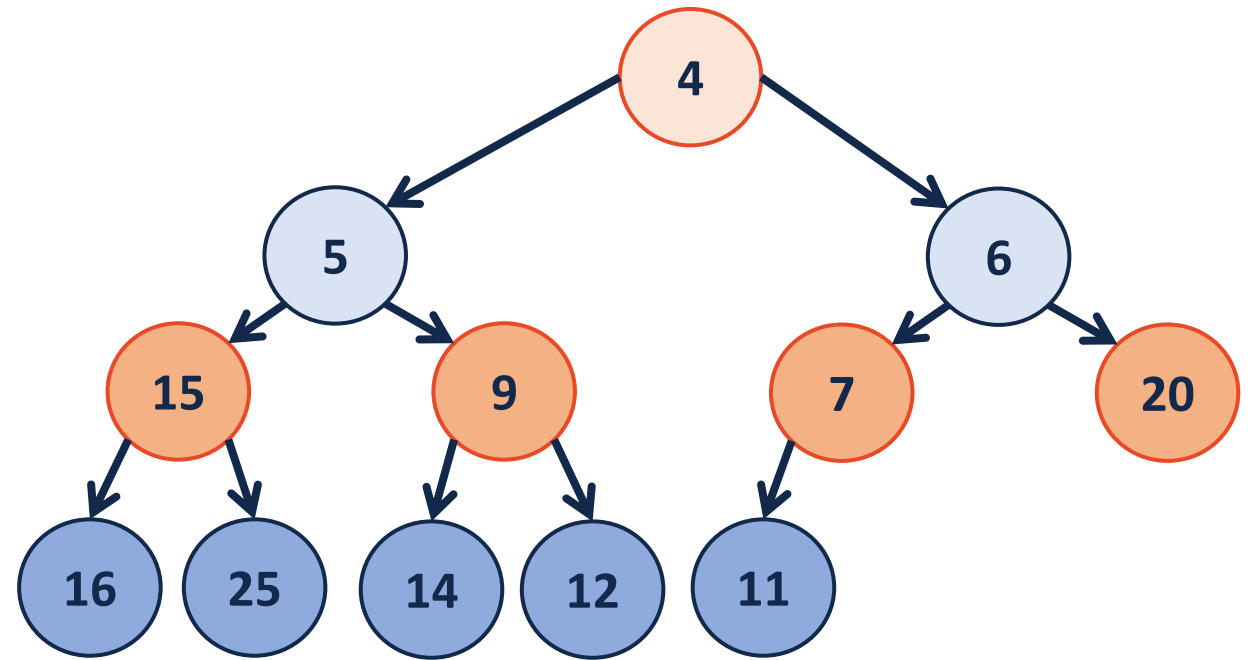
removeMin



removeMin

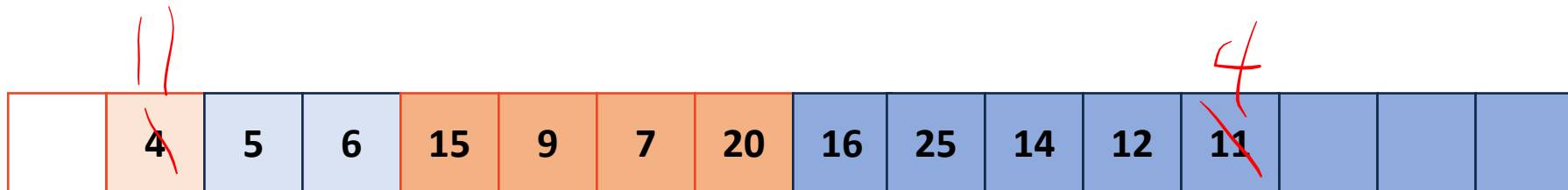
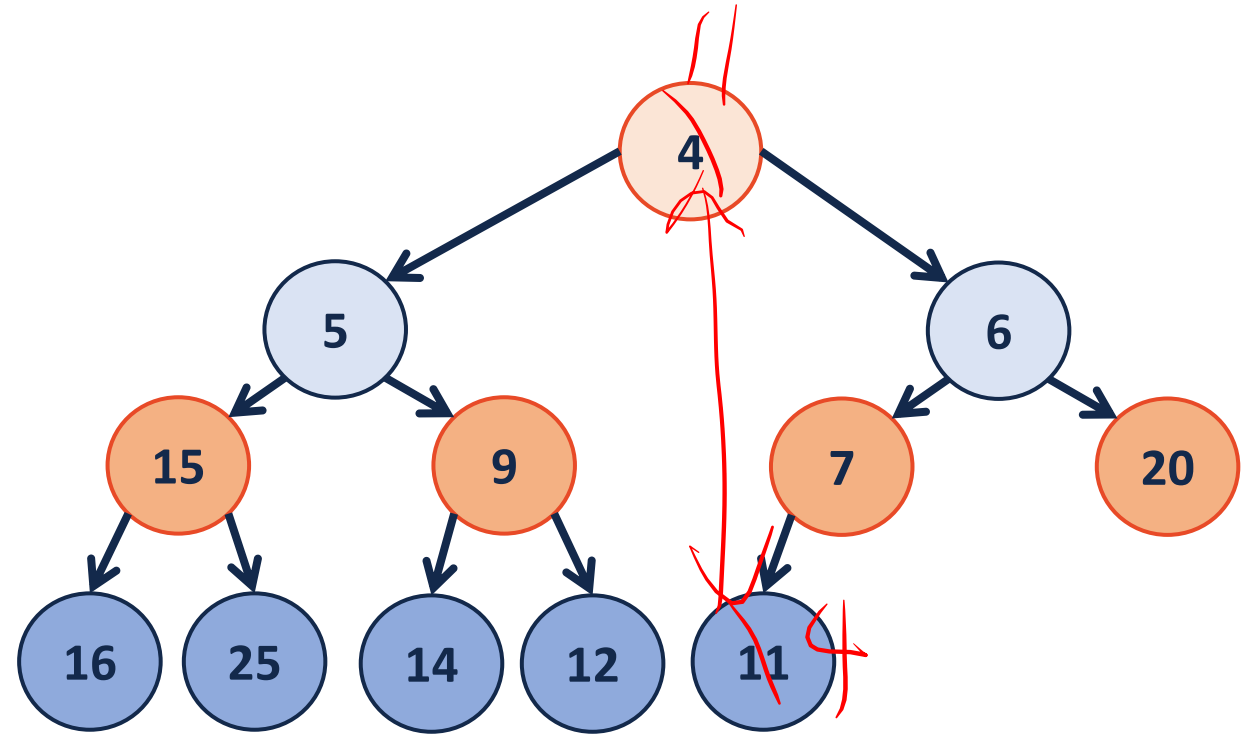


removeMin

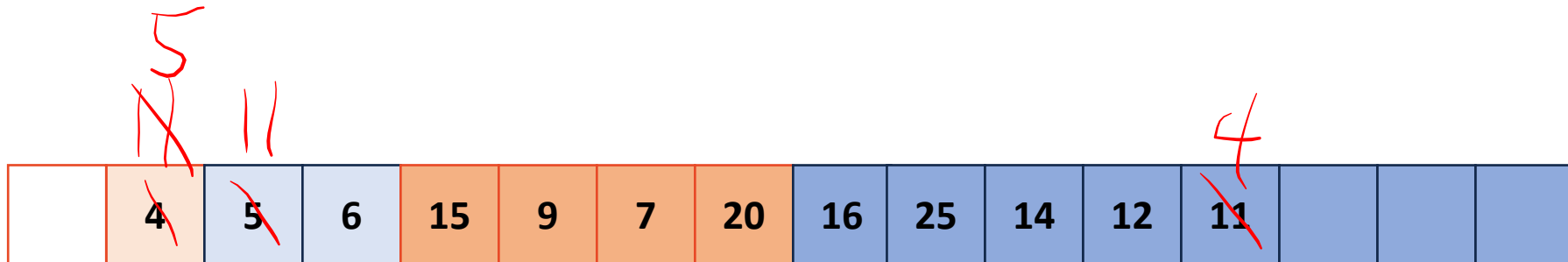
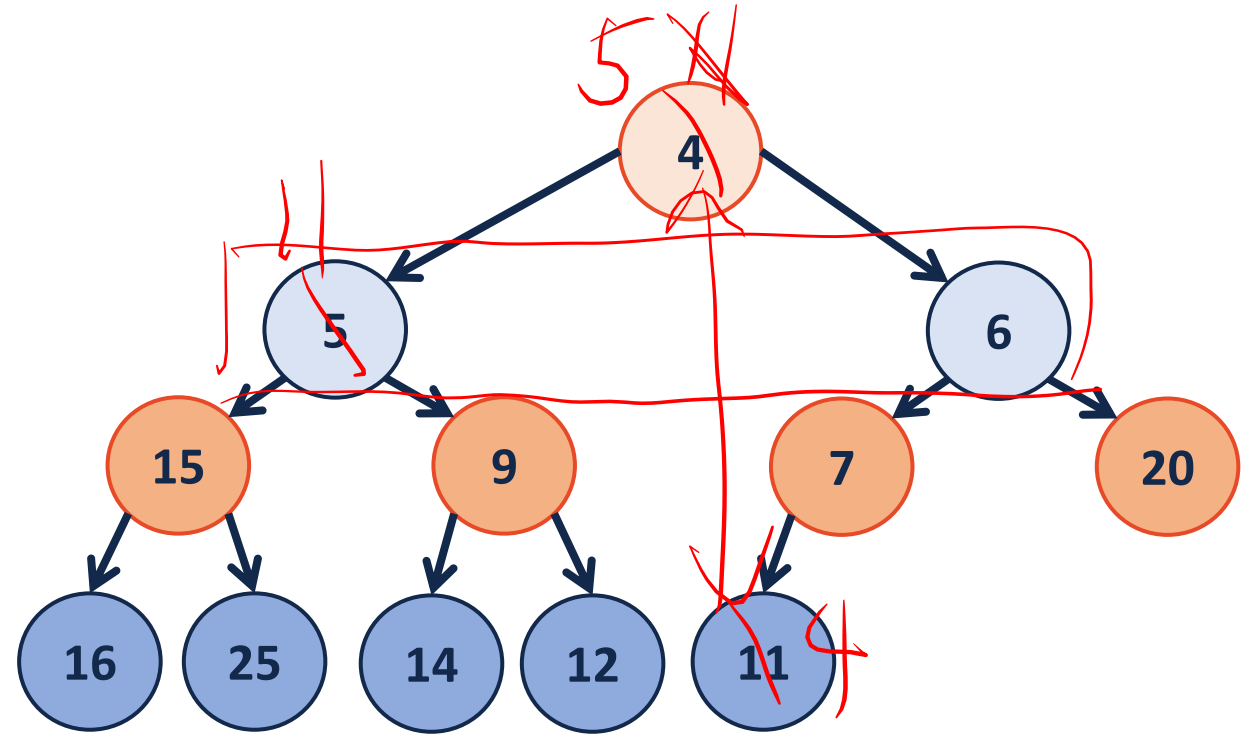


	4	5	6	15	9	7	20	16	25	14	12	11			
--	---	---	---	----	---	---	----	----	----	----	----	----	--	--	--

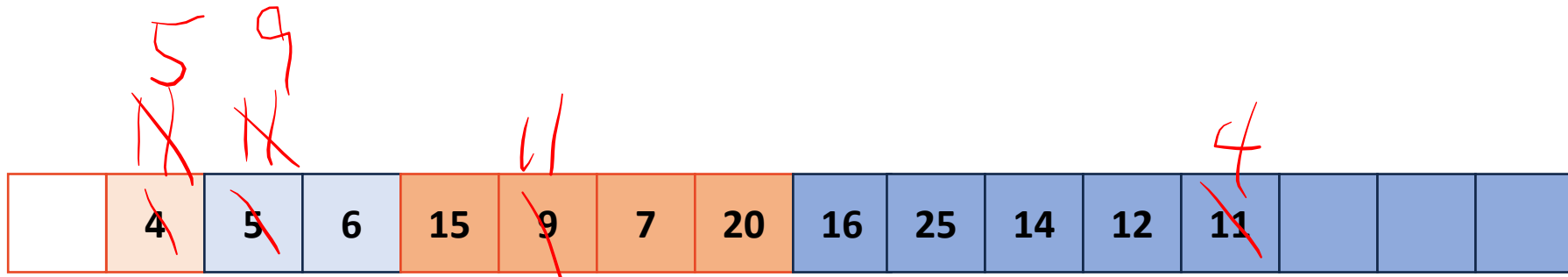
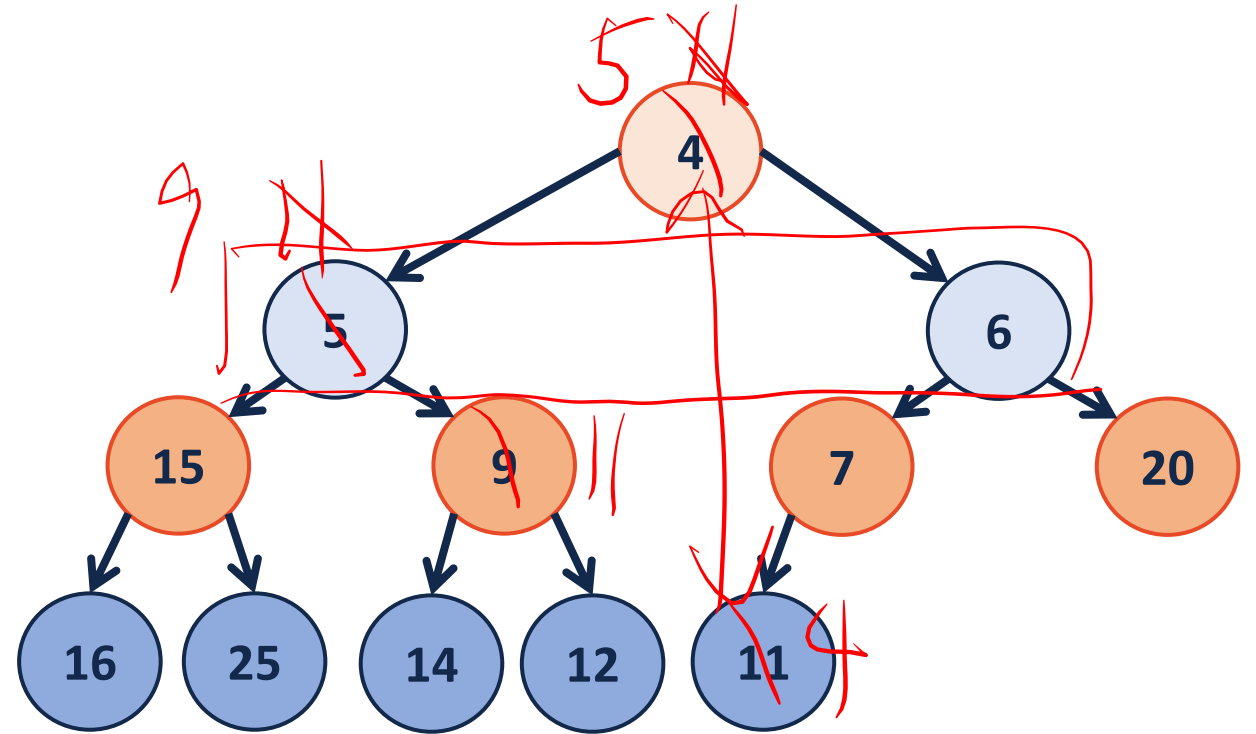
removeMin



removeMin

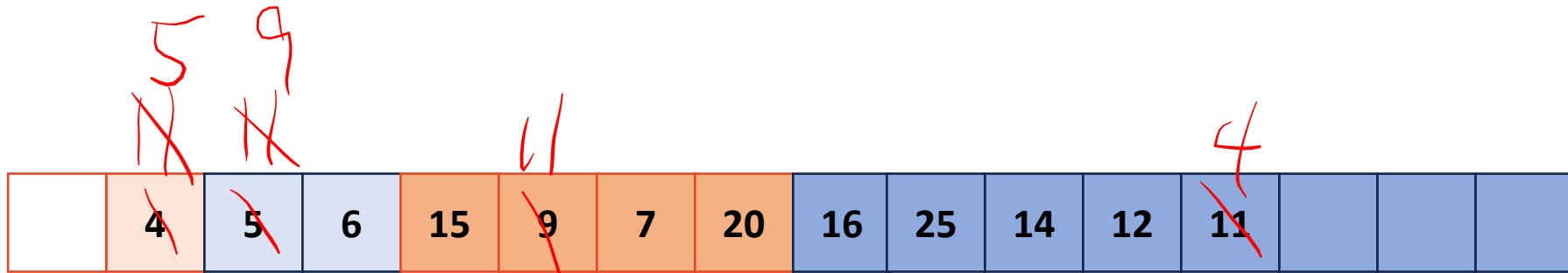
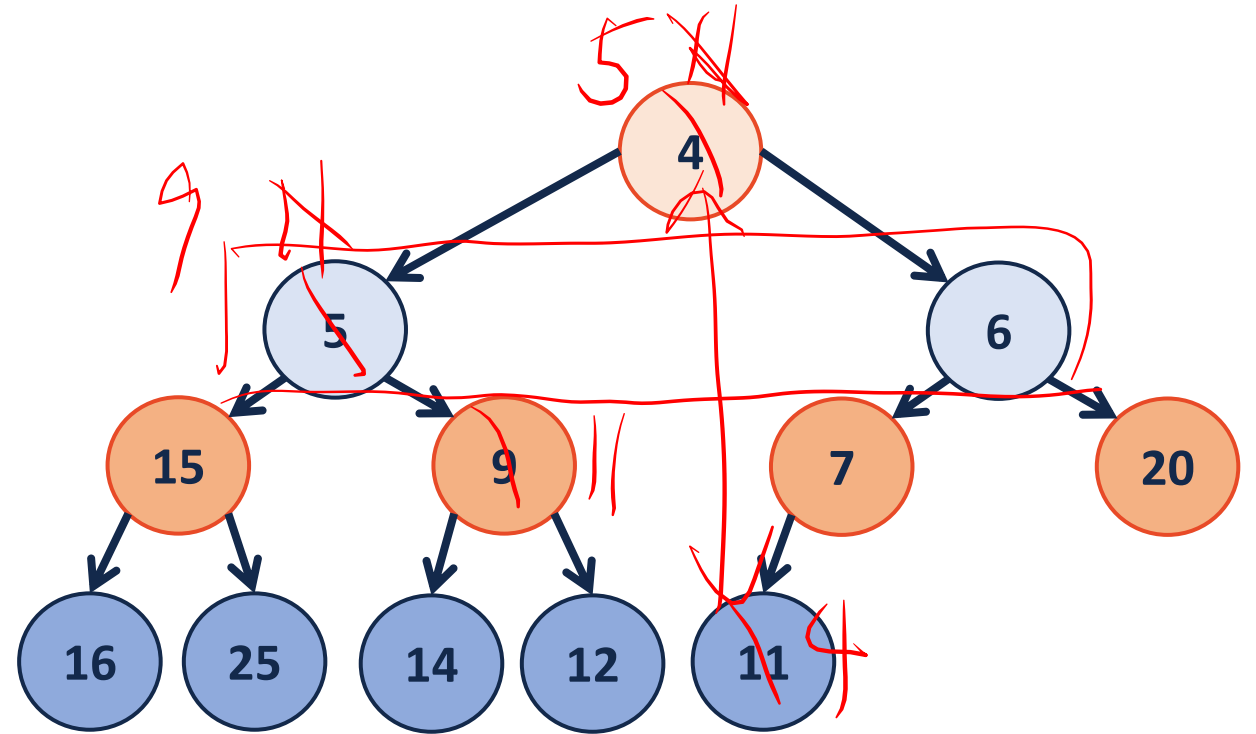


removeMin



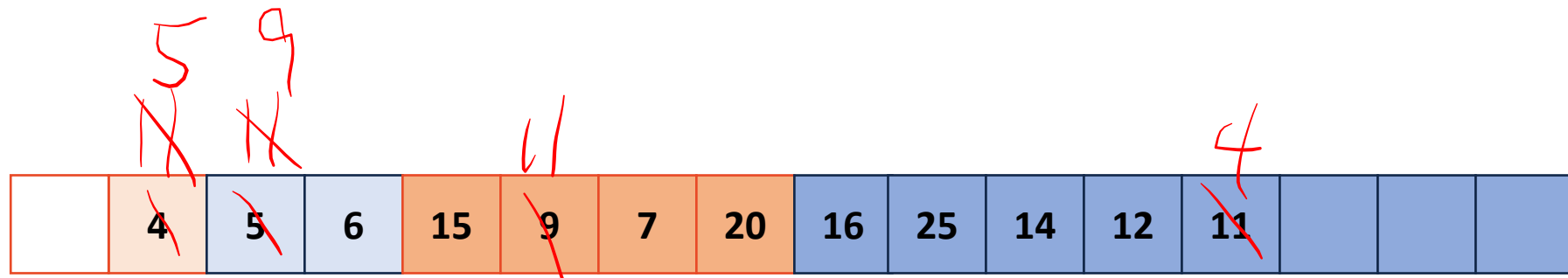
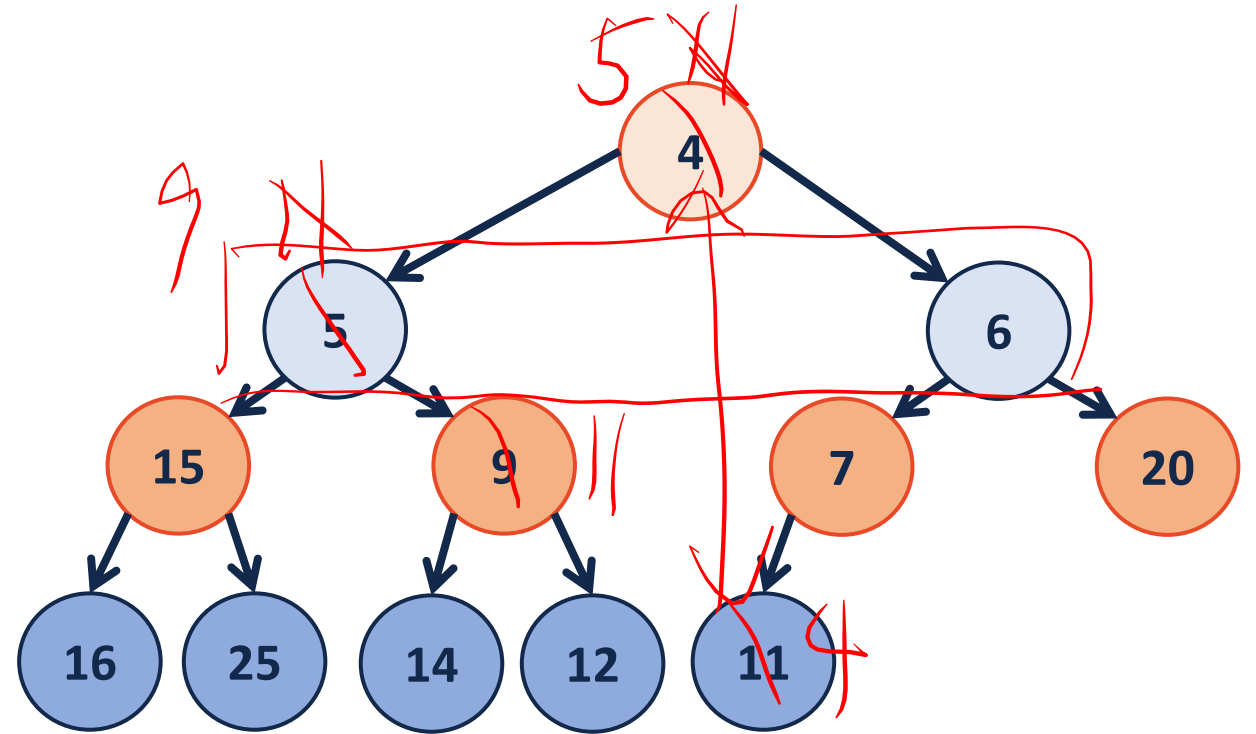
removeMin

SWAP · $O(1)$



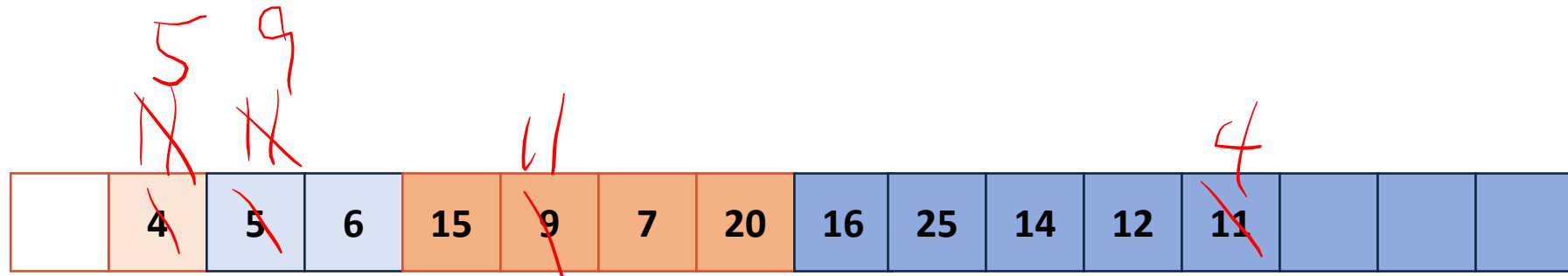
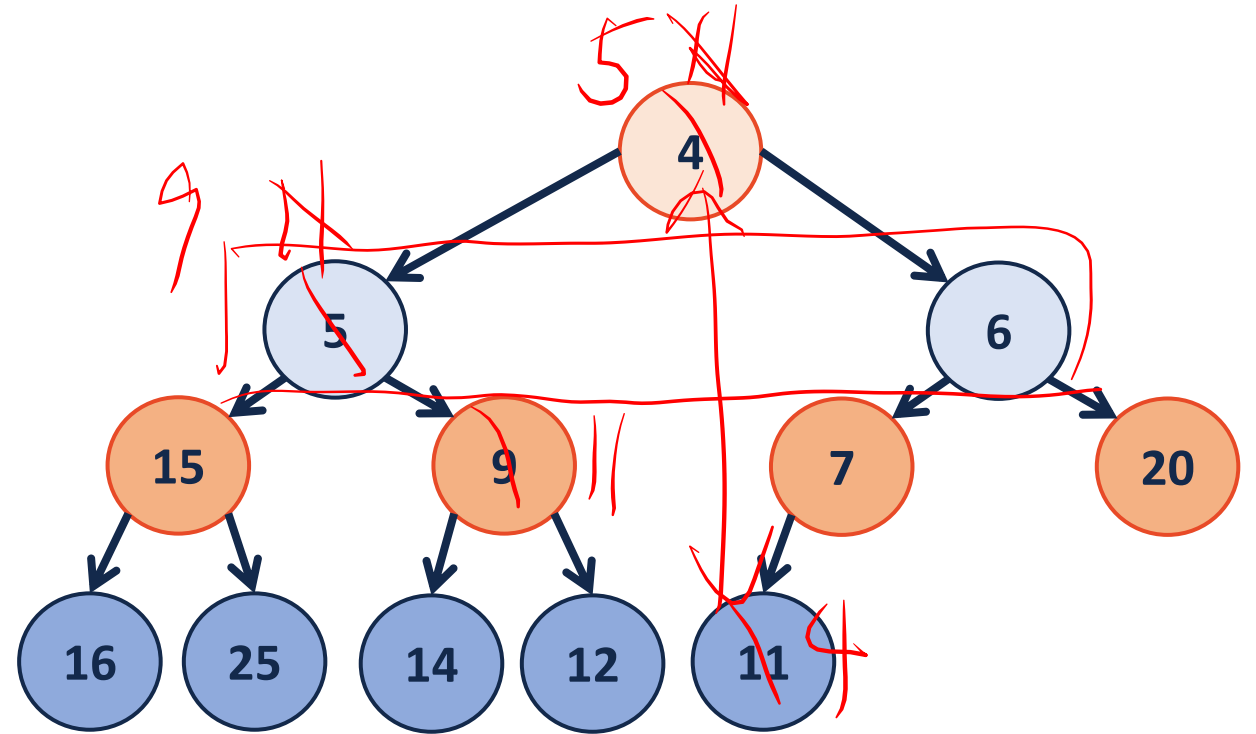
removeMin

SWAP : $O(1)$
Heapify Down : $O(1)$



removeMin

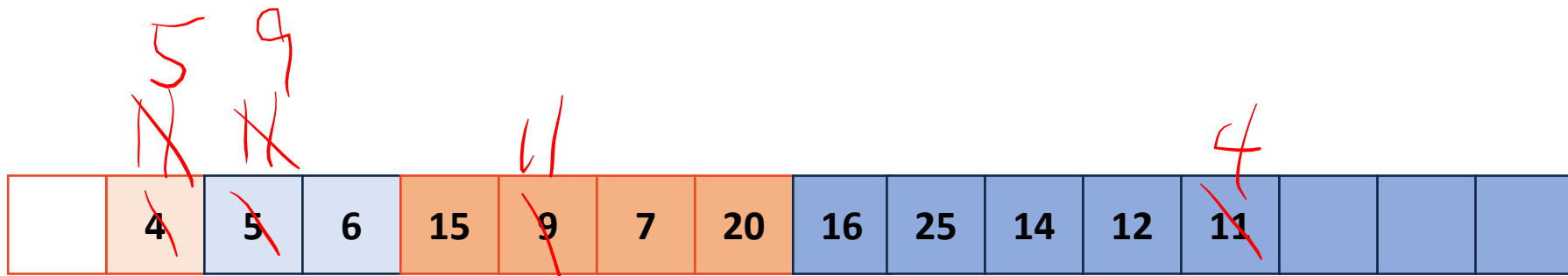
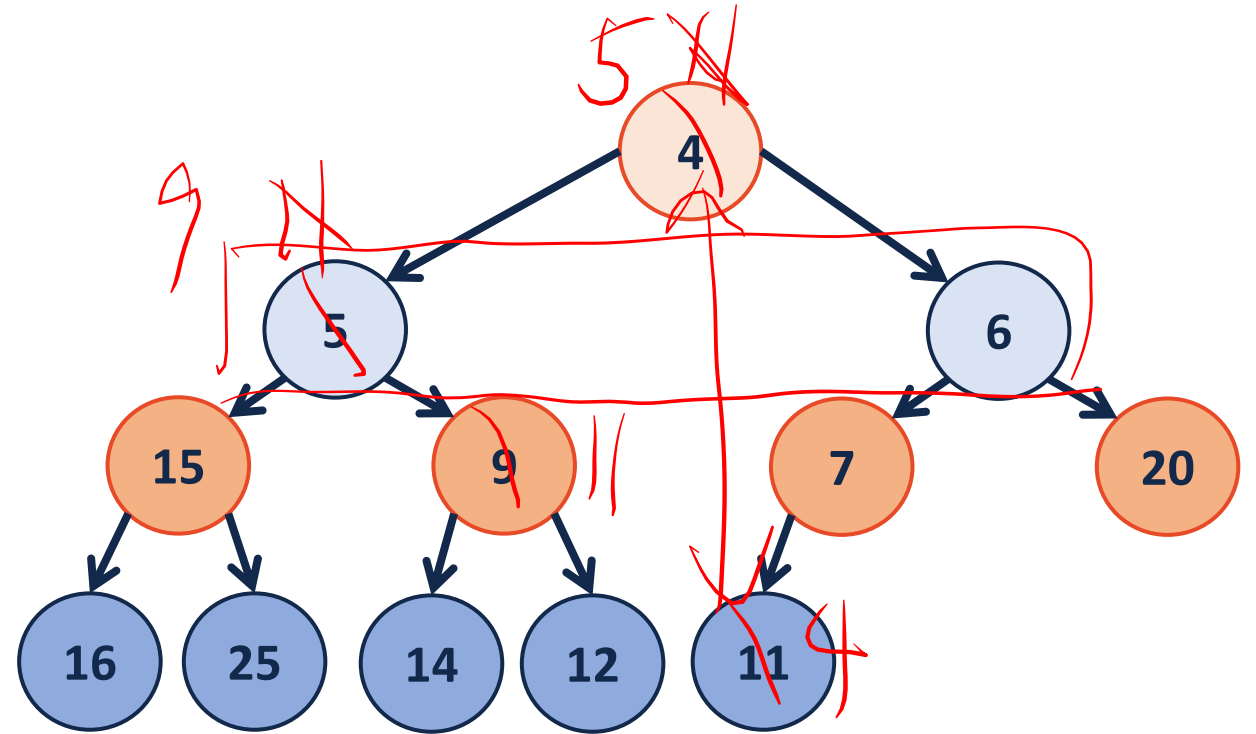
SWAP : $O(1)$
Heapify Down : $O(1)$
 $\times O(\lg n)$



removeMin

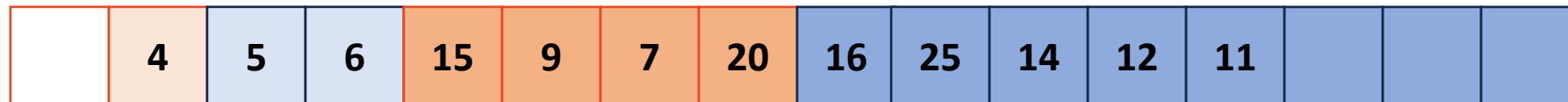
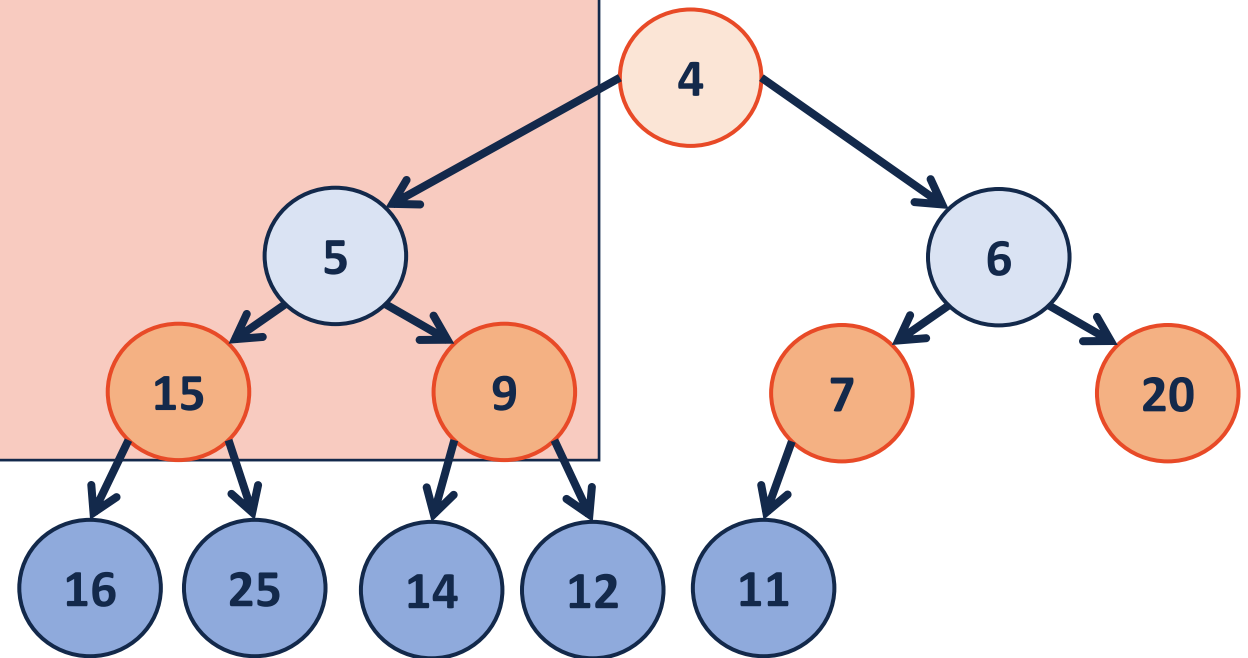
swap : $O(1)$
Heapify Down : $O(1)$
 $\times O(\lg n)$

 $O(\lg n)$



removeMin

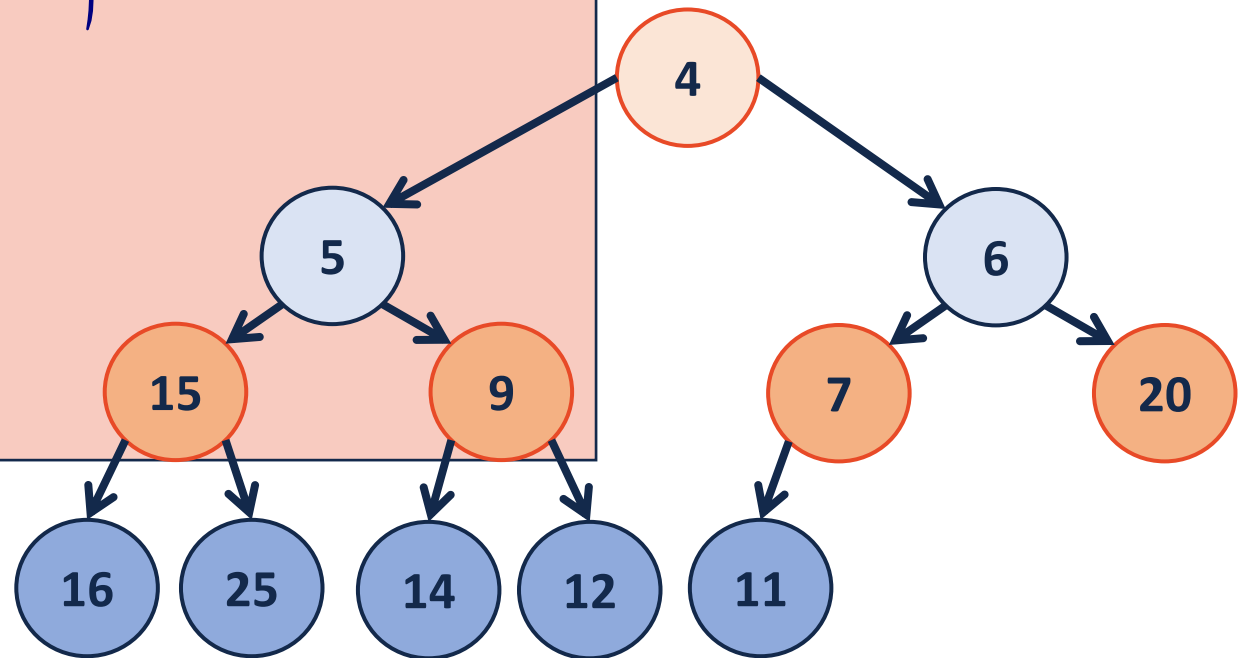
```
1  template <class T>
2  void Heap<T>::_removeMin() {
3      // Swap with the last value
4      T minValue = item_[1];
5      item_[1] = item_[size_];
6      size--;
7
8      // Restore the heap property
9      heapifyDown(1);
10
11     // Return the minimum value
12     return minValue;
13 }
```



removeMin

```
1  template <class T>
2  void Heap<T>::_removeMin() {
3      // Swap with the last value
4      T minValue = item_[1];
5      item_[1] = item_[size_];
6      size--;
7
8      // Restore the heap property
9      heapifyDown(1);
10
11     // Return the minimum value
12     return minValue;
13 }
```

→ swap w/ end

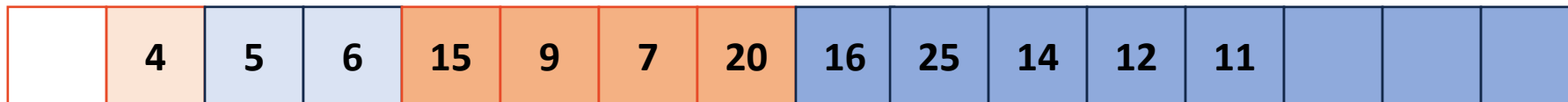
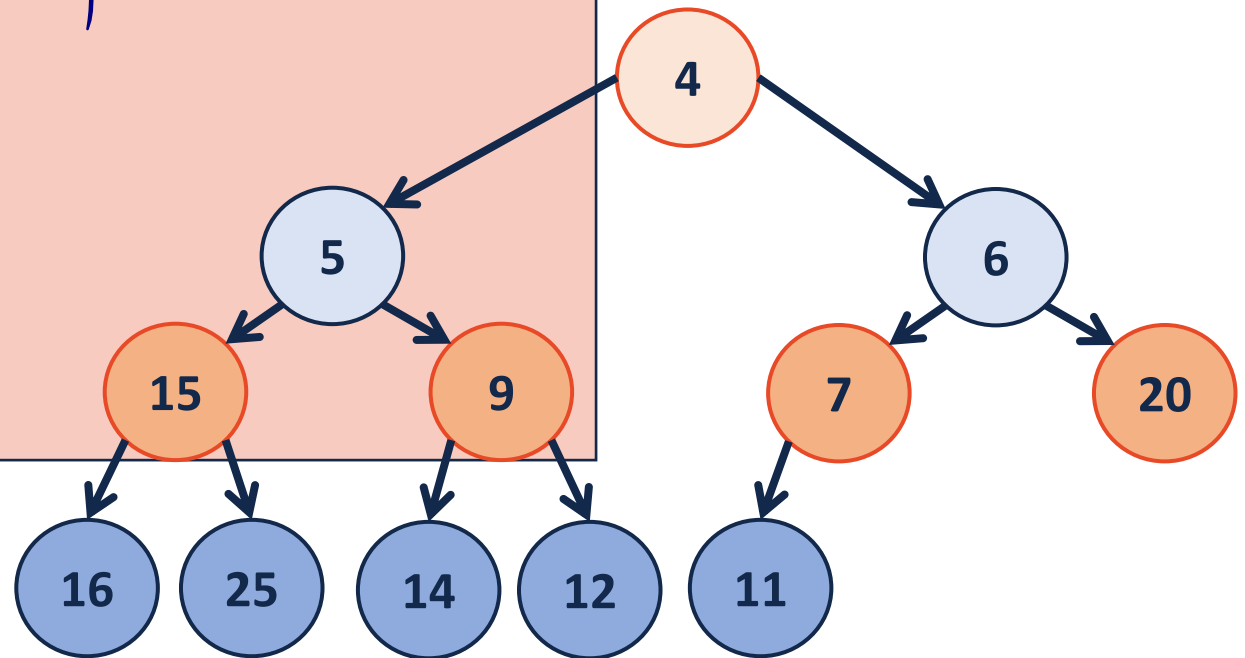


	4	5	6	15	9	7	20	16	25	14	12	11			
--	---	---	---	----	---	---	----	----	----	----	----	----	--	--	--

removeMin

```
1  template <class T>
2  void Heap<T>::_removeMin() {
3      // Swap with the last value
4      T minValue = item_[1];
5      item_[1] = item_[size_];
6      size--;
7
8      // Restore the heap property
9      heapifyDown(1);
10
11     // Return the minimum value
12     return minValue;
13 }
```

→ swap w/ end



removeMin - heapifyDown

```
1  template <class T>
2  void Heap<T>::_removeMin() {
3      // Swap with the last value
4      T minValue = item_[1];
5      item_[1] = item_[size_];
6      size--;
7
8      // Restore the heap property
9      _heapifyDown(1);
10
11     // Return the minimum value
12     return minValue;
13 }
```

```
1  template <class T>
2  void Heap<T>::_heapifyDown(int index) {
3      if ( !_isLeaf(index) ) {
4          int minChildIndex = _minChild(index);
5          if ( item_[index] > item_[minChildIndex] ) {
6              std::swap( item_[index], item_[minChildIndex] );
7              _heapifyDown( minChildIndex );
8          }
9      }
10 }
```

removeMin - heapifyDown

```
1  template <class T>
2  void Heap<T>::_removeMin() {
3      // Swap with the last value
4      T minValue = item_[1];
5      item_[1] = item_[size_];
6      size--;
7
8      // Restore the heap property
9      _heapifyDown(1);
10
11     // Return the minimum value
12     return minValue;
13 }
```

Base case check

```
1  template <class T>
2  void Heap<T>::_heapifyDown(int index) {
3      if ( !_isLeaf(index) ) {
4          int minChildIndex = _minChild(index);
5          if ( item_[index] > item_[minChildIndex] ) {
6              std::swap( item_[index], item_[minChildIndex] );
7              _heapifyDown( minChildIndex );
8          }
9      }
10 }
```

removeMin - heapifyDown

```
1  template <class T>
2  void Heap<T>::_removeMin() {
3      // Swap with the last value
4      T minValue = item_[1];
5      item_[1] = item_[size_];
6      size--;
7
8      // Restore the heap property
9      _heapifyDown(1);
10
11     // Return the minimum value
12     return minValue;
13 }
```

Base case check

```
1  template <class T>
2  void Heap<T>::_heapifyDown(int index) {
3      if ( !_isLeaf(index) ) {
4          int minChildIndex = _minChild(index);
5          if ( item_[index] > item_[minChildIndex] ) {
6              std::swap( item_[index], item_[minChildIndex] );
7              _heapifyDown( _____ );
8          }
9      }
10 }
```

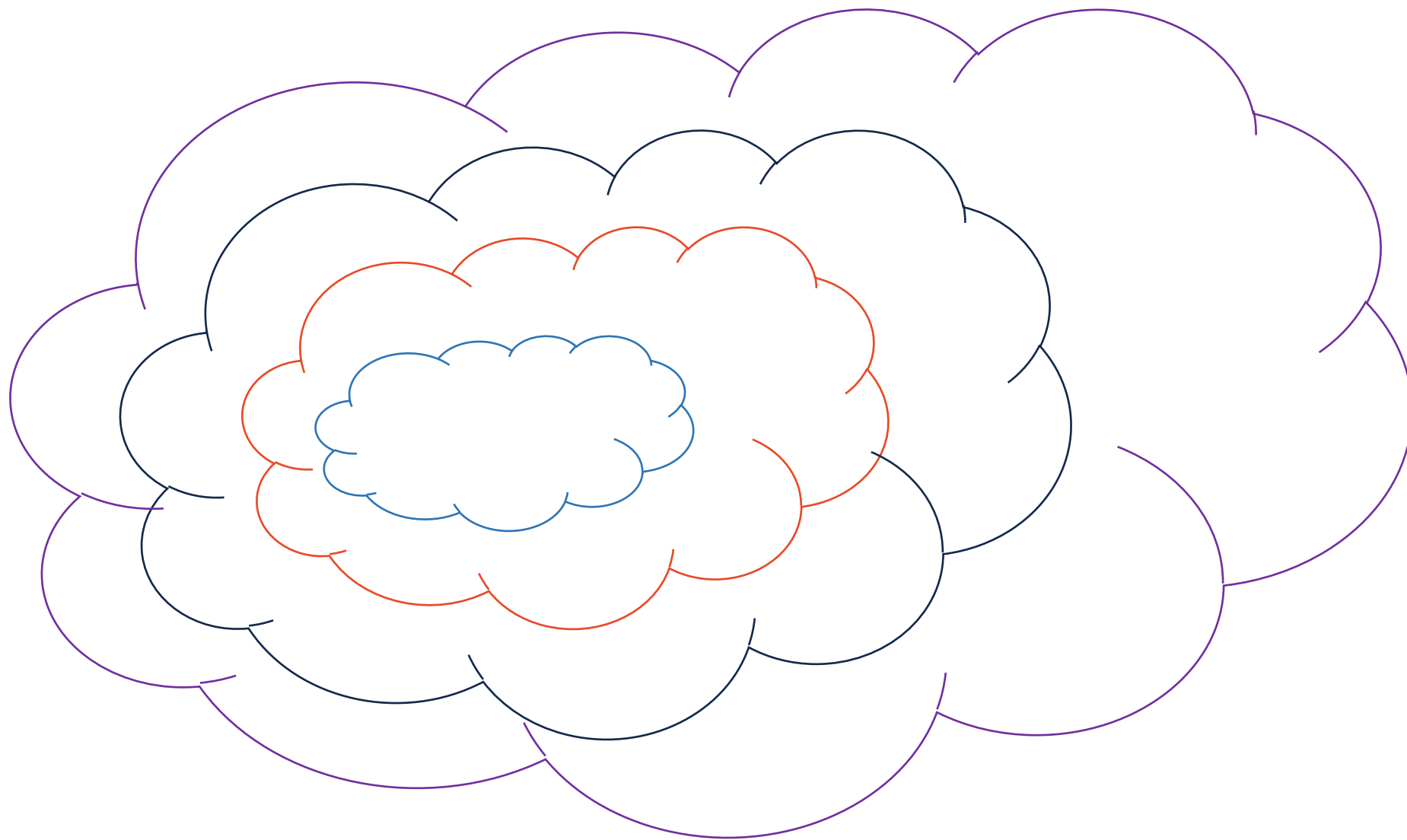
removeMin - heapifyDown

```
1  template <class T>
2  void Heap<T>::_removeMin() {
3      // Swap with the last value
4      T minValue = item_[1];
5      item_[1] = item_[size_];
6      size--;
7
8      // Restore the heap property
9      _heapifyDown(1);
10
11     // Return the minimum value
12     return minValue;
13 }
```

Base case check

```
1  template <class T>
2  void Heap<T>::_heapifyDown(int index) {
3      if ( !_isLeaf(index) ) {
4          int minChildIndex = _minChild(index);
5          if ( item_[index] > item_[minChildIndex] ) {
6              std::swap( item_[index], item_[minChildIndex] );
7              _heapifyDown( minChildIndex );
8          }
9      }
10 }
```

Array Abstractions



Array Abstractions



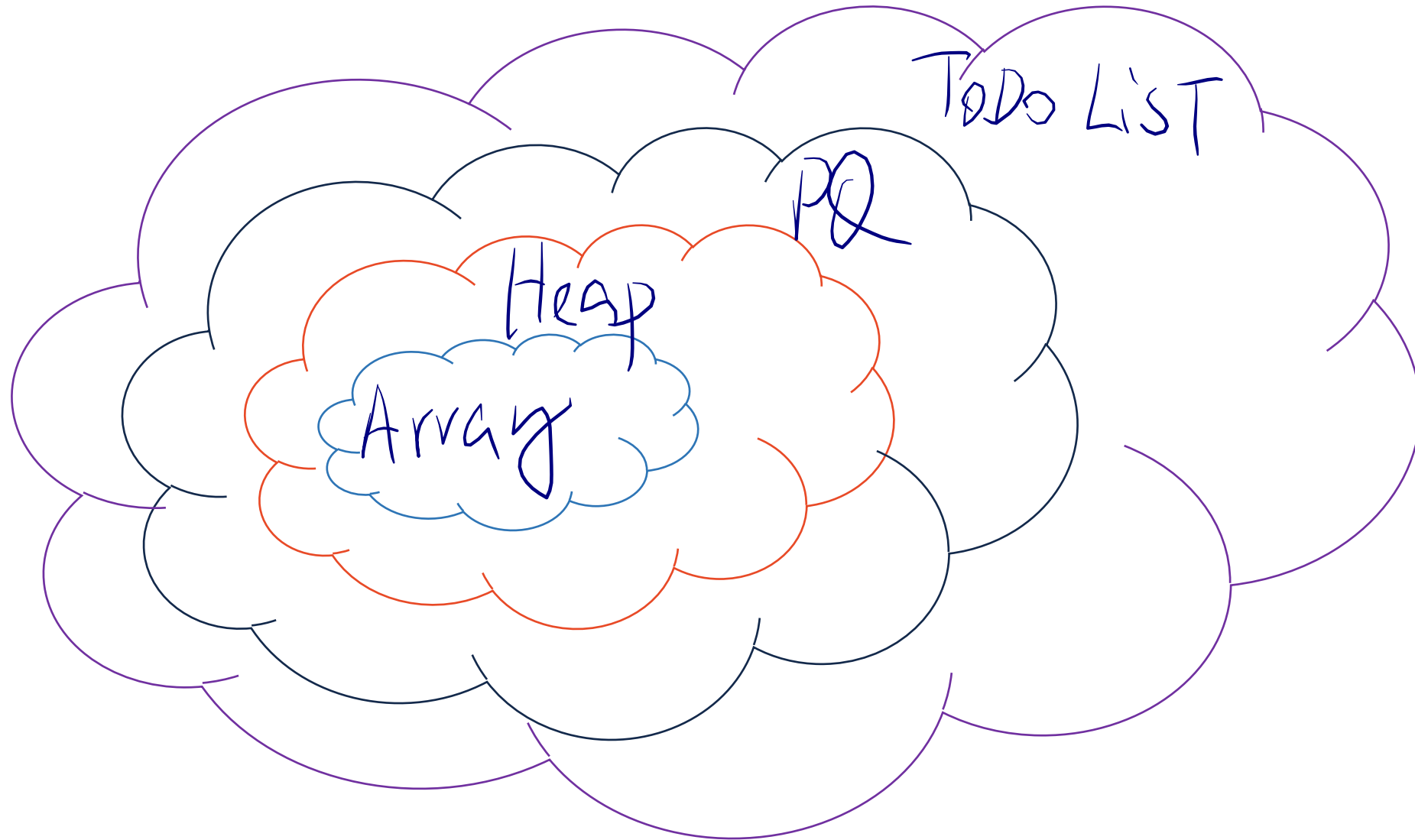
Array Abstractions



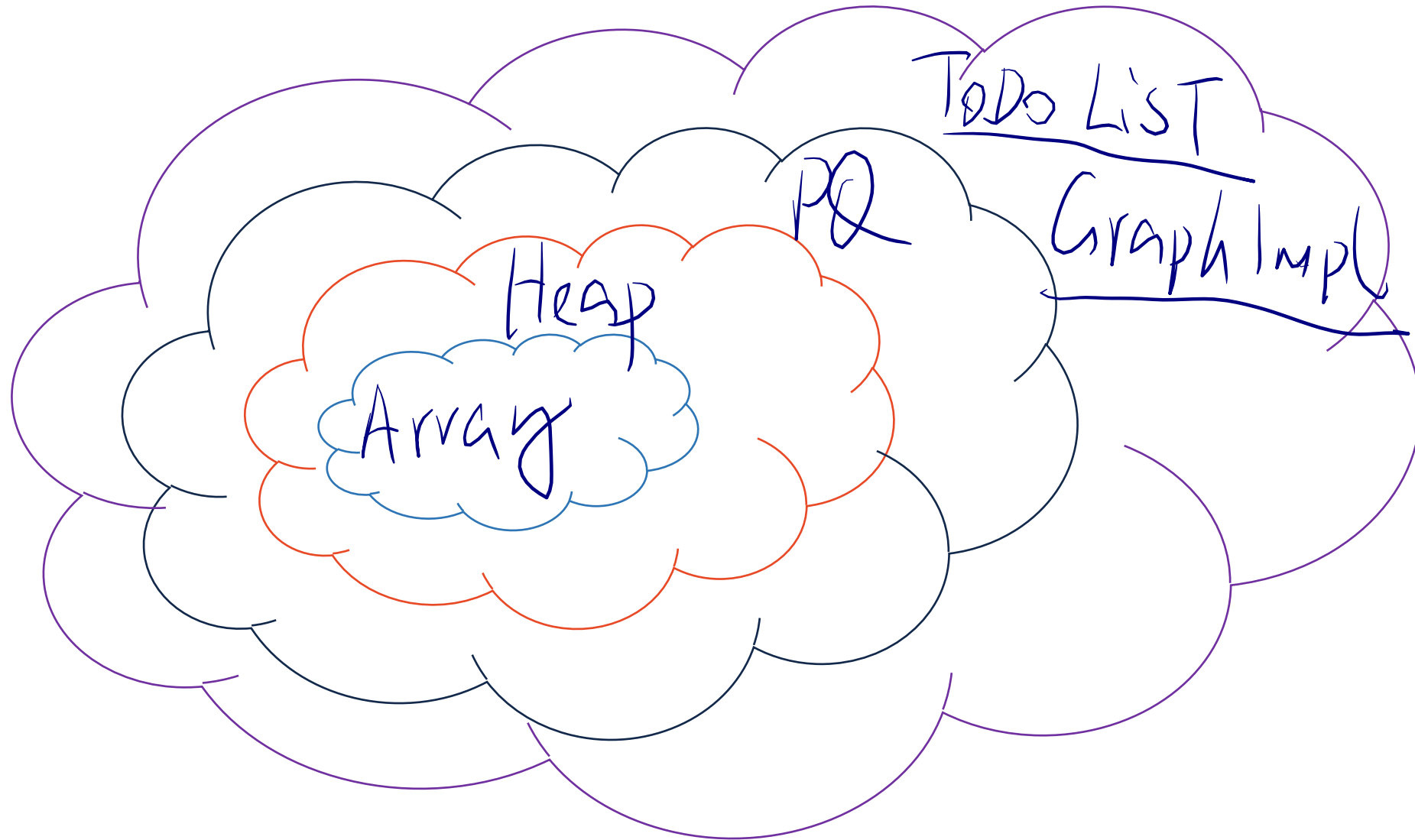
Array Abstractions



Array Abstractions



Array Abstractions



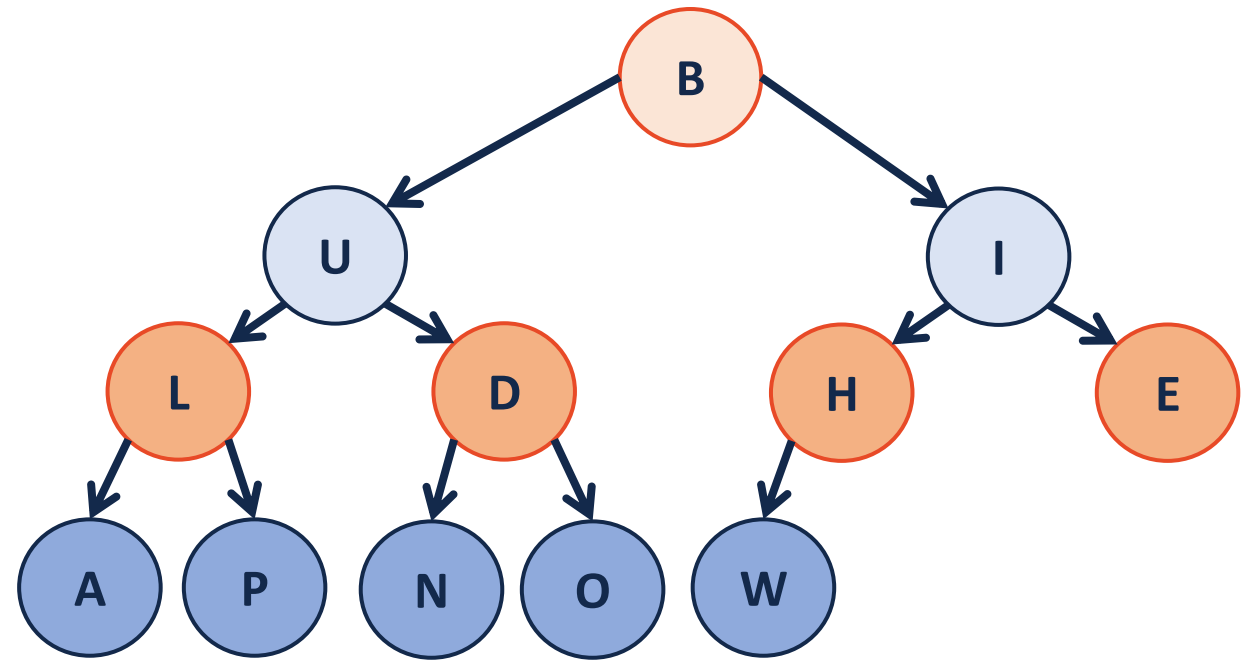
CS 225

Data Structures

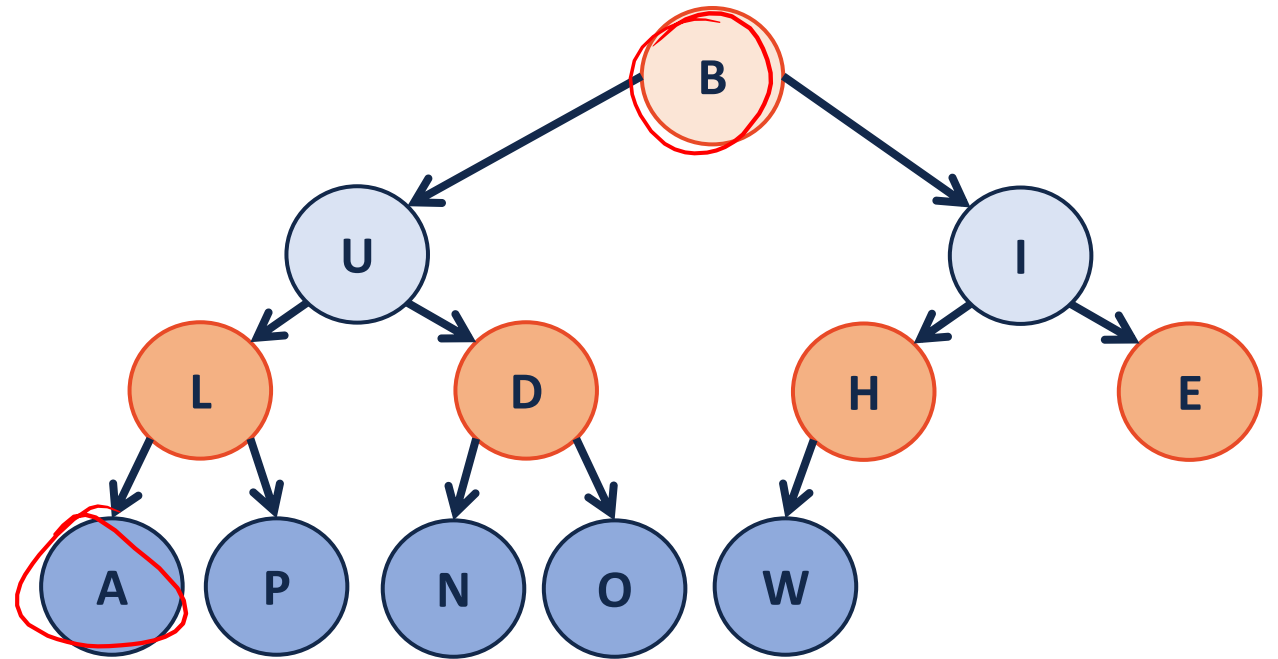
Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

buildHeap

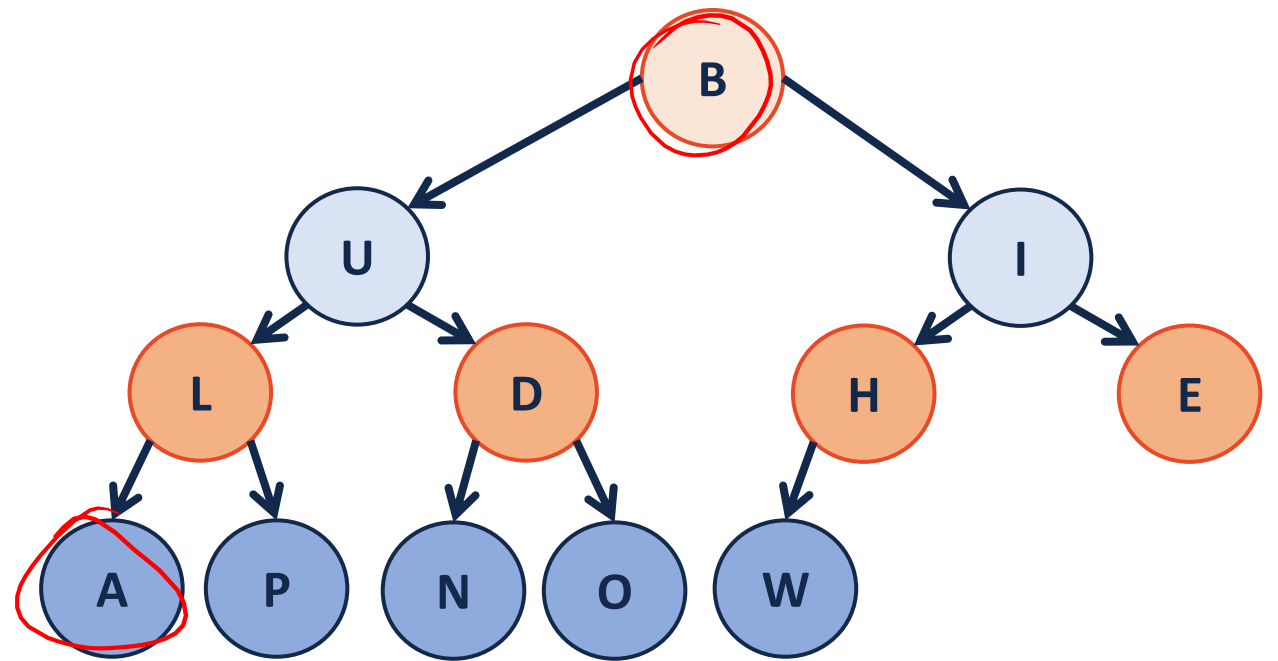


buildHeap



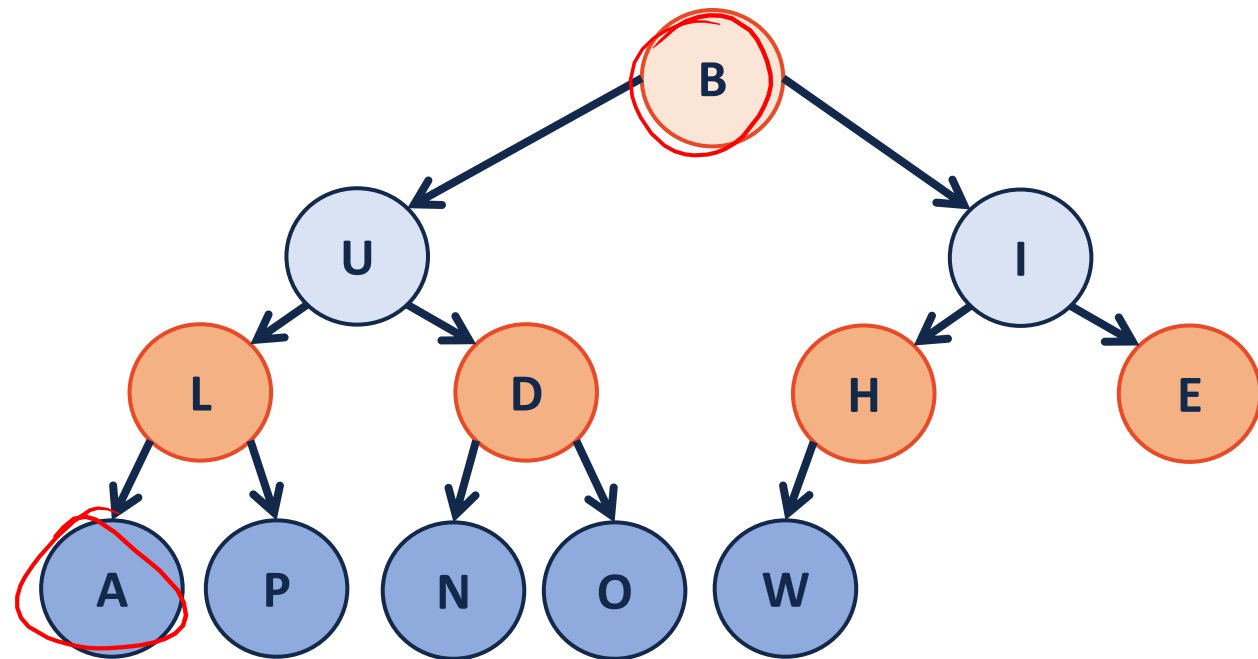
buildHeap

① Sort



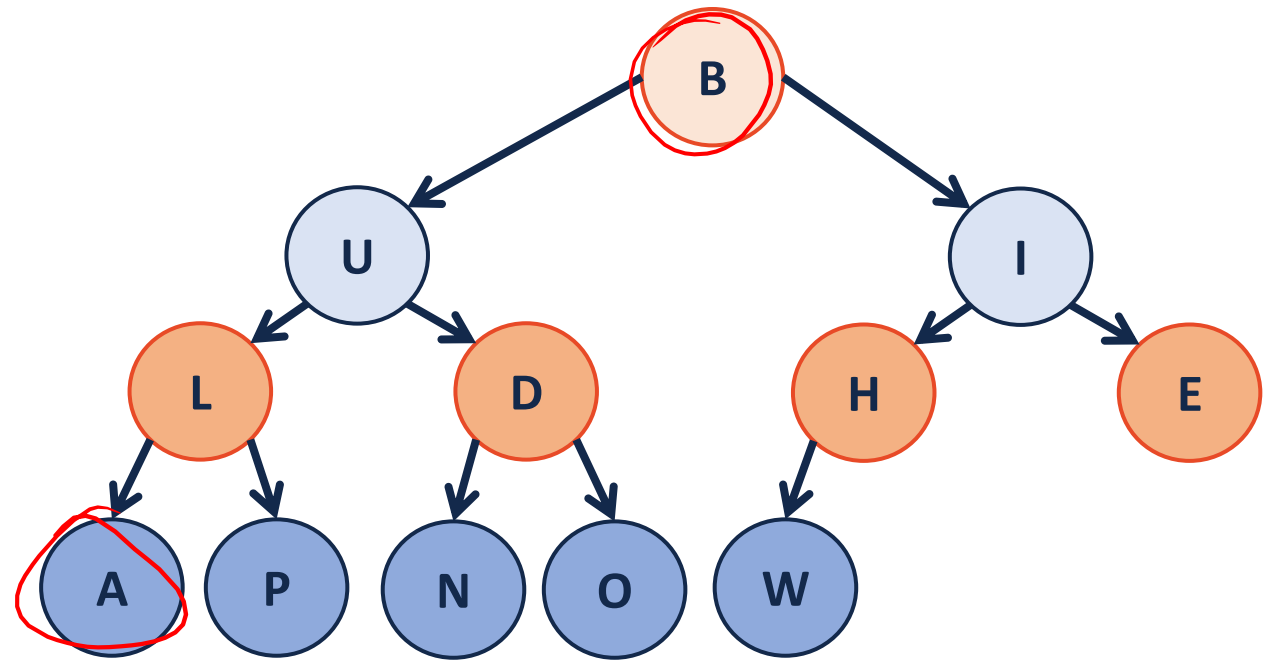
buildHeap

① Sort
- Heap requires an ordering on every path



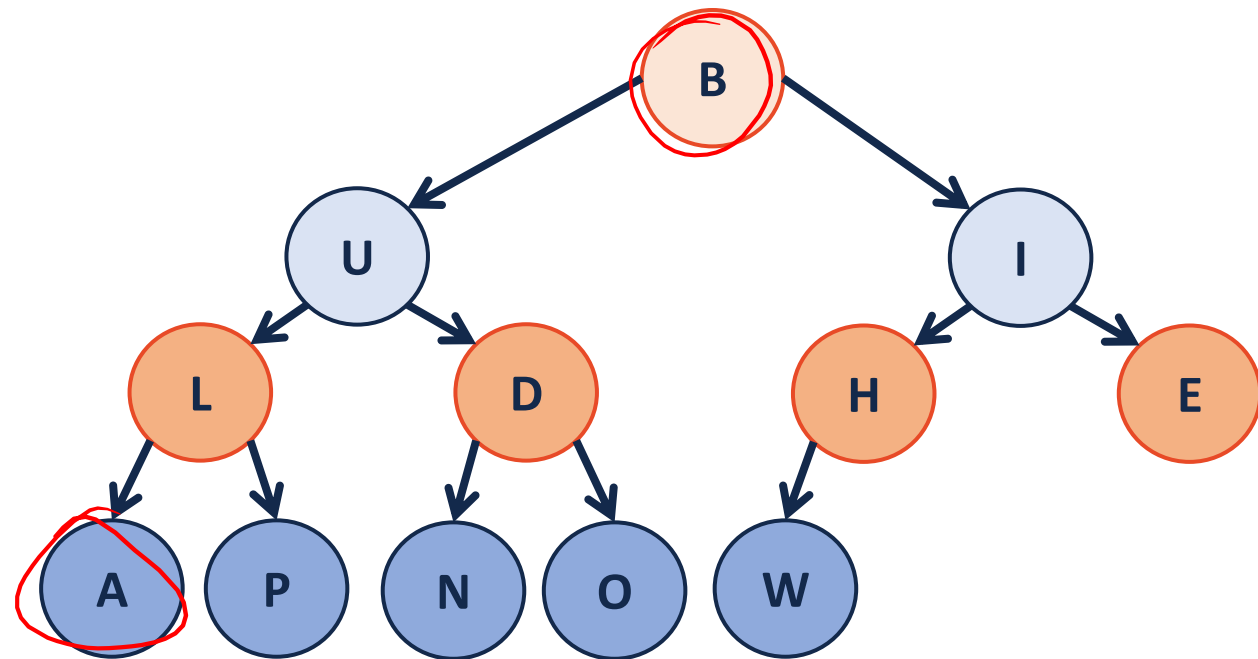
buildHeap

① Sort
- Heap requires an ordering on every path
 $O(n \lg n)$



buildHeap

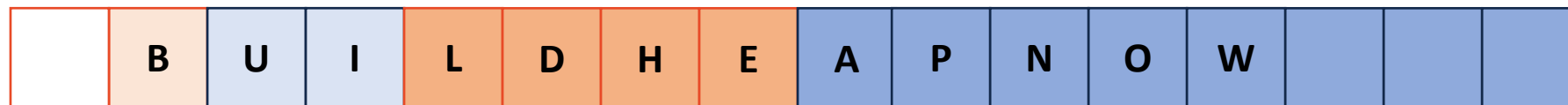
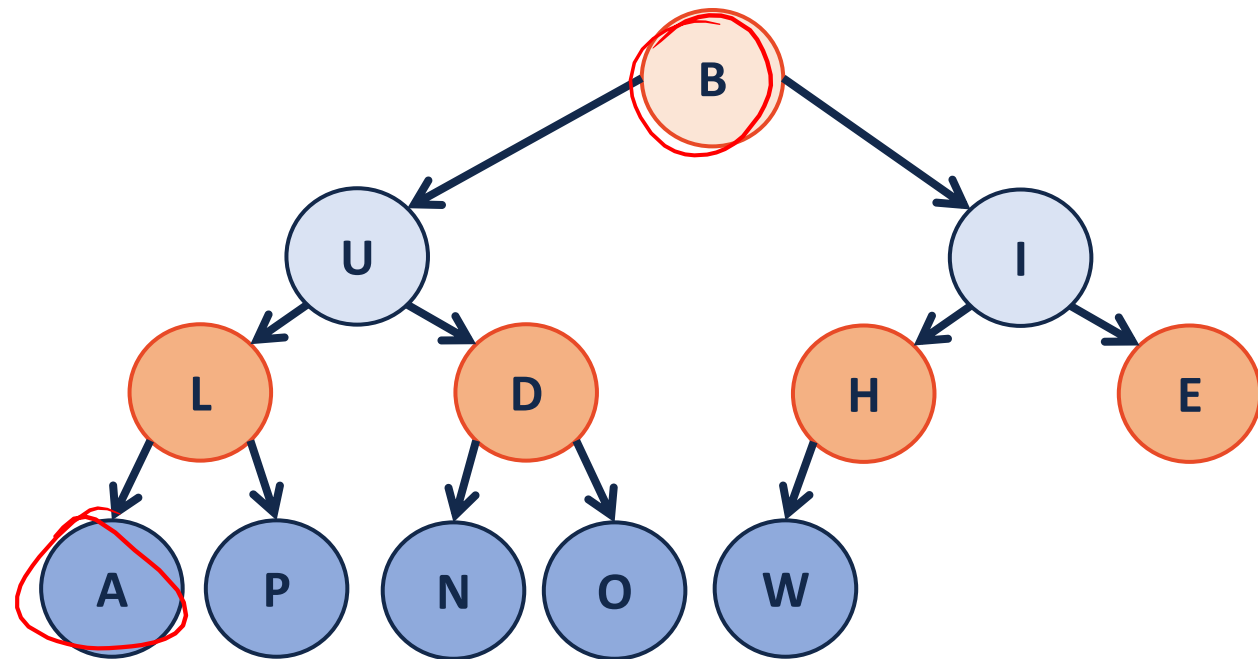
① Sort
- Heap requires an ordering on every path
 $O(n \lg n)$



② Call insert() n times

buildHeap

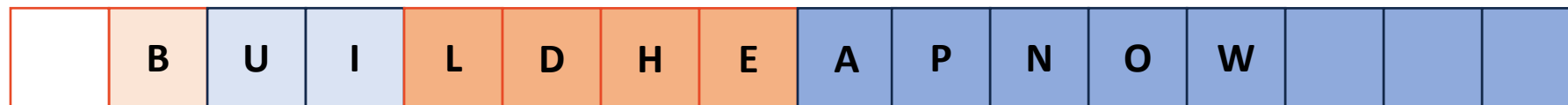
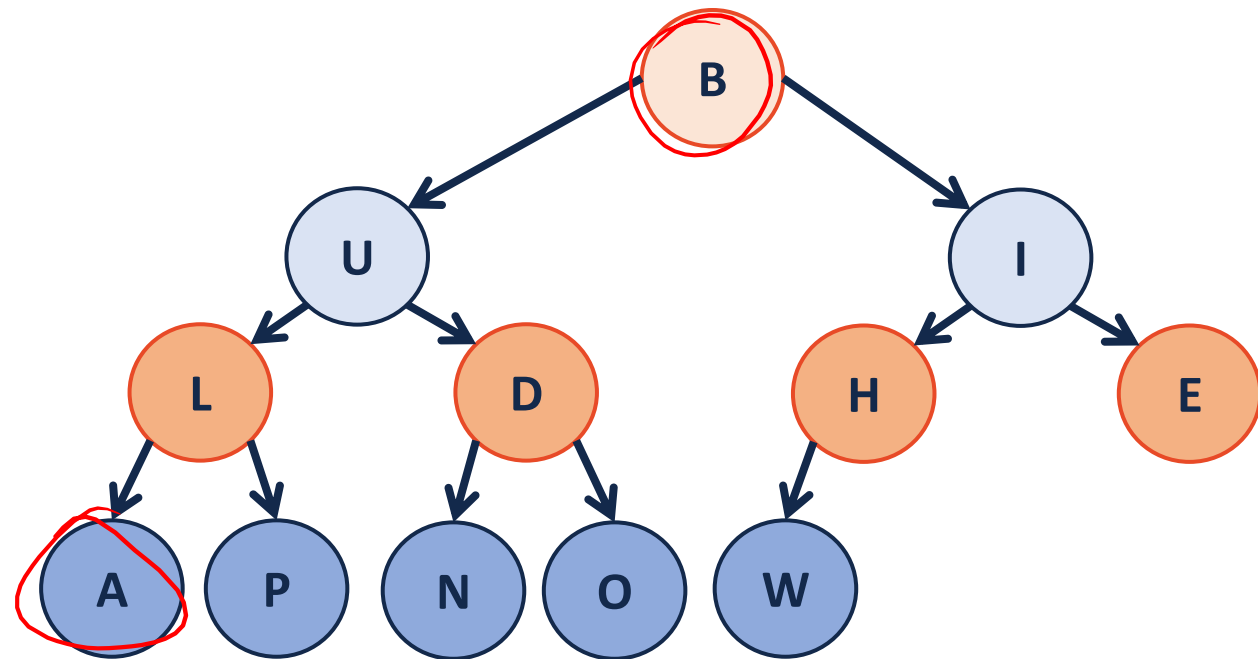
① Sort
- Heap requires an ordering on every path
 $O(n \lg n)$



② Call insert() n times $\lg n \times n = n \lg n$

buildHeap

① Sort
- Heap requires an ordering on every path
 $O(n \lg n)$



② Call insert() n times
↳ HeapifyUp

$$\lg n \times n = n \lg n$$

buildHeap

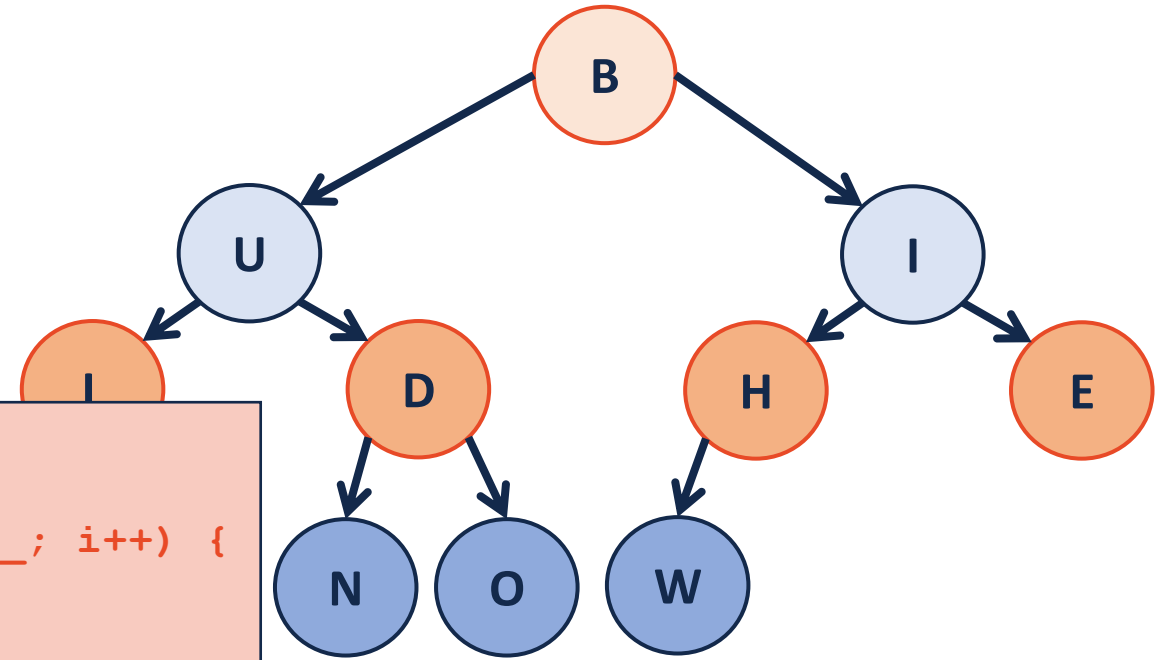
1. Sort the array – it's a heap!

2.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = 0; i <= size_; i++) {
4         heapifyUp(i);
5     }
6 }
```

3.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = parent(size); i > 0; i--) {
4         heapifyDown(i);
5     }
6 }
```



buildHeap

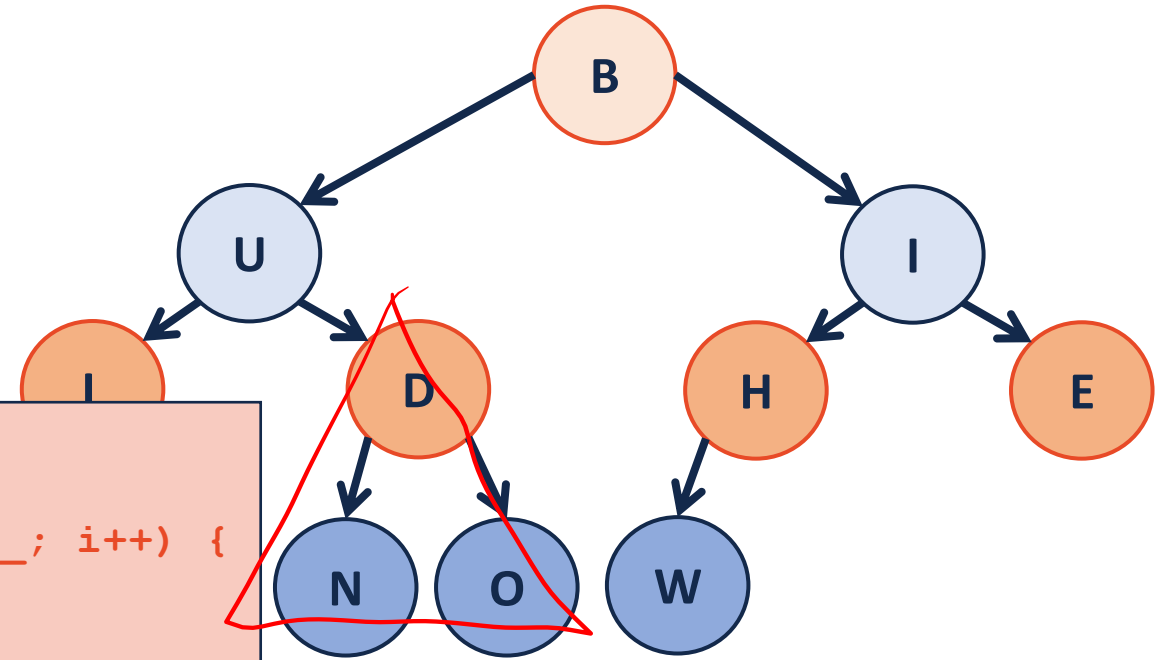
1. Sort the array – it's a heap!

2.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = 0; i <= size_; i++) {
4         heapifyUp(i);
5     }
6 }
```

3.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = parent(size); i > 0; i--) {
4         heapifyDown(i);
5     }
6 }
```



buildHeap

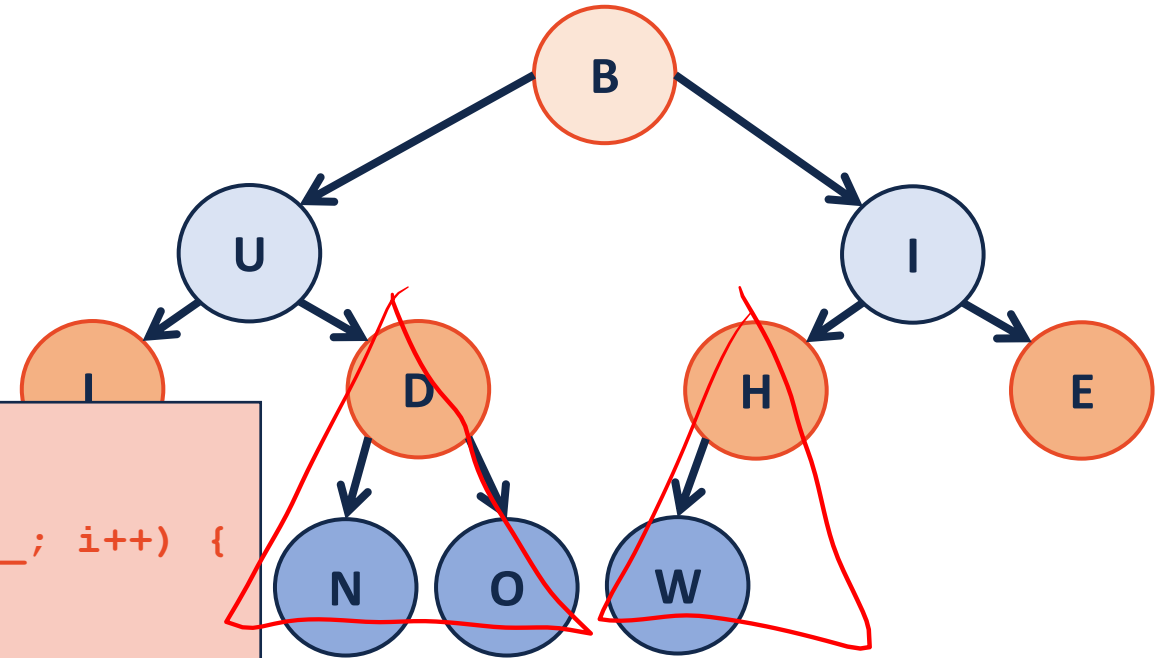
1. Sort the array – it's a heap!

2.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = 0; i <= size_; i++) {
4         heapifyUp(i);
5     }
6 }
```

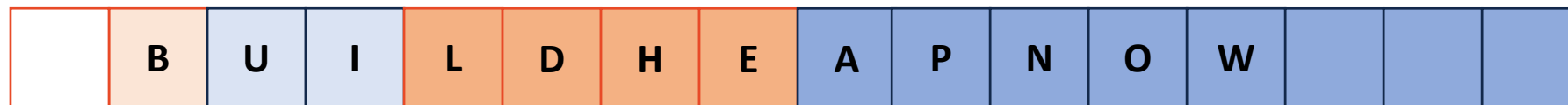
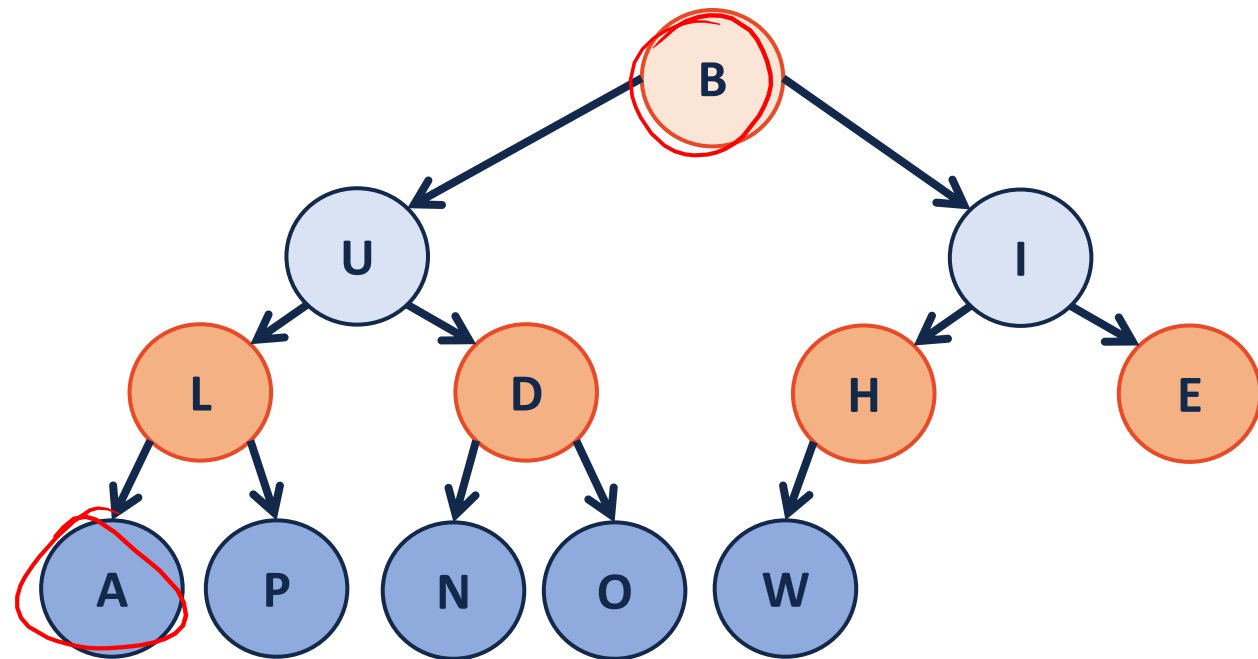
3.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = parent(size); i > 0; i--) {
4         heapifyDown(i);
5     }
6 }
```



buildHeap

① Sort
- Heap requires an ordering on every path
 $O(n \lg n)$



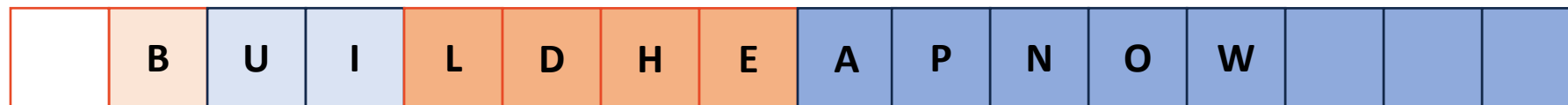
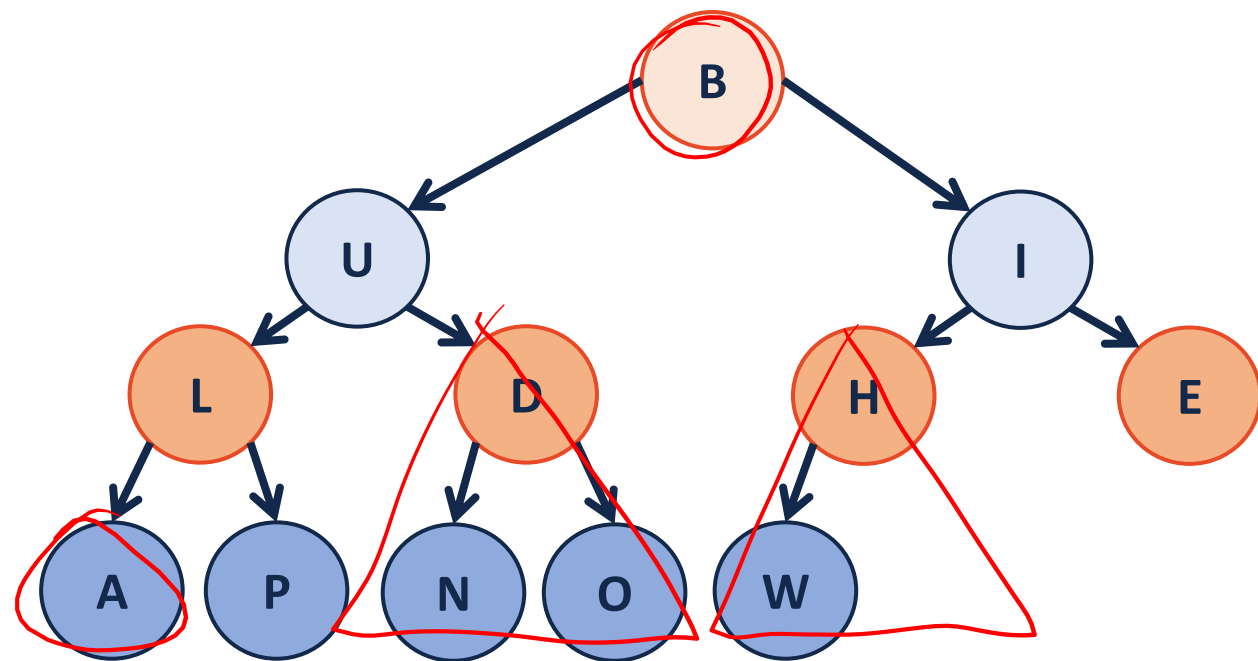
② Call insert() n times
↳ HeapifyUp

$$\lg n \times n = n \lg n$$

buildHeap

① Sort

- Heap requires an ordering on every path
 $O(n \lg n)$



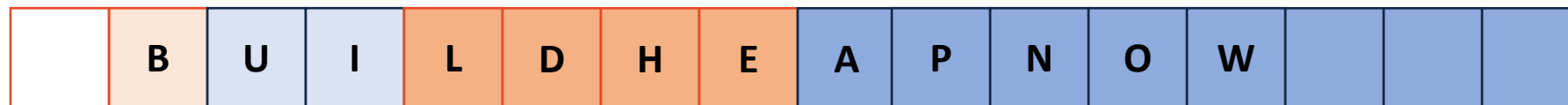
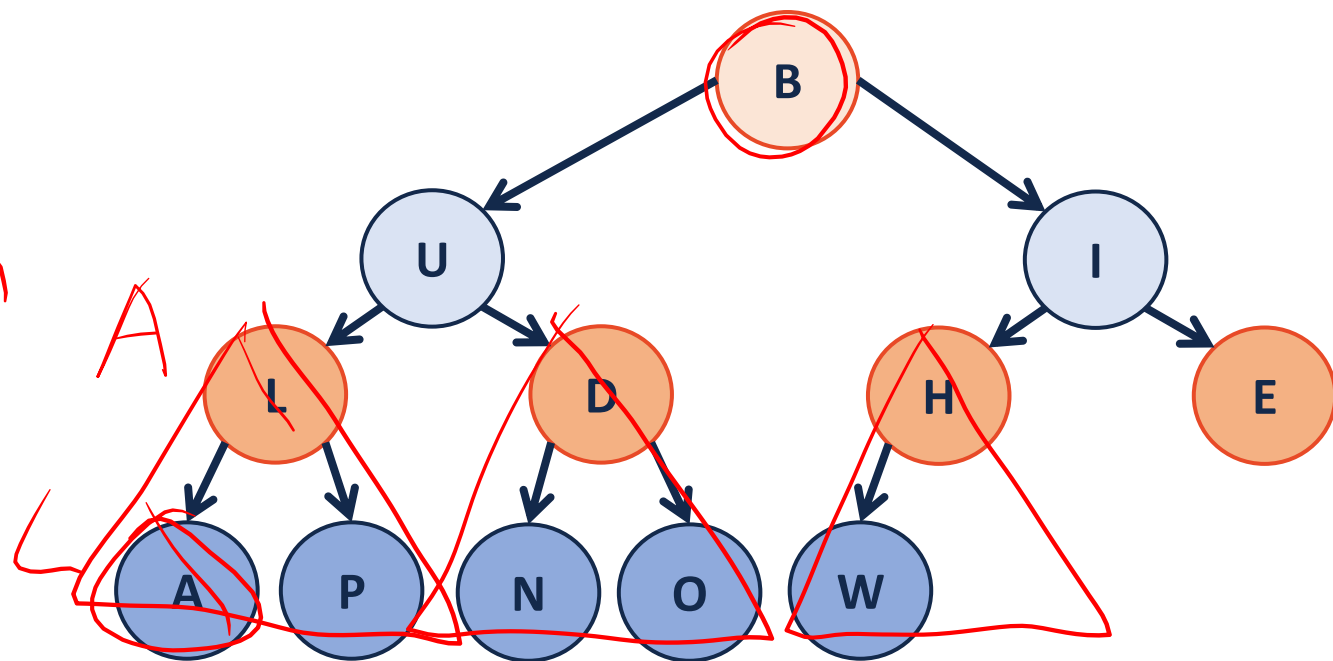
② Call insert() n times
↳ HeapifyUp

$$\lg(n) \times n = n \lg(n)$$

buildHeap

① Sort

- Heap requires an ordering on every path
 $O(n \lg n)$

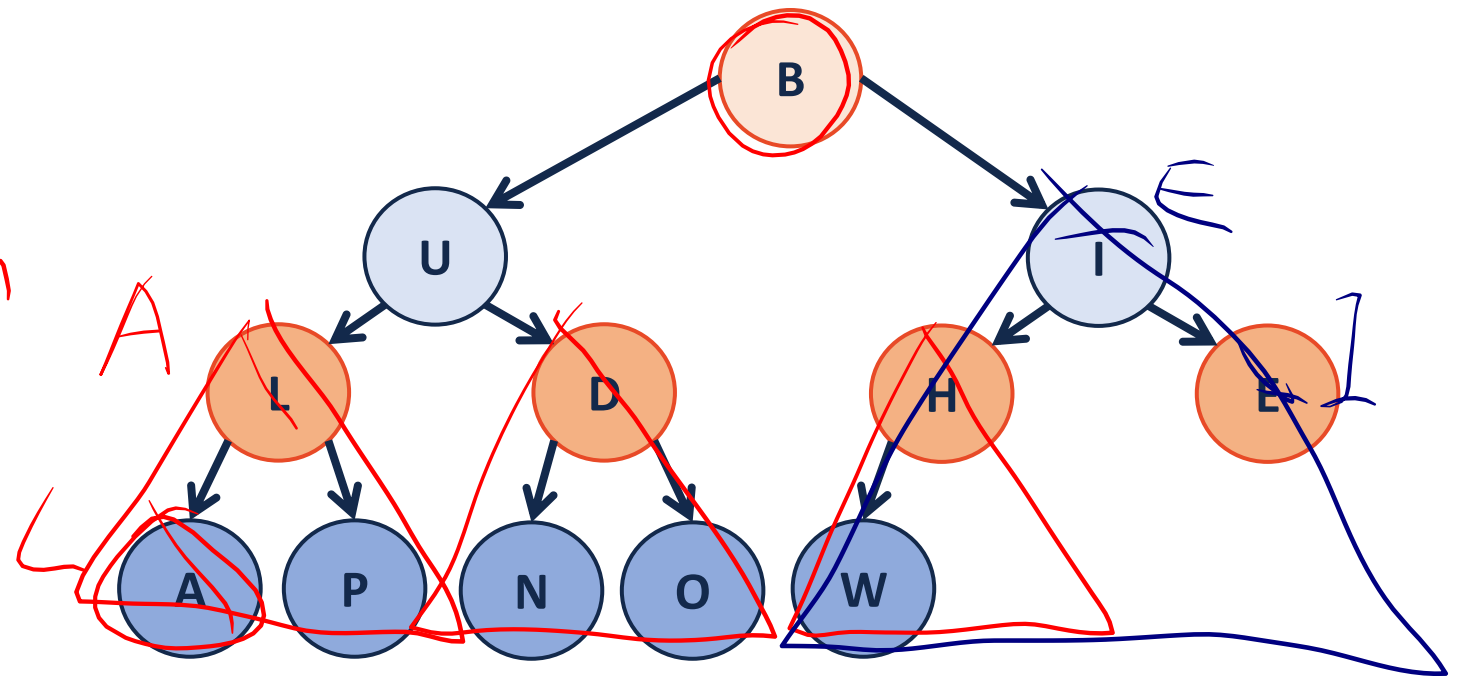


② Call insert() n times
↳ HeapifyUp

$$\lg n \times n = n \lg n$$

buildHeap

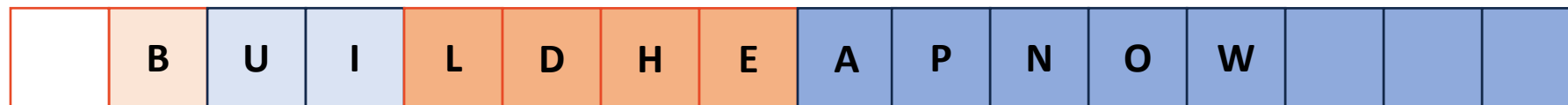
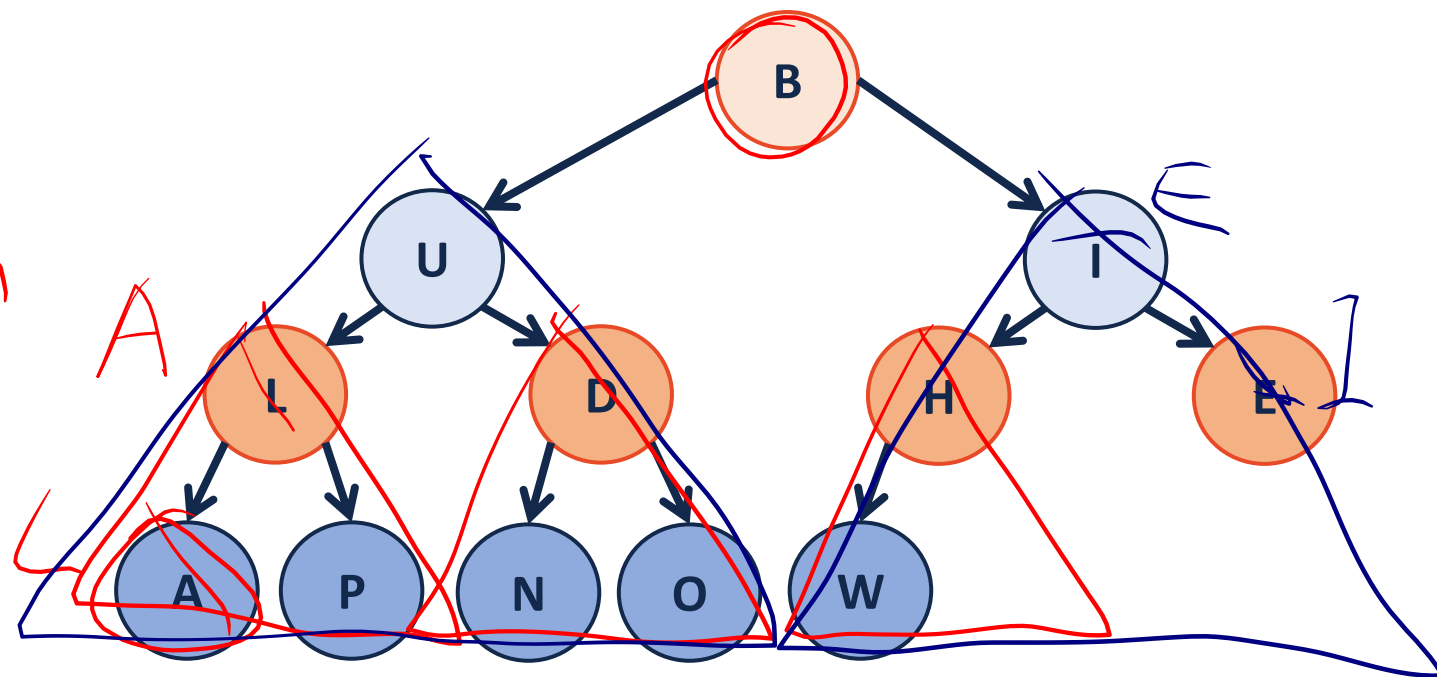
① Sort
- Heap requires an ordering on every path
 $O(n \lg n)$



② Call insert() n times $\lg n \times n = n \lg n$
 ↳ HeapifyUp

buildHeap

① Sort
- Heap requires an ordering on every path
 $O(n \lg n)$

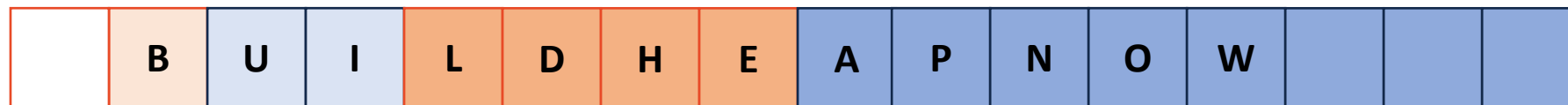
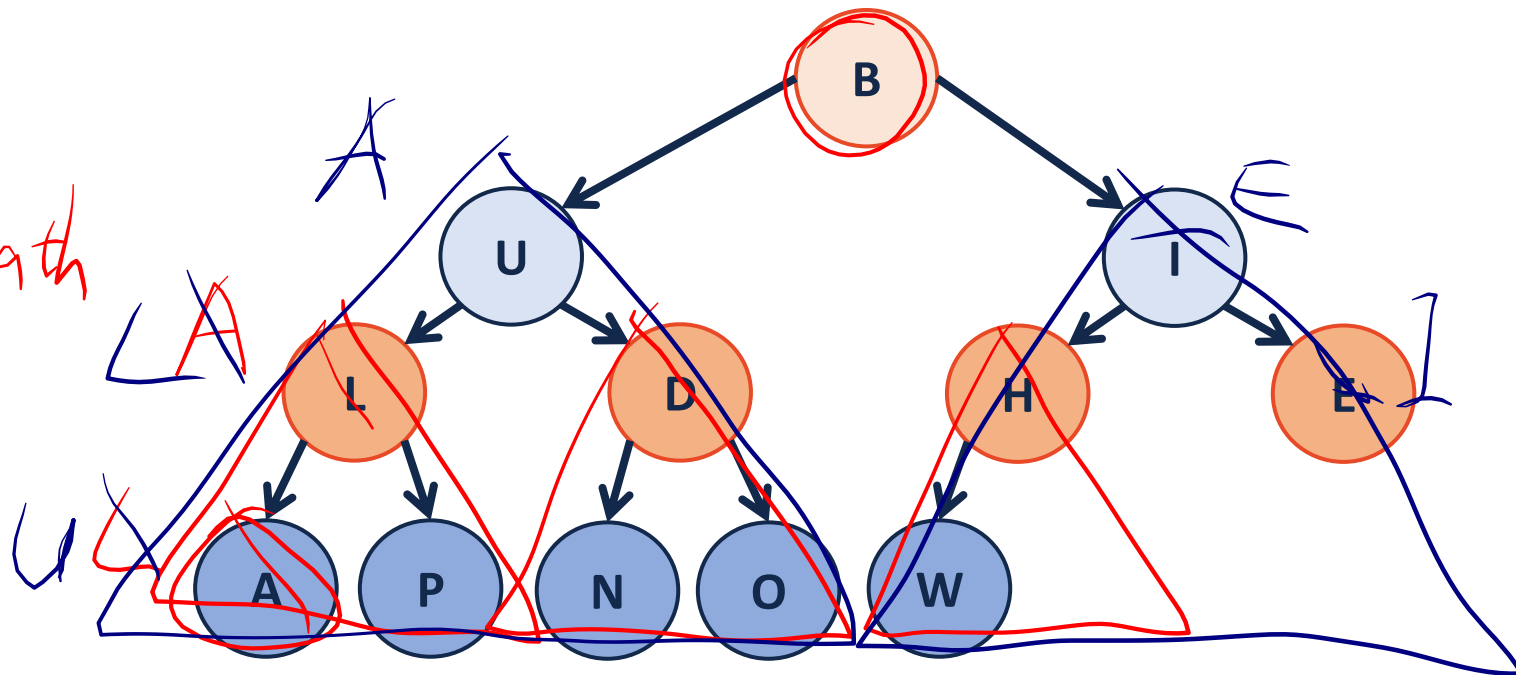


② Call insert() n times
↳ HeapifyUp

$$\lg n \times n = n \lg n$$

buildHeap

① Sort
- Heap requires an ordering on every path
 $O(n \lg n)$

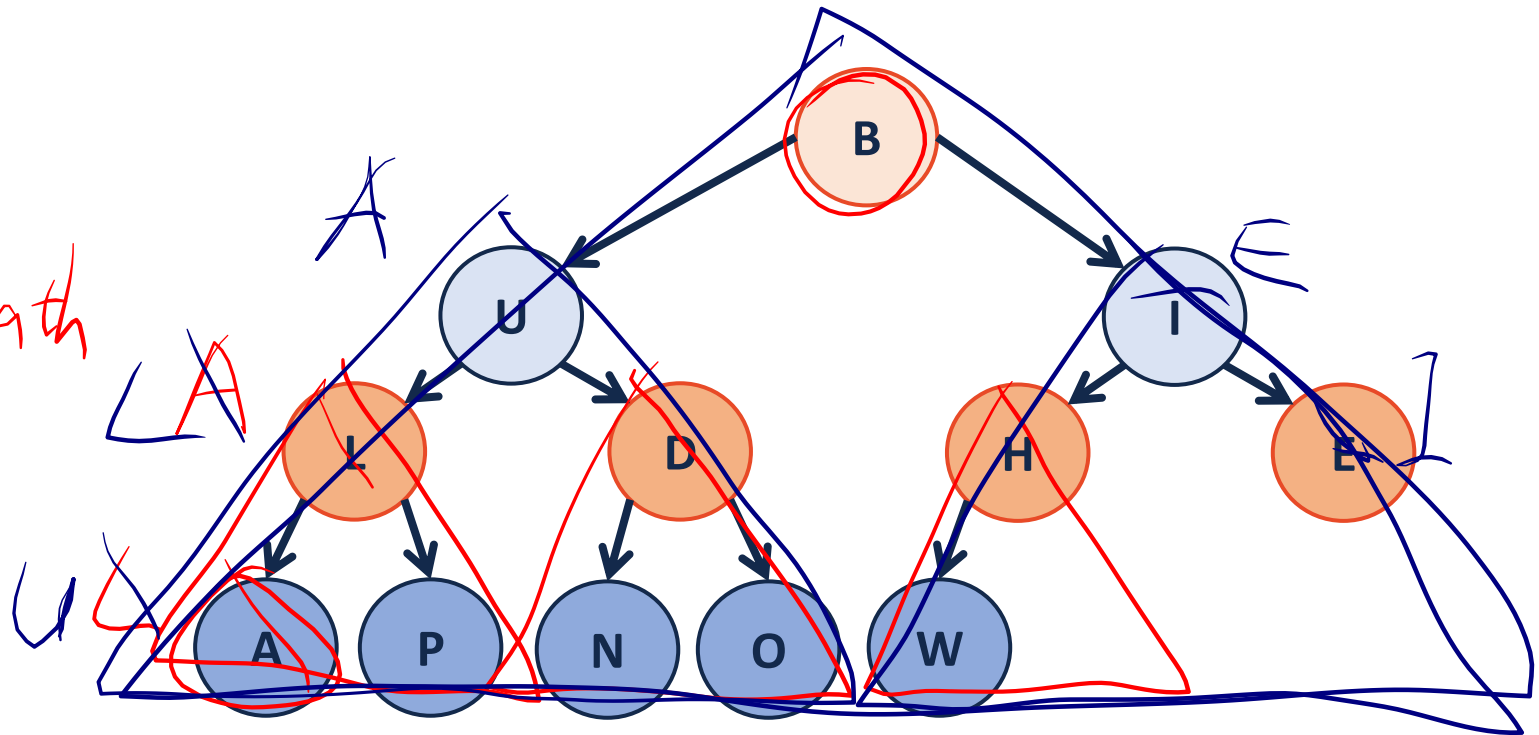


② Call insert() n times
↳ HeapifyUp

$$\lg n \times n = n \lg n$$

buildHeap

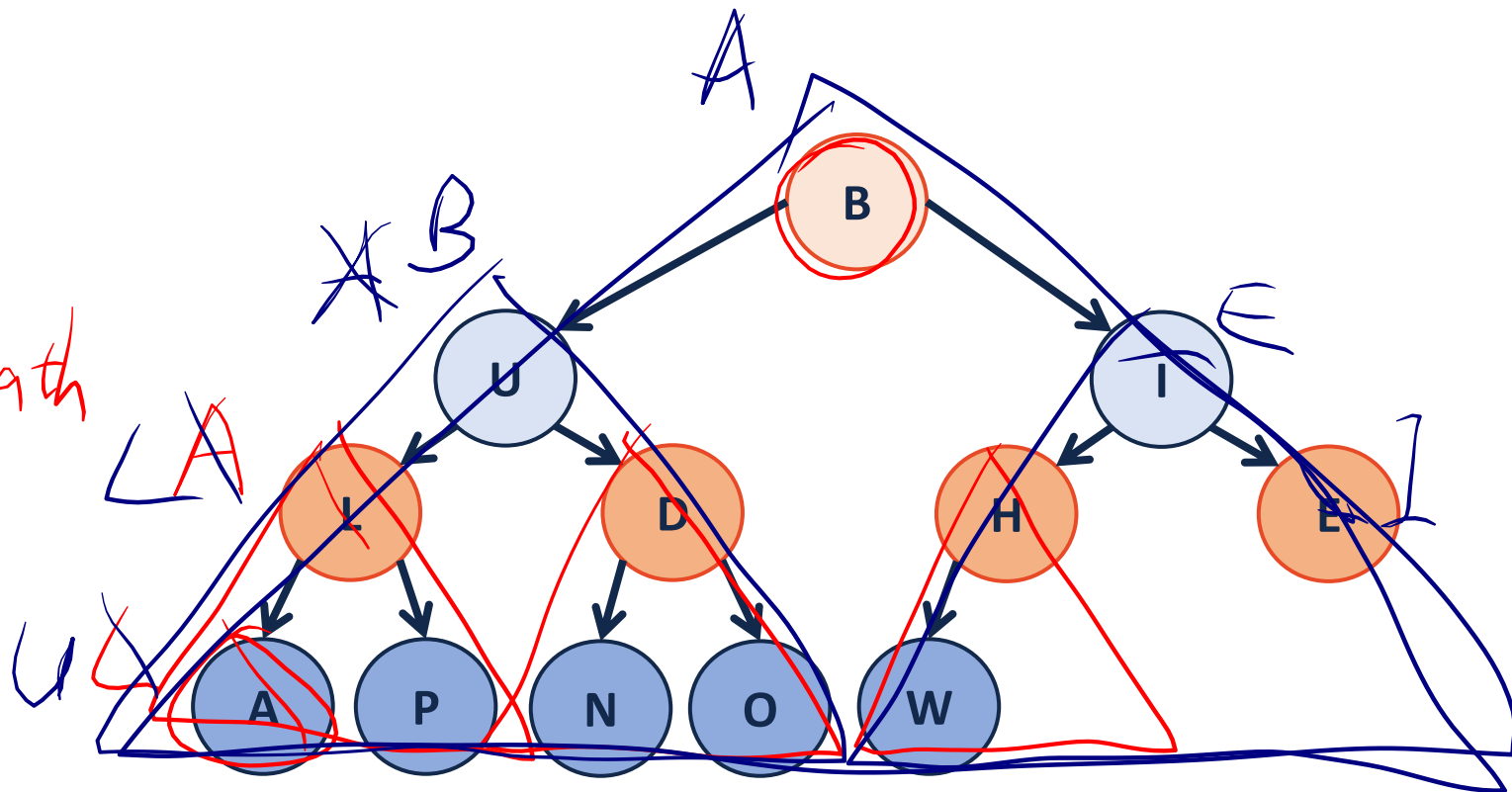
- ① Sort
- Heap requires an ordering on every path
- $O(n \lg n)$



- ② Call insert() n times $\lg n \times n = n \lg n$
↳ HeapifyUp

buildHeap

- ① Sort
- Heap requires an ordering on every path
- $O(n \lg n)$



	B	U	I	L	D	H	E	A	P	N	O	W			
--	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--

- ② Call insert() n times $\lg n \times n = n \lg n$
 ↳ HeapifyUp

buildHeap

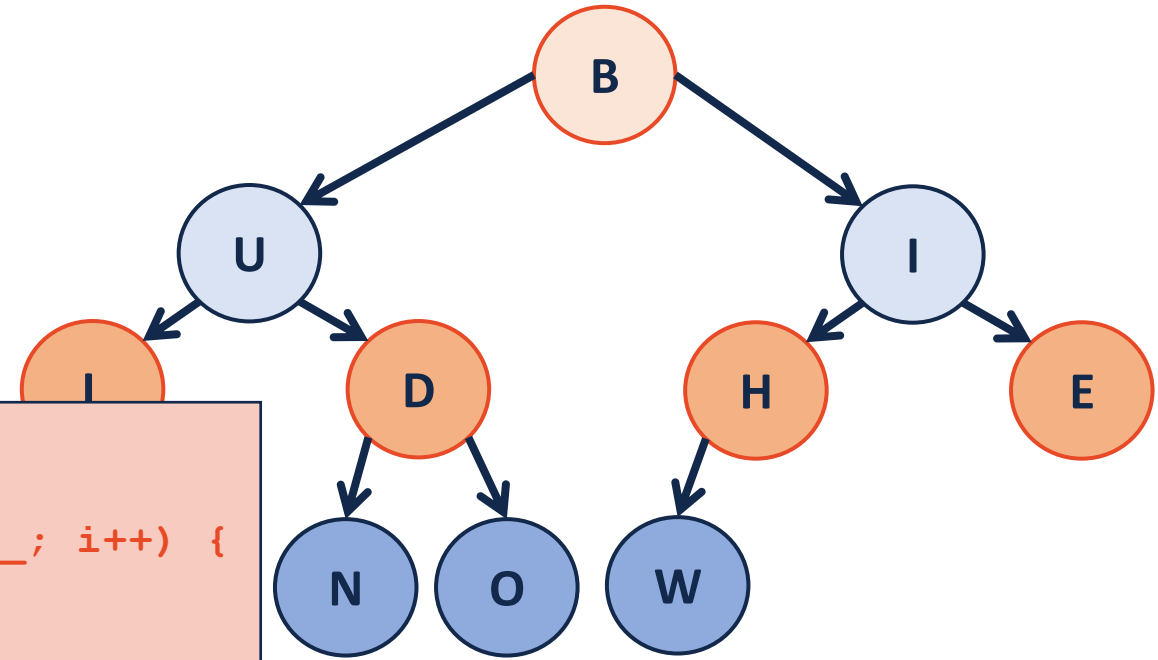
1. Sort the array – it's a heap!

2.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = 0; i <= size_; i++) {
4         heapifyUp(i);
5     }
6 }
```

3.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = parent(size); i > 0; i--) {
4         heapifyDown(i);
5     }
6 }
```



buildHeap

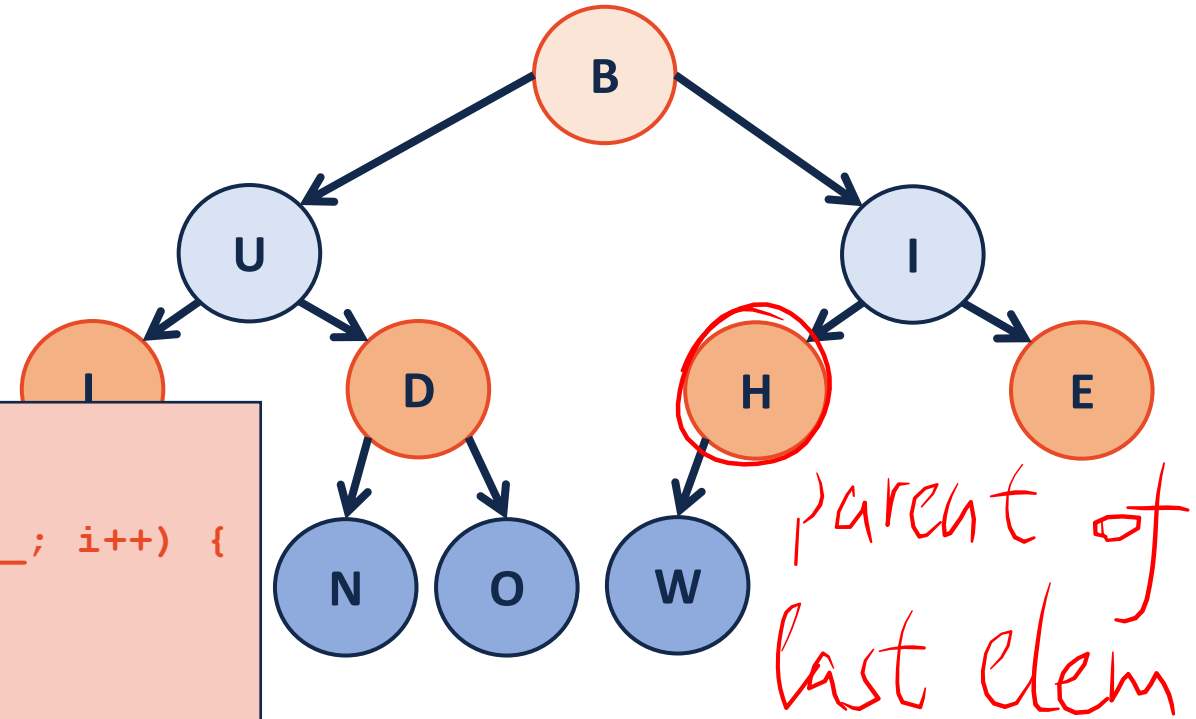
1. Sort the array – it's a heap!

2.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = 0; i <= size_; i++) {
4         heapifyUp(i);
5     }
6 }
```

3.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = parent(size); i > 0; i--) {
4         heapifyDown(i);
5     }
6 }
```



buildHeap

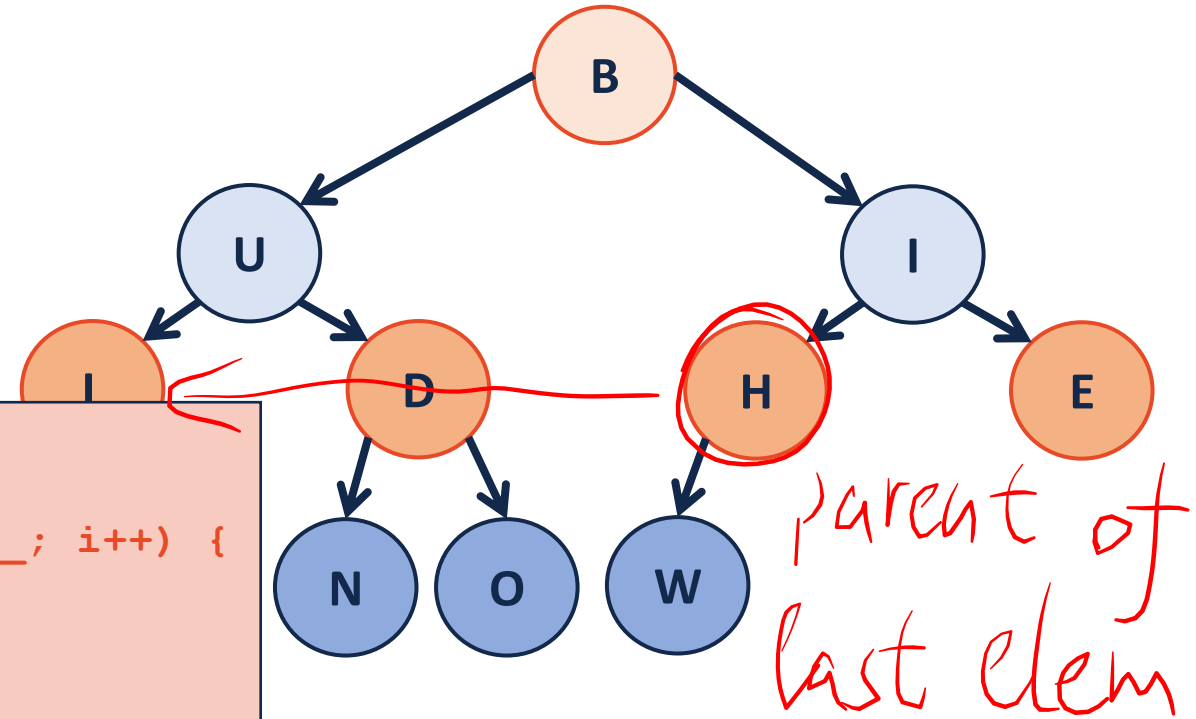
1. Sort the array – it's a heap!

2.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = 0; i <= size_; i++) {
4         heapifyUp(i);
5     }
6 }
```

3.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = parent(size); i > 0; i--) {
4         heapifyDown(i);
5     }
6 }
```



buildHeap

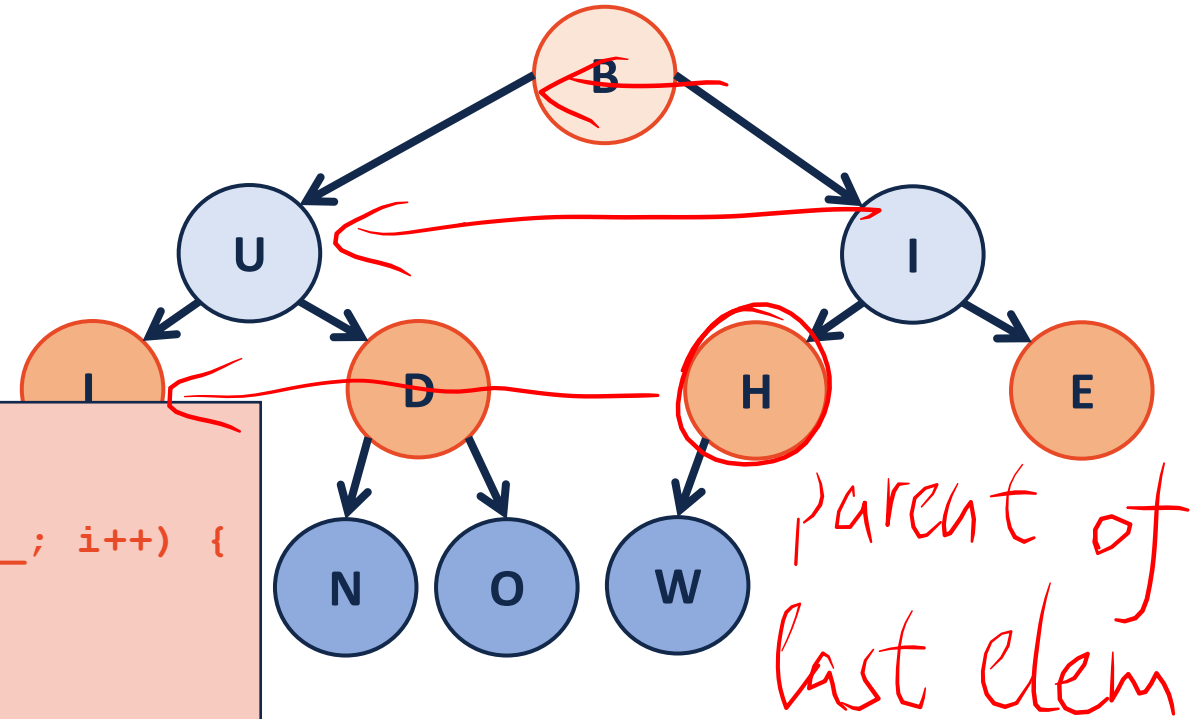
1. Sort the array – it's a heap!

2.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = 0; i <= size_; i++) {
4         heapifyUp(i);
5     }
6 }
```

3.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = parent(size); i > 0; i--) {
4         heapifyDown(i);
5     }
6 }
```



buildHeap

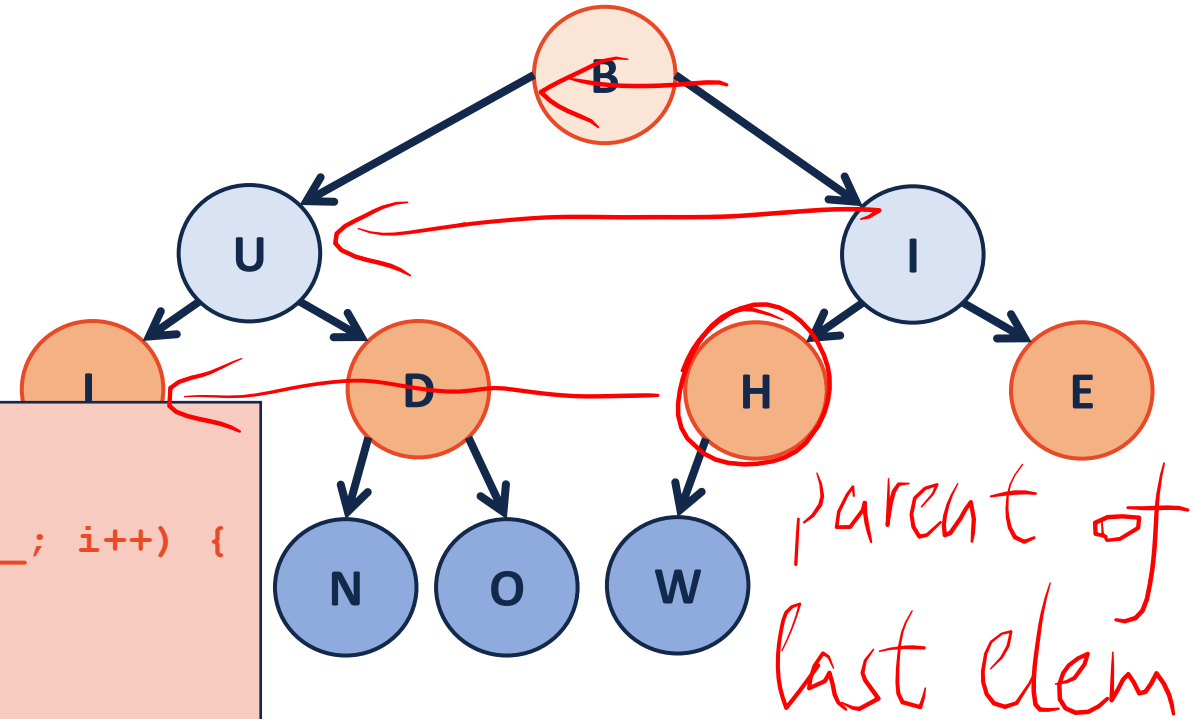
1. Sort the array – it's a heap!

2.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = 0; i <= size_; i++) {
4         heapifyUp(i);
5     }
6 }
```

3.

```
1 template <class T>
2 void Heap<T>::buildHeap() {
3     for (unsigned i = parent(size); i > 0; i--) {
4         heapifyDown(i);
5     }
6 }
```



$$\left(\frac{n}{2}\right) \times \lg(n) = \frac{n \lg n}{2}$$



Proving buildHeap Running Time

Theorem: The running time of buildHeap on array of size n is:
_____.

Strategy:

- We know that constant work is done based on the distance a node is away from the root (eg: it's height).
- Therefore, the running time is proportional to the sum of the heights of the nodes involved in heapifyDown operation.
- We will work towards creating a proof around the sum of the heights of all such nodes.

Proving buildHeap Running Time

Theorem: The running time of buildHeap on array of size n is:

$O(n \log n)$

Strategy:

- We know that constant work is done based on the distance a node is away from the root (eg: it's height).
- Therefore, the running time is proportional to the sum of the heights of the nodes involved in heapifyDown operation.
- We will work towards creating a proof around the sum of the heights of all such nodes.

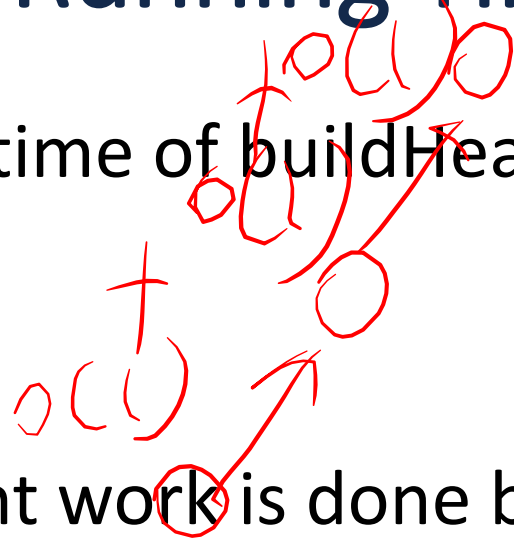
Proving buildHeap Running Time

Theorem: The running time of buildHeap on array of size n is:

$O(n)$

Strategy:

- We know that constant work is done based on the distance a node is away from the root (eg: it's height).
- Therefore, the running time is proportional to the sum of the heights of the nodes involved in heapifyDown operation.
- We will work towards creating a proof around the sum of the heights of all such nodes.



Proving buildHeap Running Time

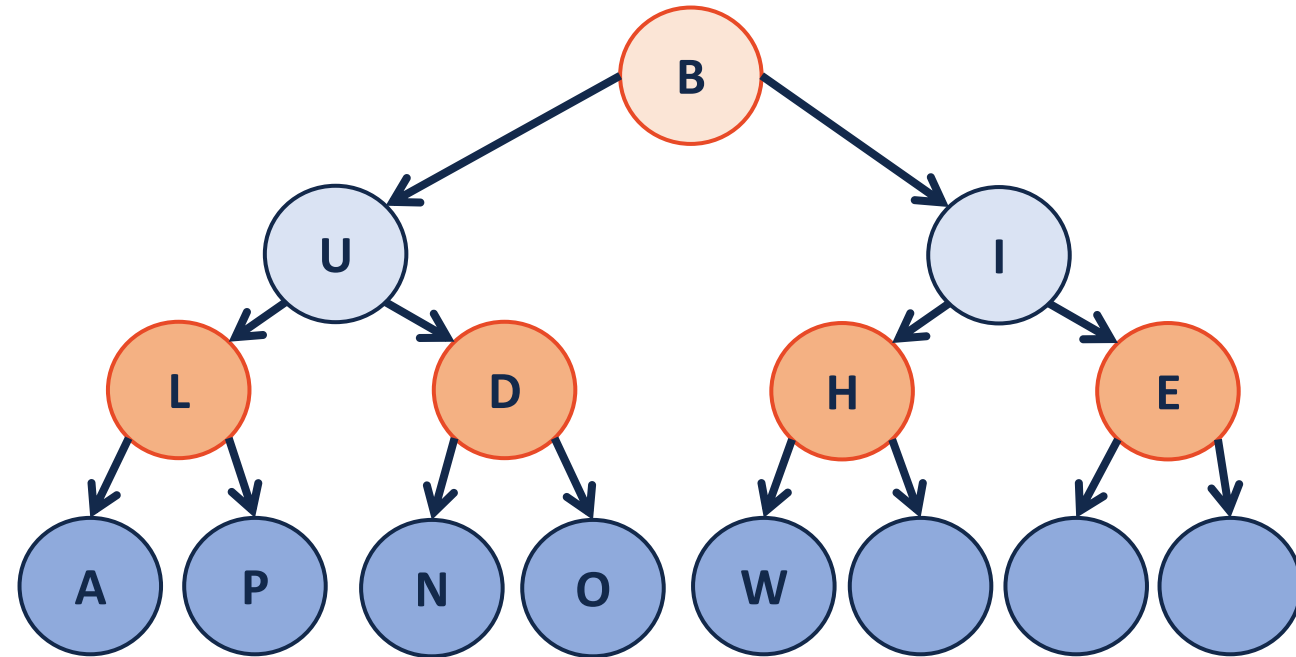
$S(h)$: Sum of the heights of all nodes in a perfect tree of height h .

$S(0) =$

$S(1) =$

$S(2) =$

$S(h) =$



Proving buildHeap Running Time

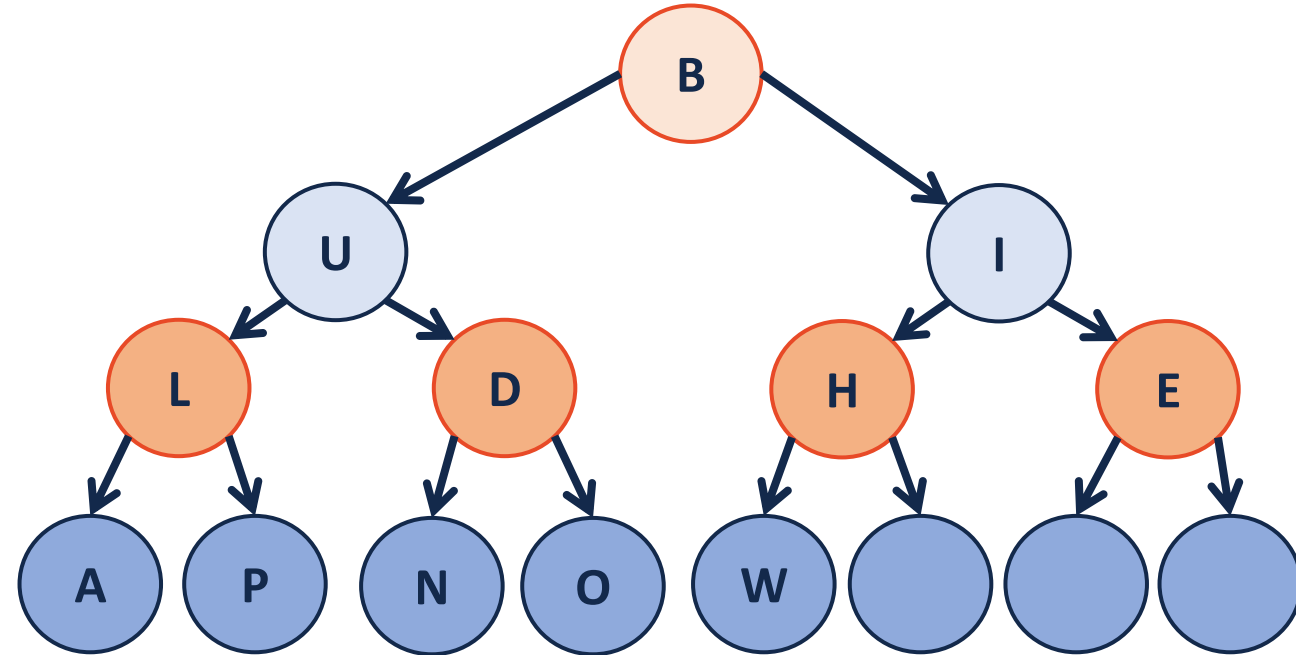
$S(h)$: Sum of the heights of all nodes in a perfect tree of height h .

$$S(0) = 0$$

$$S(1) = 0 + 1 = 1$$

$$S(2) =$$

$$S(h) =$$



Proving buildHeap Running Time

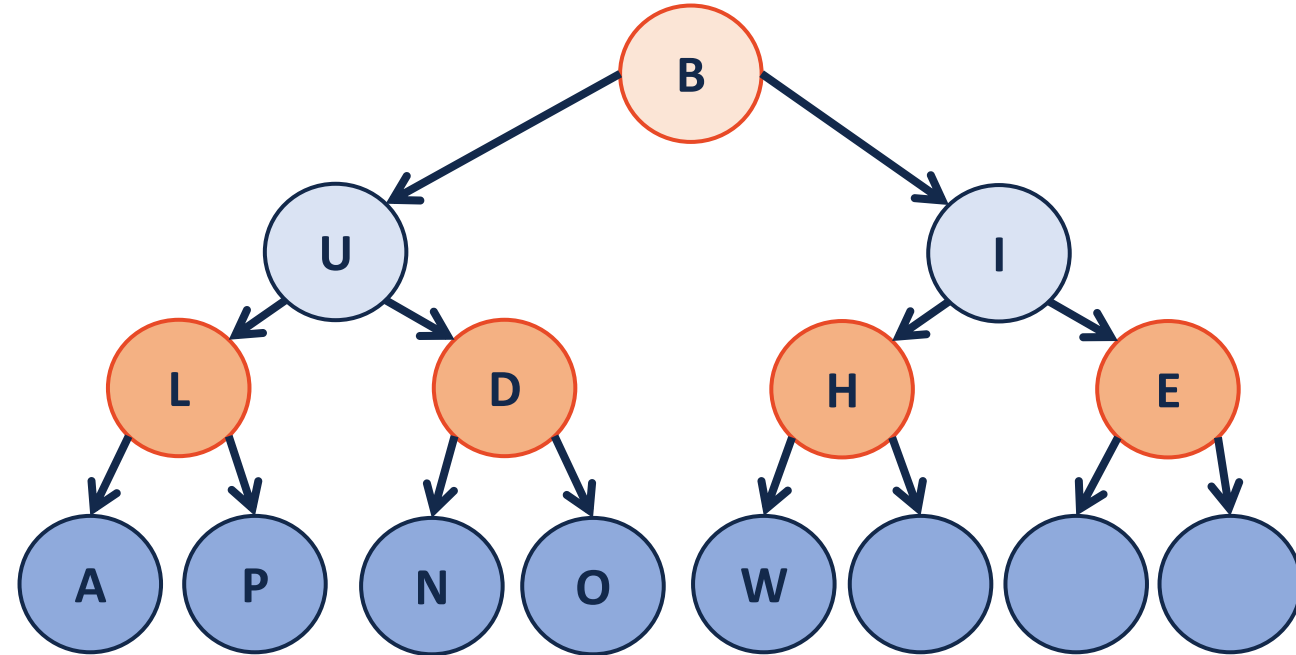
$S(h)$: Sum of the heights of all nodes in a perfect tree of height h .

$$S(0) = 0$$

$$S(1) = 0 + 1 = 1$$

$$S(2) = 1 + 1 + 2 = 4$$

$$S(h) =$$



Proving buildHeap Running Time

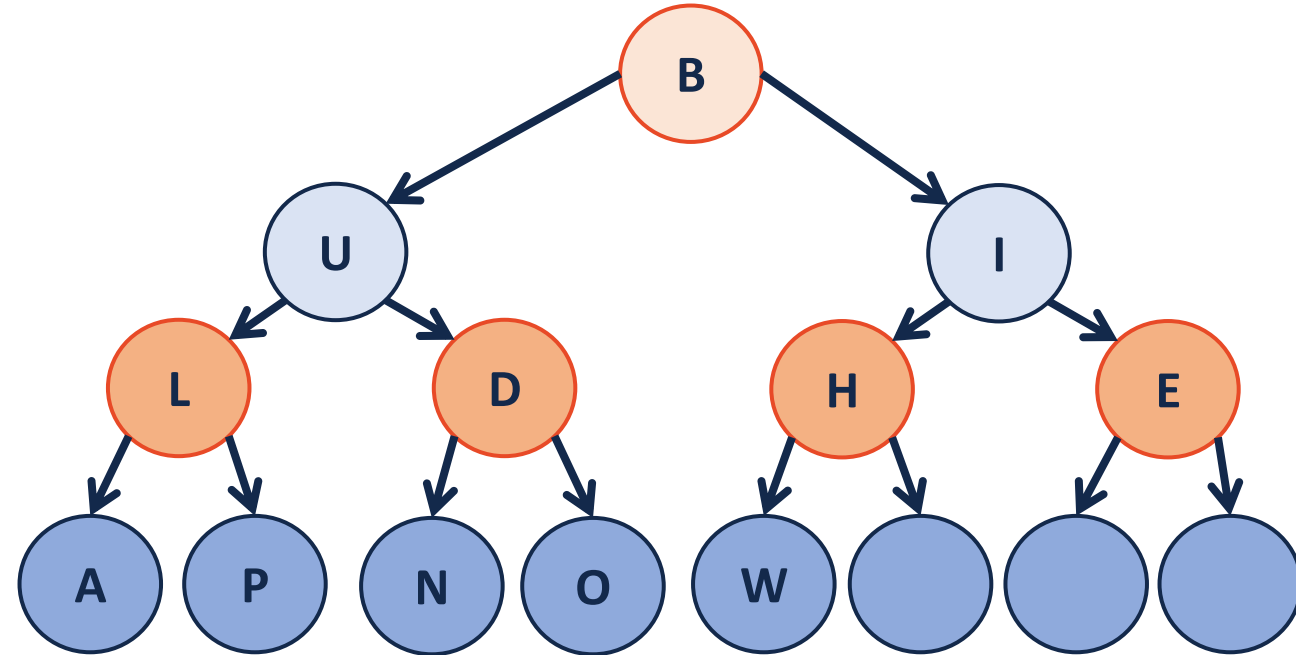
$S(h)$: Sum of the heights of all nodes in a perfect tree of height h .

$$S(0) = 0$$

$$S(1) = 0 + 1 = 1$$

$$S(2) = 1 + 1 + 2 = 4$$

$$S(h) = S(h-1) + S(h-1) + h = 2S(h-1) + h$$



Proving buildHeap Running Time

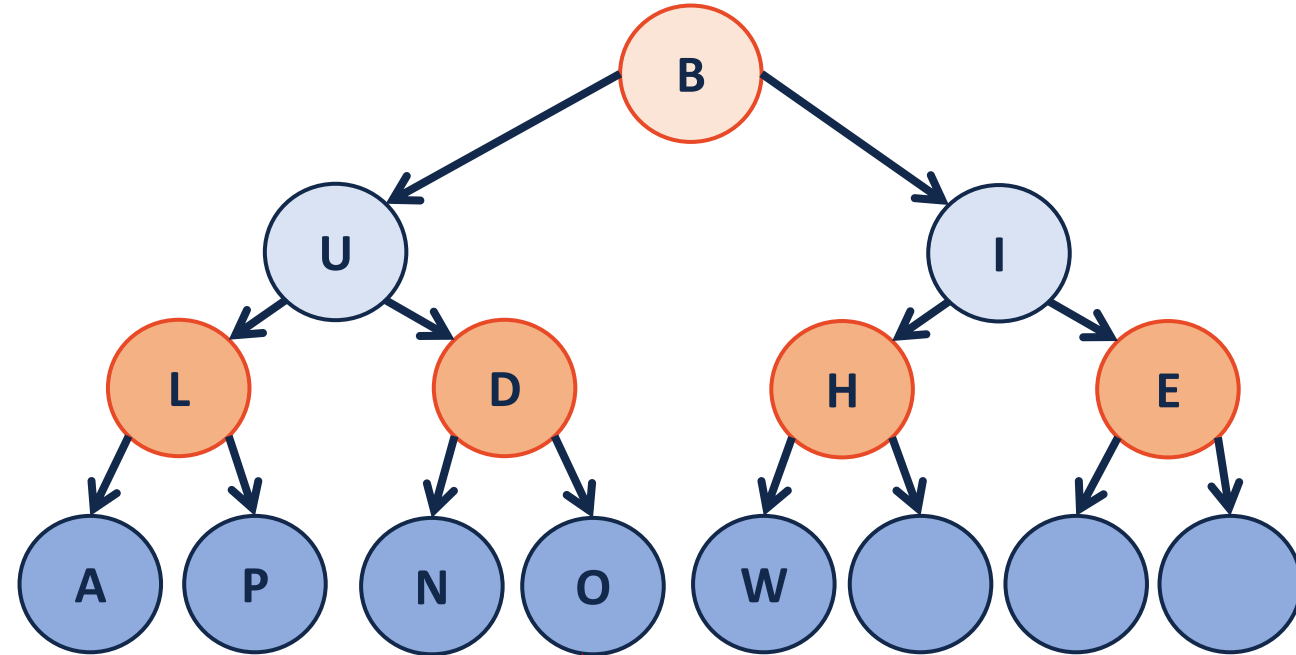
$S(h)$: Sum of the heights of all nodes in a perfect tree of height h .

$$S(0) = 0$$

$$S(1) = 0 + 1 = 1$$

$$S(2) = 1 + 1 + 2 = 4$$

$$S(h) = S(h-1) + S(h-1) + h = 2S(h-1) + h$$



Proving buildHeap Running Time

Proof the recursion:

$$S(h) = 2S(h-1) + h = 2^{(h+1)} - 2 - h$$

Base case:

General case:

Proving buildHeap Running Time

Proof the recursion:

$$S(h) = 2S(h-1) + h = 2^{(h+1)} - 2 - h$$

Base case:

$$S(0) = 2^1 - 2 - 0 = 0$$

General case:

Proving buildHeap Running Time

Proof the recursion:

$$S(h) = 2S(h-1) + h = 2^{(h+1)} - 2 - h$$

Base case:

$$S(0) = 2^1 - 2 - 0 = 0$$

General case:

Inductive hypothesis : $\forall j < h, S(j) = 2^{j+1} - 2 - j$

Proving buildHeap Running Time

Proof the recursion:

$$S(h) = 2S(h-1) + h = 2^{(h+1)} - 2 - h$$

Base case:

$$S(0) = 2^1 - 2 - 0 = 0$$

General case:

Inductive hypothesis : $\forall j < h, S(j) = 2^{j+1} - 2 - j$
 $j = h-1 \Rightarrow$

Proving buildHeap Running Time

Proof the recursion:

$$S(h) = 2S(h-1) + h = 2^{(h+1)} - 2 - h$$

Base case:

$$S(0) = 2^1 - 2 - 0 = 0$$

General case:

Inductive hypothesis : $\forall j < h, S(j) = 2^{j+1} - 2 - j$

$$j = h-1 \Rightarrow S(h) = 2S(h-1) + h$$

Proving buildHeap Running Time

Proof the recursion:

$$S(h) = 2S(h-1) + h = 2^{(h+1)} - 2 - h$$

Base case:

$$S(0) = 2^1 - 2 - 0 = 0$$

General case:

Inductive hypothesis : $\forall j < h, S(j) = 2^{j+1} - 2 - j$

$$\begin{aligned} j = h-1 \Rightarrow S(h) &= 2S(h-1) + h \\ &= 2[2^{h-1+1} - 2 - (h-1)] + h \end{aligned}$$

Proving buildHeap Running Time

Proof the recursion:

$$S(h) = 2S(h-1) + h = 2^{(h+1)} - 2 - h$$

Base case:

$$S(0) = 2^1 - 2 - 0 = 0$$

General case:

Inductive hypothesis : $\forall j < h, S(j) = 2^{j+1} - 2 - j$

$$\begin{aligned} j = h-1 \Rightarrow S(h) &= 2S(h-1) + h \\ &= 2[2^{h-1+1} - 2 - (h-1)] + h = 2^{h+1} - 2 - h \end{aligned}$$

Proving buildHeap Running Time

No one cares about things in terms of height:

$$S(h) = 2^{(h+1)} - 2 - h$$

We know that the nodes in a perfect tree of height **h** is:

$$n =$$

Proving buildHeap Running Time

No one cares about things in terms of height:

$$S(h) = 2^{(h+1)} - 2 - h$$

We know that the nodes in a perfect tree of height **h** is:

n =

$$2^{h+1} - 1$$

Proving buildHeap Running Time

No one cares about things in terms of height:

$$S(h) = 2^{(h+1)} - 2 - h$$

We know that the nodes in a perfect tree of height **h** is:

$$n = 2^{h+1} - 1 \quad \bigcirc \quad 2^{h+1} - 2 - h$$

Proving buildHeap Running Time

No one cares about things in terms of height:

$$S(h) = 2^{(h+1)} - 2 - h$$

We know that the nodes in a perfect tree of height **h** is:

$$n = 2^{h+1} - 1 > 2^{h+1} - 2 - h$$

Proving buildHeap Running Time

No one cares about things in terms of height:

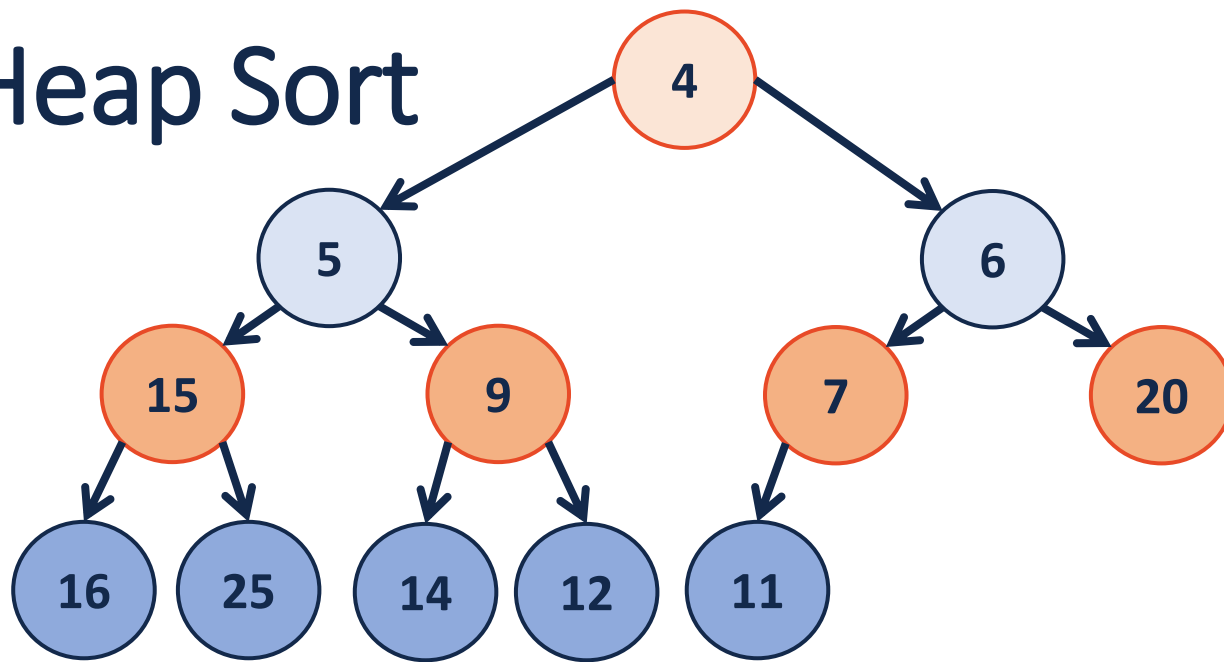
$$S(h) = 2^{(h+1)} - 2 - h$$

We know that the nodes in a perfect tree of height **h** is:

$$n = 2^{h+1} - 1 \quad \textcircled{>} \quad 2^{h+1} - 2 - h$$

$\therefore O(N)$

Heap Sort



1.

2.

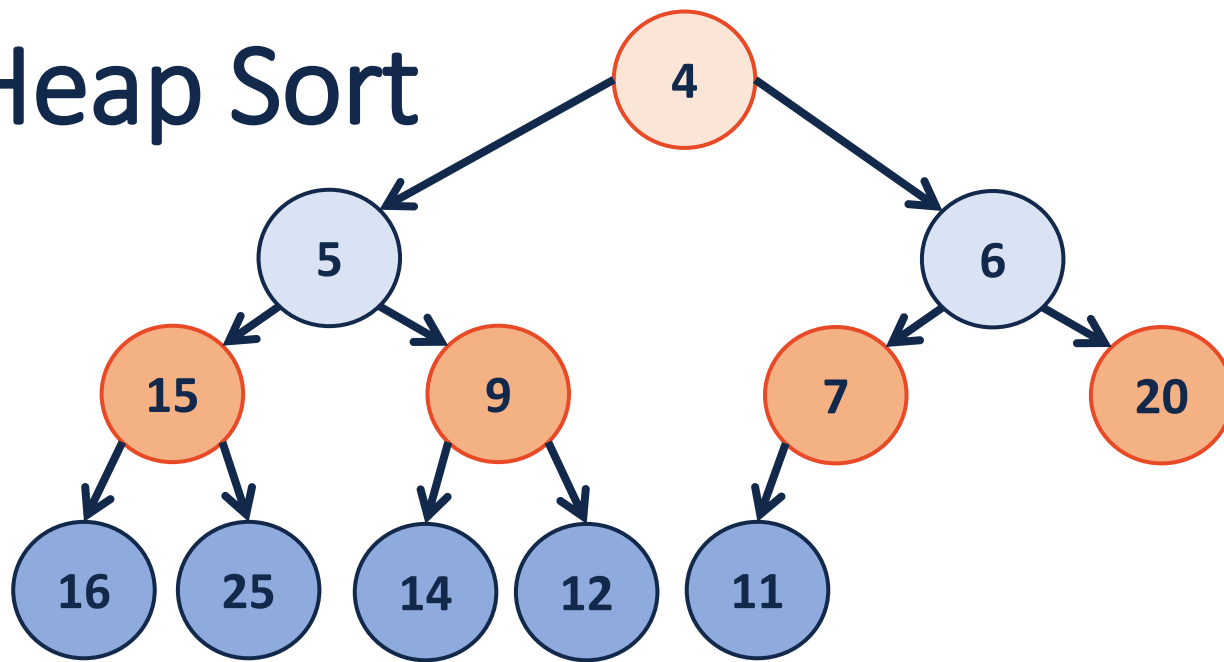
3.



Running Time?

Why do we care about another sort?

Heap Sort



1. *buildHeap()*

2.

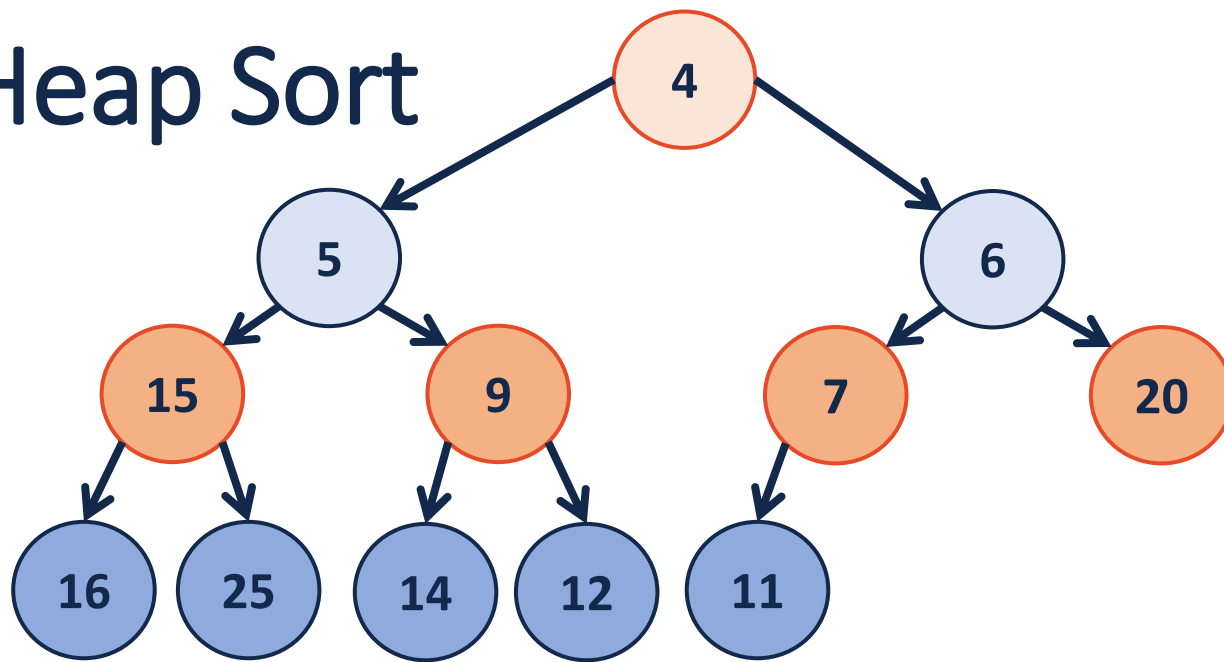
3.



Running Time?

Why do we care about another sort?

Heap Sort



1. *buildHeap()* $O(n)$

2.

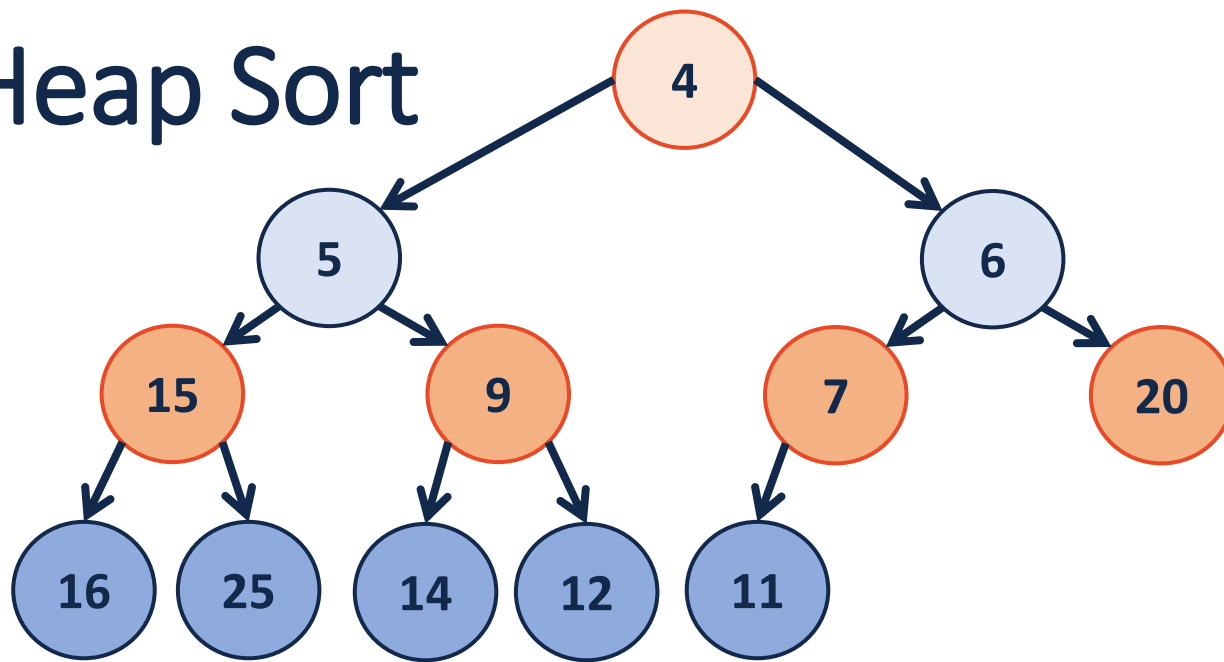
3.



Running Time?

Why do we care about another sort?

Heap Sort



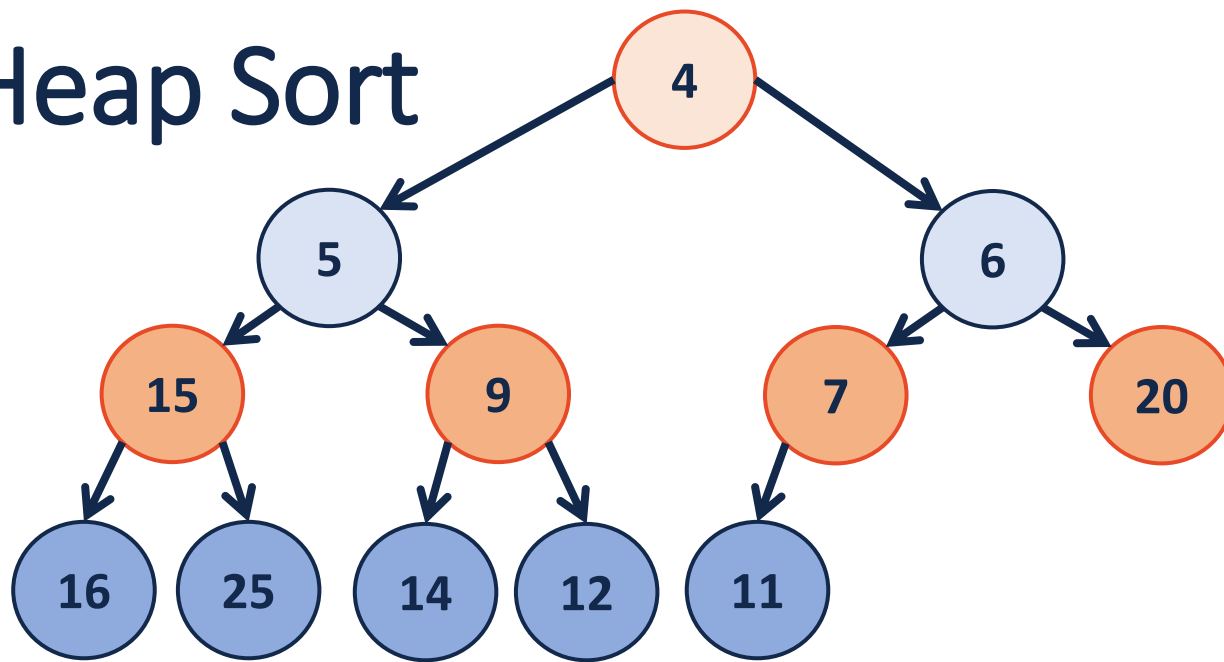
1. *buildHeap()* $O(n)$
2. *removeMin*
- 3.



Running Time?

Why do we care about another sort?

Heap Sort



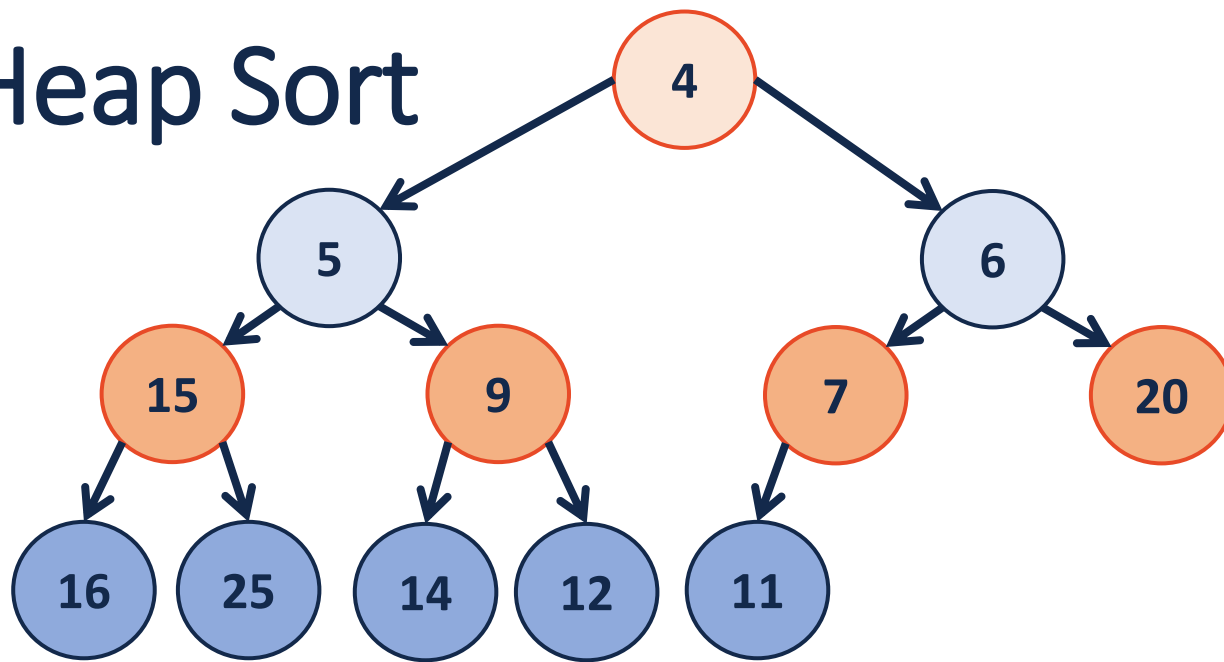
1. *buildHeap()* $O(n)$
2. *removeMin* $n \times O(\lg n)$
- 3.



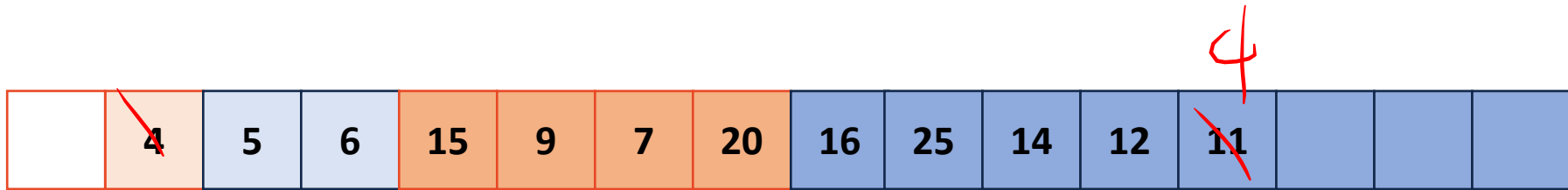
Running Time?

Why do we care about another sort?

Heap Sort



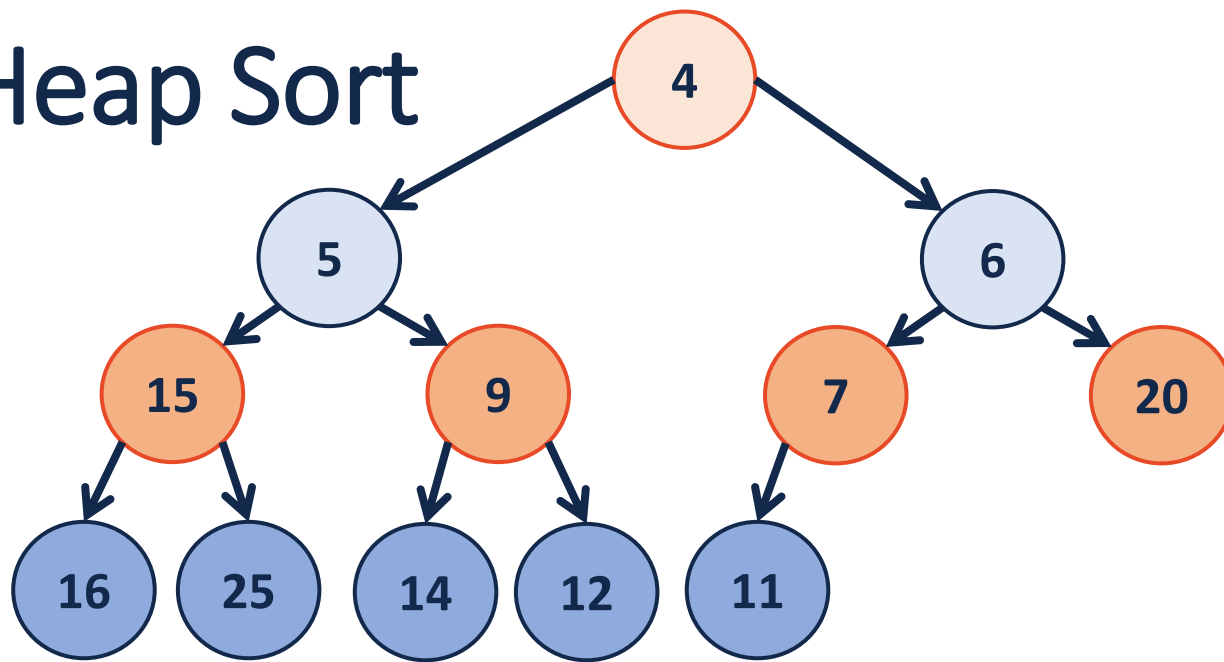
1. *buildHeap()* $O(n)$
2. *removeMin* $n \times O(\lg n)$
- 3.



Running Time?

Why do we care about another sort?

Heap Sort



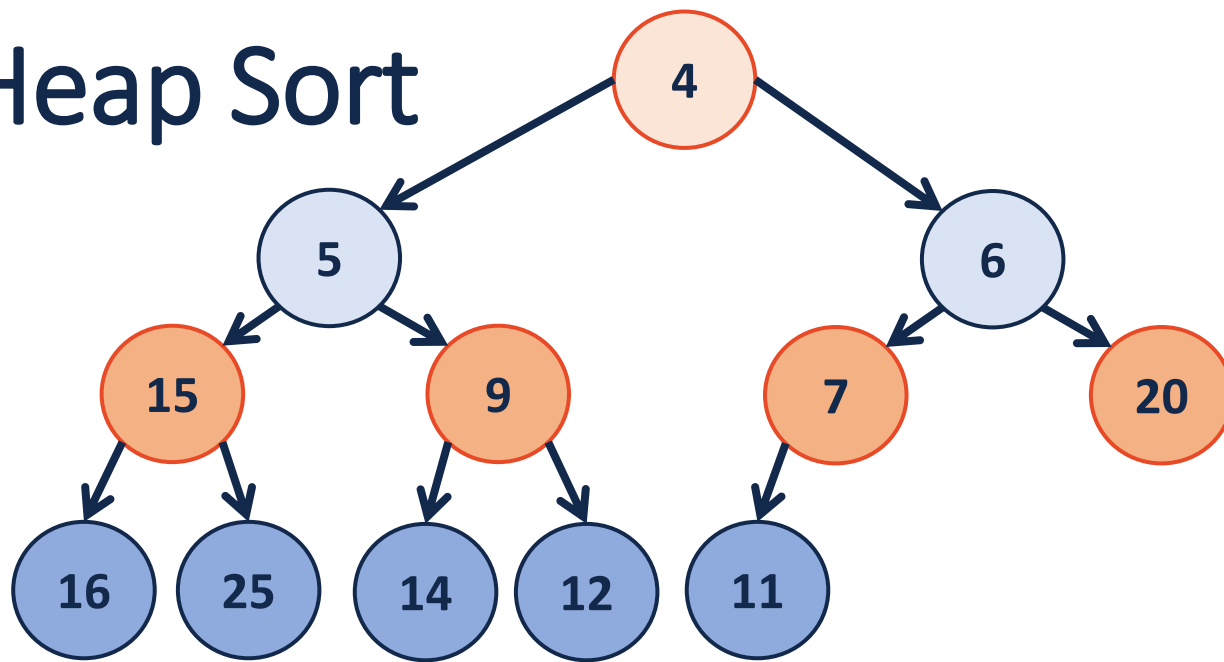
1. *buildHeap()* $O(n)$
2. *removeMin* $n \times O(\lg n)$
- 3.



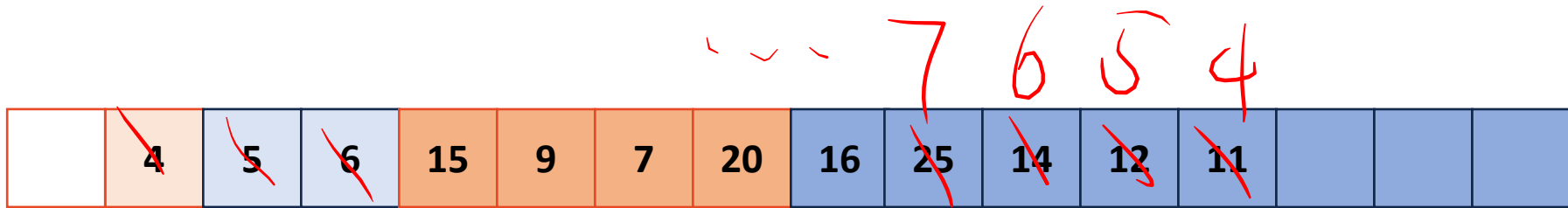
Running Time?

Why do we care about another sort?

Heap Sort



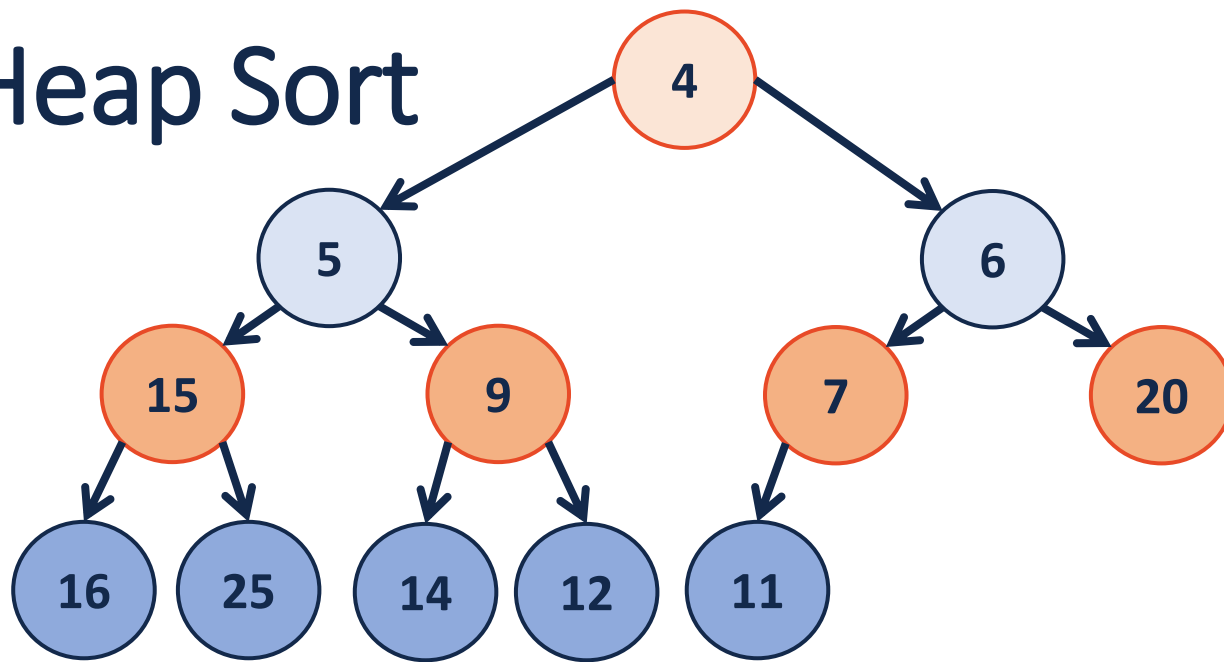
1. *buildHeap()* $O(n)$
2. *removeMin* $n \times O(\lg n)$
- 3.



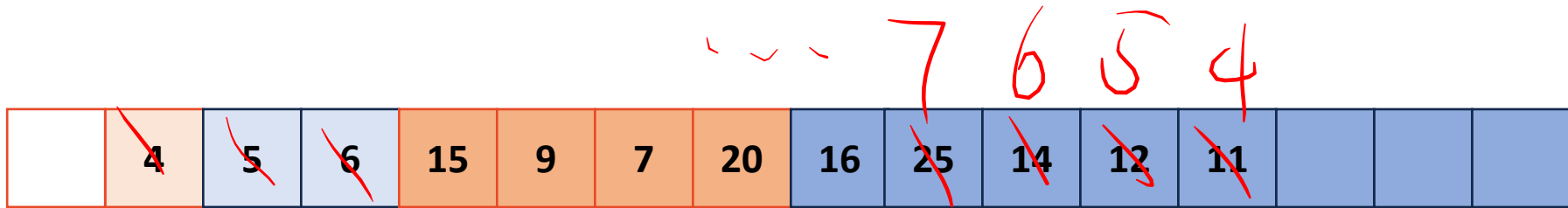
Running Time?

Why do we care about another sort?

Heap Sort



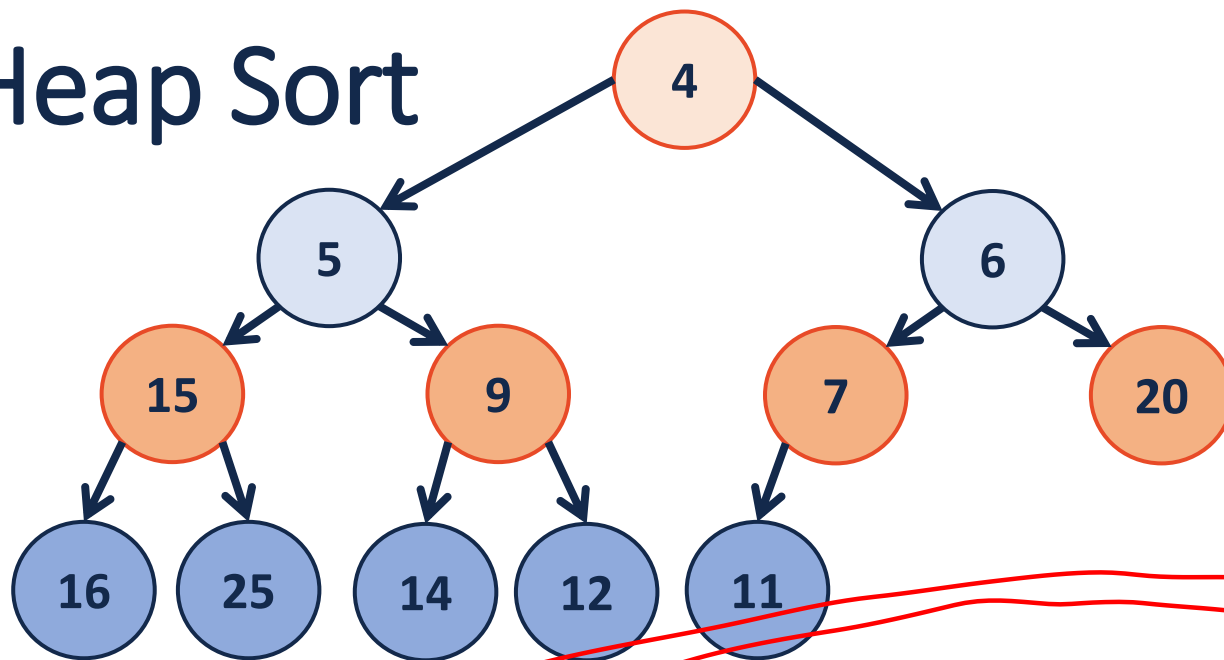
1. *buildHeap()* $O(n)$
2. *removeMin* $n \times O(\lg n)$
3. *Swap elem starting w/ [0]*



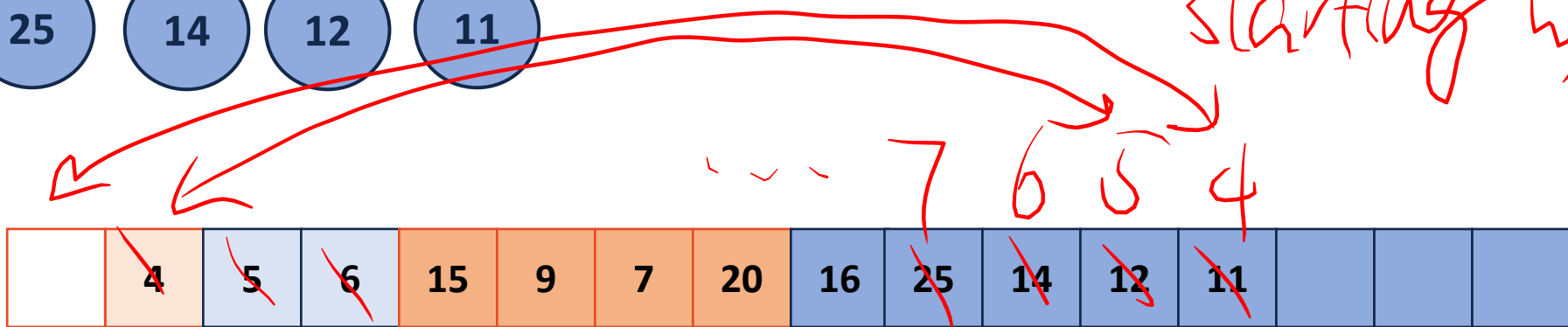
Running Time?

Why do we care about another sort?

Heap Sort



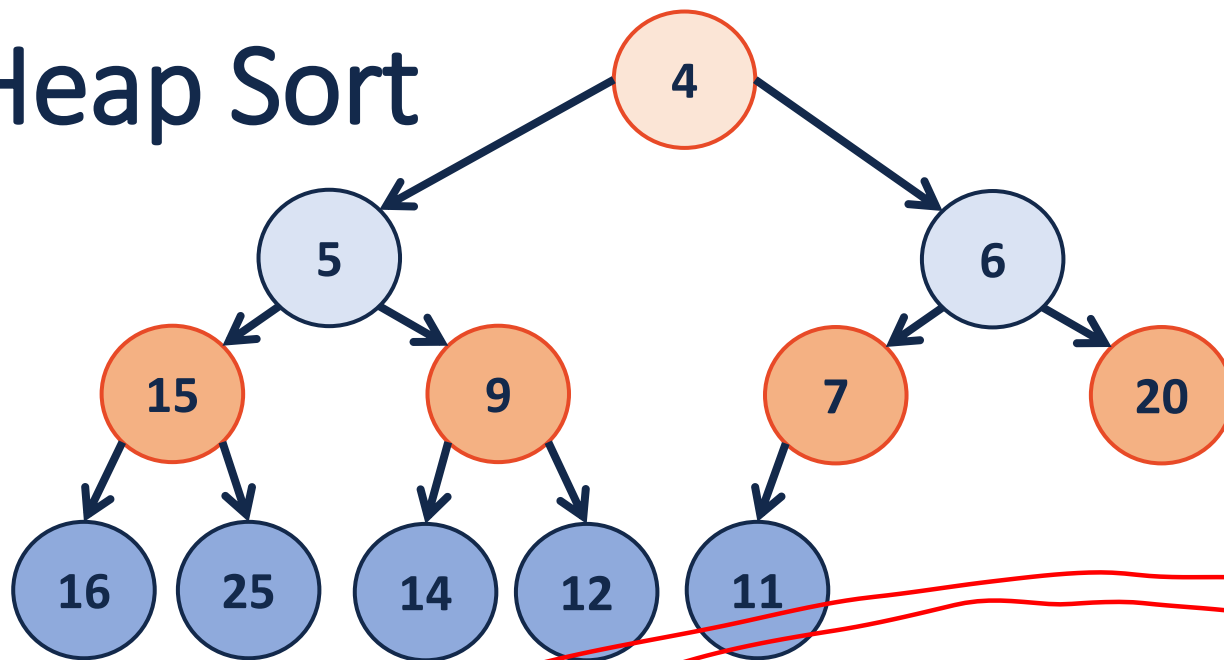
1. *buildHeap()* $O(n)$
2. *removeMin* $n \times O(\lg n)$
3. *Swap elem starting w/ [0]*



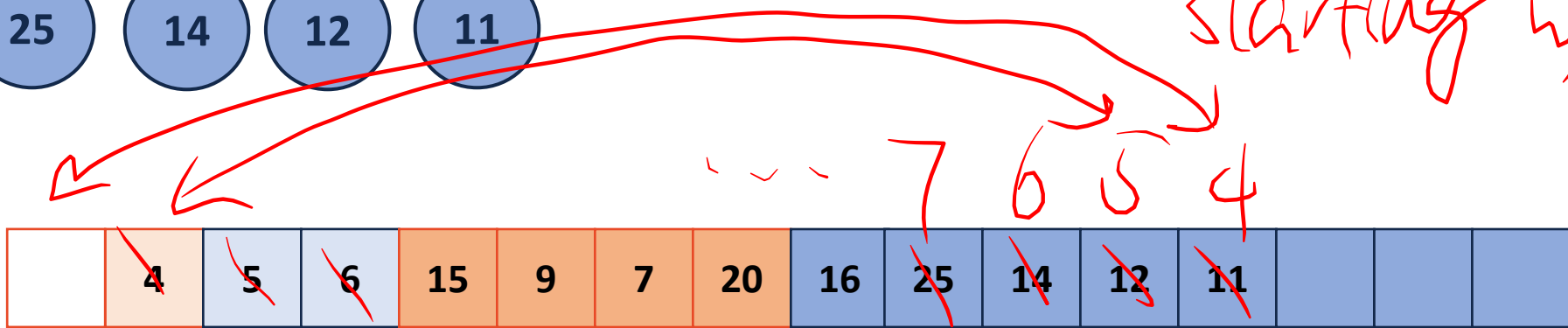
Running Time?

Why do we care about another sort?

Heap Sort



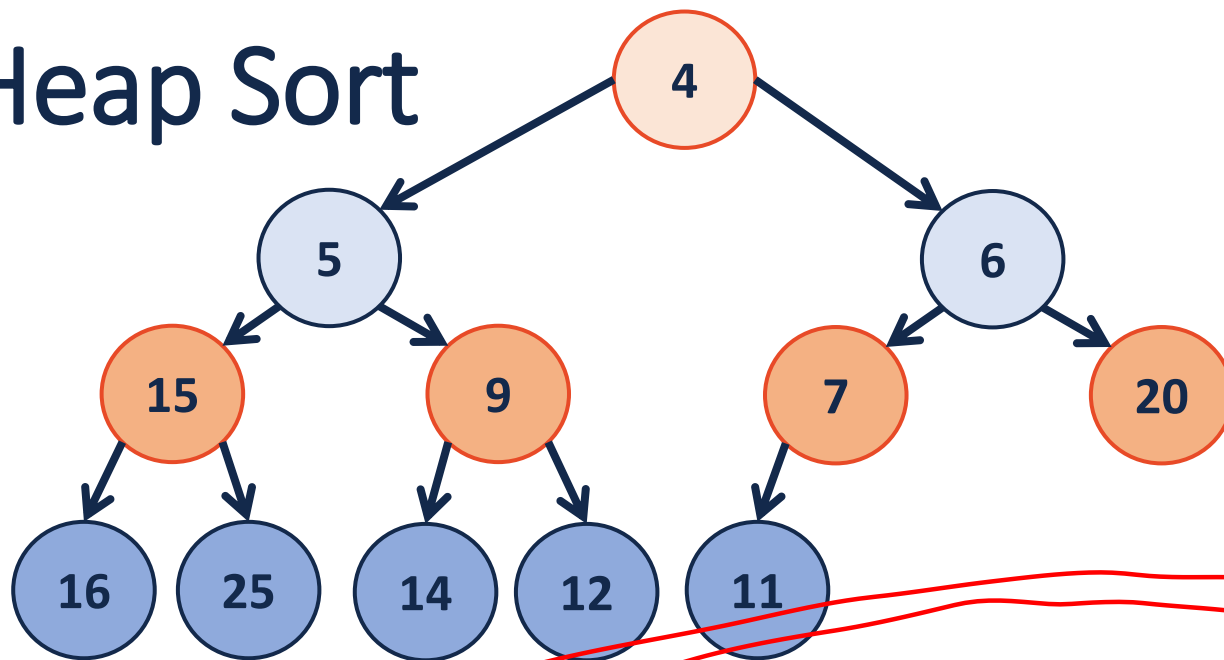
1. *buildHeap()* $O(n)$
2. *removeMin* $n \times O(\lg n)$
3. *Swap elem* $O(n)$ starting w/ [0]



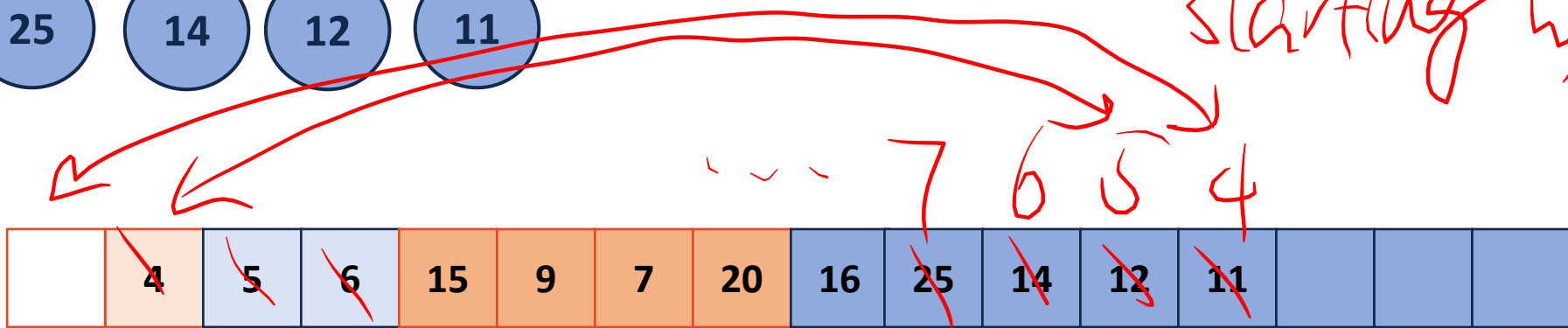
Running Time?

Why do we care about another sort?

Heap Sort



1. *buildHeap()* $O(n)$
2. *removeMin* $n \times O(\lg n)$
3. *Swap elem* $O(n)$ starting w/ [0]

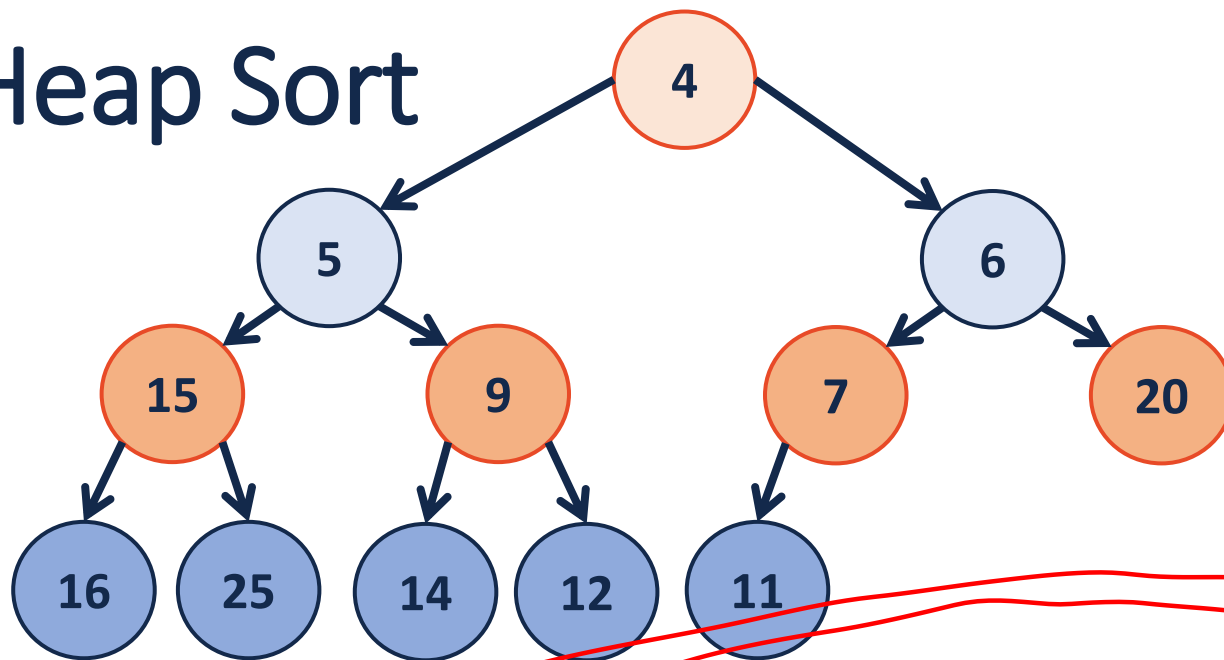


Running Time?

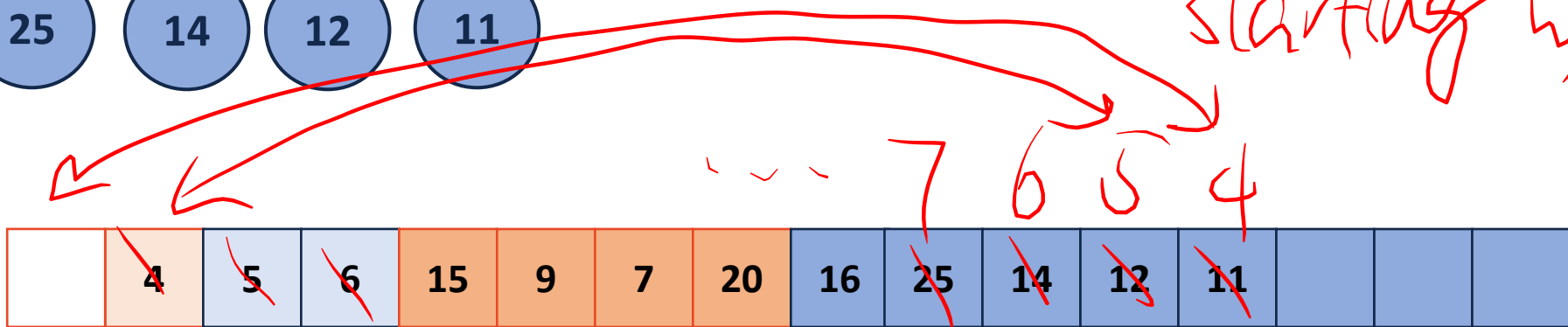
$O(n \lg n)$

Why do we care about another sort?

Heap Sort



1. *buildHeap()* $O(n)$
2. *removeMin* $n \times O(\lg n)$
3. *Swap elem* $O(n)$ starting w/ [0]



Running Time?

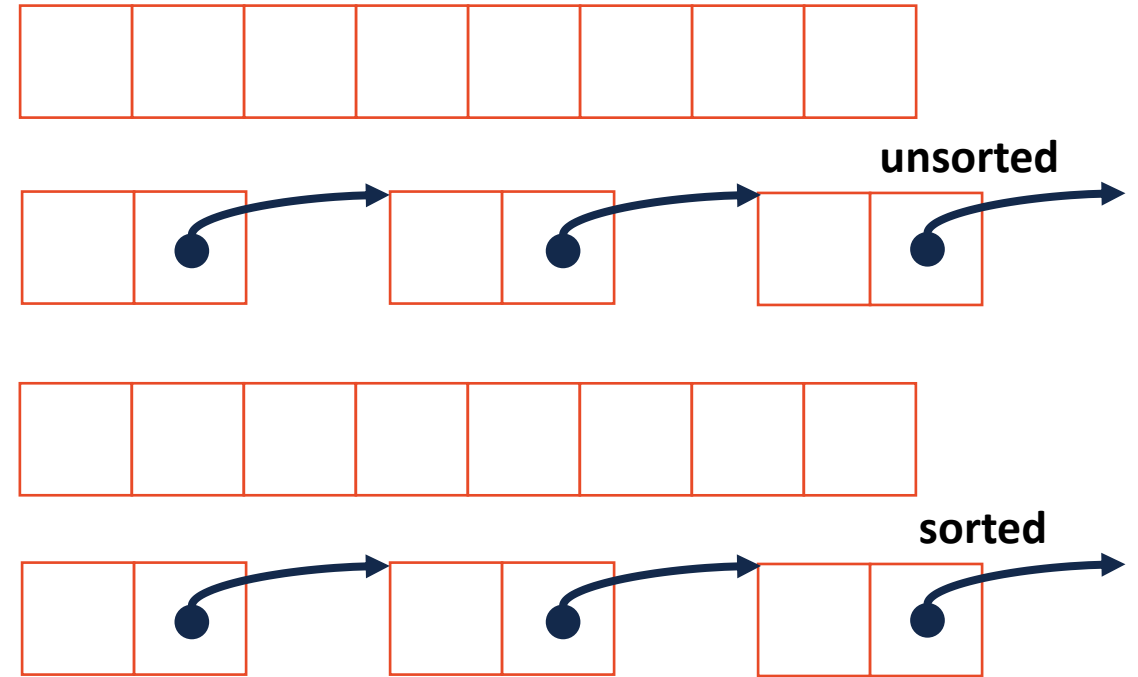
$O(n \lg n)$

Why do we care about another sort?

In-place sort!

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	
$O(1)$	$O(n)$	
$O(n)$	$O(1)$	
$O(n)$	$O(1)$	

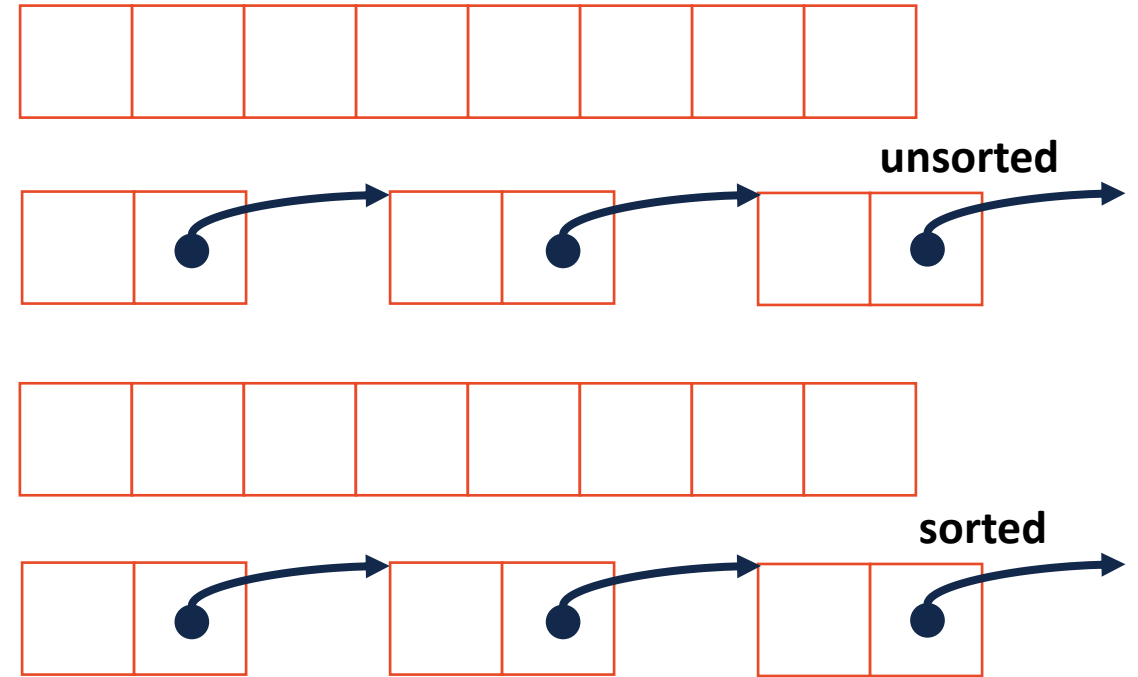


AVL Tree

Heap

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	
$O(n)$	$O(1)$	

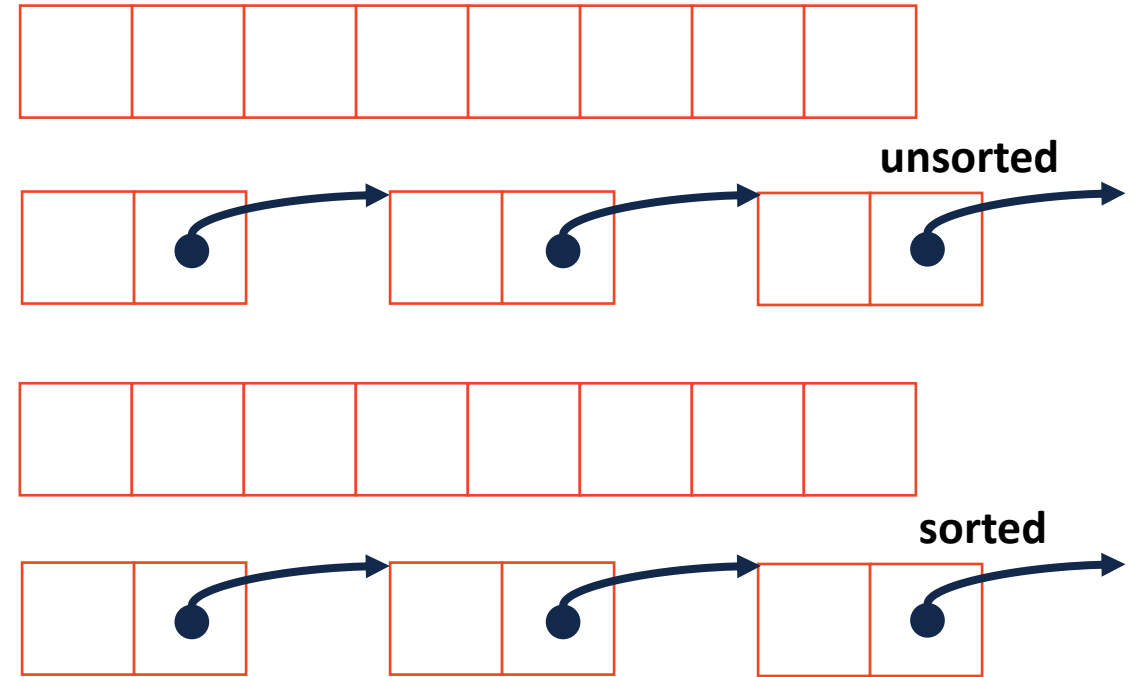


AVL Tree

Heap

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	$O(n \lg n)$
$O(n)$	$O(1)$	

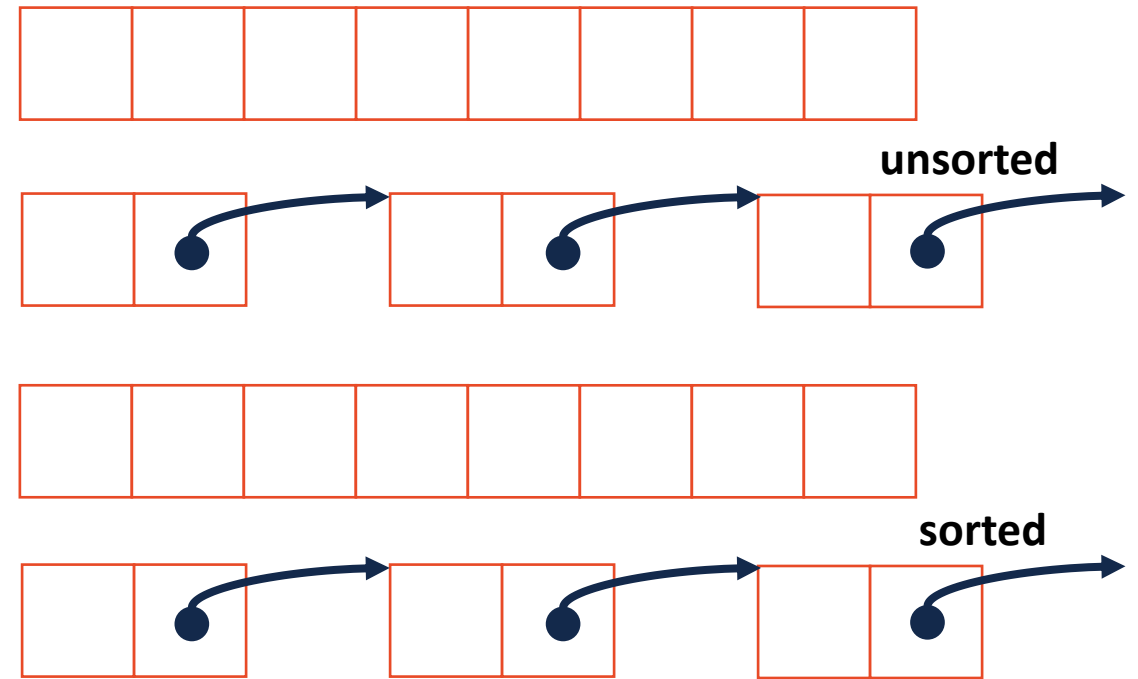


AVL Tree

Heap

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	$O(\lg(n))$
$O(n)$	$O(1)$	$O(\lg(n))$



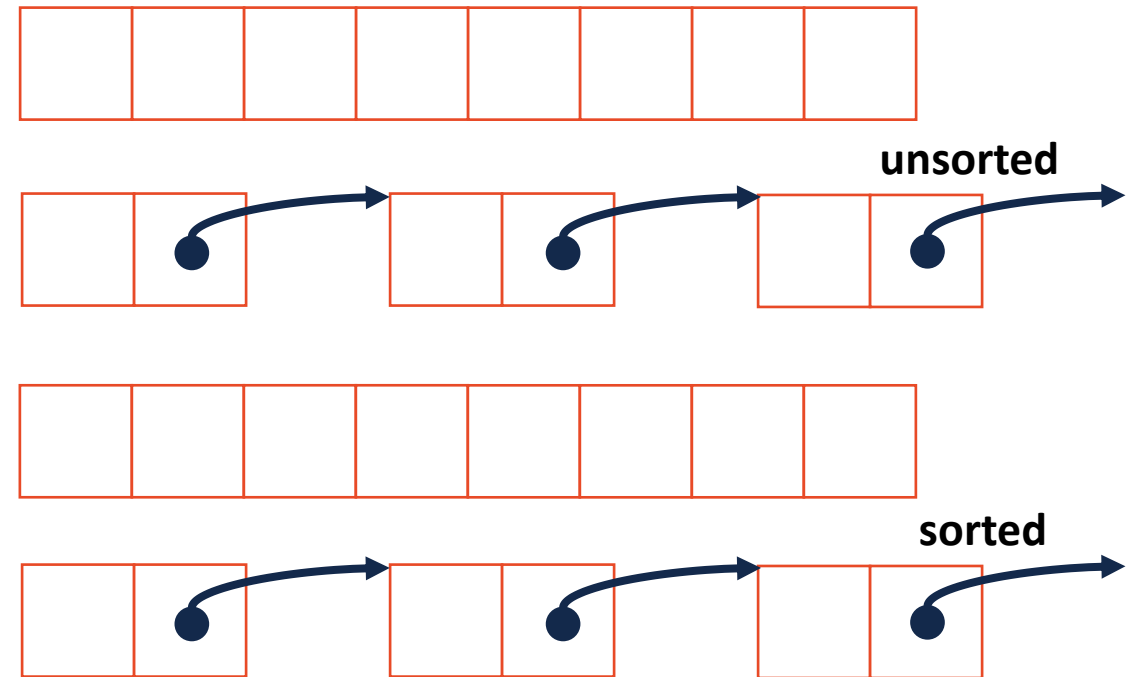
AVL Tree

Heap

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	$O(\lg(n))$
$O(n)$	$O(1)$	$O(\lg(n))$

$O(\lg(n))$
 $O(\lg(n))$

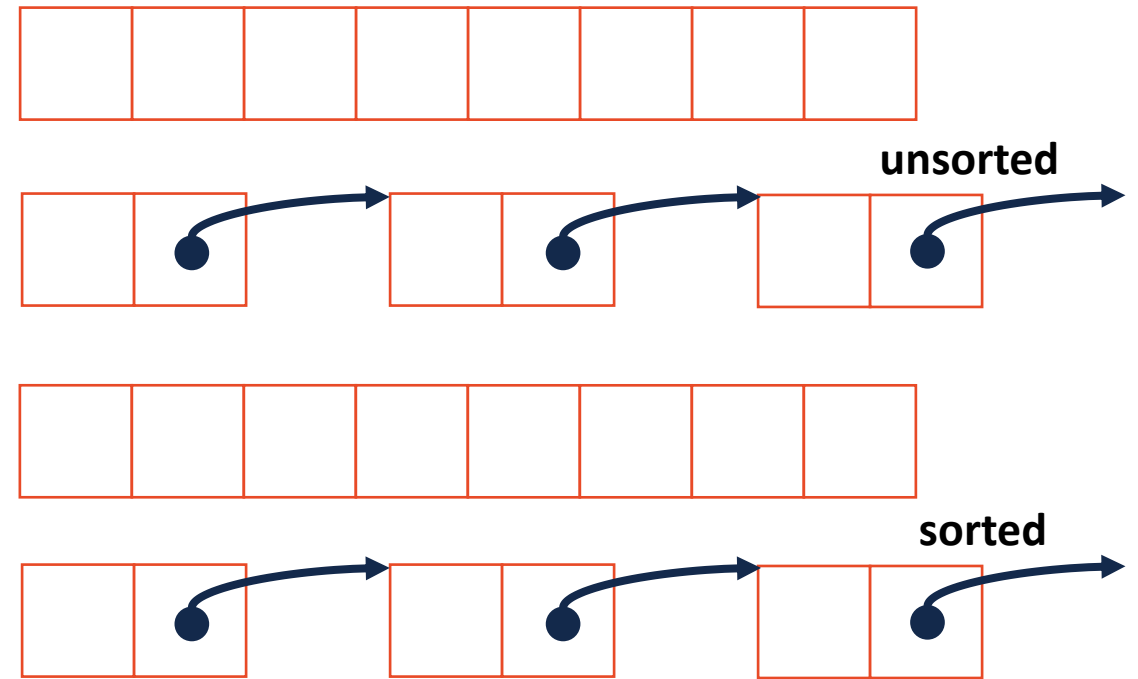


AVL Tree

Heap

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(\lg n)$		

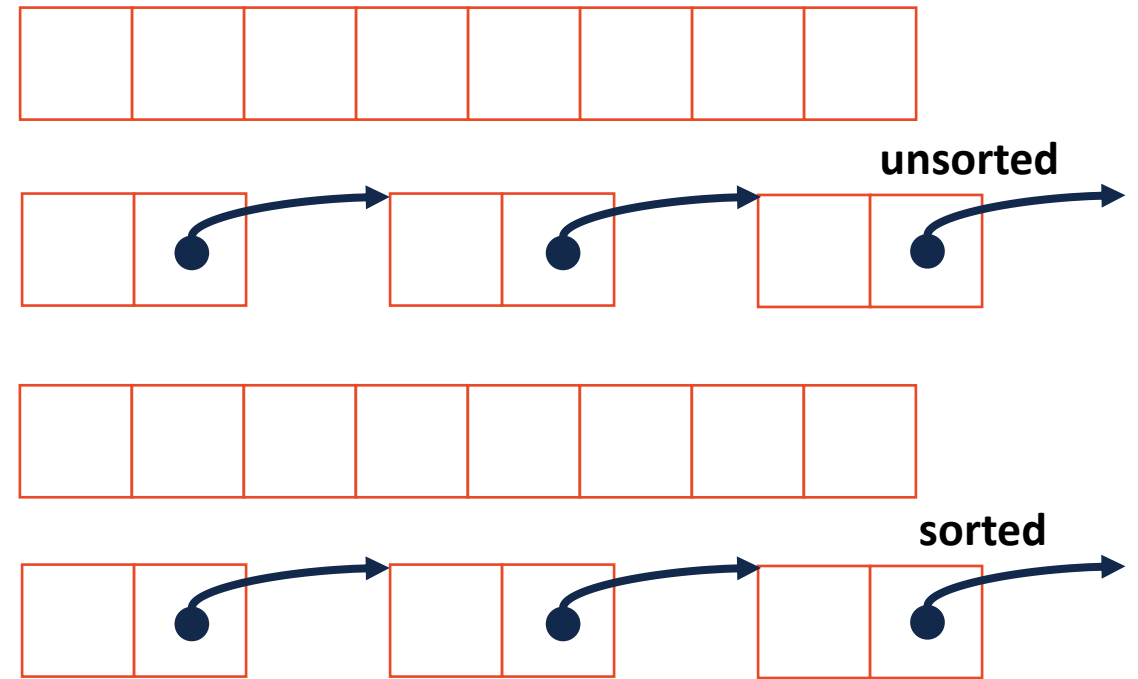


AVL Tree

Heap

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	$O(\lg(n))$
$O(n)$	$O(1)$	$O(\lg(n))$
$O(\lg(n))$	$O(\lg(n))$	

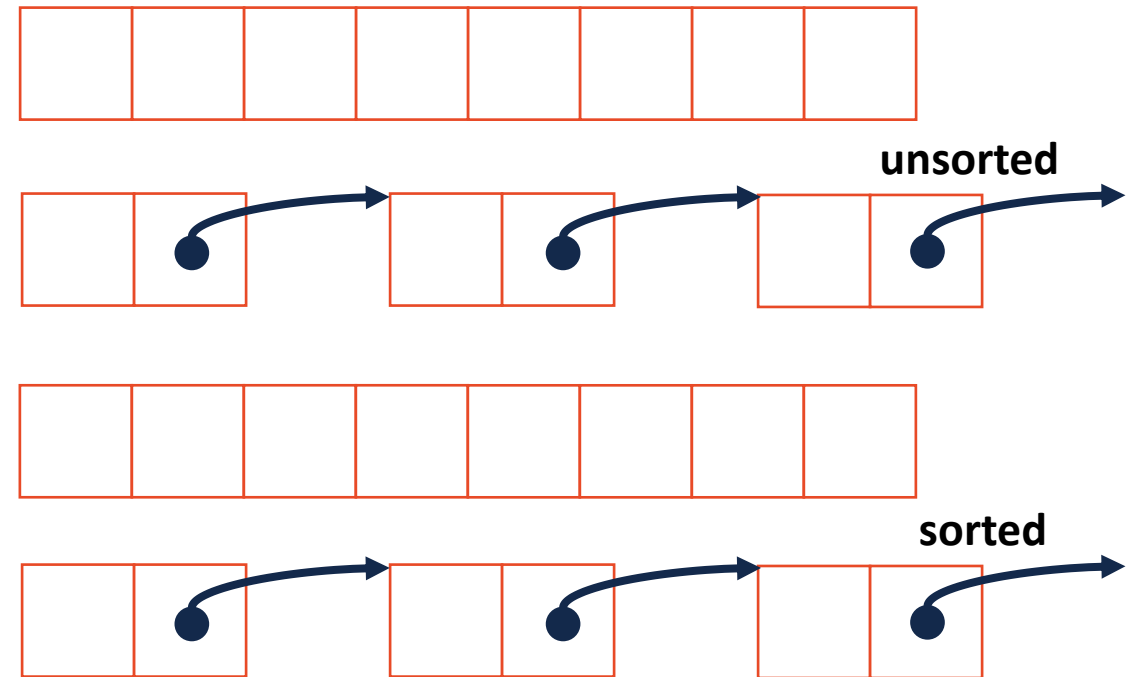


AVL Tree

Heap

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(\lg n)$	$O(\lg n)$	$O(\lg n)$

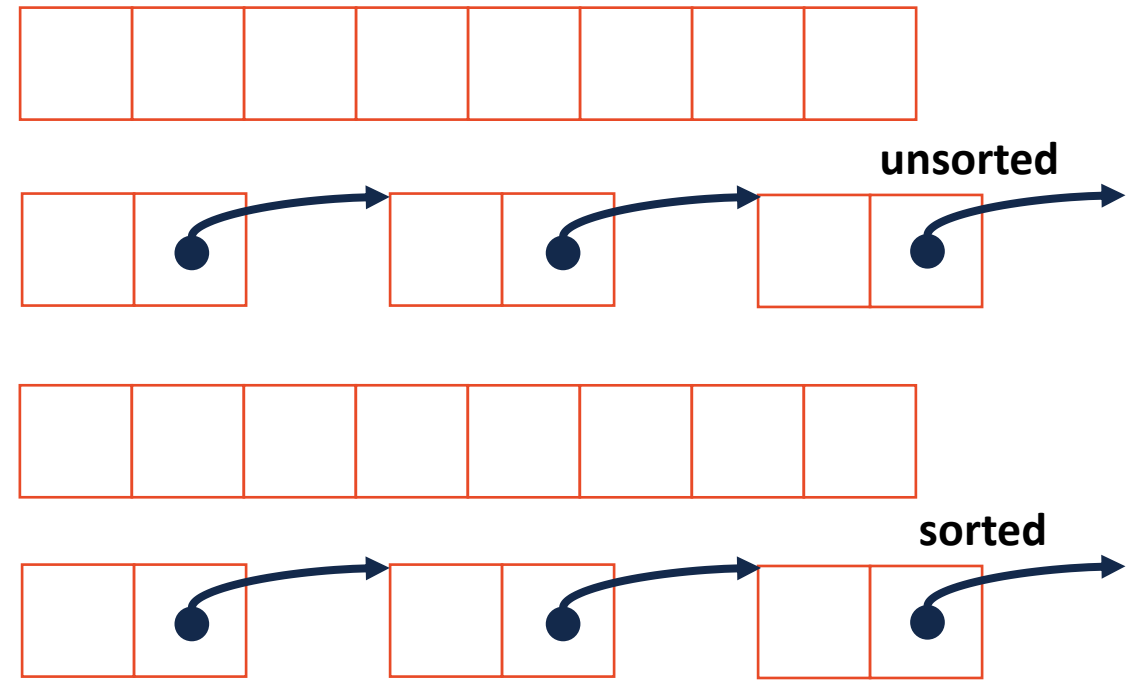


AVL Tree

Heap

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(\lg n)$	$O(\lg n)$	$O(\lg n)$
$O(\lg n)$		

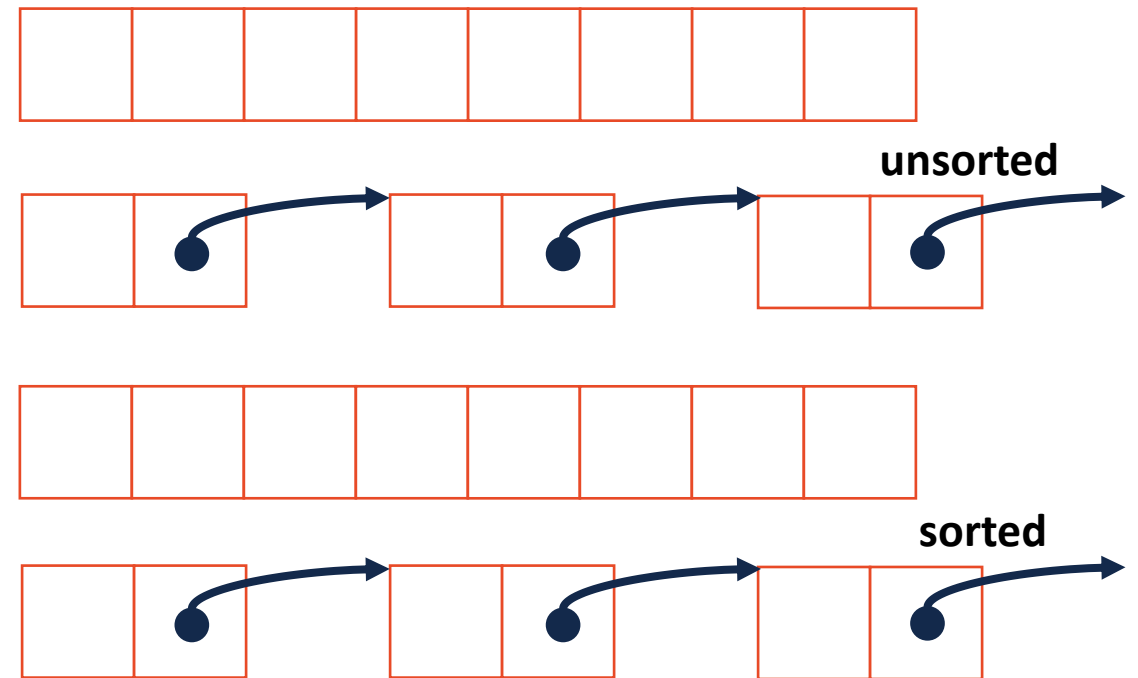


AVL Tree

Heap

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(\lg n)$	$O(\lg n)$	$O(\lg n)$
$O(\lg n)$	$O(\lg n)$	$O(\lg n)$

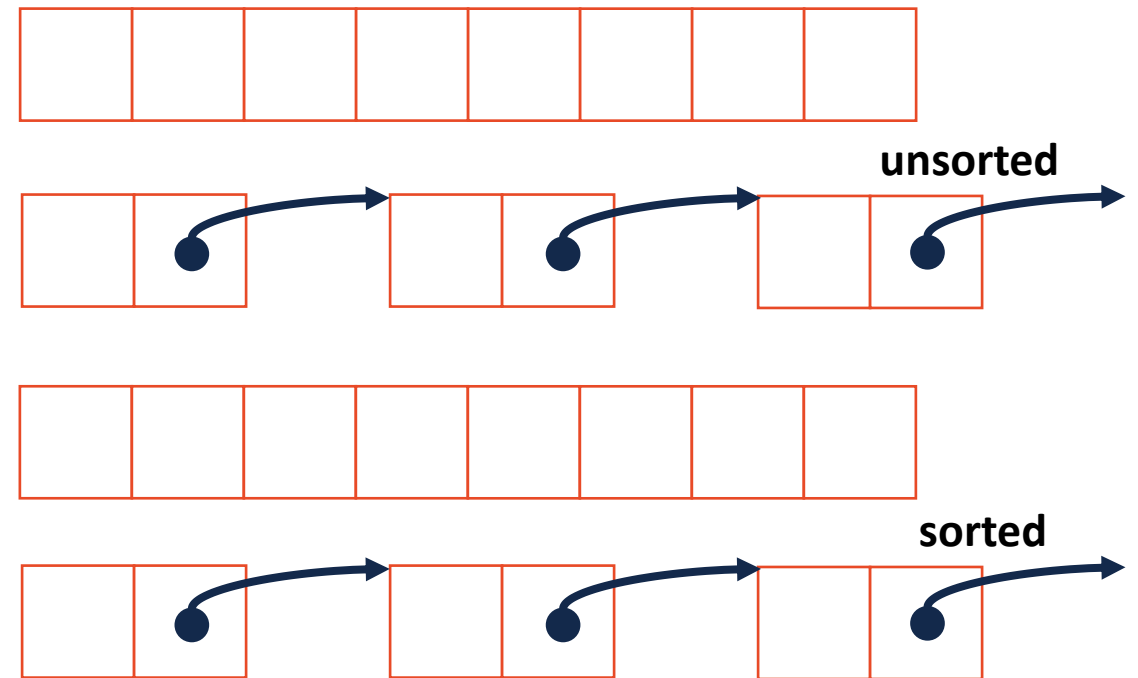


AVL Tree

Heap

Priority Queue Implementation

insert	removeMin	build
$O(1)^A$	$O(n)$	$O(n)$
$O(1)$	$O(n)$	$O(n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(n)$	$O(1)$	$O(\lg n)$
$O(\lg n)$	$O(\lg n)$	$O(\lg n)$
$O(\lg n)$	$O(\lg n)$	$O(n)$



AVL Tree

Heap



CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

A(nother) throwback to Math 213...

Let $\mathbf{R}$ be an equivalence relation on us where $(s, t) \in \mathbf{R}$ if s and t have the same favorite among:

{ _____, _____, _____, _____, _____, _____ }

A(nother) throwback to Math 213...

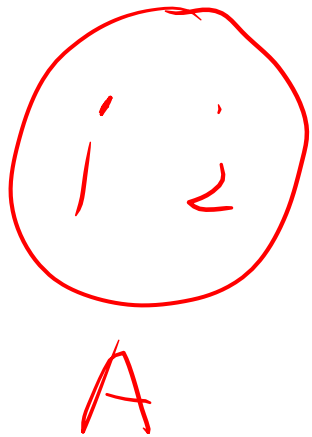
Let R be an equivalence relation on us where $(s, t) \in R$ if s and t have the same favorite among:

$\{ \underline{A}, \underline{B}, \underline{C}, \underline{D}, \underline{E}, \underline{F} \}$

A(nother) throwback to Math 213...

Let R be an equivalence relation on us where $(s, t) \in R$ if s and t have the same favorite among:

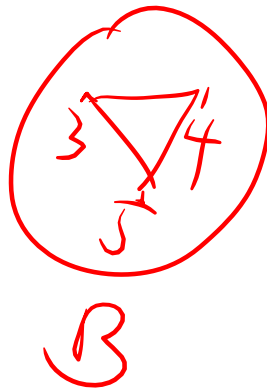
$\{ \text{A, B, C, D, E, F} \}$



A(nother) throwback to Math 213...

Let R be an equivalence relation on us where $(s, t) \in R$ if s and t have the same favorite among:

$\{ \underline{\text{A}}, \underline{\text{B}}, \underline{\text{C}}, \underline{\text{D}}, \underline{\text{E}}, \underline{\text{F}} \}$



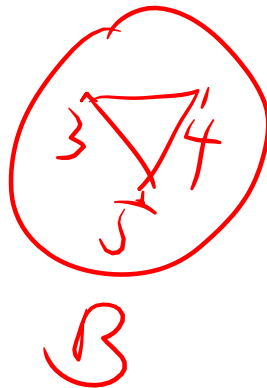
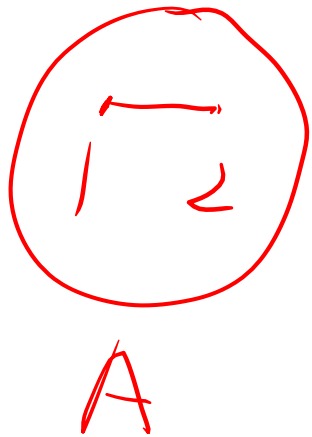
...

$[s]_R : s \in R$

A(nother) throwback to Math 213...

Let R be an equivalence relation on us where $(s, t) \in R$ if s and t have the same favorite among:

$\{ \underline{\text{A}}, \underline{\text{B}}, \underline{\text{C}}, \underline{\text{D}}, \underline{\text{E}}, \underline{\text{F}} \}$

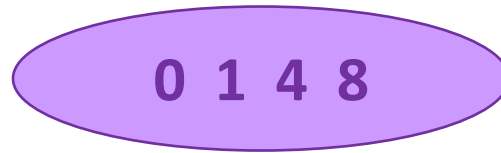
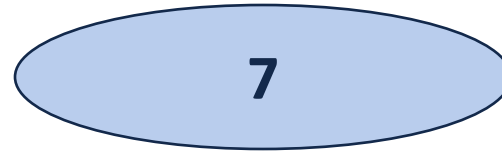
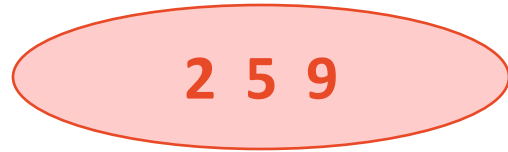


...

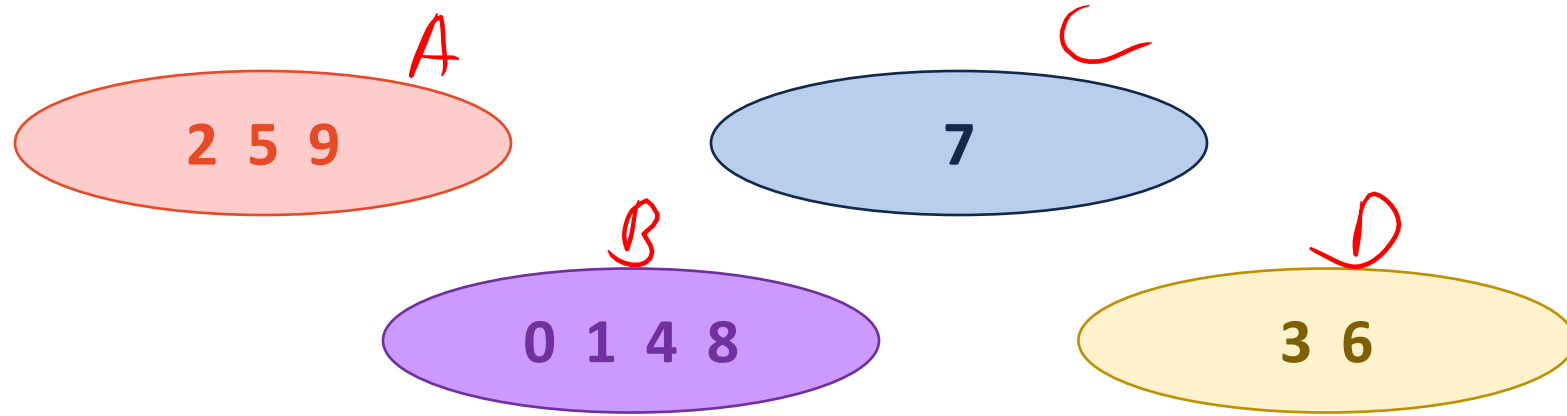
$$[s]_R : s \in R$$

$$[3]_R \equiv [4]_R$$

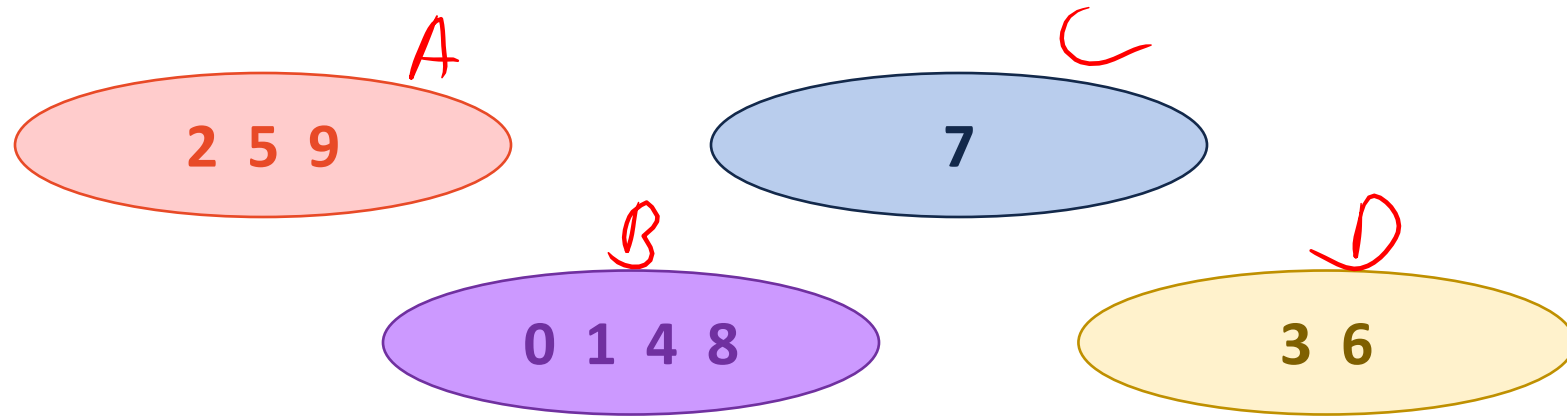
Disjoint Sets



Disjoint Sets

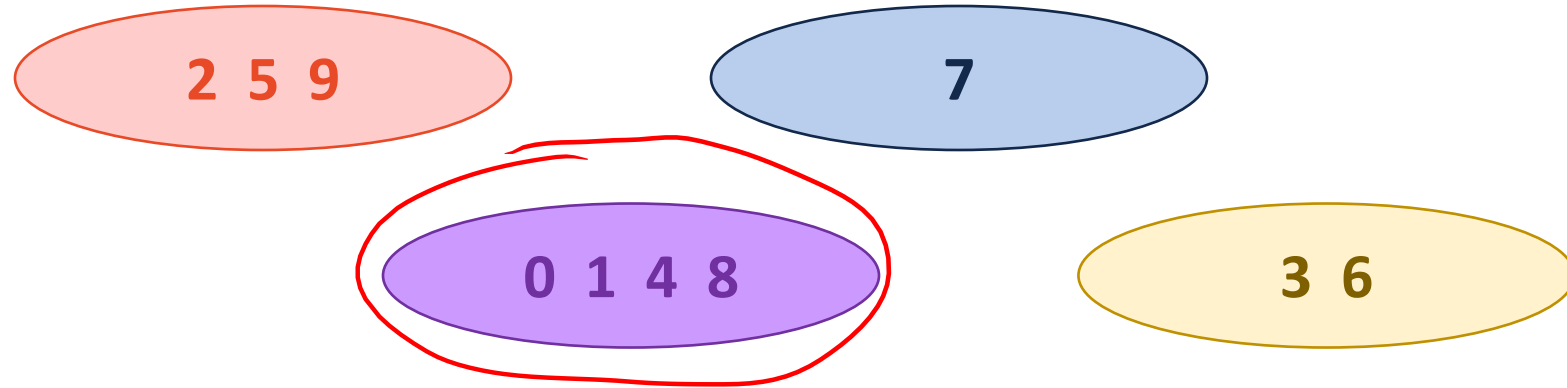


Disjoint Sets



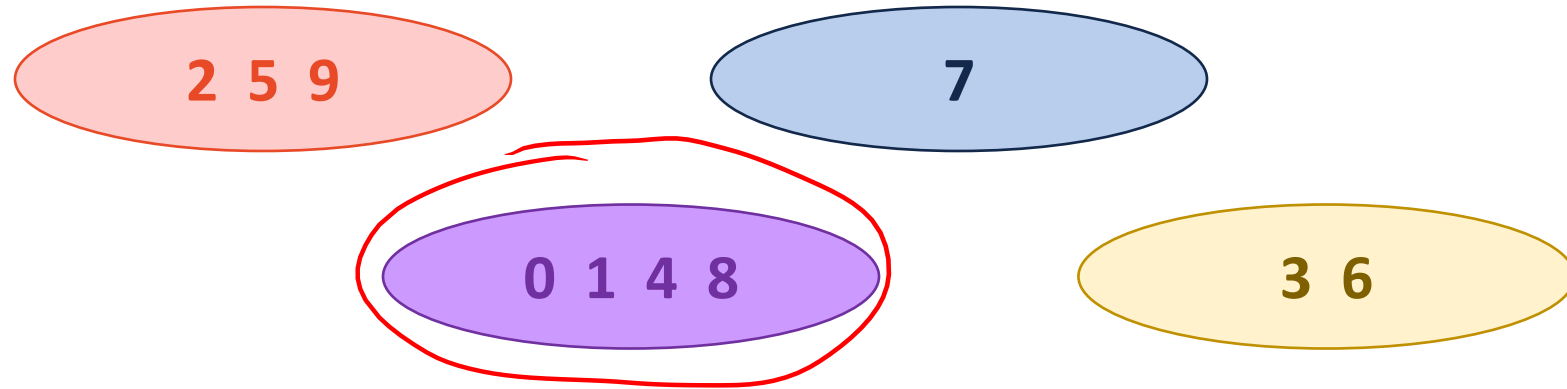
Every elem is in exactly one set

Disjoint Sets



Operation: find(4)

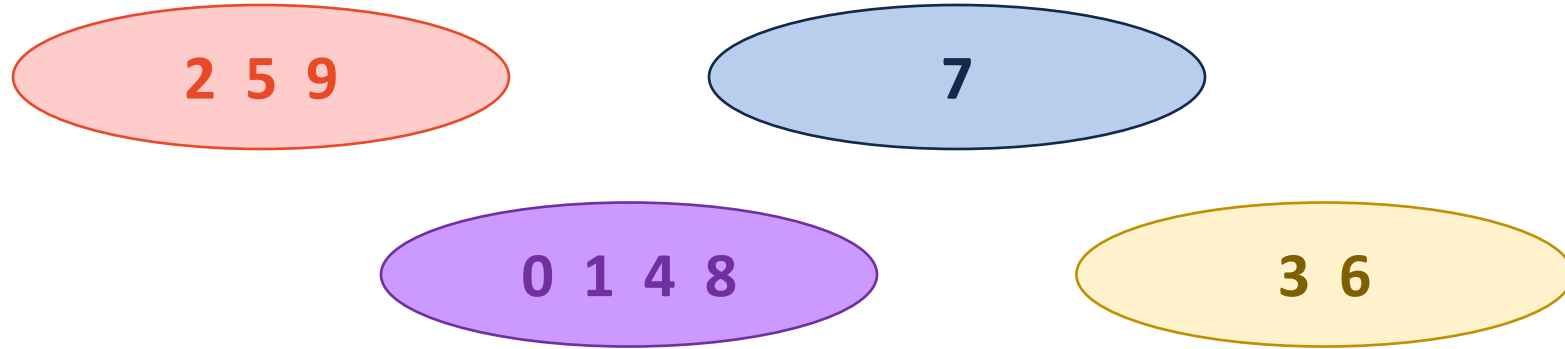
Disjoint Sets



Operation: find(4)

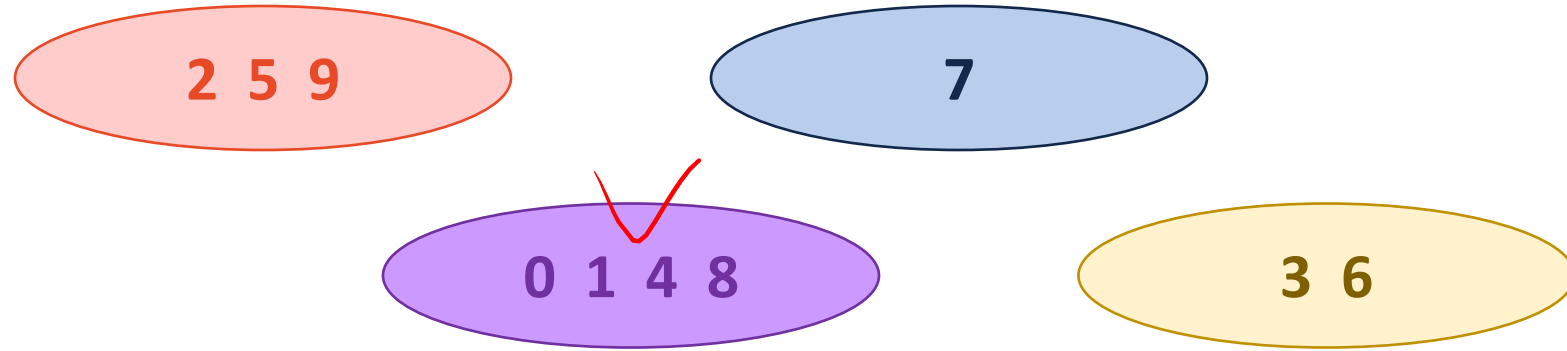
$\equiv 4$

Disjoint Sets



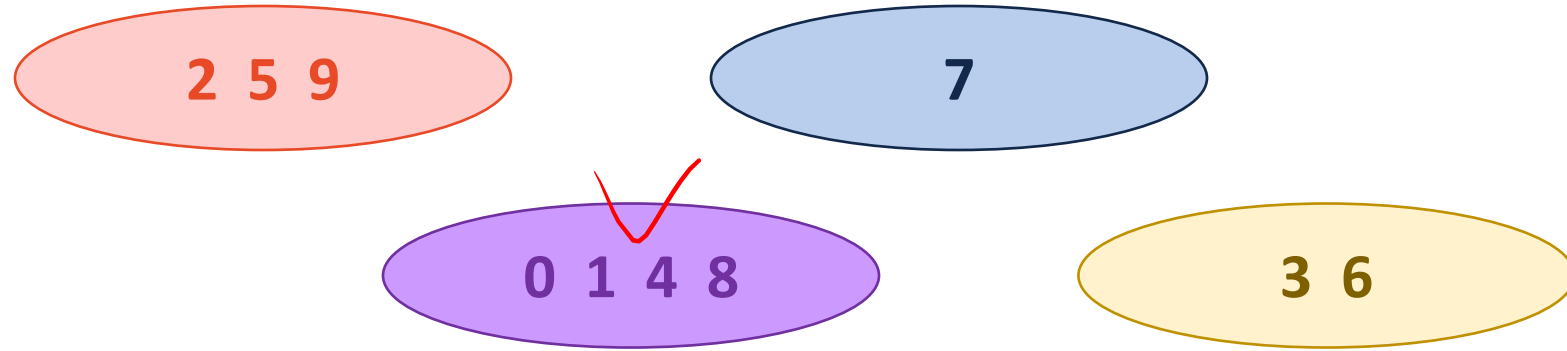
Operation: $\text{find}(4) == \text{find}(8)$

Disjoint Sets



Operation: $\text{find}(4) == \text{find}(8)$

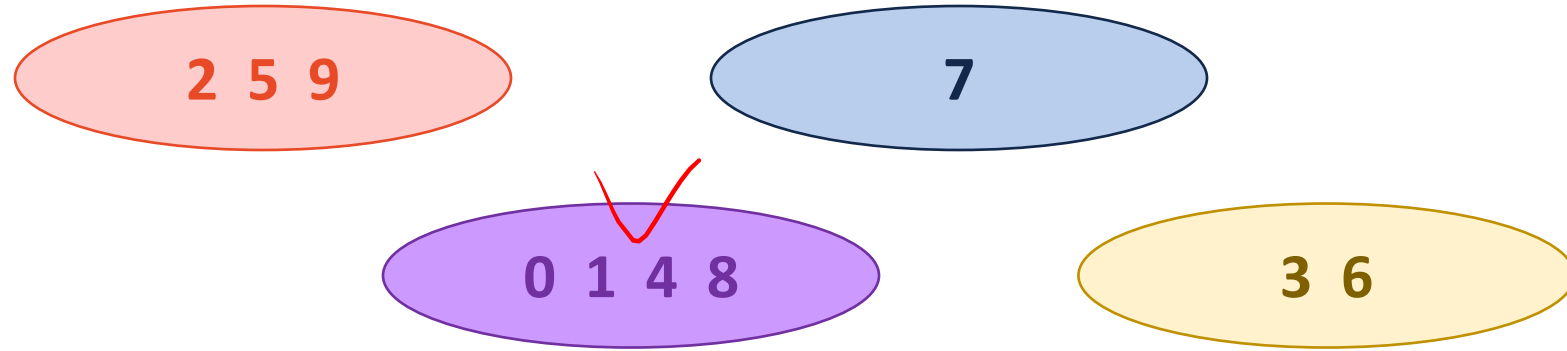
Disjoint Sets



Operation: $\text{find}(4) == \text{find}(8)$

Rep elem of the set must be the same

Disjoint Sets



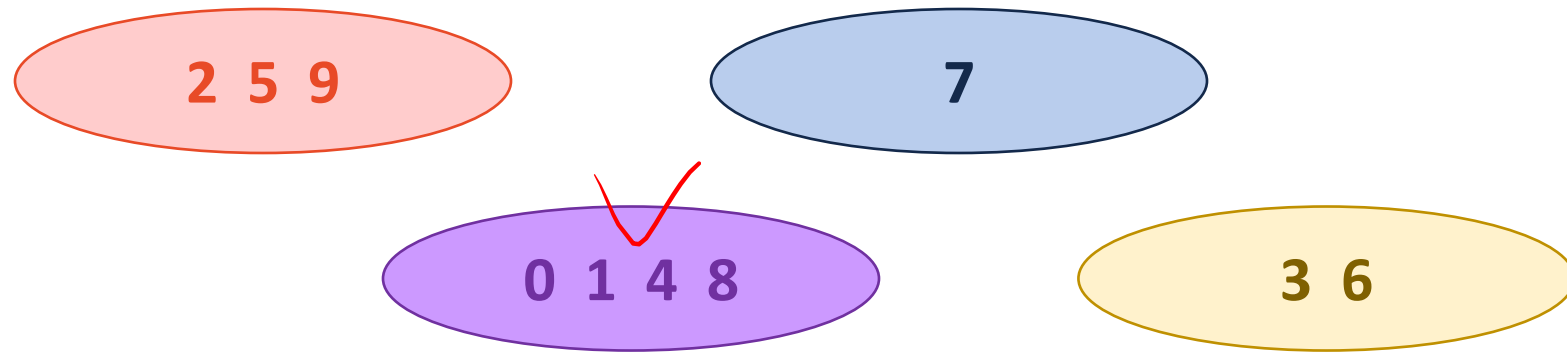
Operation: $\text{find}(4) == \text{find}(8)$

Rep elem of the set must be the same

4

4

Disjoint Sets



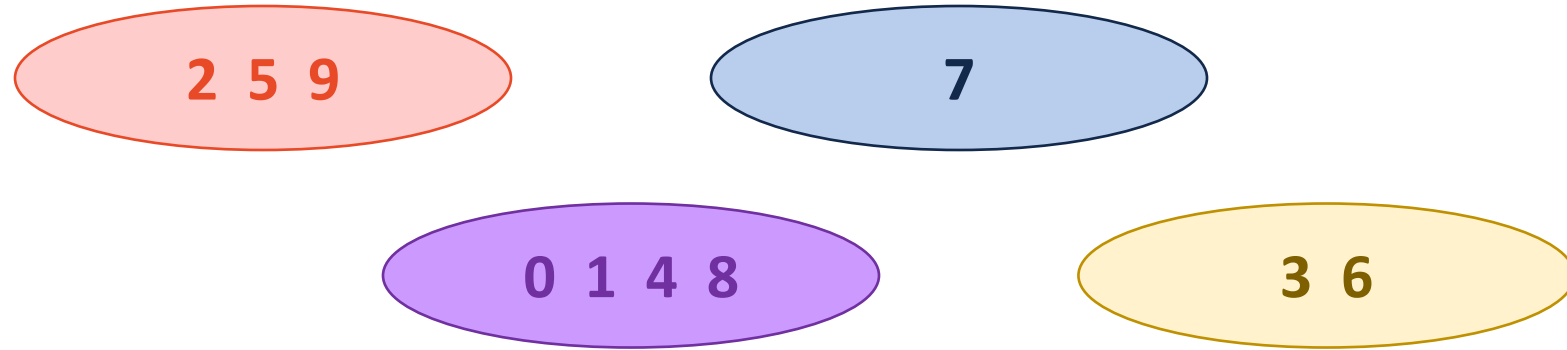
Operation: $\text{find}(4) == \text{find}(8)$

Rep elem of the set must be the same

$\downarrow$ $\downarrow$

4 == 4

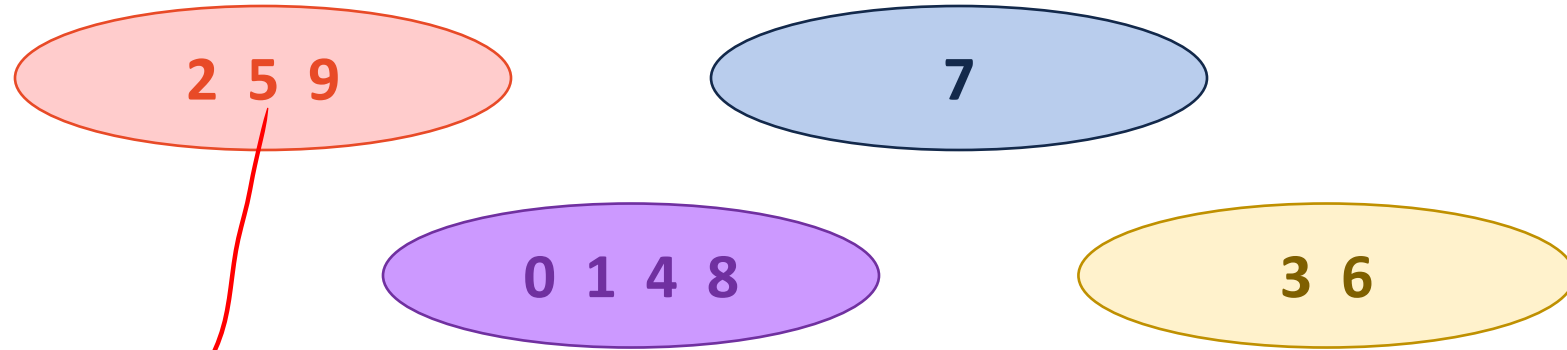
Disjoint Sets



Operation:

```
if ( find(2) != find(7) ) {  
    union( find(2), find(7) );  
}
```

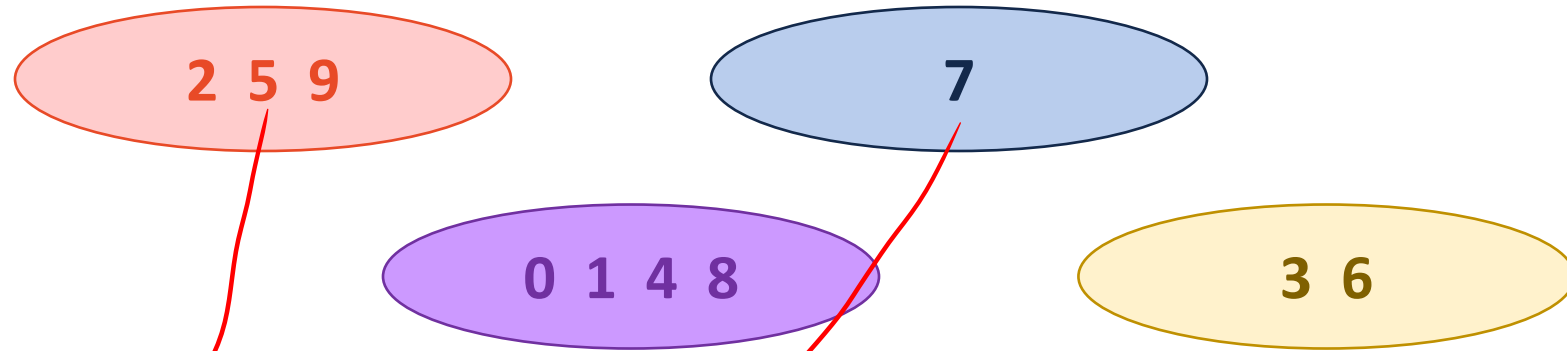
Disjoint Sets



Operation:

```
if ( find(2) != find(7) ) {  
    union( find(2), find(7) );  
}
```

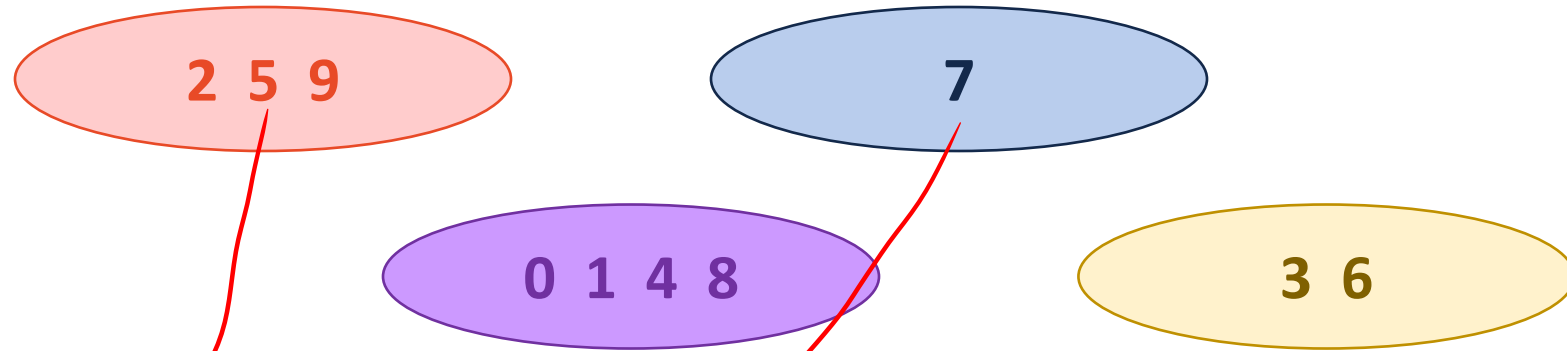
Disjoint Sets



Operation:

```
if ( find(2) != find(7) ) {  
    union( find(2), find(7) );  
}
```

Disjoint Sets

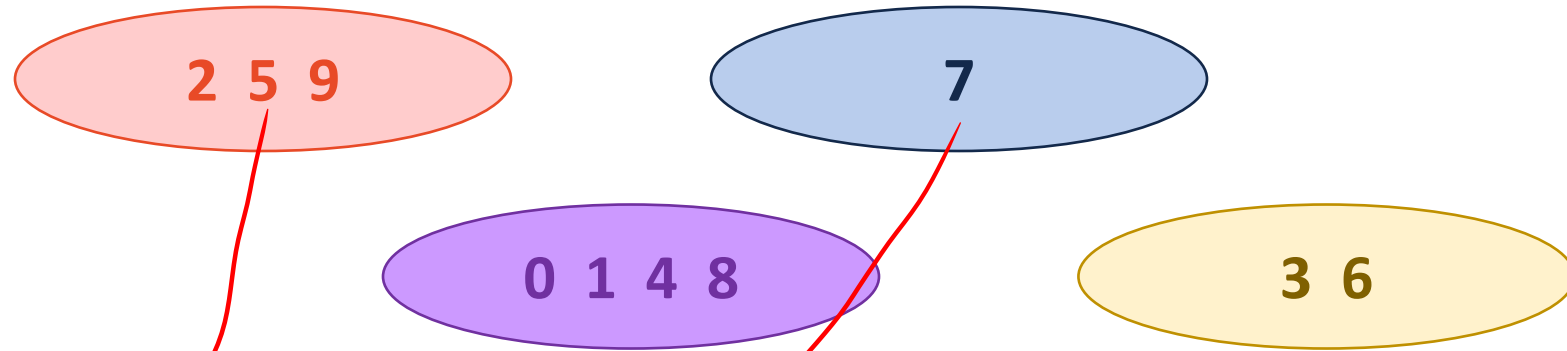


Operation:

Handwritten red text: $9 \neq 7$ with arrows pointing to the red oval and the blue oval.

```
if ( find(2) != find(7) ) {  
    union( find(2), find(7) );  
}
```

Disjoint Sets



Operation:

$9 \neq 7$

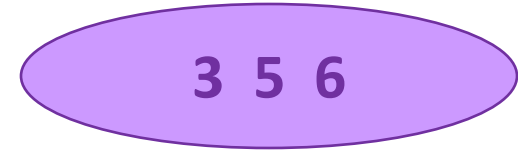
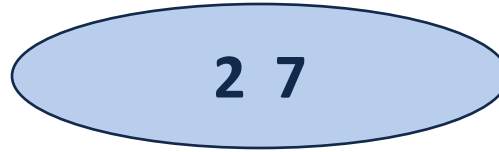
```
if ( find(2) != find(7) ) {  
    union( find(2), find(7) );  
}
```

Disjoint Sets ADT

- Maintain a collection $S = \{s_0, s_1, \dots, s_k\}$
- Each set has a representative member.
- API:

```
void makeSet(const T & t);  
void union(const T & k1, const T & k2);  
T & find(const T & k);
```


Implementation #1

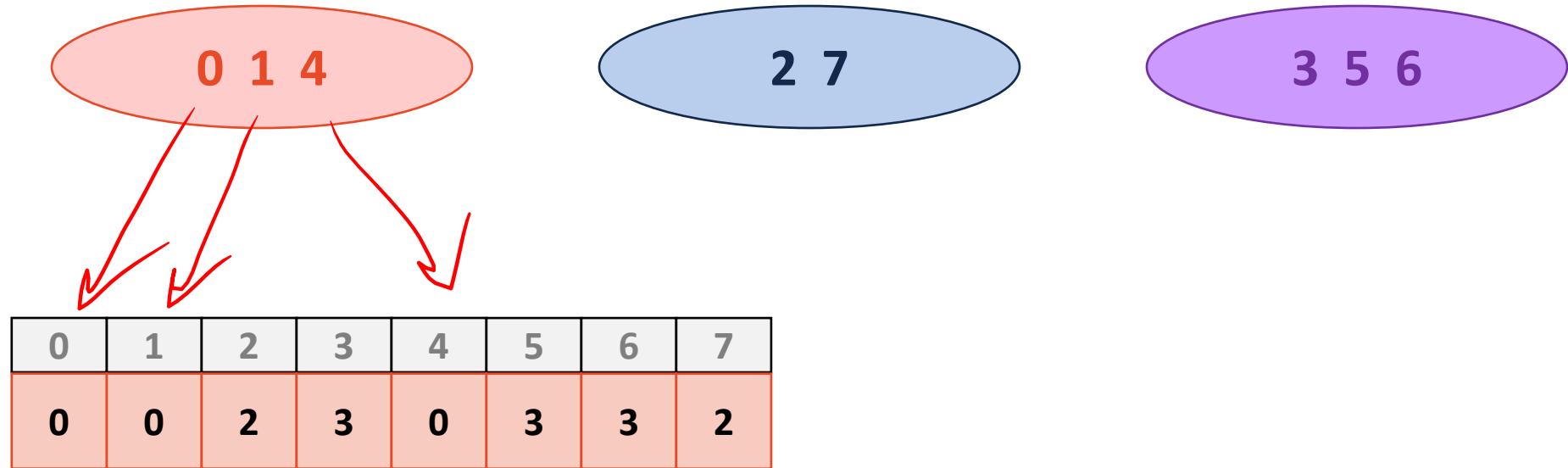


0	1	2	3	4	5	6	7
0	0	2	3	0	3	3	2

Find(k):

Union(k1, k2):

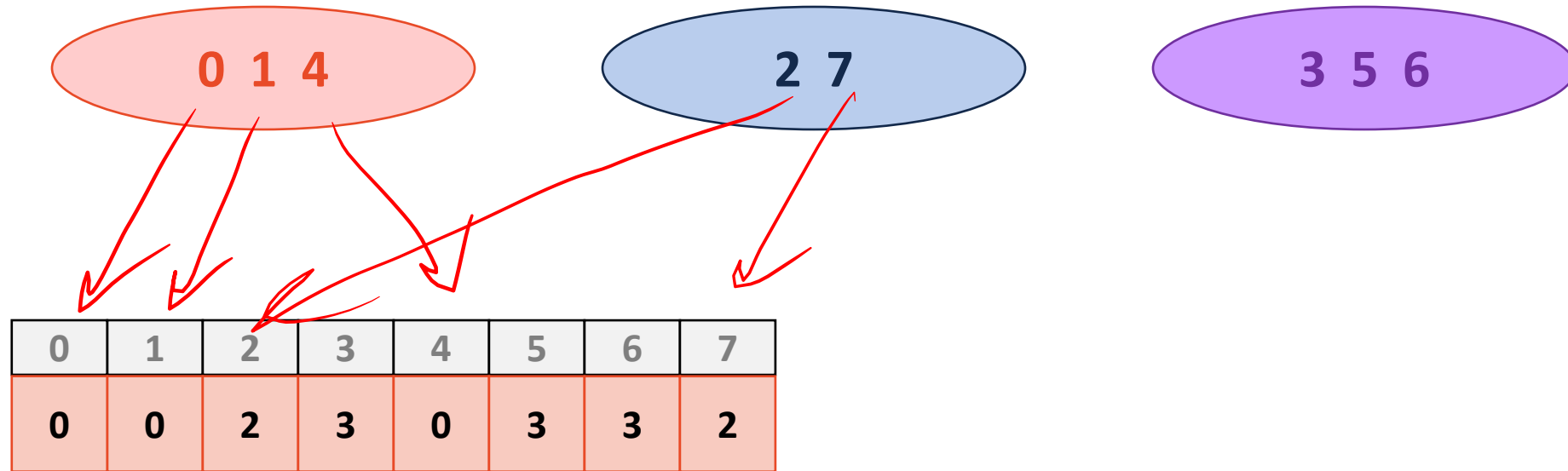
Implementation #1



Find(k):

Union(k1, k2):

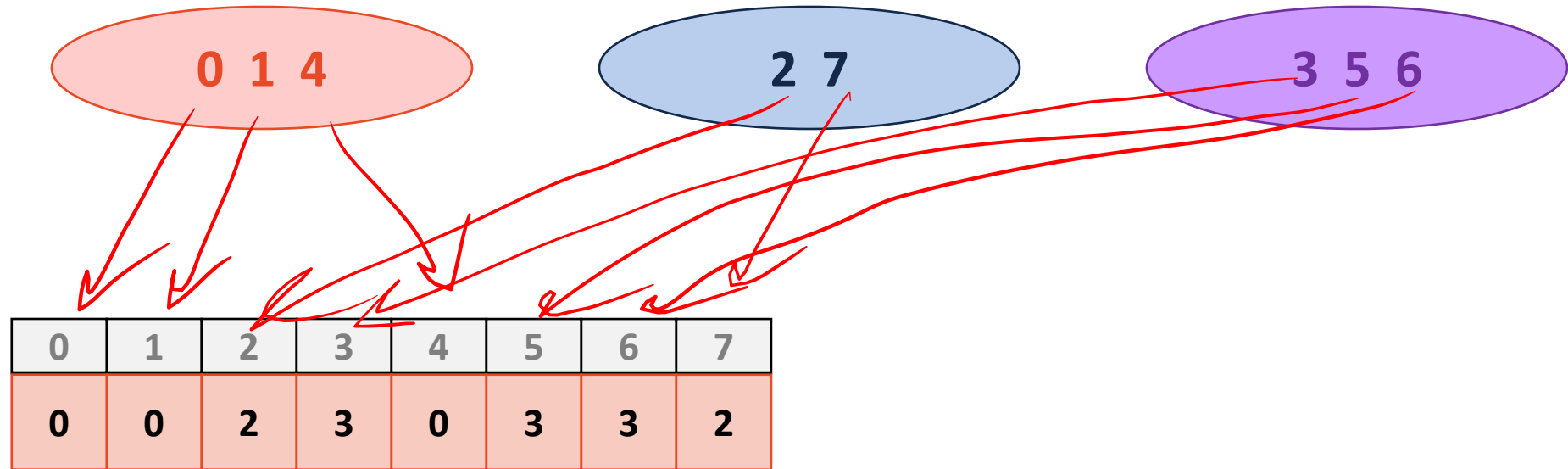
Implementation #1



Find(k):

Union(k1, k2):

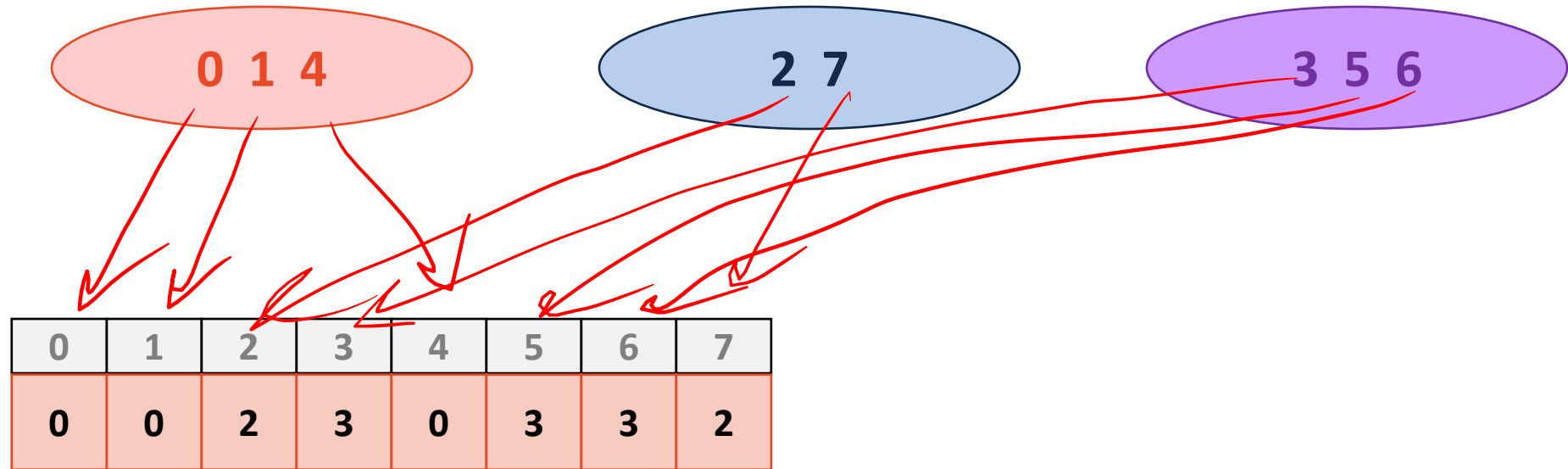
Implementation #1



Find(k):

Union(k1, k2):

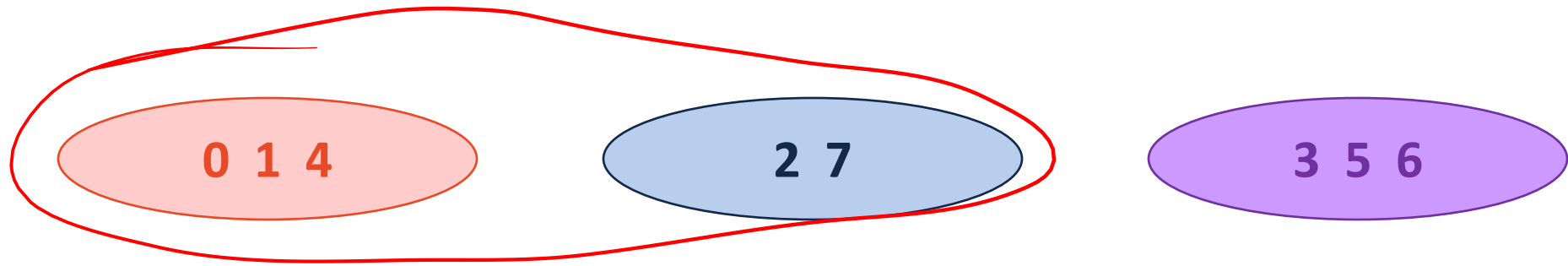
Implementation #1



Find(k): $O(1)$

Union(k1, k2):

Implementation #1

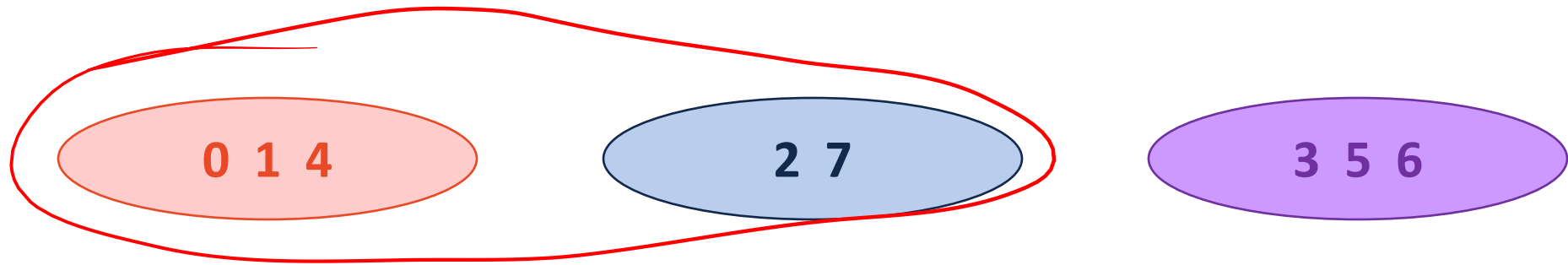


0	1	2	3	4	5	6	7
0	0	2	3	0	3	3	2

Find(k): $O(1)$

Union(k1, k2):

Implementation #1



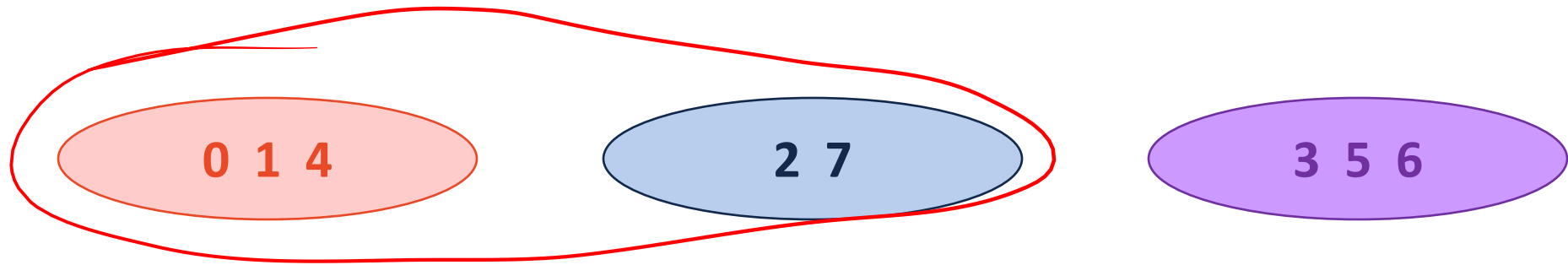
0	1	2	3	4	5	6	7
0	0	2	3	0	3	3	2

0

Find(k): $O(1)$

Union(k1, k2):

Implementation #1



0	1	2	3	4	5	6	7
0	0	2	3	0	3	3	2

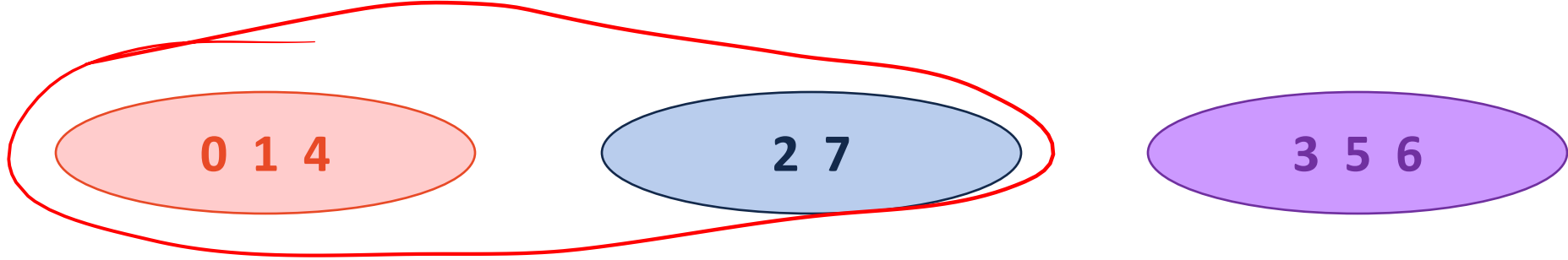
0

0

Find(k): $O(1)$

Union(k1, k2):

Implementation #1



0	1	2	3	4	5	6	7
0	0	2	3	0	3	3	2

0

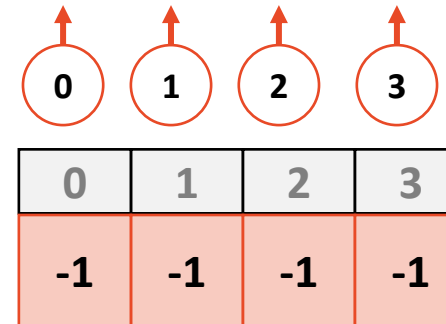
0

Find(k): $O(1)$

Union(k1, k2): $O(n)$

Implementation #2

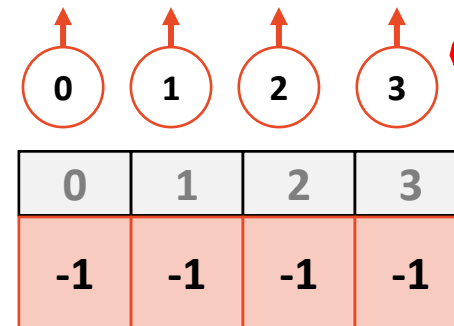
- We will continue to use an array where the index is the key
- The value of the array is:
 - **-1**, if we have found the representative element
 - **The index of the parent**, if we haven't found the rep. element
- We will call theses **UpTrees**:



Array
Impl

Implementation #2

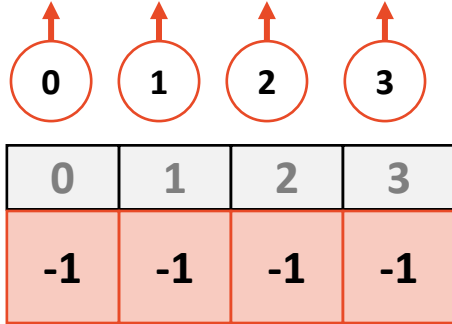
- We will continue to use an array where the index is the key
- The value of the array is:
 - **-1**, if we have found the representative element
 - **The index of the parent**, if we haven't found the rep. element
- We will call theses **UpTrees**:



Array
impl

visual
model

UpTrees

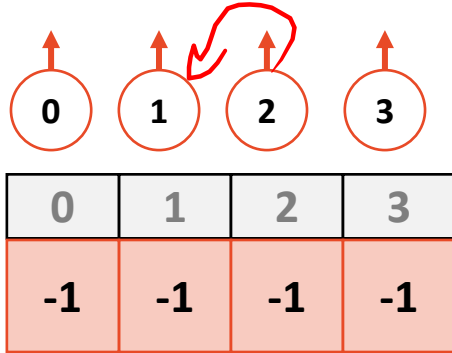


0	1	2	3

0	1	2	3

0	1	2	3

UpTrees

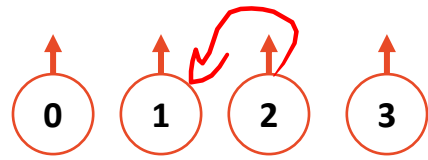


0	1	2	3

0	1	2	3

0	1	2	3

UpTrees



0	1	2	3
-1	-1	-1	-1

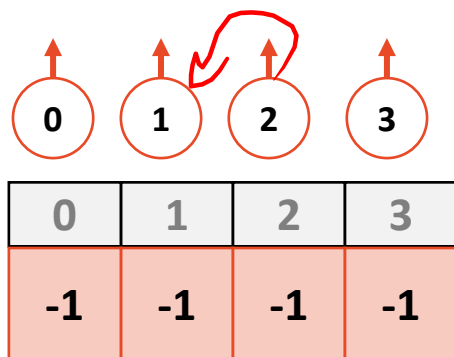
union(1, 2)

0	1	2	3

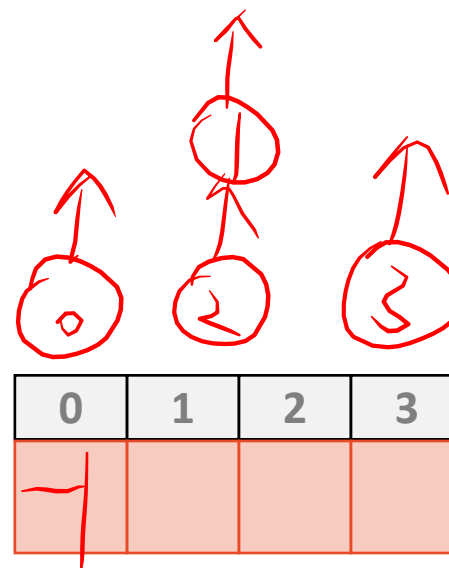
0	1	2	3

0	1	2	3

UpTrees



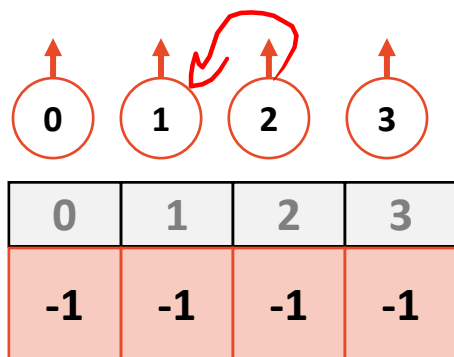
union(1, 2)



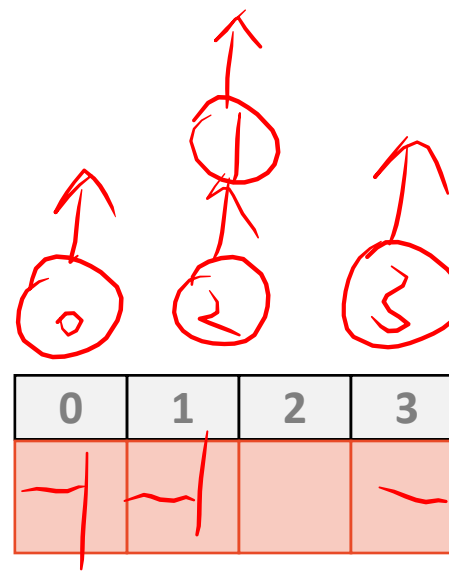
0	1	2	3

0	1	2	3

UpTrees



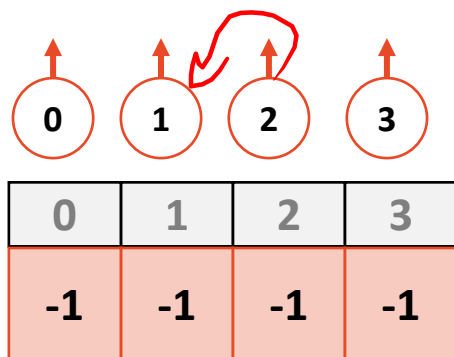
union(1, 2)



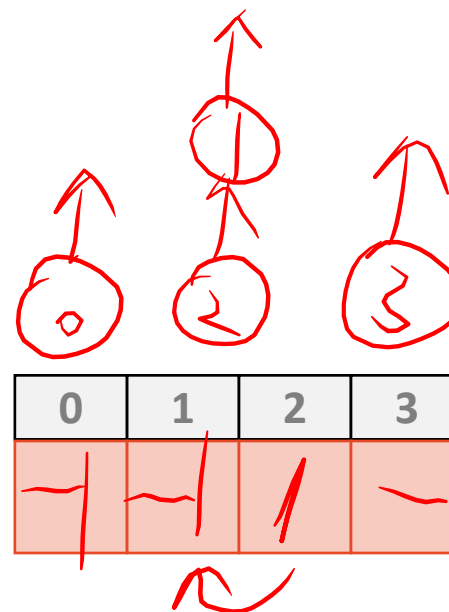
0	1	2	3

0	1	2	3

UpTrees



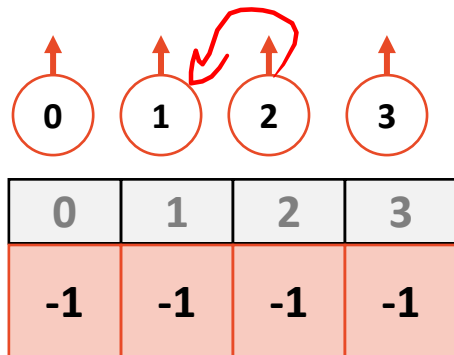
union(1, 2)



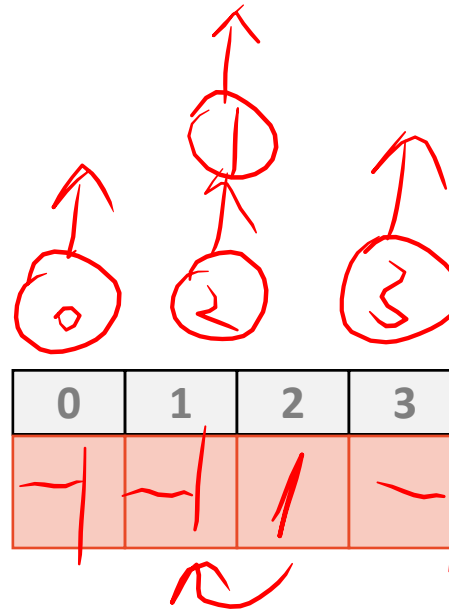
0	1	2	3

0	1	2	3

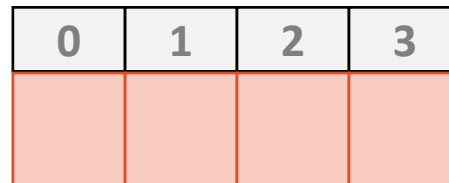
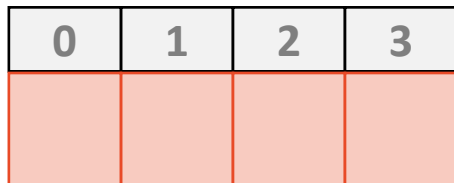
UpTrees



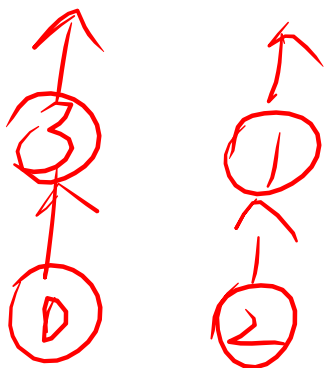
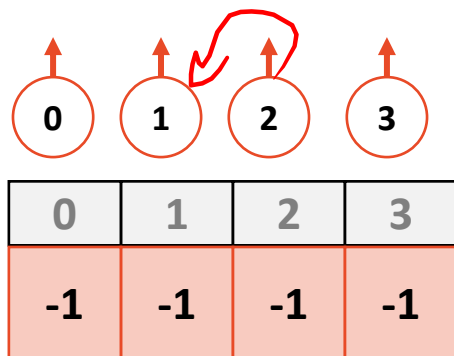
union(1, 2)



union(0, 3)

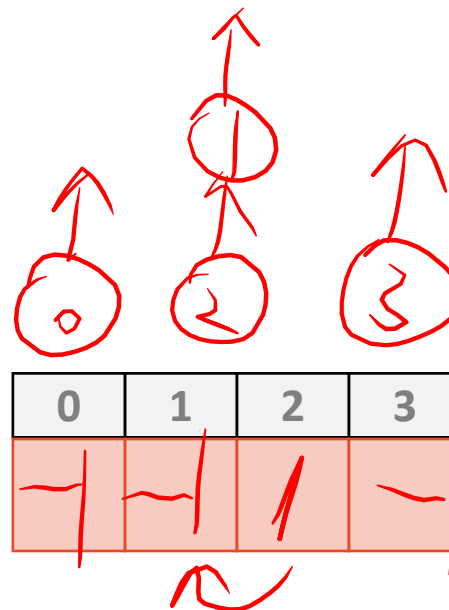


UpTrees



0	1	2	3

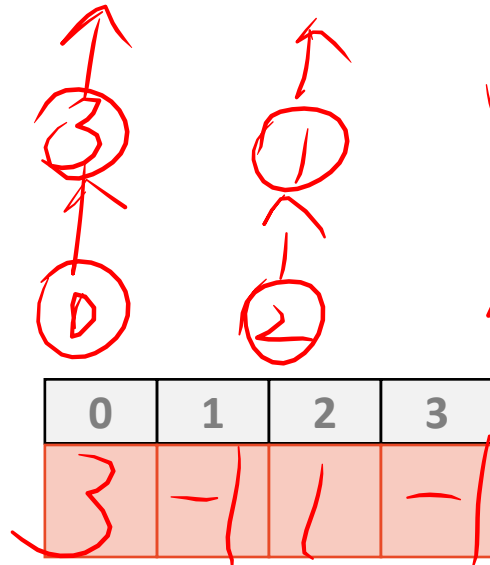
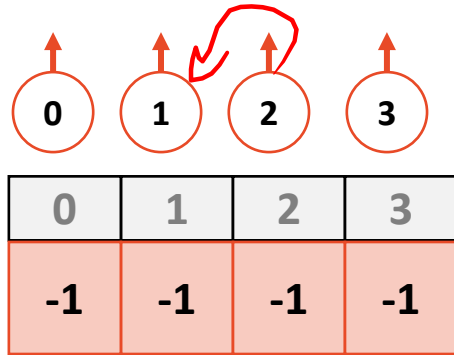
union(1, 2)



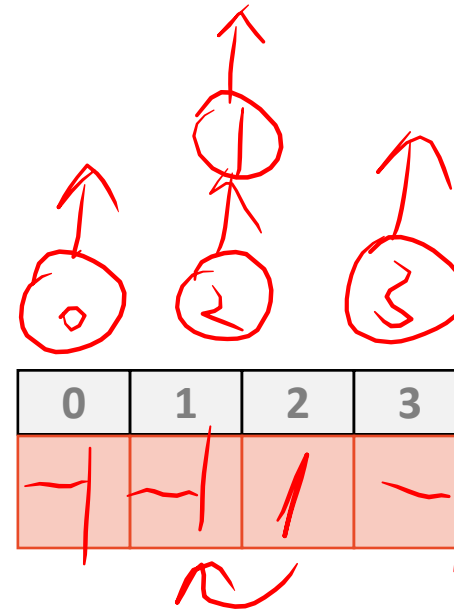
union(0, 3)

0	1	2	3

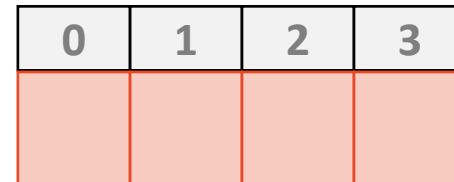
UpTrees



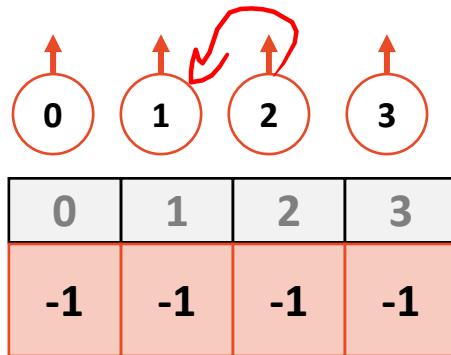
union(1,2)



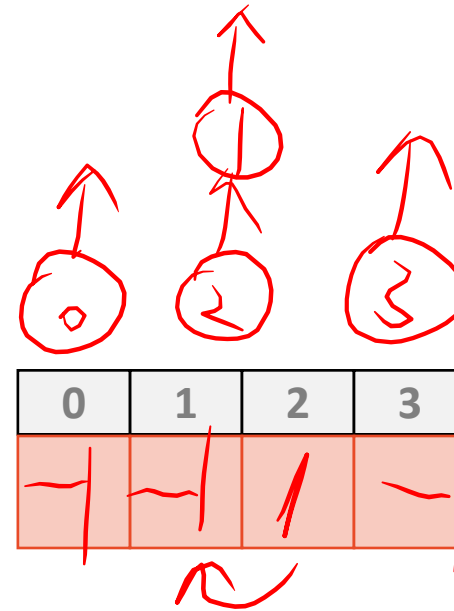
union(0,3)



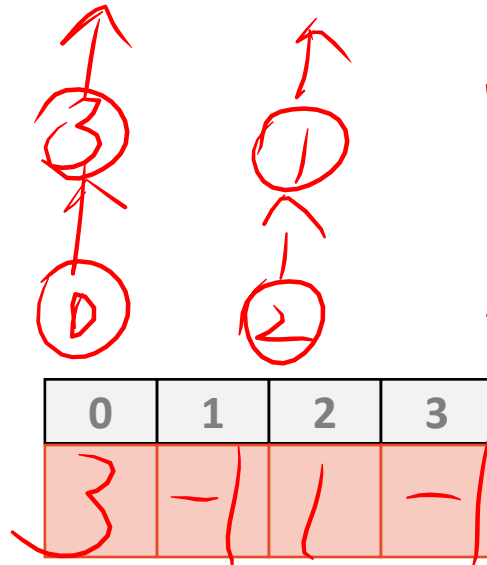
UpTrees



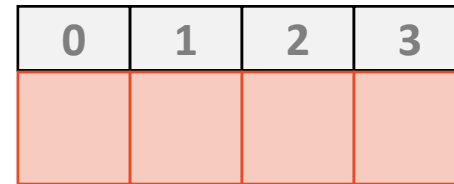
$\text{union}(1, 2)$



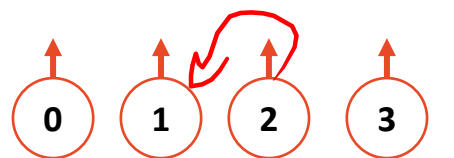
$\text{union}(0, 3)$



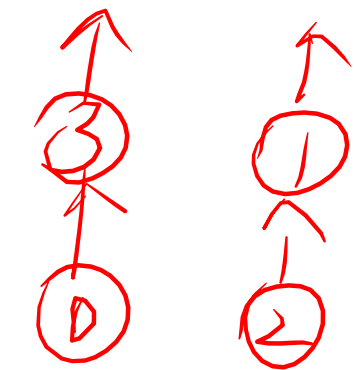
$\text{union}(3, 1)$



UpTrees

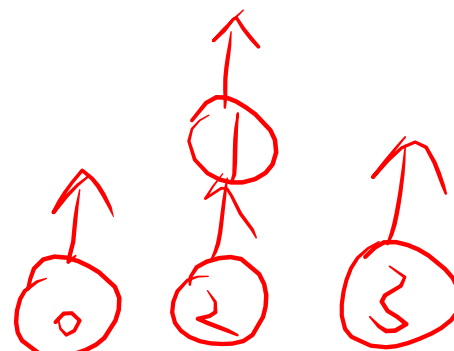


0	1	2	3
-1	-1	-1	-1

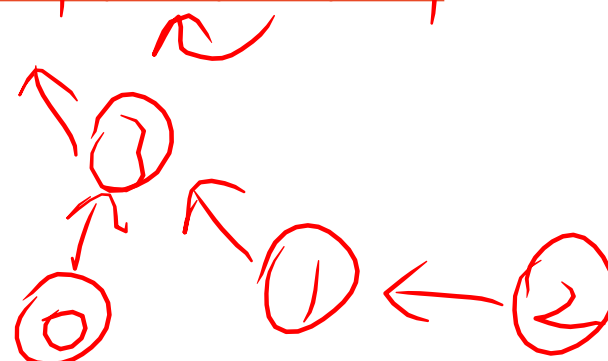


0	1	2	3
3	-1	-1	-1

$\text{union}(1, 2)$



0	1	2	3
3	2	2	3

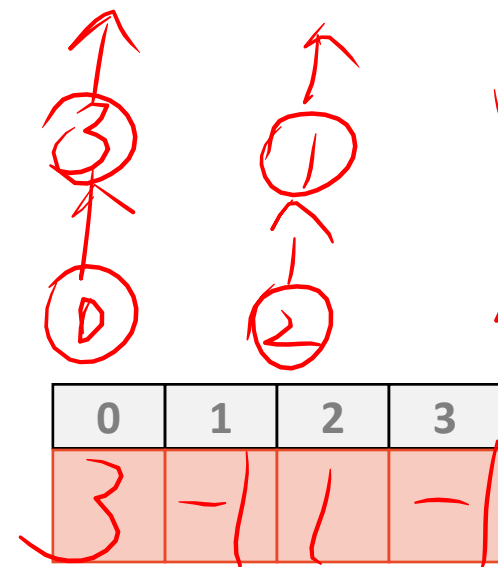
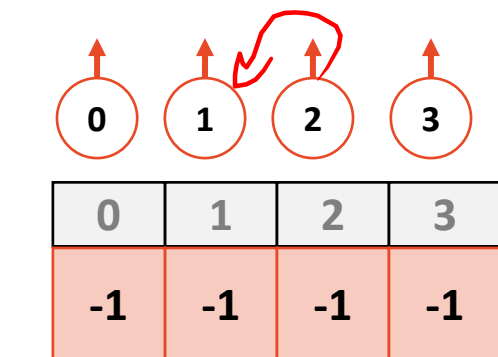


0	1	2	3
3	3	3	3

$\text{union}(0, 3)$

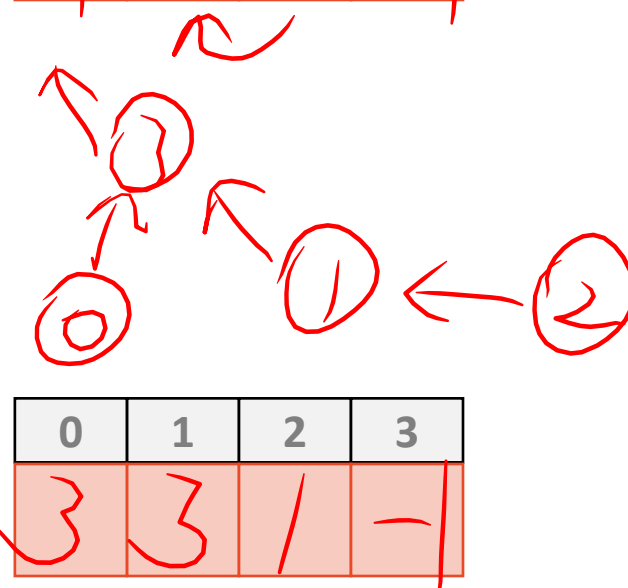
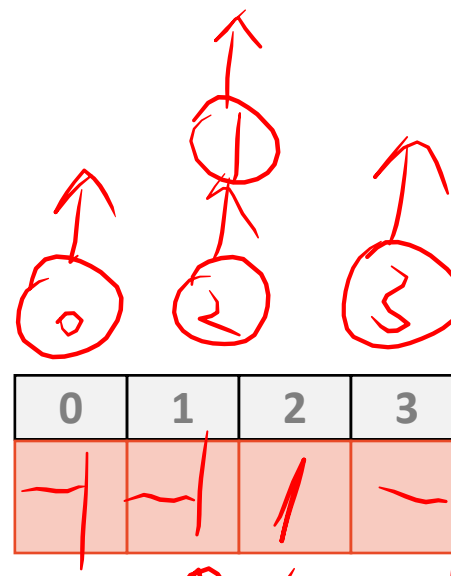
$\text{union}(3, 1)$

UpTrees

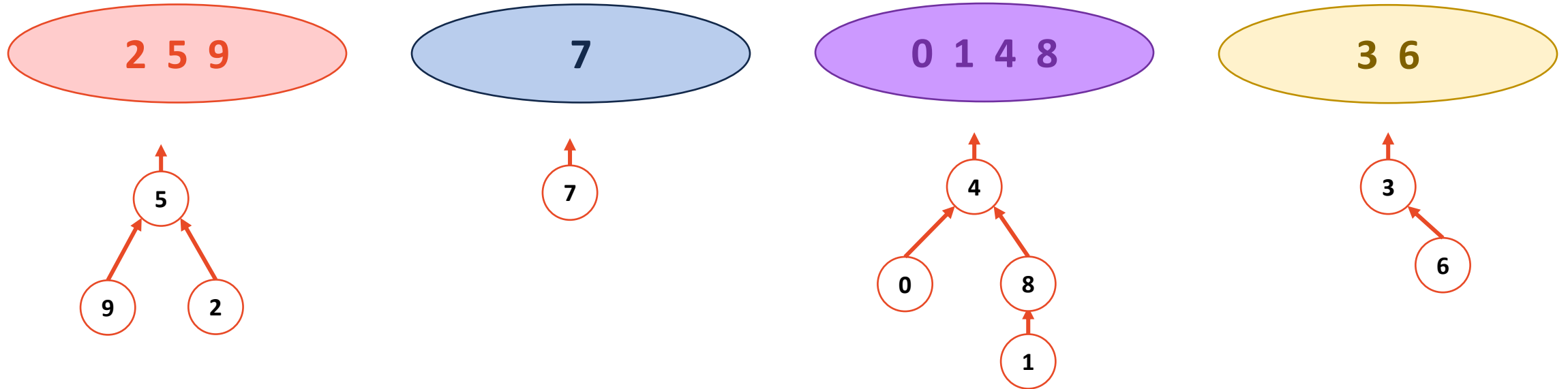


$\text{union}(0, 3)$

$\text{union}(3, 1)$

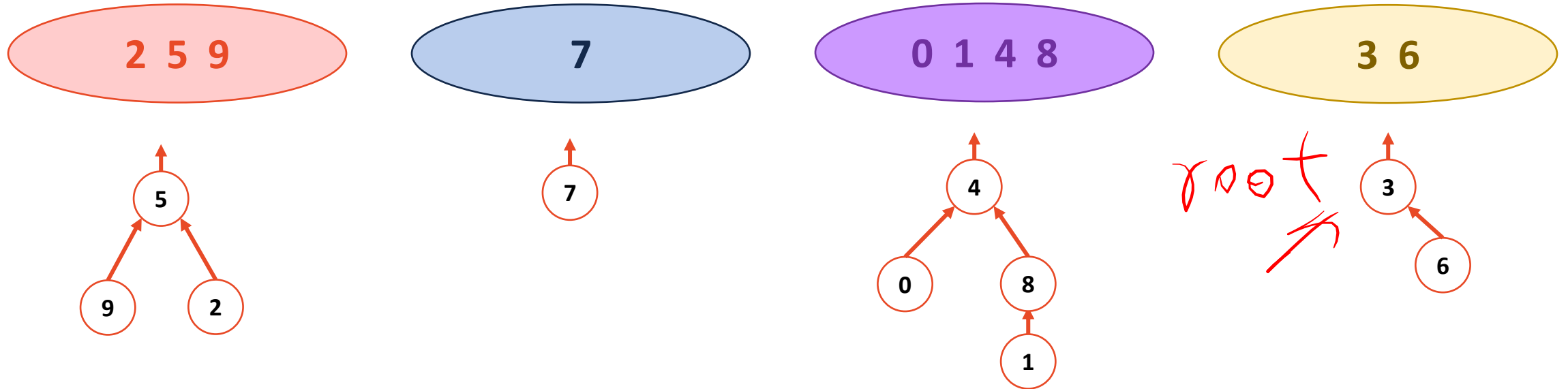


Disjoint Sets



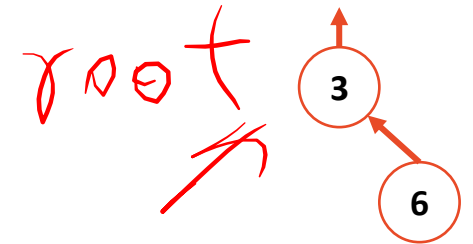
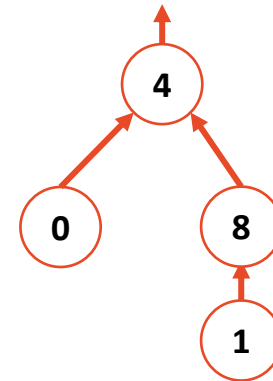
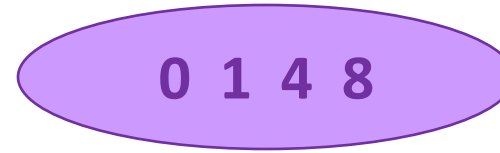
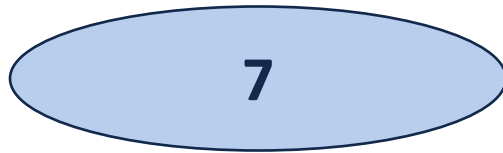
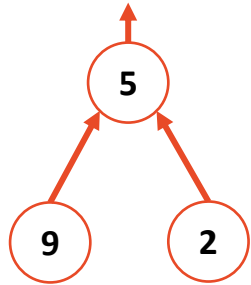
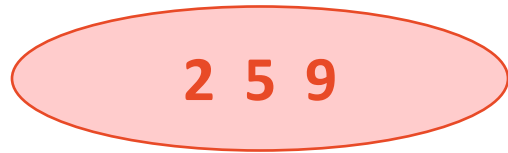
0	1	2	3	4	5	6	7	8	9
4	8	5	6	-1	-1	-1	-1	4	5

Disjoint Sets



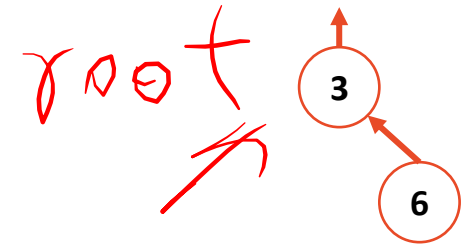
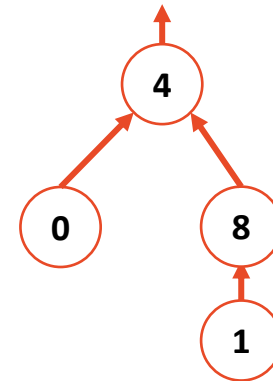
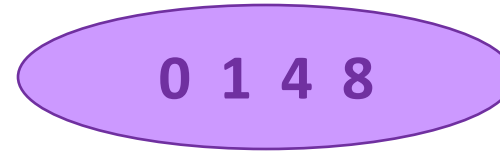
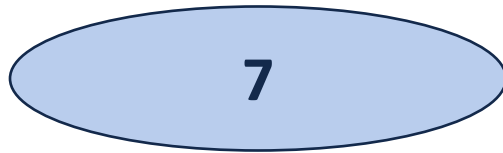
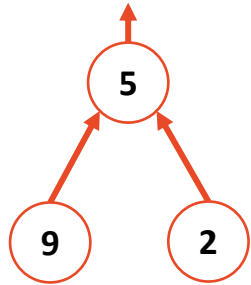
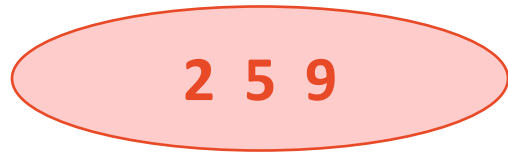
0	1	2	3	4	5	6	7	8	9
4	8	5	6	-1	-1	-1	-1	4	5

Disjoint Sets



0	1	2	3	4	5	6	7	8	9
4	8	5	6	-1	-1	-1	-1	4	5

Disjoint Sets



0	1	2	3	4	5	6	7	8	9
4	8	5	6	-1	-1	-1	-1	4	5

Handwritten red marks below the table: a bracket under the row containing 6 and -1, and a large '3' under the row containing -1 and -1.

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

Running time?

What is the ideal UpTree?

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

← returns rep elem.

Running time?

What is the ideal UpTree?

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

← returns rep elem.

Running time?

$O(u)$

What is the ideal UpTree?

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

← returns rep elem.

Running time?

$O(N)$



What is the ideal UpTree?

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

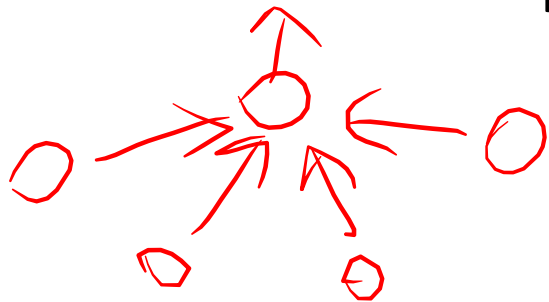
← returns rep elem.

Running time?

$O(N)$

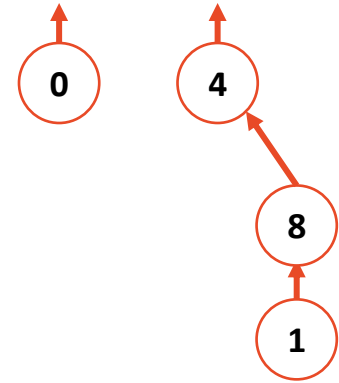


What is the ideal UpTree?



Disjoint Sets Union

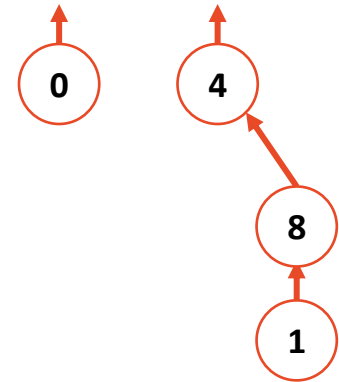
```
1 void DisjointSets::union(int r1, int r2) {  
2  
3  
4 }
```



Disjoint Sets Union

```
1 void DisjointSets::union(int r1, int r2) {  
2  
3  
4 }
```

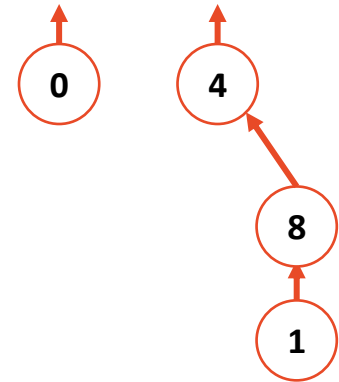
Union(0, 4)



Disjoint Sets Union

```
1 void DisjointSets::union(int r1, int r2) {  
2      $S[r2] = r1$ ;  
3  
4 }
```

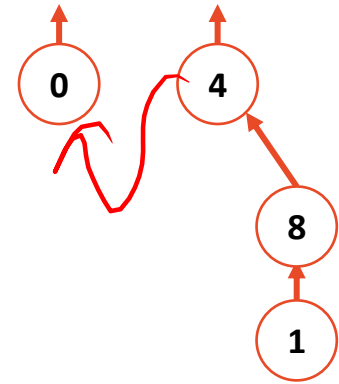
$\text{union}(0, 4)$



Disjoint Sets Union

```
1 void DisjointSets::union(int r1, int r2) {  
2      $S[r2] = r1$ ;  
3  
4 }
```

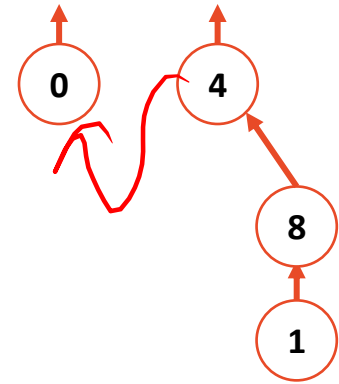
union(0, 4)



Disjoint Sets Union

Union(0, 4)

```
1 void DisjointSets::union(int r1, int r2) {  
2     S[r2] = r1;  
3 }  
4
```

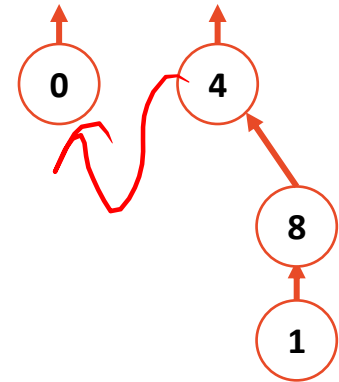


① Union the tree w/ the smallest height

Disjoint Sets Union

Union(0, 4)

```
1 void DisjointSets::union(int r1, int r2) {  
2     S[r2] = r1;  
3 }  
4
```



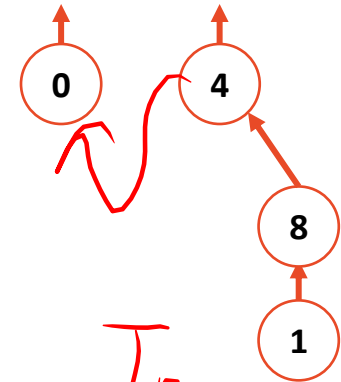
① Union the tree w/ the smallest height

②

Disjoint Sets Union

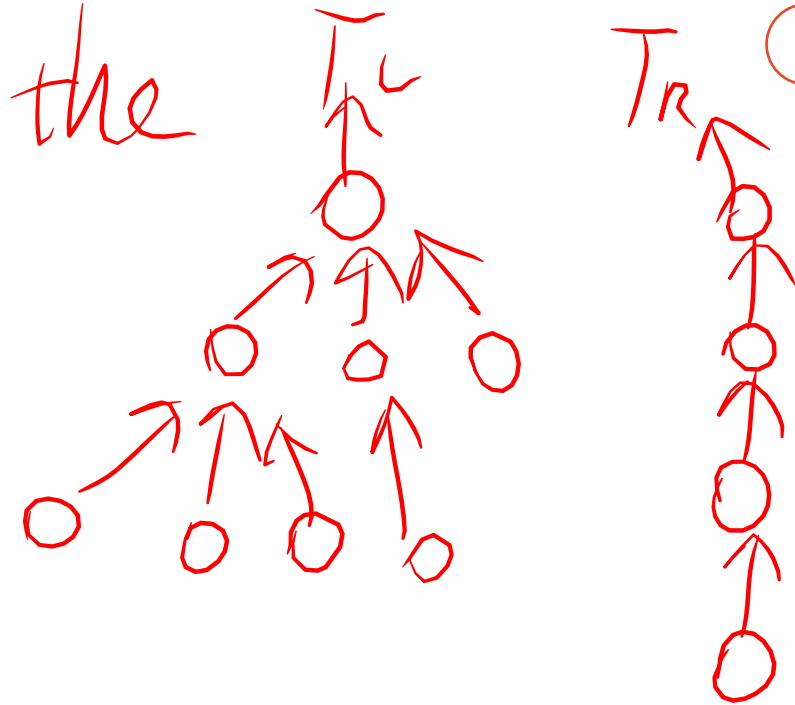
Union(0, 4)

```
1 void DisjointSets::union(int r1, int r2) {  
2     S[r2] = r1;  
3 }  
4
```



① Union the tree w/ the smallest height

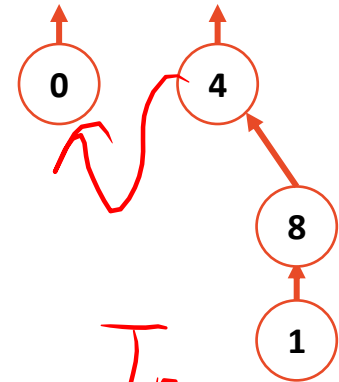
②



Disjoint Sets Union

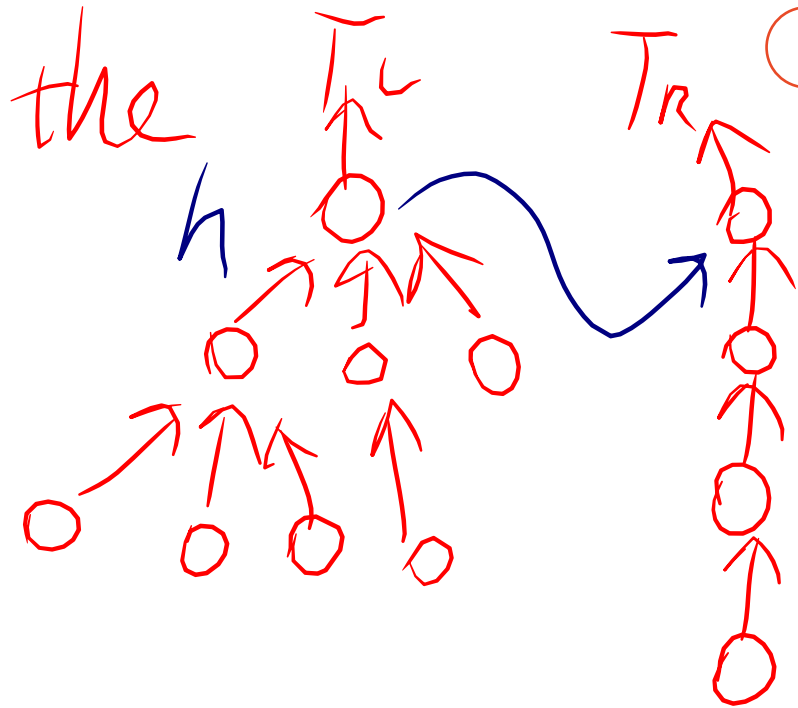
Union(0, 4)

```
1 void DisjointSets::union(int r1, int r2) {  
2     S[r2] = r1;  
3 }  
4
```



① Union the tree w/ the smallest height

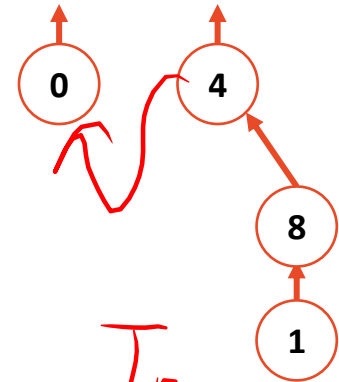
②



Disjoint Sets Union

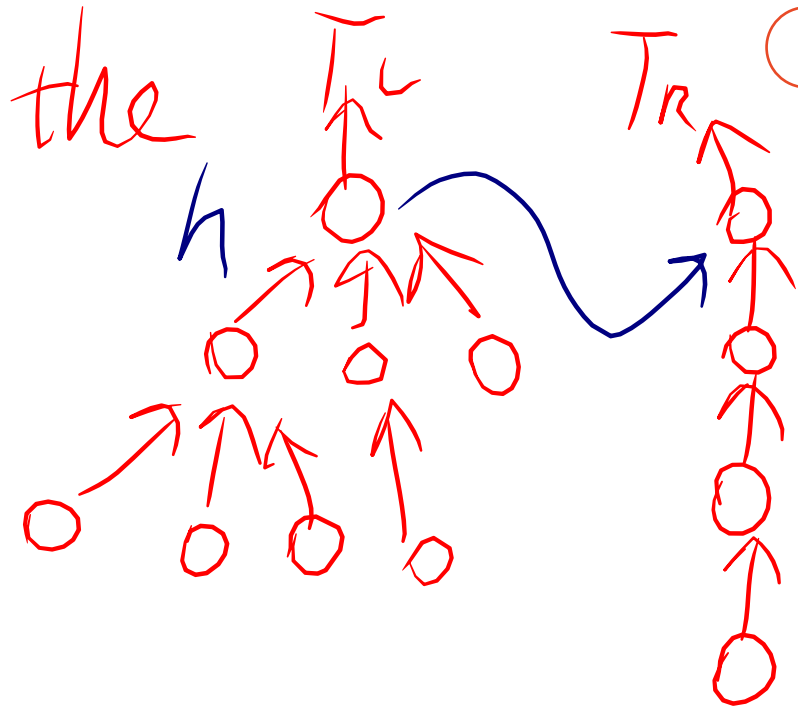
union(0, 4)

```
1 void DisjointSets::union(int r1, int r2) {  
2     S[r2] = r1;  
3 }  
4
```



① Union the tree w/ the smallest height

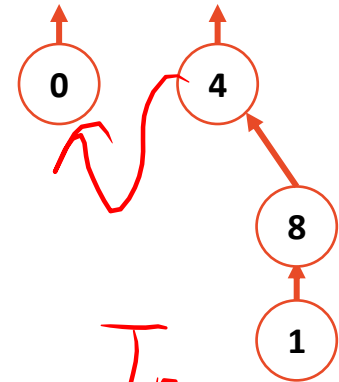
② |S₁| vs |S₂|



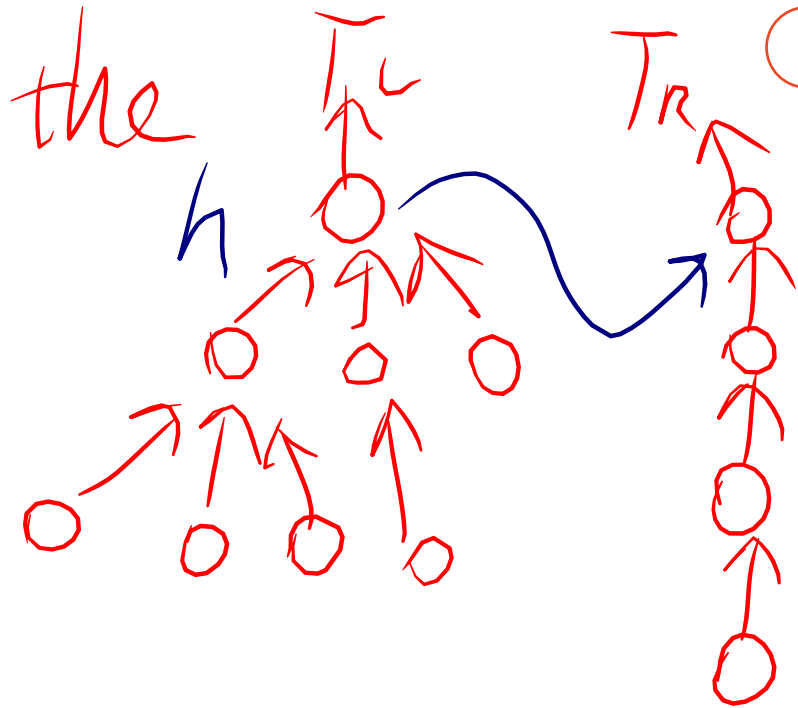
Disjoint Sets Union

Union(0, 4)

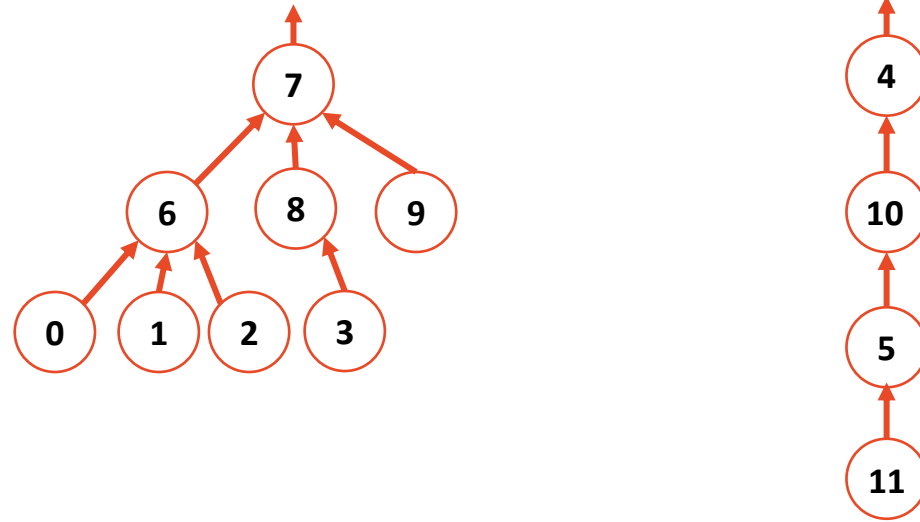
```
1 void DisjointSets::union(int r1, int r2) {  
2     S[r2] = r1;  
3 }  
4
```



- ① Union the tree w/ the smallest height
- ② $|S_1|$ vs $|S_2|$.
Union by set size



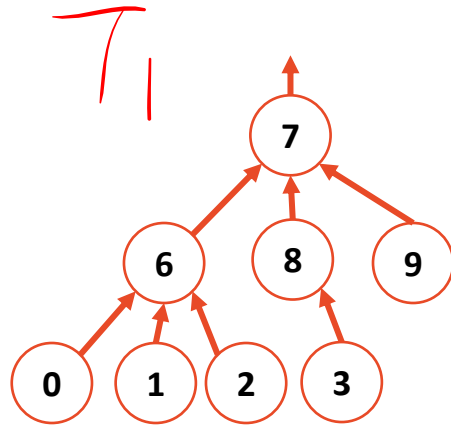
Disjoint Sets – Union



0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	-1	10	7	-1	7	7	4	5

Disjoint Sets – Union

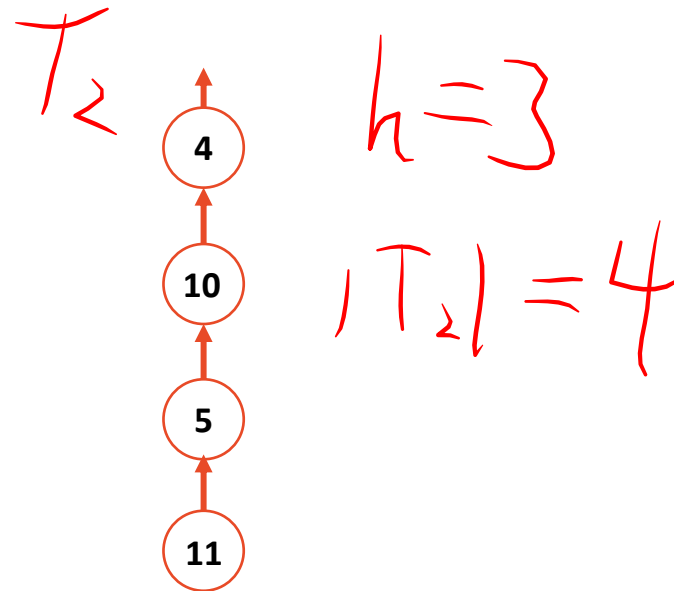
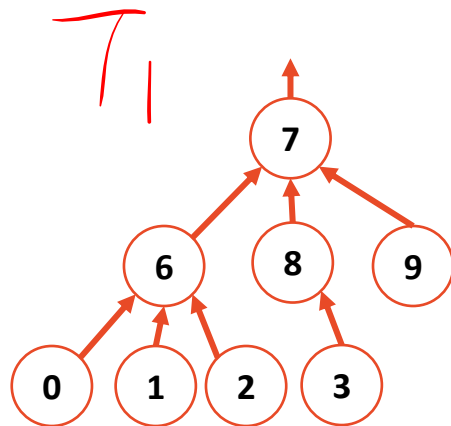
$h=2$
 $|T_1|=8$



0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	-1	10	7	-1	7	7	4	5

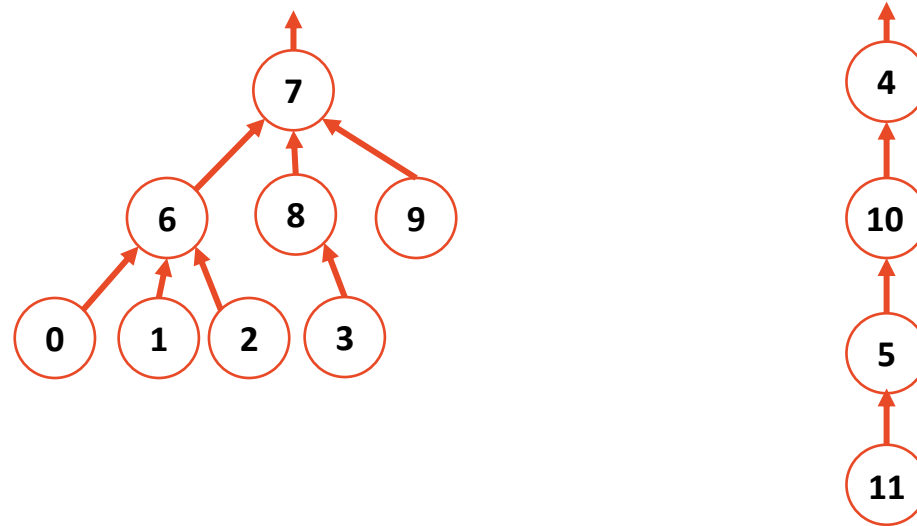
Disjoint Sets – Union

$h=2$
 $|T_1|=8$



0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	-1	10	7	-1	7	7	4	5

Disjoint Sets – Smart Union



Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Keep the height of the tree as small as possible.

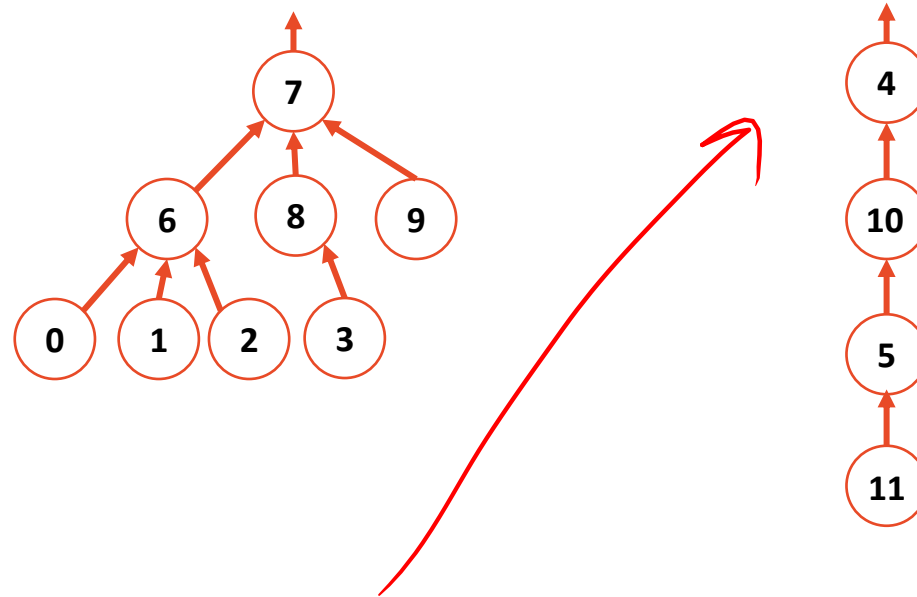
Union by size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union



Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	3	10	7		7	7	4	5

Idea: Keep the height of the tree as small as possible.

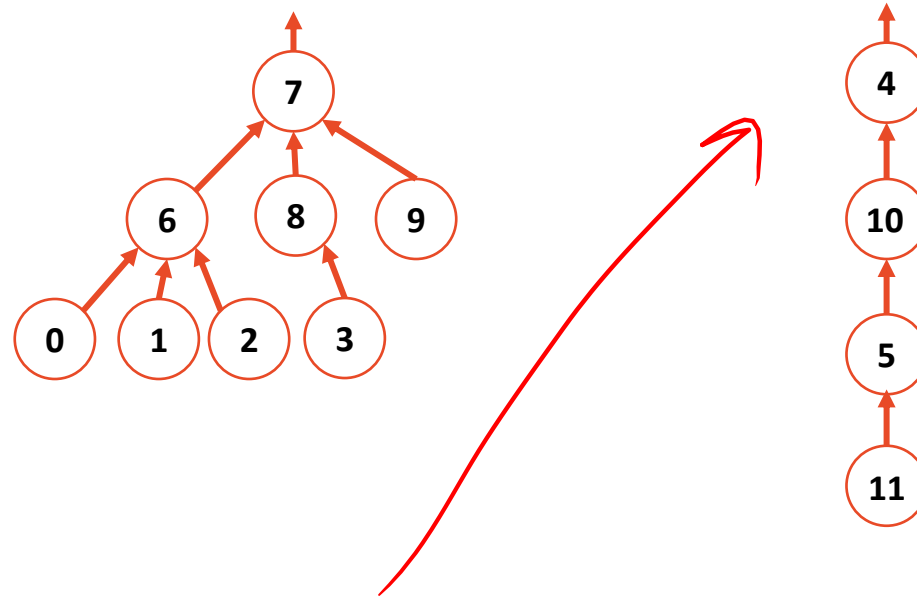
Union by size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union



Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	3	10	7	7	7	7	4	5

Idea: Keep the height of the tree as small as possible.

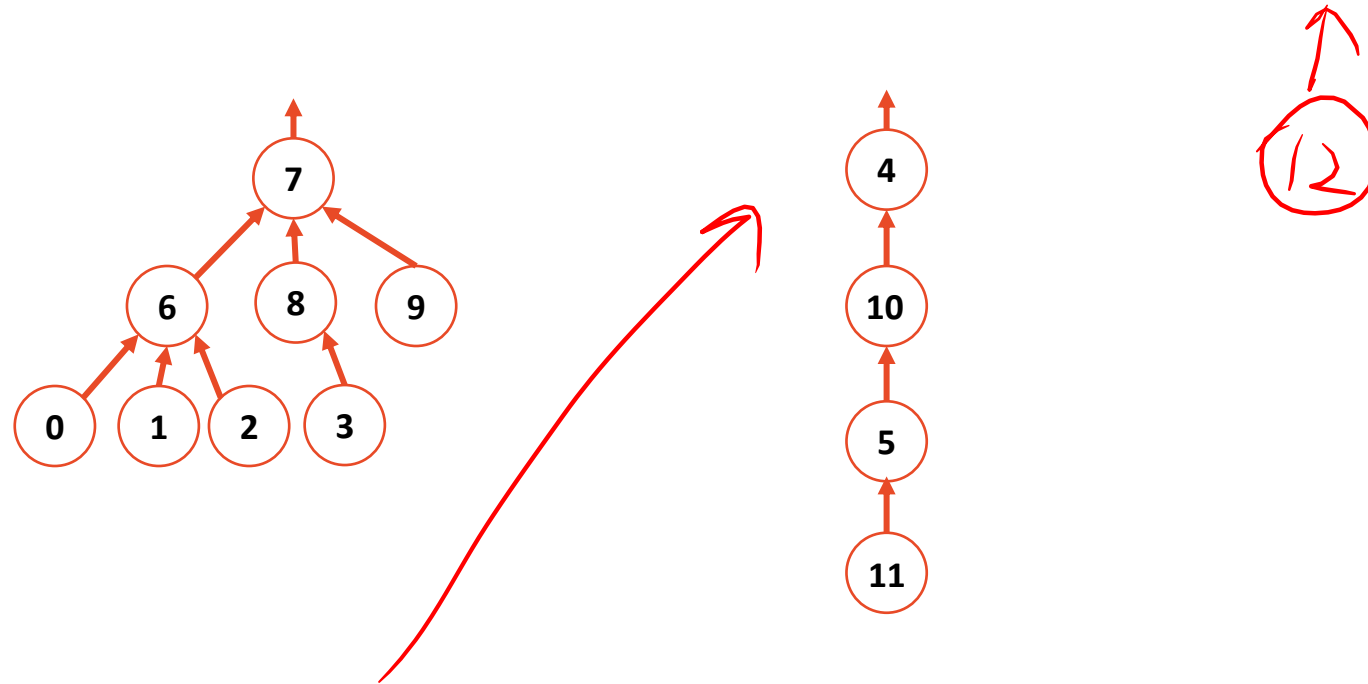
Union by size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union



Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	3	10	7	7	7	7	4	5

Idea: Keep the height of the tree as small as possible.

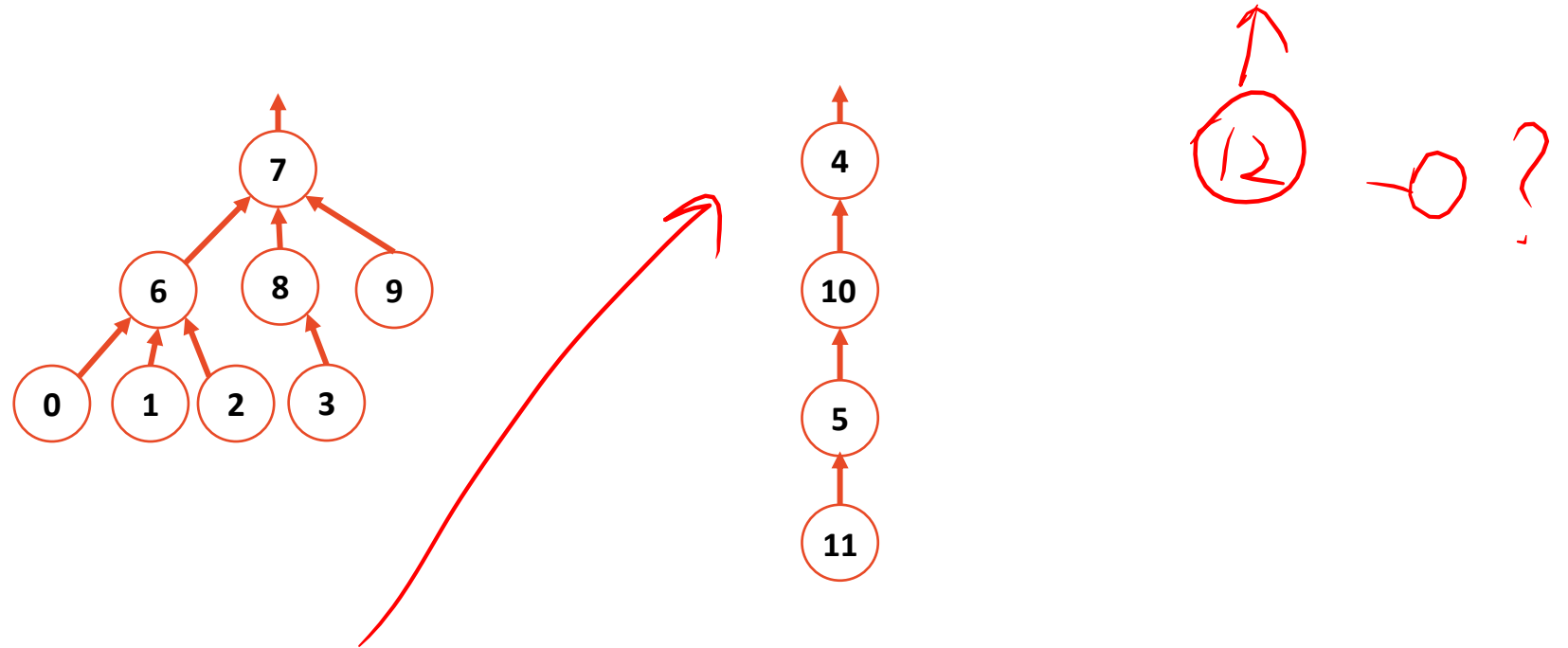
Union by size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union



Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	3	10	7	7	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

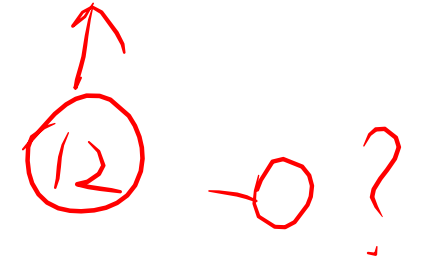
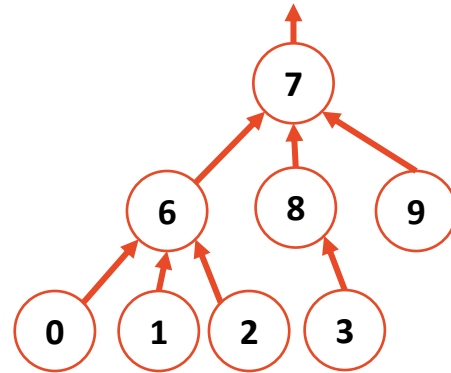
0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union

Store $-h-1$



Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	3	10	7	7	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

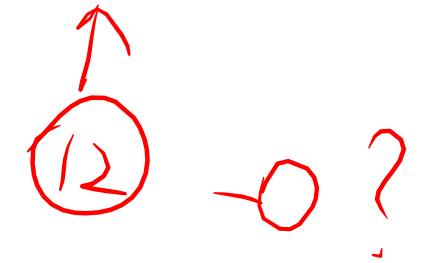
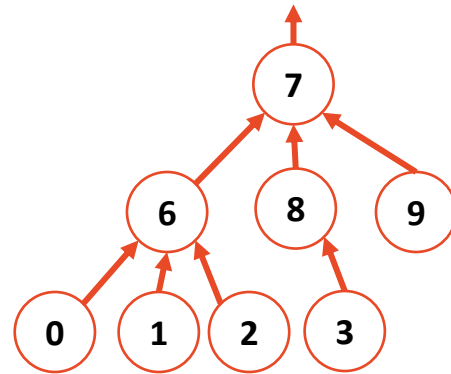
0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union

Store $-h-1$



Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	3	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

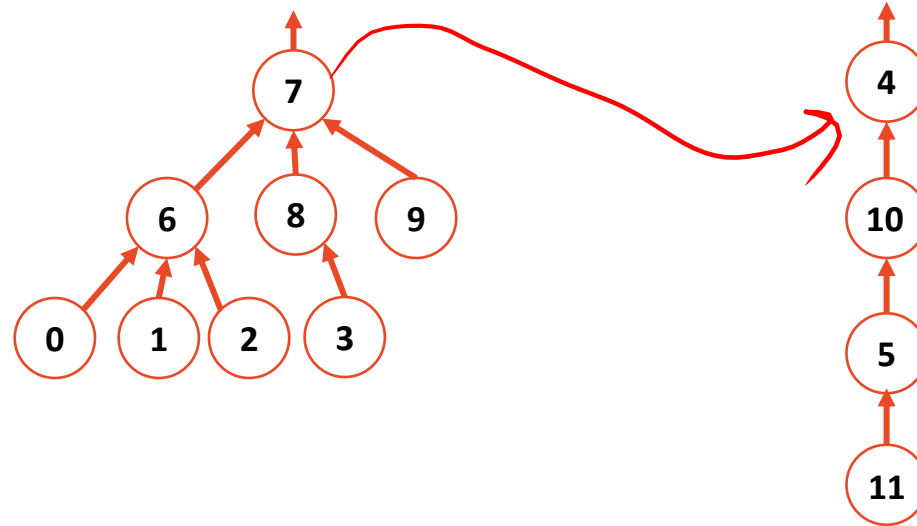
0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union

Store $-h-1$



12 - 0?

Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	3	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

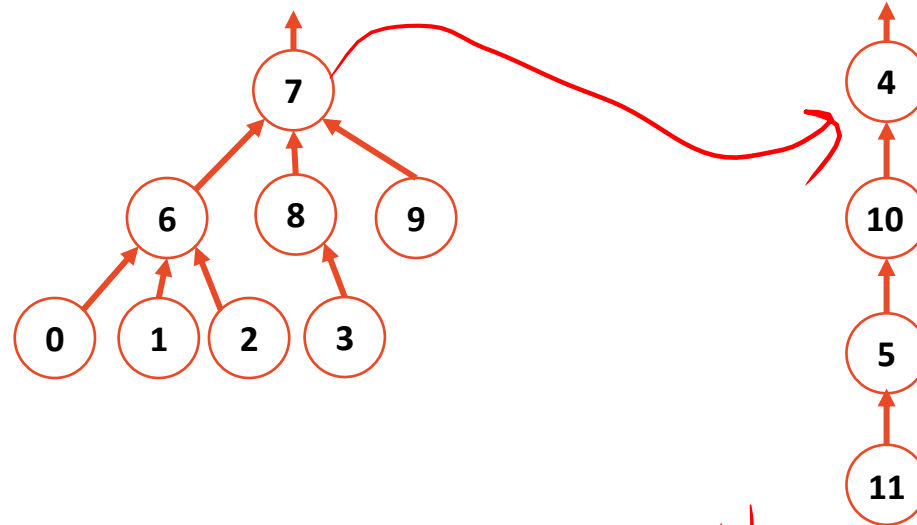
0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union

Store $-h-1$



12 - 0?

Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

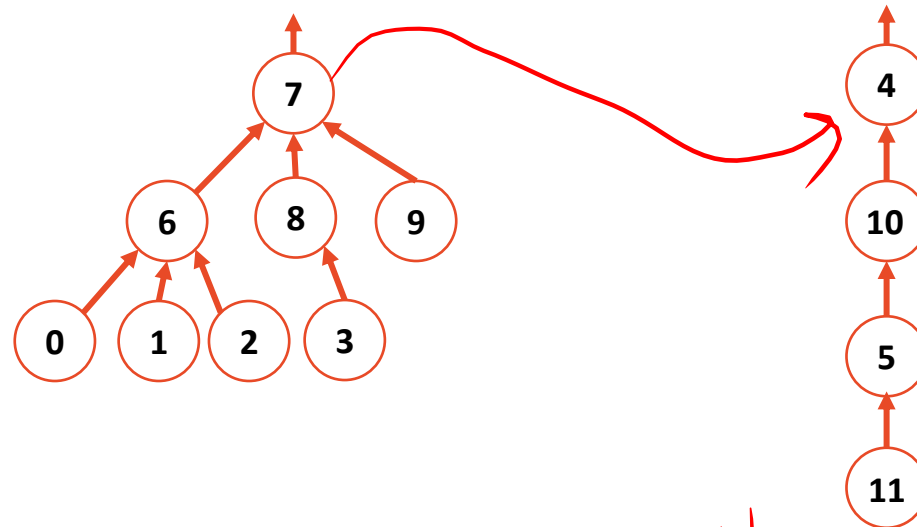
0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union

Store $-h-1$



12 - 0?

Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

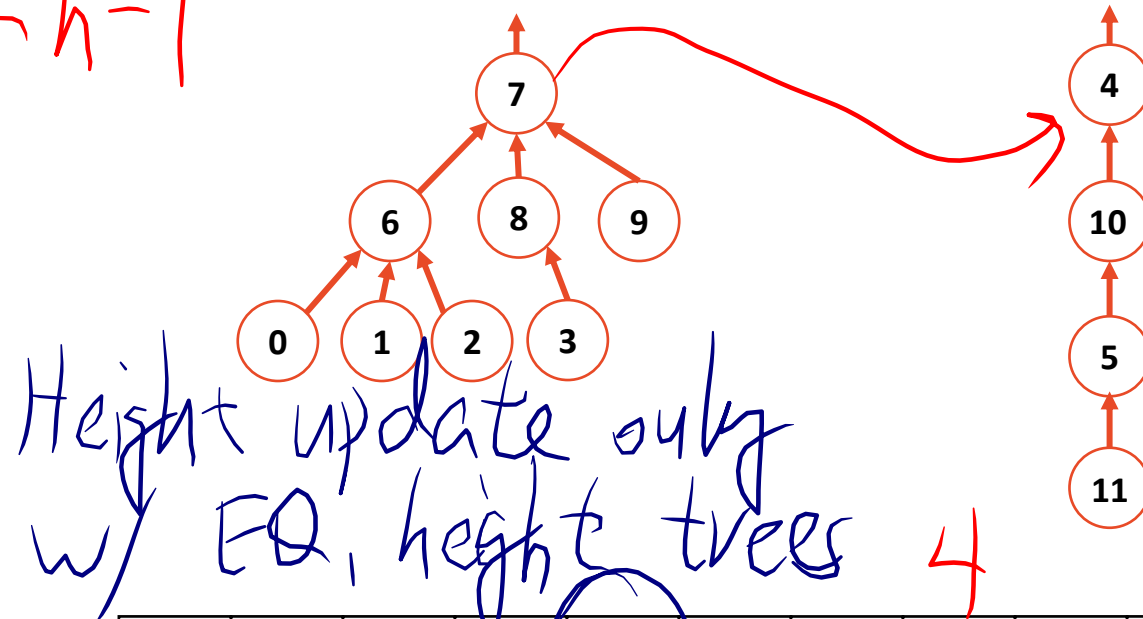
0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union

Store $-h-1$



12 - 0?

Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

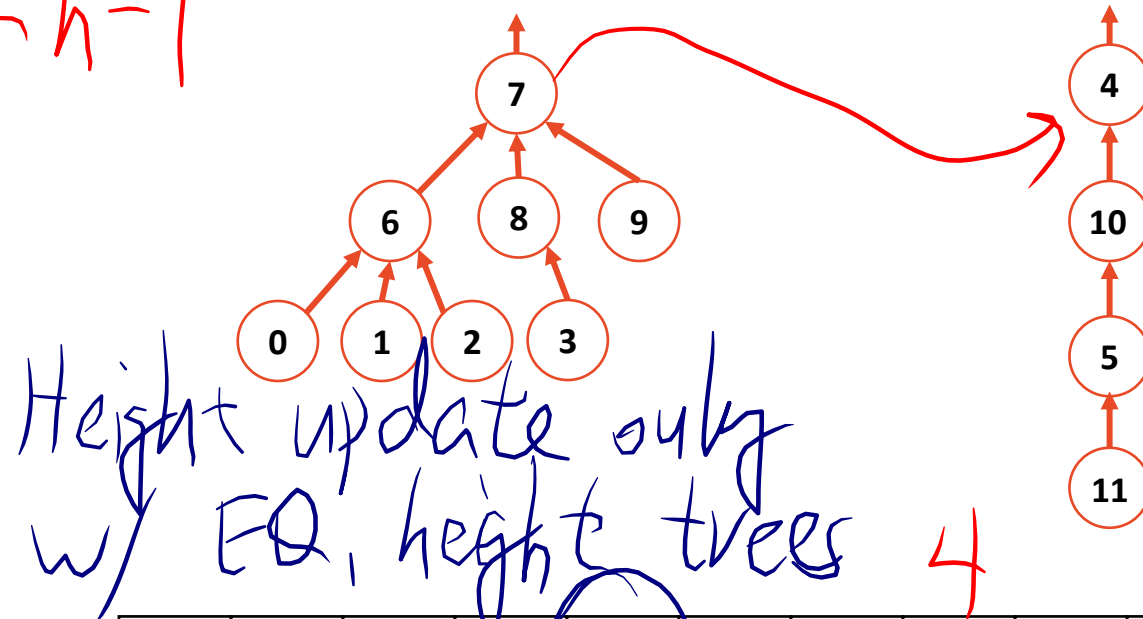
0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union

Store $-h-1$



12 → 0?

Union by height

size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

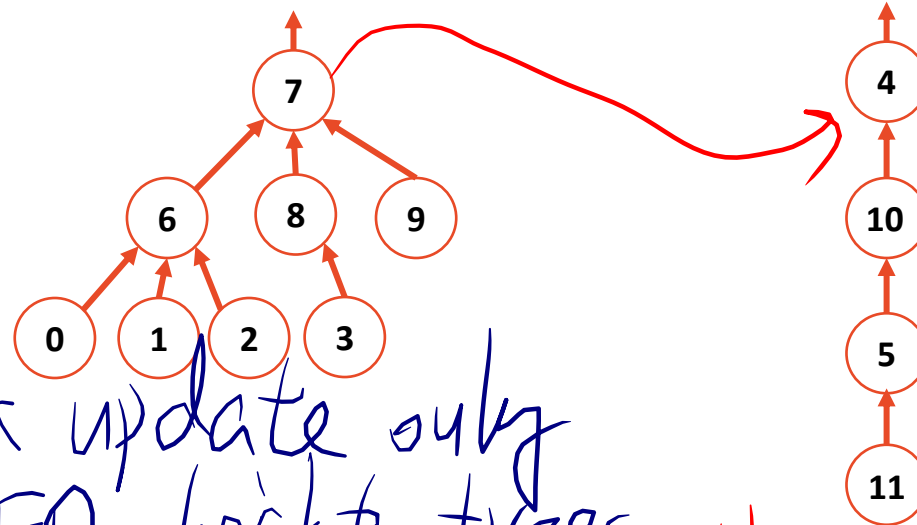
0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8		10	7		7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union

Store $-h-1$



Height update only
w/ EQ, height trees

Union by height

-size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

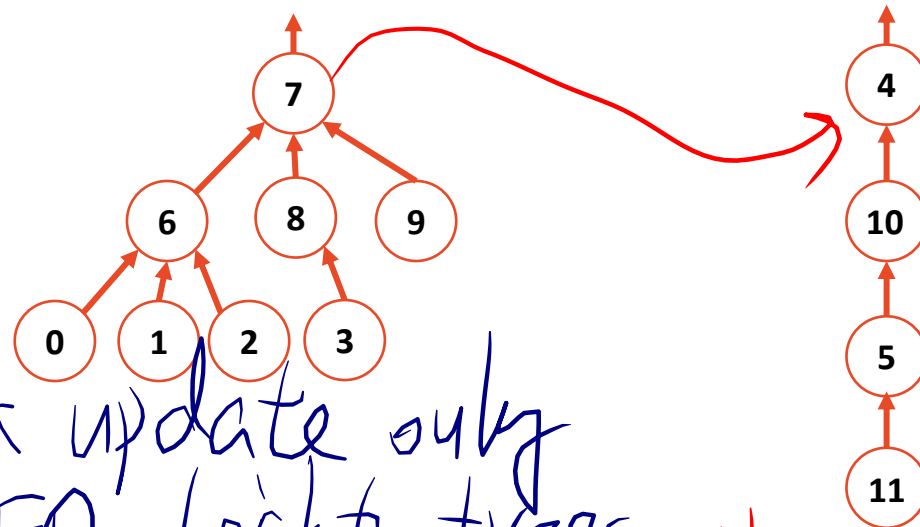
0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union

Store $-h-1$



12 - 0?

Height update only
w/ EQ, height trees

Union by height

-size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

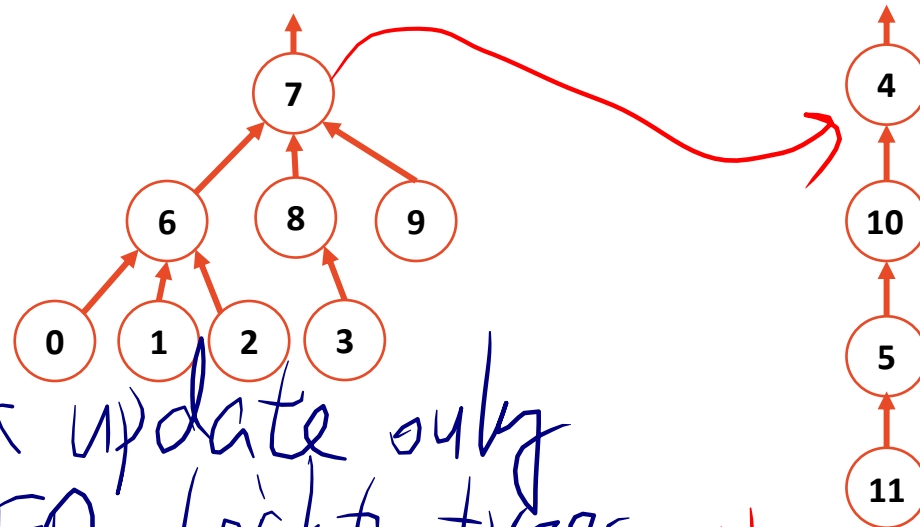
0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: _____.

Disjoint Sets – Smart Union

Store $-h-1$



Height update only
w/ EQ, height trees

Union by height

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Keep the height of the tree as small as possible.

Union by size

0	1	2	3	4	5	6	7	8	9	10	11
6	6	6	8	4	10	7	8	7	7	4	5

Idea: Minimize the number of nodes that increase in height

Both guarantee the height of the tree is: $O(\lg n)$

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

```
1 void DisjointSets::unionBySize(int root1, int root2) {  
2     int newSize = arr_[root1] + arr_[root2];  
3  
4     // If arr_[root1] is less than (more negative), it is the larger set;  
5     // we union the smaller set, root2, with root1.  
6     if ( arr_[root1] < arr_[root2] ) {  
7         arr_[root2] = root1;  
8         arr_[root1] = newSize;  
9     }  
10  
11     // Otherwise, do the opposite:  
12     else {  
13         arr_[root1] = root2;  
14         arr_[root2] = newSize;  
15     }  
16 }
```

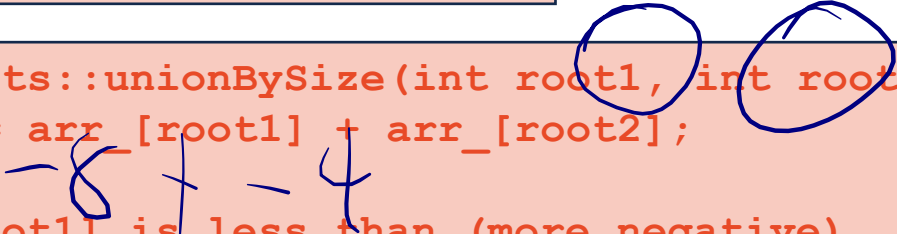
Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

```
1 void DisjointSets::unionBySize(int root1, int root2) {  
2     int newSize = arr_[root1] + arr_[root2];  
3  
4     // If arr_[root1] is less than (more negative), it is the larger set;  
5     // we union the smaller set, root2, with root1.  
6     if ( arr_[root1] < arr_[root2] ) {  
7         arr_[root2] = root1;  
8         arr_[root1] = newSize;  
9     }  
10  
11     // Otherwise, do the opposite:  
12     else {  
13         arr_[root1] = root2;  
14         arr_[root2] = newSize;  
15     }  
16 }
```

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

```
1 void DisjointSets::unionBySize(int root1, int root2) {  
2     int newSize = arr[root1] + arr[root2];  
3       
4     // If arr[root1] is less than (more negative), it is the larger set;  
5     // we union the smaller set, root2, with root1.  
6     if ( arr[root1] < arr[root2] ) {  
7         arr[root2] = root1;  
8         arr[root1] = newSize;  
9     }  
10  
11     // Otherwise, do the opposite:  
12     else {  
13         arr[root1] = root2;  
14         arr[root2] = newSize;  
15     }  
16 }
```

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

```
1 void DisjointSets::unionBySize(int root1, int root2) {  
2     int newSize = arr[root1] + arr[root2];  
3  
4     // If arr[root1] is less than (more negative), it is the larger set;  
5     // we union the smaller set, root2, with root1.  
6     if ( arr[root1] < arr[root2] ) {  
7         arr[root2] = root1;  
8         arr[root1] = newSize;  
9     }  
10  
11     // Otherwise, do the opposite:  
12     else {  
13         arr[root1] = root2;  
14         arr[root2] = newSize;  
15     }  
16 }
```

Handwritten notes:

- Handwritten calculation: $-8 + -4$
- Handwritten circles around `root1` and `root2` in the function signature.
- Handwritten circles around the `if` block (lines 6-9) and the `else` block (lines 12-15).

Disjoint Sets Find

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

```
1 void DisjointSets::unionBySize(int root1, int root2) {  
2     int newSize = arr[root1] + arr[root2];  
3  
4     // If arr[root1] is less than (more negative), it is the larger set;  
5     // we union the smaller set, root2, with root1.  
6     if ( arr[root1] < arr[root2] ) {  
7         arr[root2] = root1;  
8         arr[root1] = newSize;  
9     }  
10  
11     // Otherwise, do the opposite:  
12     else {  
13         arr[root1] = root2;  
14         arr[root2] = newSize;  
15     }  
16 }
```

Given two roots 0(1)

Disjoint Sets Find

Find runs in $O(\log n)$

```
1 int DisjointSets::find(int i) {  
2     if ( s[i] < 0 ) { return i; }  
3     else { return find( s[i] ); }  
4 }
```

```
1 void DisjointSets::unionBySize(int root1, int root2) {  
2     int newSize = arr[root1] + arr[root2];  
3  
4     // If arr[root1] is less than (more negative), it is the larger set;  
5     // we union the smaller set, root2, with root1.  
6     if ( arr[root1] < arr[root2] ) {  
7         arr[root2] = root1;  
8         arr[root1] = newSize;  
9     }  
10  
11     // Otherwise, do the opposite:  
12     else {  
13         arr[root1] = root2;  
14         arr[root2] = newSize;  
15     }  
16 }
```

Given two roots $O(1)$